

ISC26 Tutorial: In-Network Computing with SmartNICs for HPC Applications

Presenters: Jeffrey Young, Antonio J. Peña, Elie Kfoury, Richard Graham
Special Thanks to: Paul Ruth, Muhammad Usman, Aaron Jezghani

Audience Survey

Please follow presenter instructions to take a short survey on SmartNIC usage!



Menti.com
1548 8849



Tutorial Agenda

Agenda (Timing)

See the updated agenda at <https://github.com/gt-crnc-h-rg/smartnic-tutorial-isc26>



Please share your feedback in a short 4 question survey!

Time	Topic	Details
9:00-9:10	Introduction	Attendee Survey; intro to Hands-On
9:10-9:25	Communication Offloading	SmartNIC overview DPU examples DPU programming models
9:25-9:40	SmartNIC Use Cases	Packet processing, cyber-security, AI/HPC
9:40-9:55	Infrastructure SW – DOCA and P4 Introduction	Brief introduction to DOCA/P4
10:00-10:30	HPC Programming – Part 1	MPI Offload; DPA Programming
10:30-11:00	HPC Programming – Part 2	OpenMP Offload
11:00-11:30	BREAK	
11:30-11:45	Hands-on DOCA/P4	Attendees follow along in compiling and running test examples
11:45-11:55	Hands-on DPA Demo	
11:55-12:15	Hands-on HPC Demo – OpenMP/MPI Offload	
12:15-12:30	Storage Acceleration	Discussion of use case for SmartNICs with storage systems for AI
12:30-12:35	Tutorial Survey	
12:35-1:00	Continued Hands-on Activities	Time to work with the instructors on the presented demos and test cases



Hands-On Preparation

Hands-On Preparation

Users can test the following codes during today's tutorial:

- ODOS-MPI, DOCA, and DPA test cases

Please check the instructions for accessing the testbed from our public GitHub repo at:

<https://github.com/gt-crunch-rg/smartnic-tutorial-isc26/>



Introduction

Tutorial Objectives

Describe how SmartNICs can be used as HPC/AI accelerators

Describe how users can benefit from acceleration engines in their current and future applications

Describe how one can use one family of SmartNICs to accelerate applications and communication libraries

Share application and programming experiences using SmartNICs with canned examples and some hands-on examples

Anatomy of a supercomputer

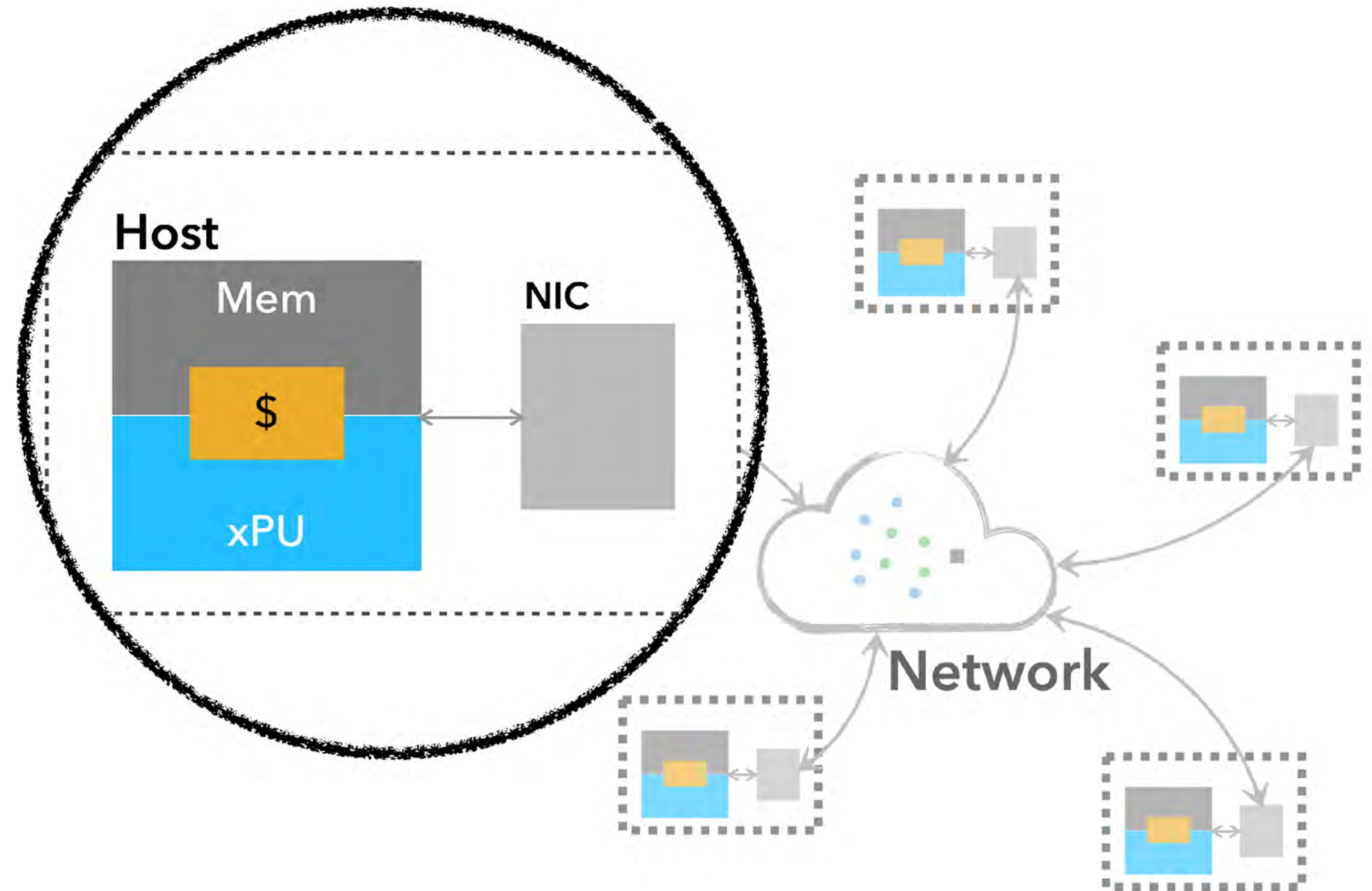
The basic building block of a distributed-memory cluster or supercomputer is a node.

Each node includes a host, which is a processor (xPU) + memory hierarchy.

The host can communicate with other hosts via its NIC (network interface controller).

A [network](#) connects the nodes. The nodes may be arranged in some topology, which determines the network's carrying capacity and cost.

Node



DPU in modern clusters

The basic building block of a distributed-memory cluster or supercomputer is a node.

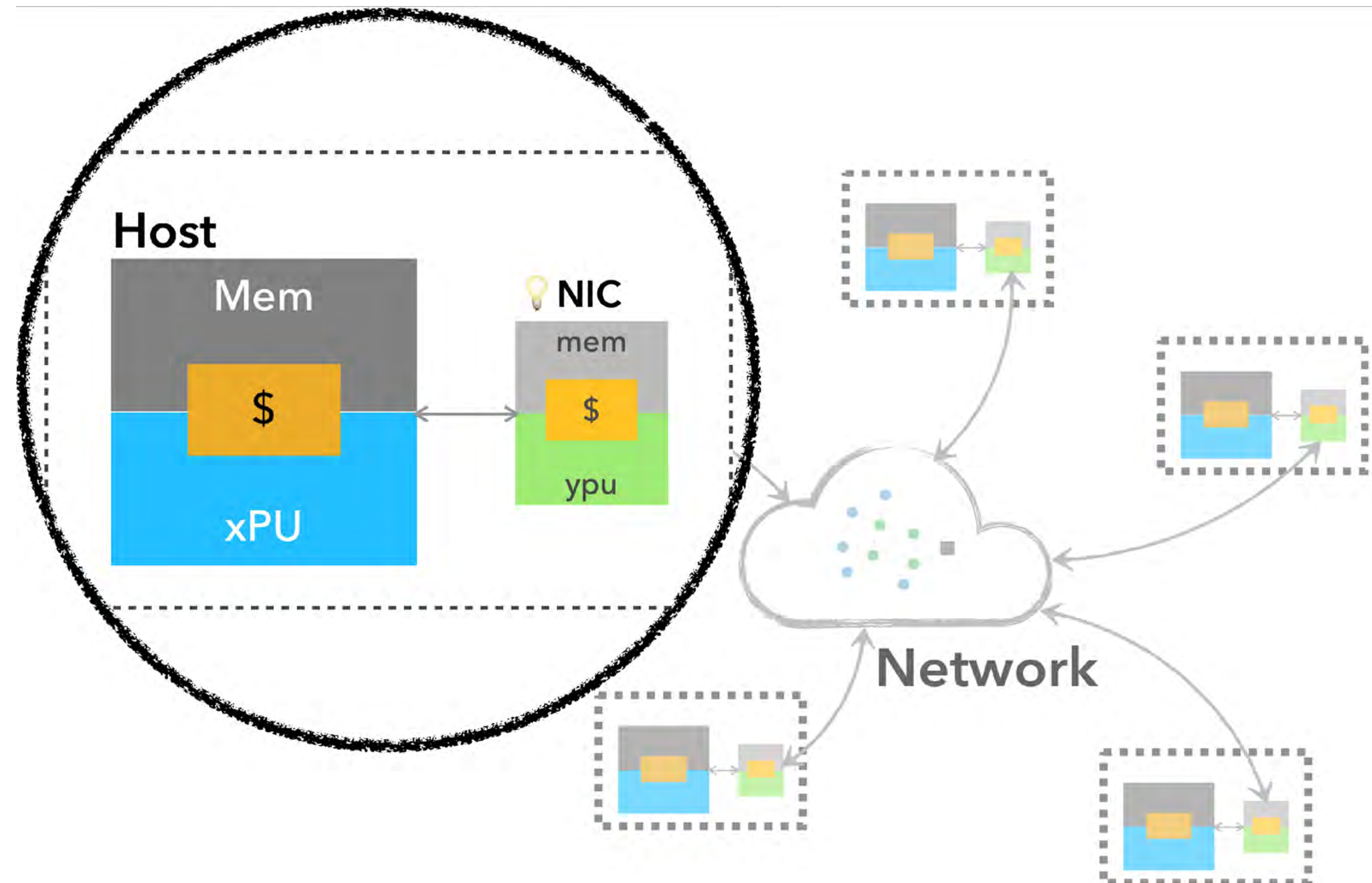
Each node includes a host, which is a processor (xPU) + memory hierarchy.

The host can communicate with other hosts via its NIC (network interface controller).

A network connects the nodes. The nodes may be arranged in some topology, which determines the network's carrying capacity and cost.

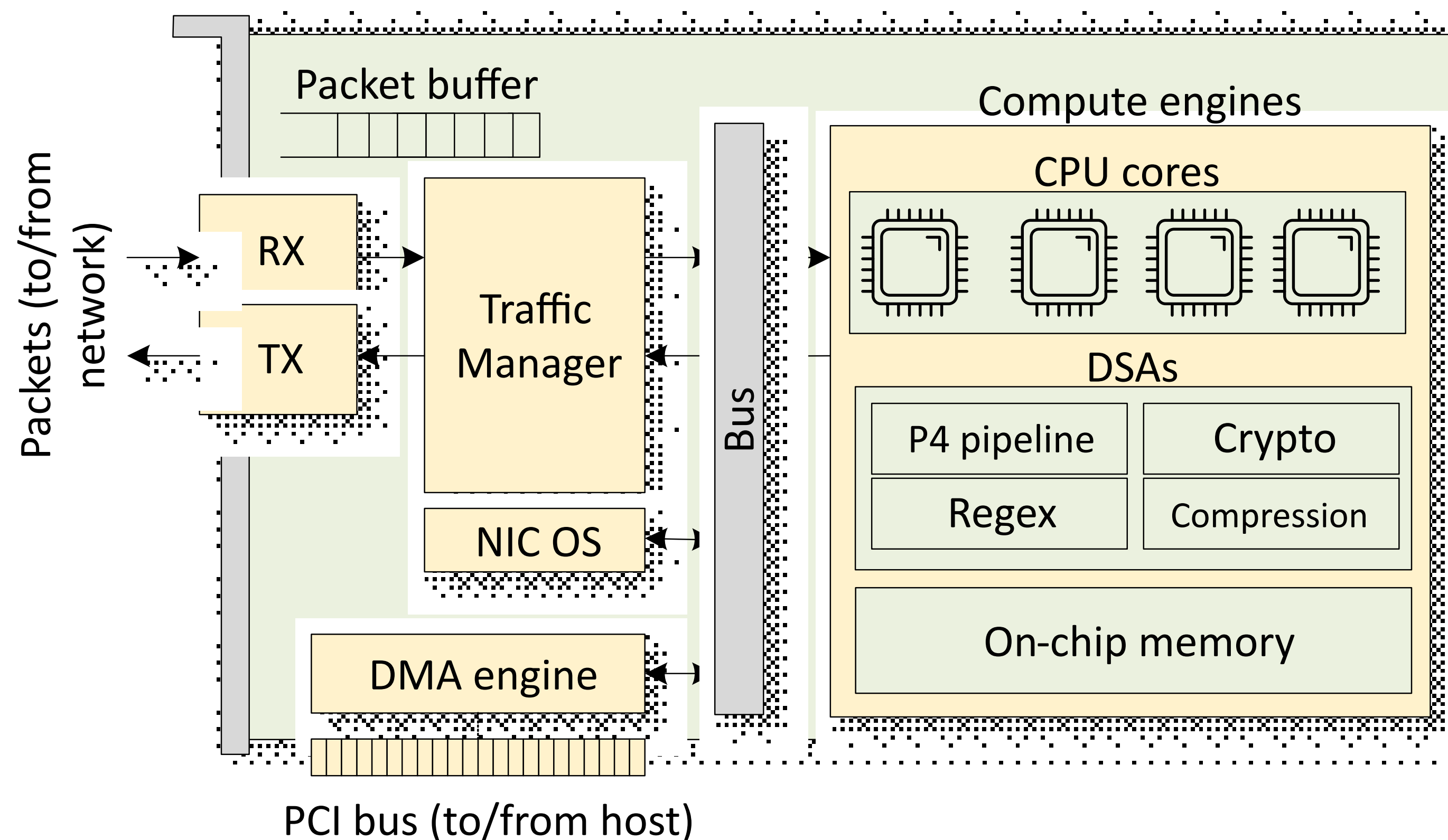
In a DPU, the NIC becomes “host-like” via the addition of processing (ypu), memory and other accelerations engines.

Node



Data Processing Units (DPUs)

- DPUs use Domain Specific Accelerators (DSAs) to offload processing from general purpose CPUs
 - Programmable packet processing pipeline, regular expression, encryption/decryption, etc.
 - The DSAs are typically implemented on ASIC or FPGAs
- DPUs also include general-purpose CPU cores for managing the system





SmartNICs and Data Processing Unit (DPU) Background

SmartNICs Available Today

NVIDIA – BlueField family of Data Processing Units (ARM CPU)

AMD – Pensando and Alveo FPGA

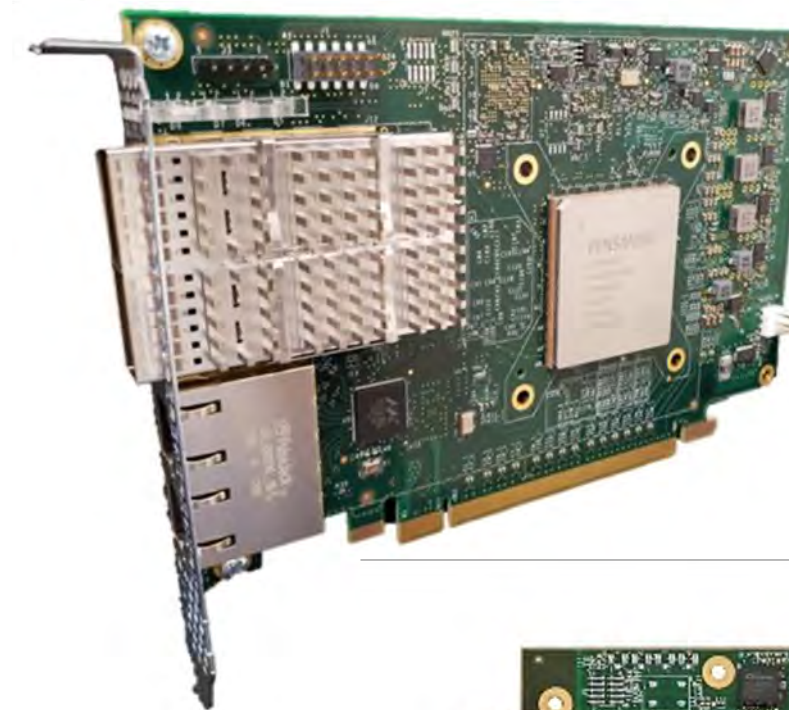
Mangoboost

Intel Infrastructure Processing Units (IPUs)

Intel FPGA variants (Silicom)

Marvell Octeon (DPU with ARM based CPU)

Research projects like sPIN



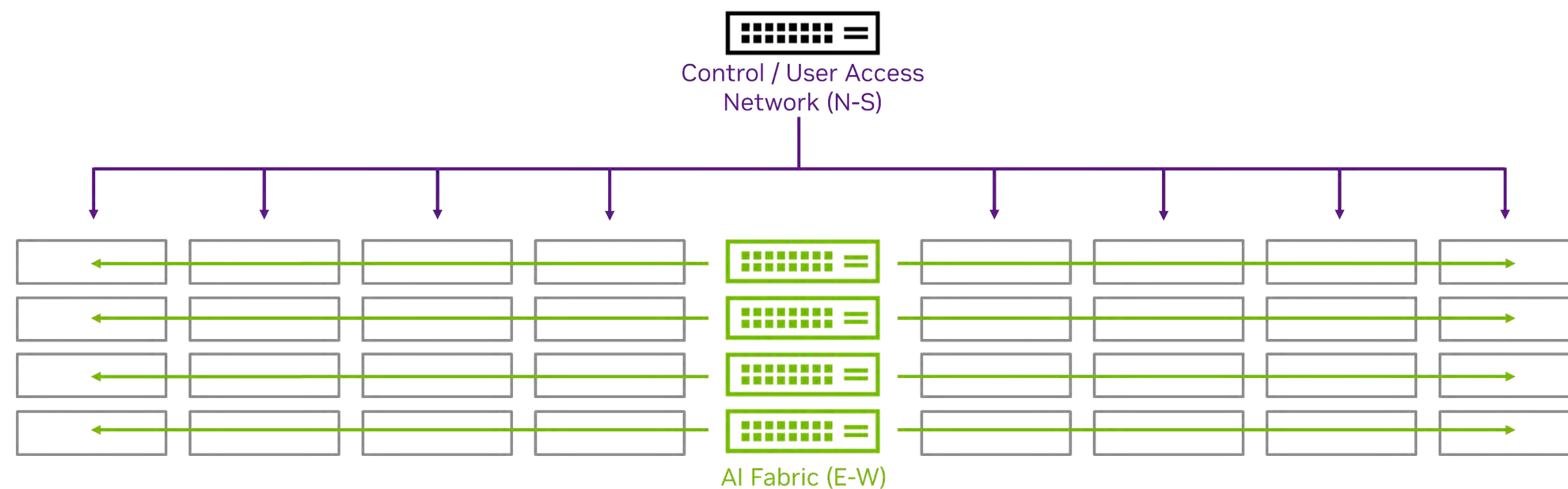
SmartNICs in Tomorrow's Supercomputers

ORNL's Lux Supercomputer will include Pensando DPUs

NVIDIA AI Factories have BlueField-4 DPUs as an integral component



SmartNIC Networking Communication Types



Control / User Access Network (N-S)

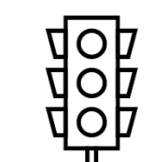
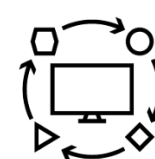
Loosely-Coupled Applications, No Isolation Required

TCP (Low Bandwidth Flows and Utilization)

High Jitter Tolerance, average as key indicator

Heterogeneous Traffic, Statistical Multi-Pathing

Low fabric utilization, forgiving of link failures



Computational Fabric (E-W)

Tightly-Coupled Processes, Tenant Isolation Required

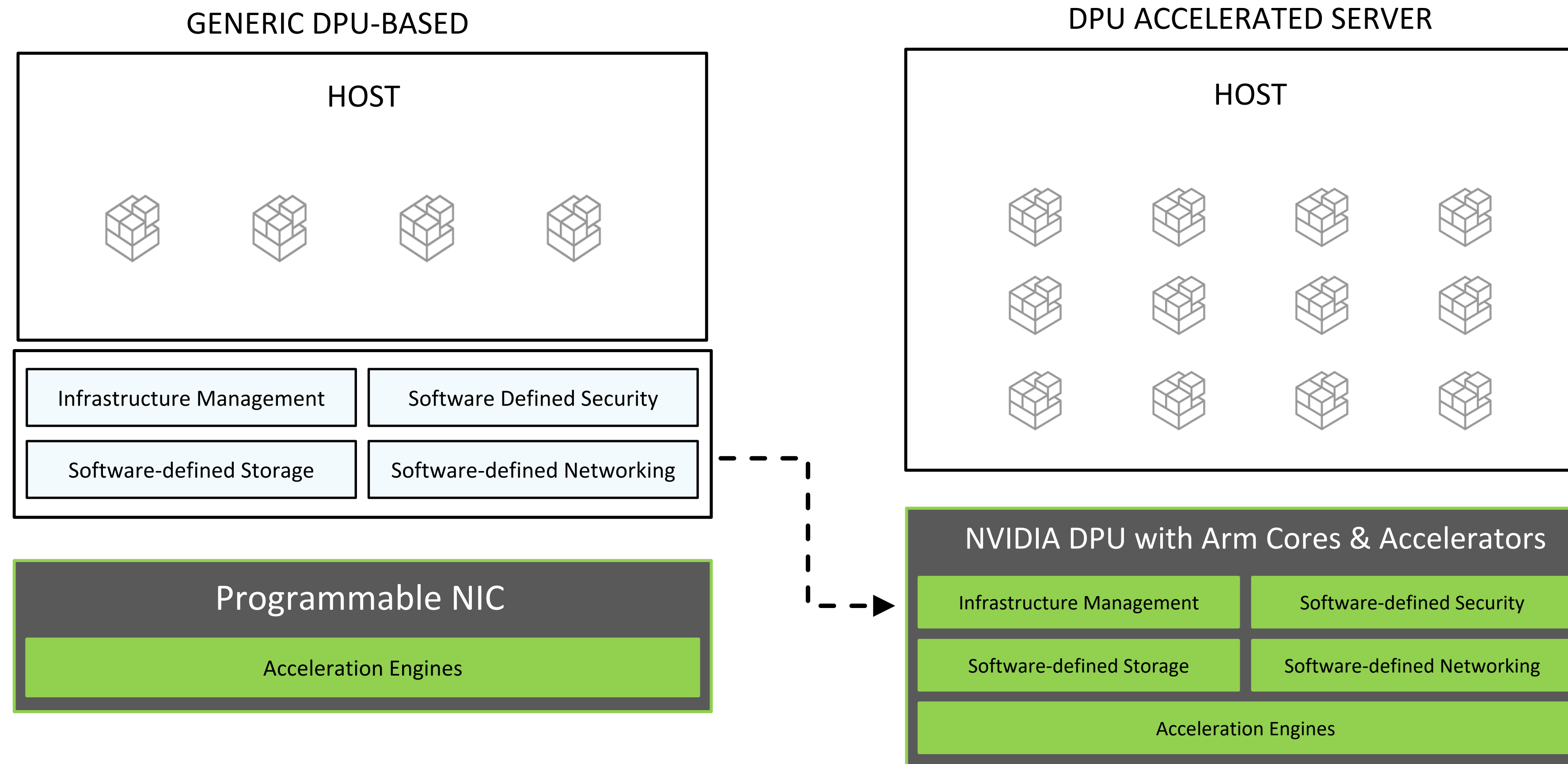
RDMA (High Bandwidth Flows and Utilization)

Low Jitter Tolerance, P99 is what matters

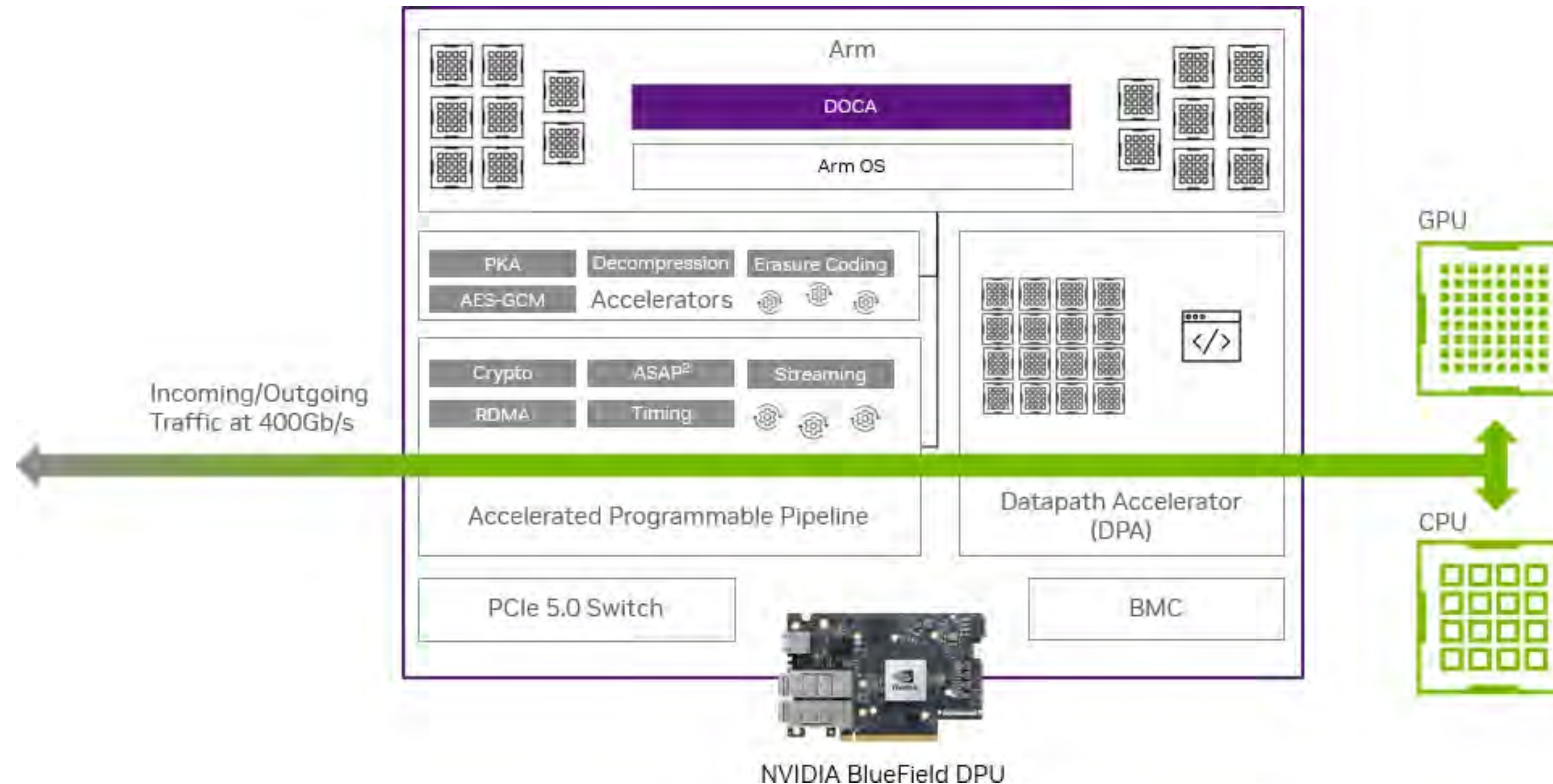
Bursty Network Capacity, Predictive Performance

High link utilization, high penalty for failure and slow recovery

NVIDIA's BlueField Data Processing Unit

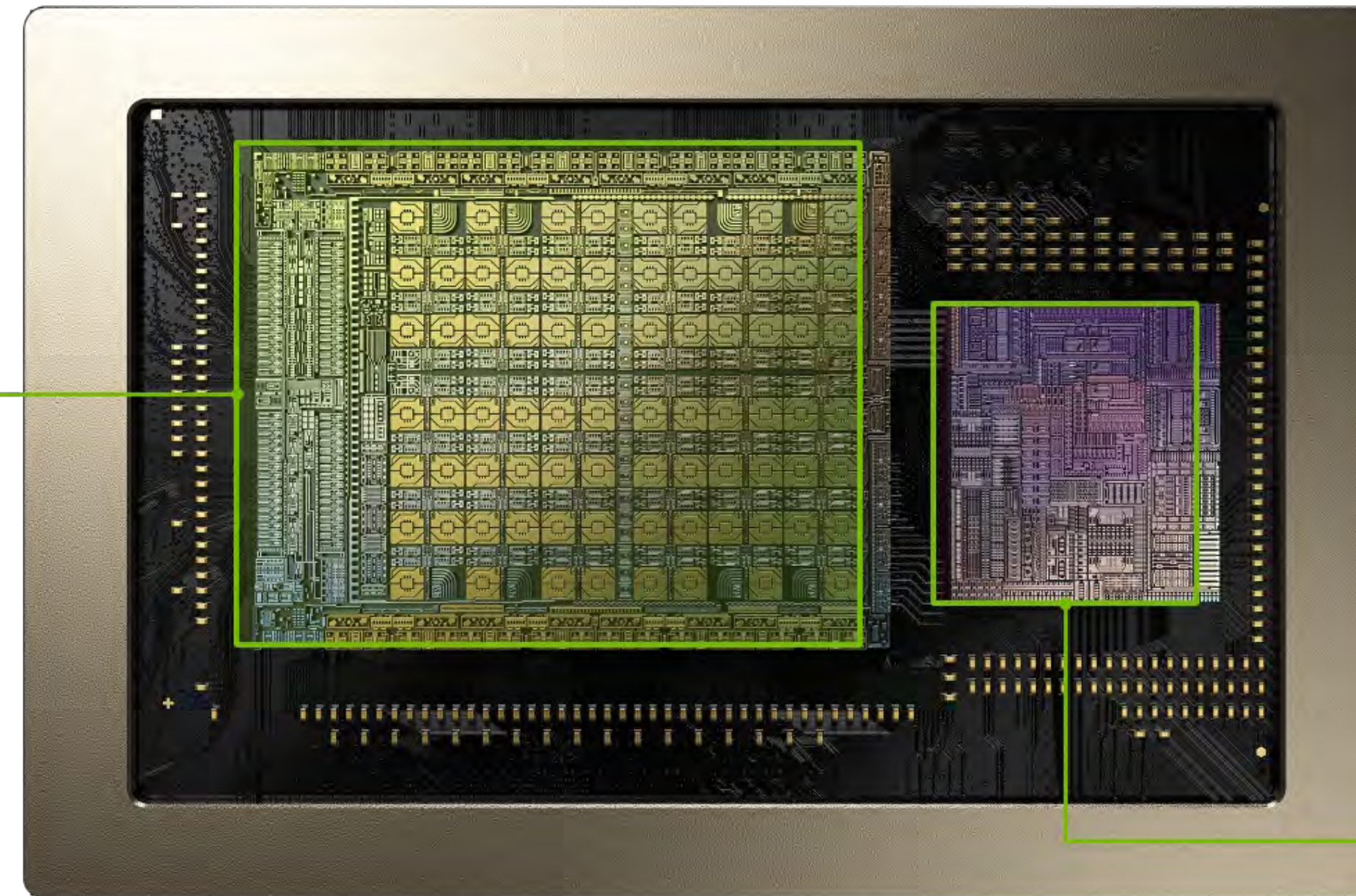


BlueField-3 Architecture



The BlueField-3 includes programmable pipelines (P4), datapath accelerators (DPAs), Arm cores, and DSAs
 Different accelerators match better to north-south (Arm cores) or east-west communication (DPAs, DSAs)

NVIDIA BlueField-4 Architecture



NVIDIA CPU Compute

- 64 Arm Neoverse V2™ @1.7GHz
- 114 MB shared L3 cache
- 128GB LPDDR5x@8533MT/s

800G BW Ethernet / InfiniBand
 200G PAM4 SerDes
 PCIe Gen6 x16

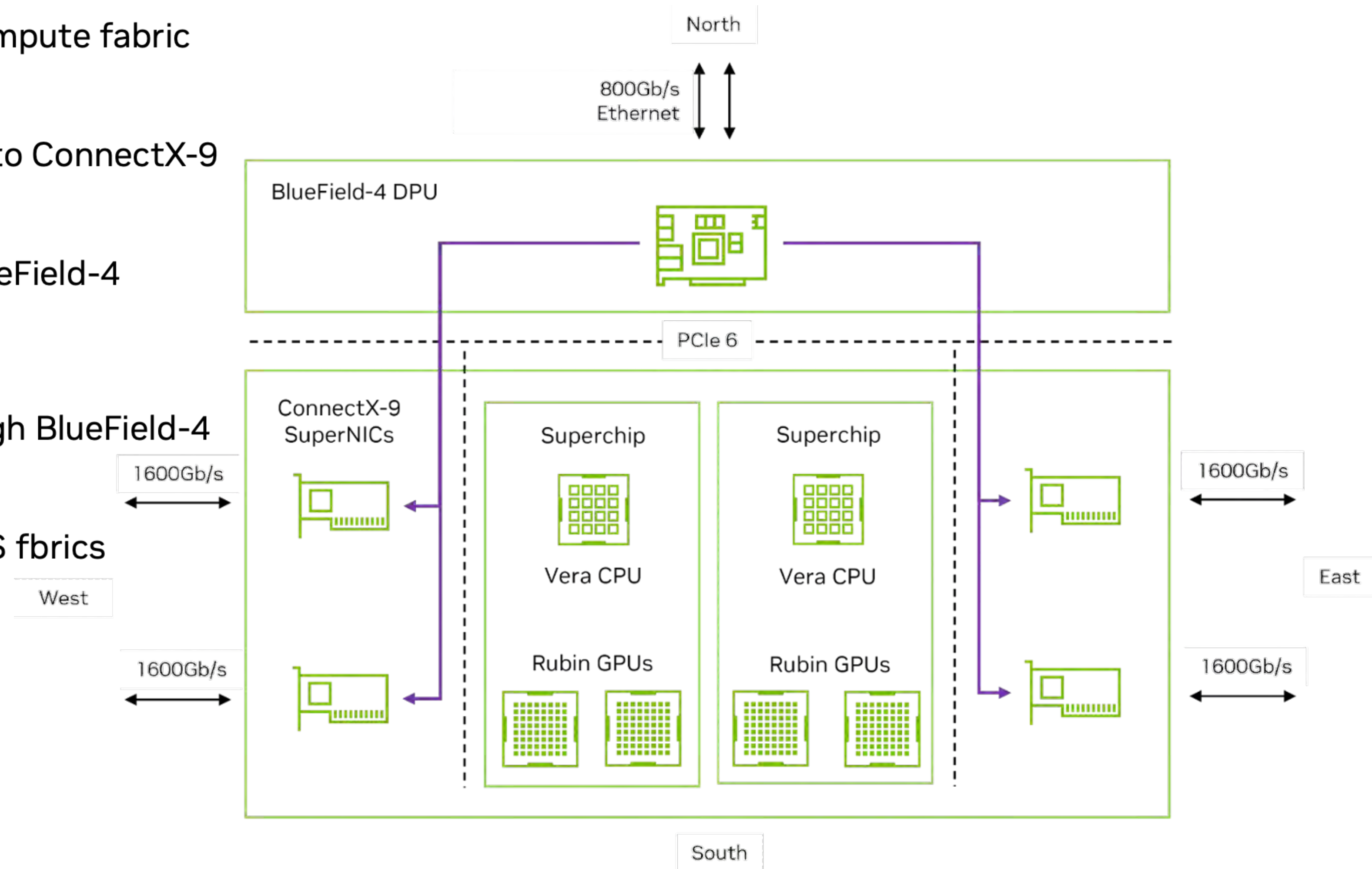
NVIDIA ConnectX-9 Networking

- RDMA/ROCE, Congestion Control
- Programmable Transport
- Inline encryption (PSP, TLS, IPSEC)

1GbE out-of-band management
 PCIe x2 interface for
 M.2 512GB SSD

BlueField-4 ASTRA

- Secure provisioning and management of AI compute fabric
- Dedicated OOB connectivity from BlueField-4 to ConnectX-9
- Management functions isolated within the BlueField-4 domain, separate from host and tenants
- Unified control of N-S and E-W domains through BlueField-4
- Consistent security model across E-W and N-S fabrics





DPU Programming Models and Frameworks

High-level HPC Programming Models

Three types of HPC programming models are available to program DPUs:

Distributed and distributed-shared memory

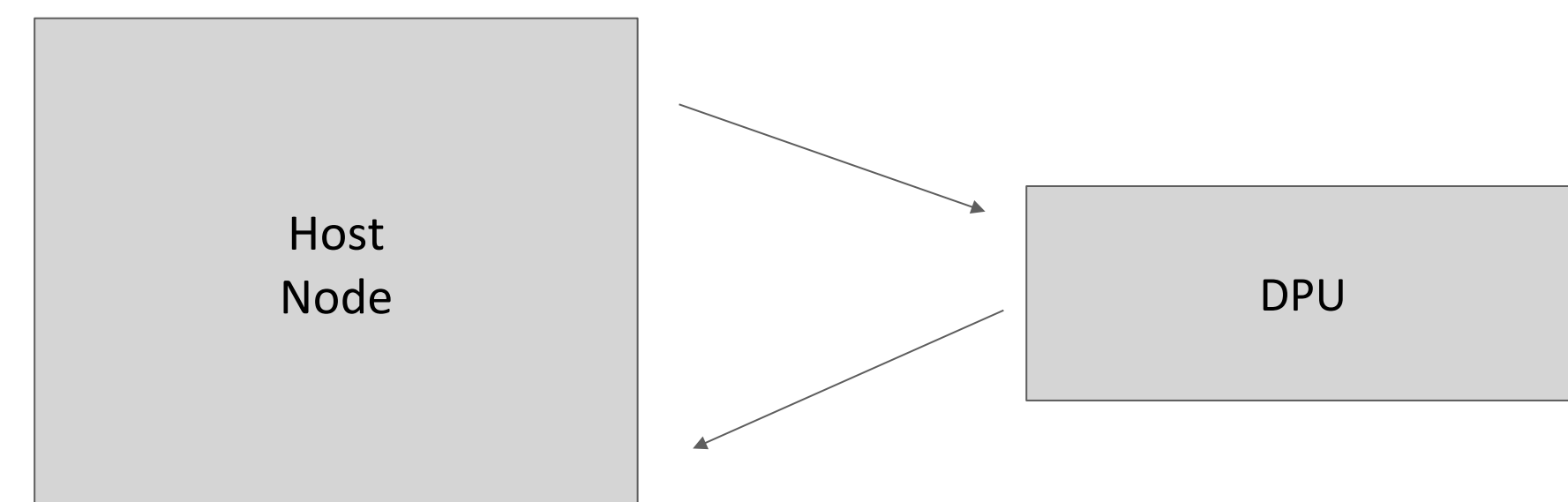
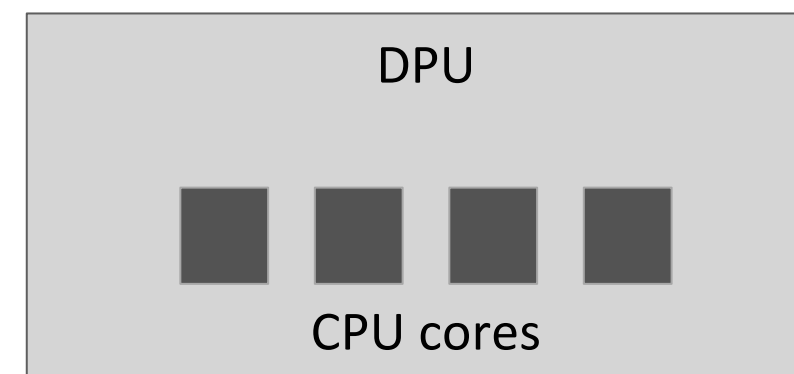
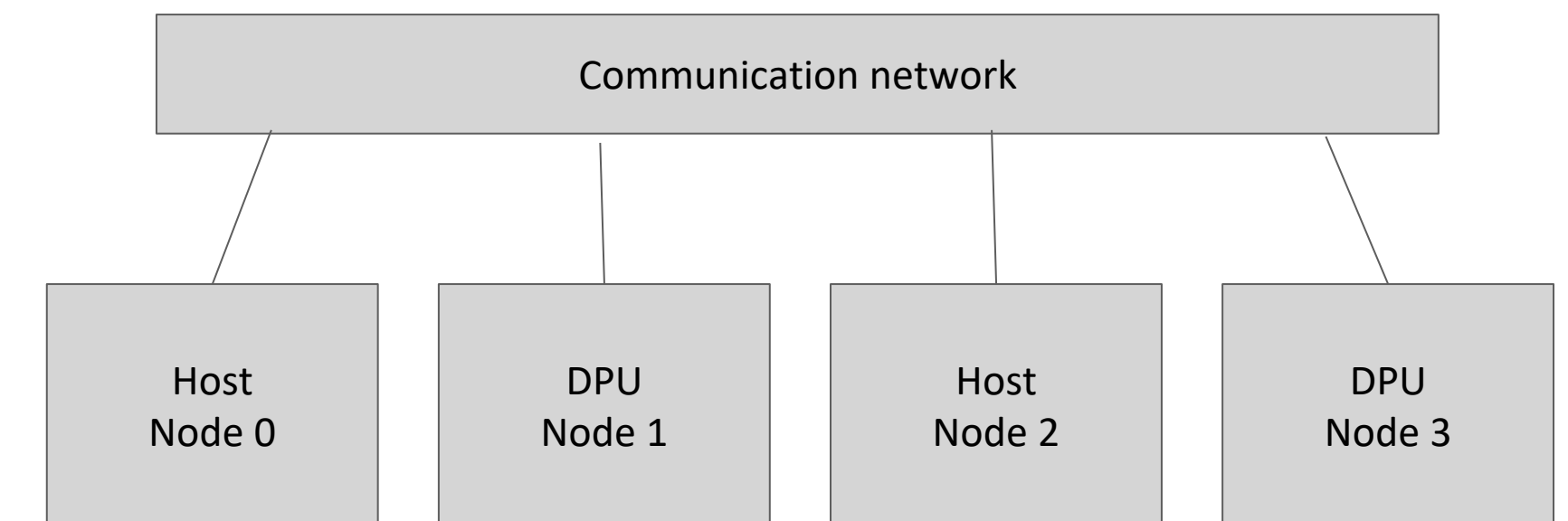
- **MPI – send/receive based**
- OpenSHMEM – put/get based

Shared memory

- OpenMP (shared memory)

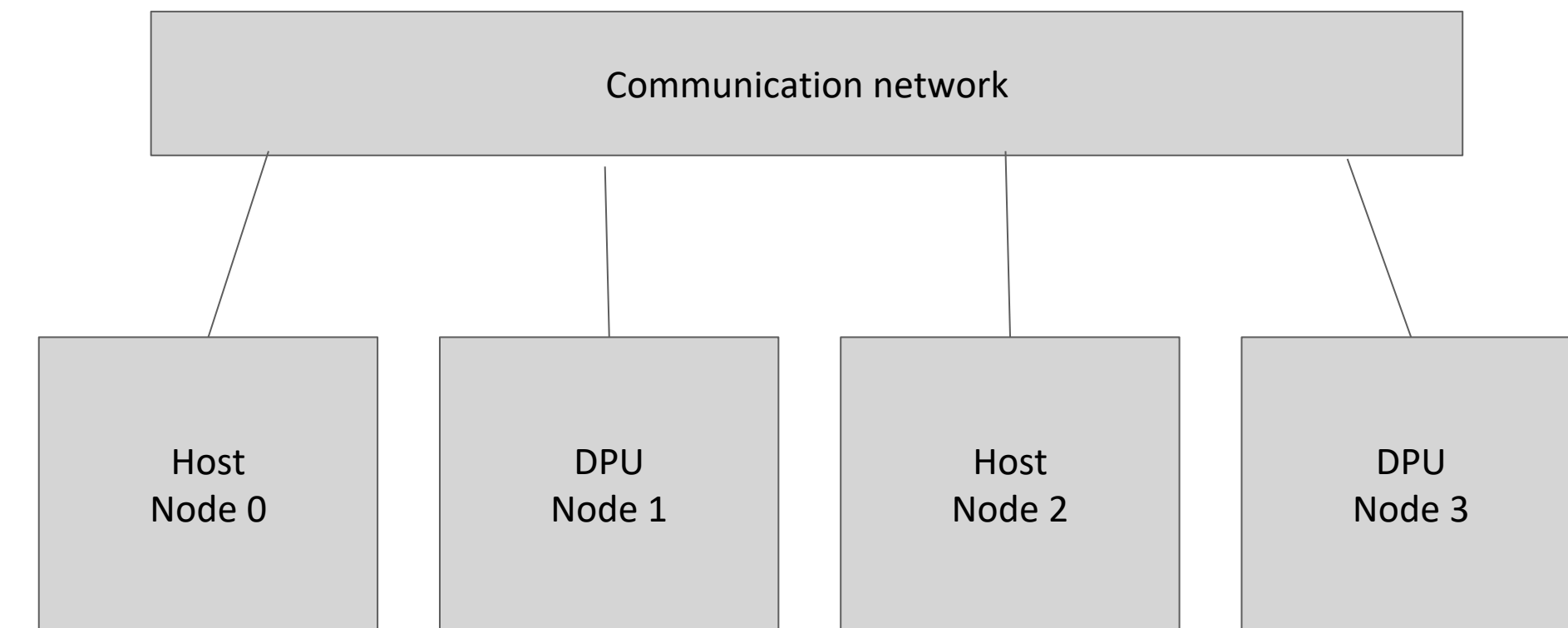
Accelerator offloading

- MPI, OpenSHMEM
- **OpenMP offload**



“Offloading network operations”

Distributed Programming Model



Ranks are mapped to compute nodes and DPUs.

A DPU is treated as another node in the system and its rank can be part of a communicator or sub communicator.

DPU ranks will have different computing capabilities

Potential use-cases:

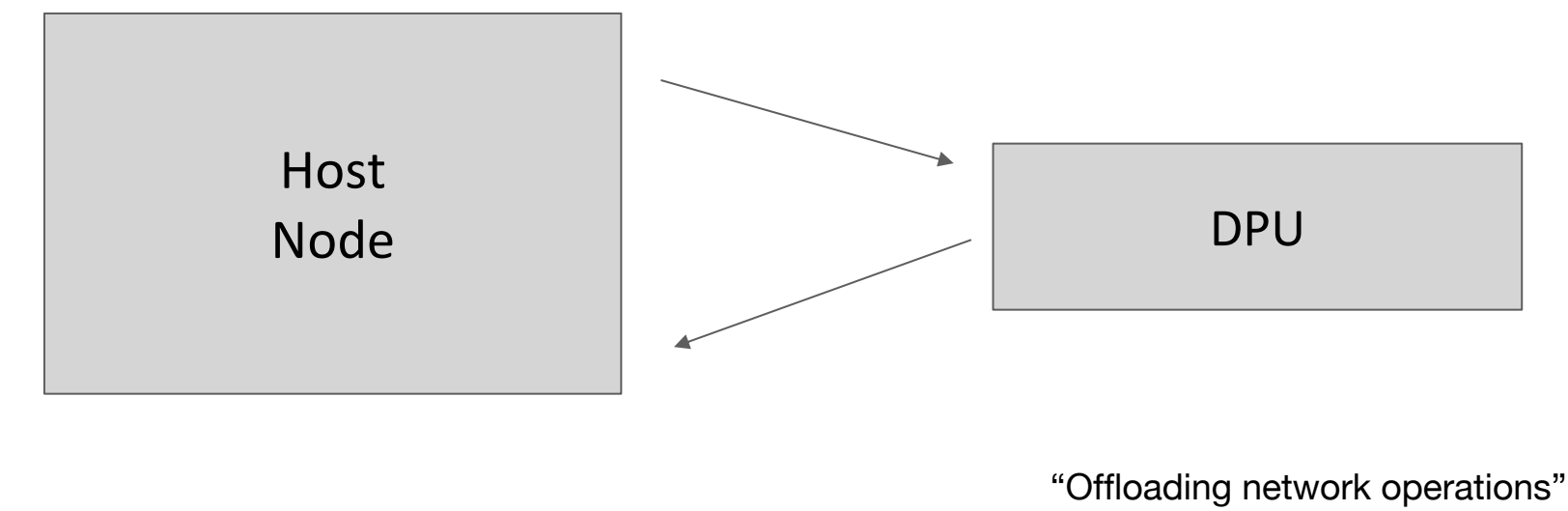
- Application is decomposed to use DPUs for computation or I/O

- Examples:

 - DPUs cores are used for computation

 - DPUs handle the I/O ranks of an application

Accelerator Offload Model



Two model views:

- Programming model exposes the accelerator

 - When accelerator is exposed

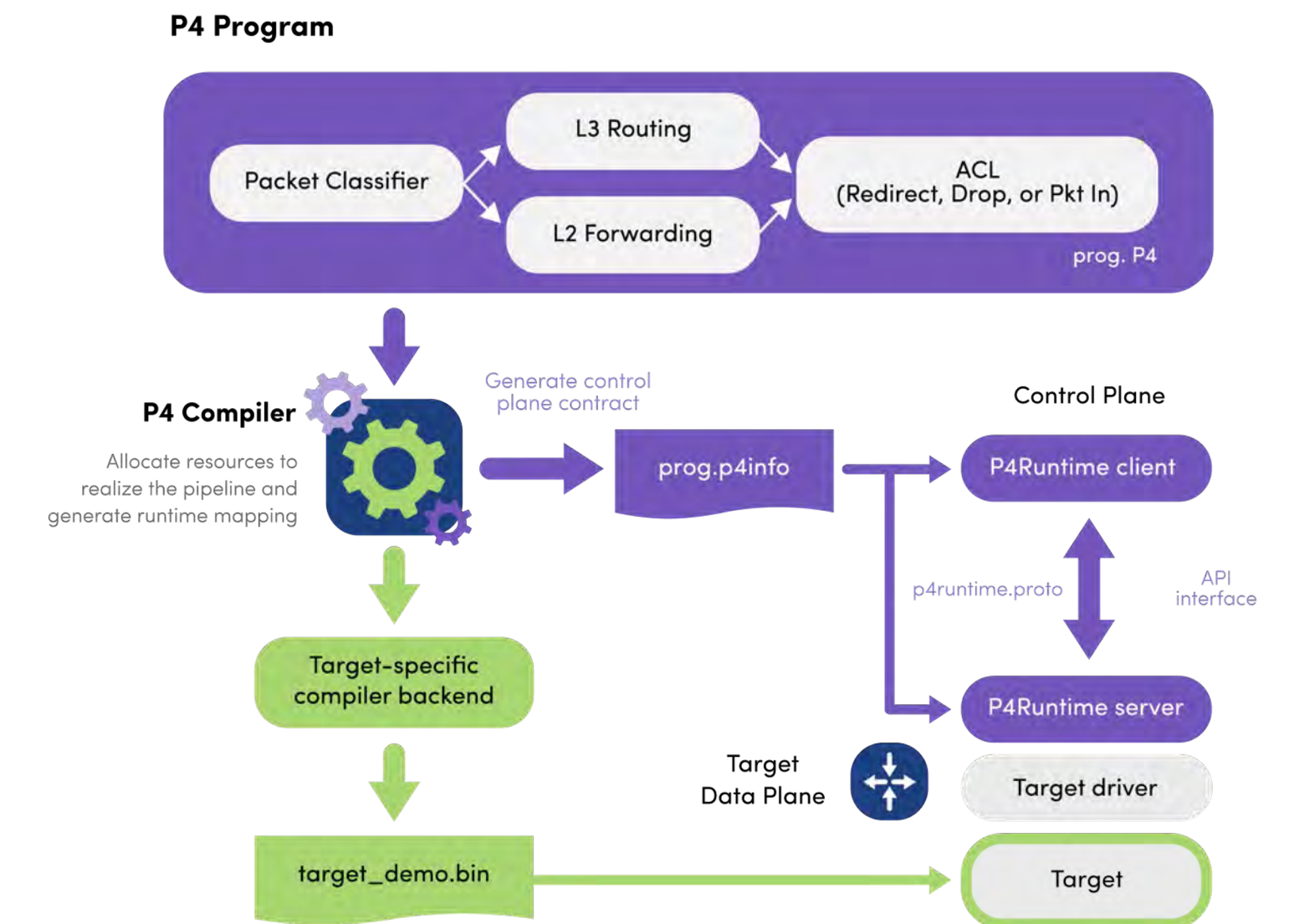
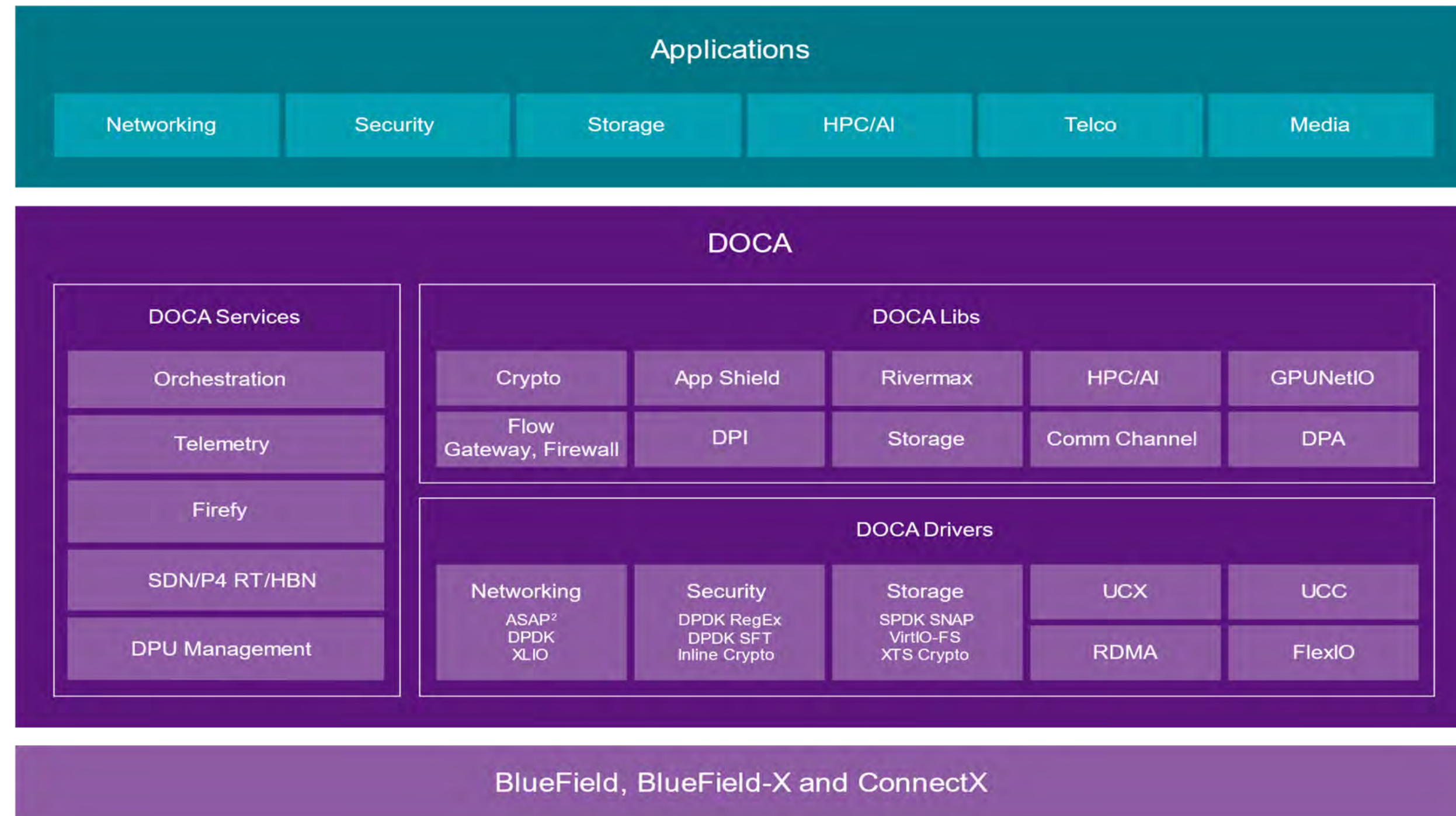
 - Program explicitly interacts with the accelerator, such as with an MPI library implementation

- Programming model does not expose the accelerator

 - Accelerator is used by services the application uses such as MPI, OpenSHMEM and storage

 - Services explicitly interact with the accelerator

Other Frameworks for DPUs/DPAs



DOCA

NVIDIA-specific SDK for application developers
Supports Data Plane Development Kit (DPDK), RDMA, compression, etc.

P4

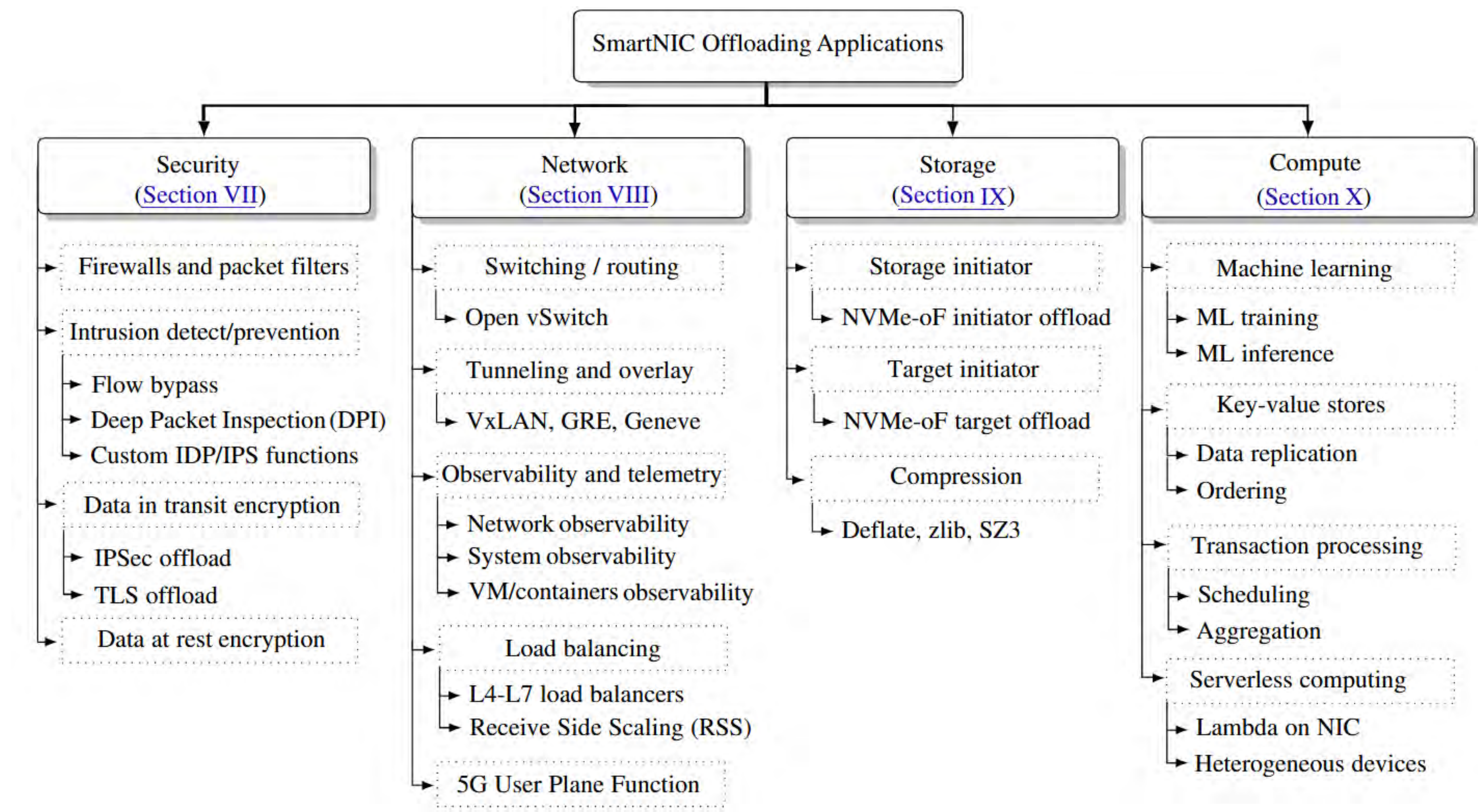
Provides a vendor-neutral specification for packet processing and manipulation



SmartNIC Applications and Use Cases

Surveys on SmartNICs and DPUs

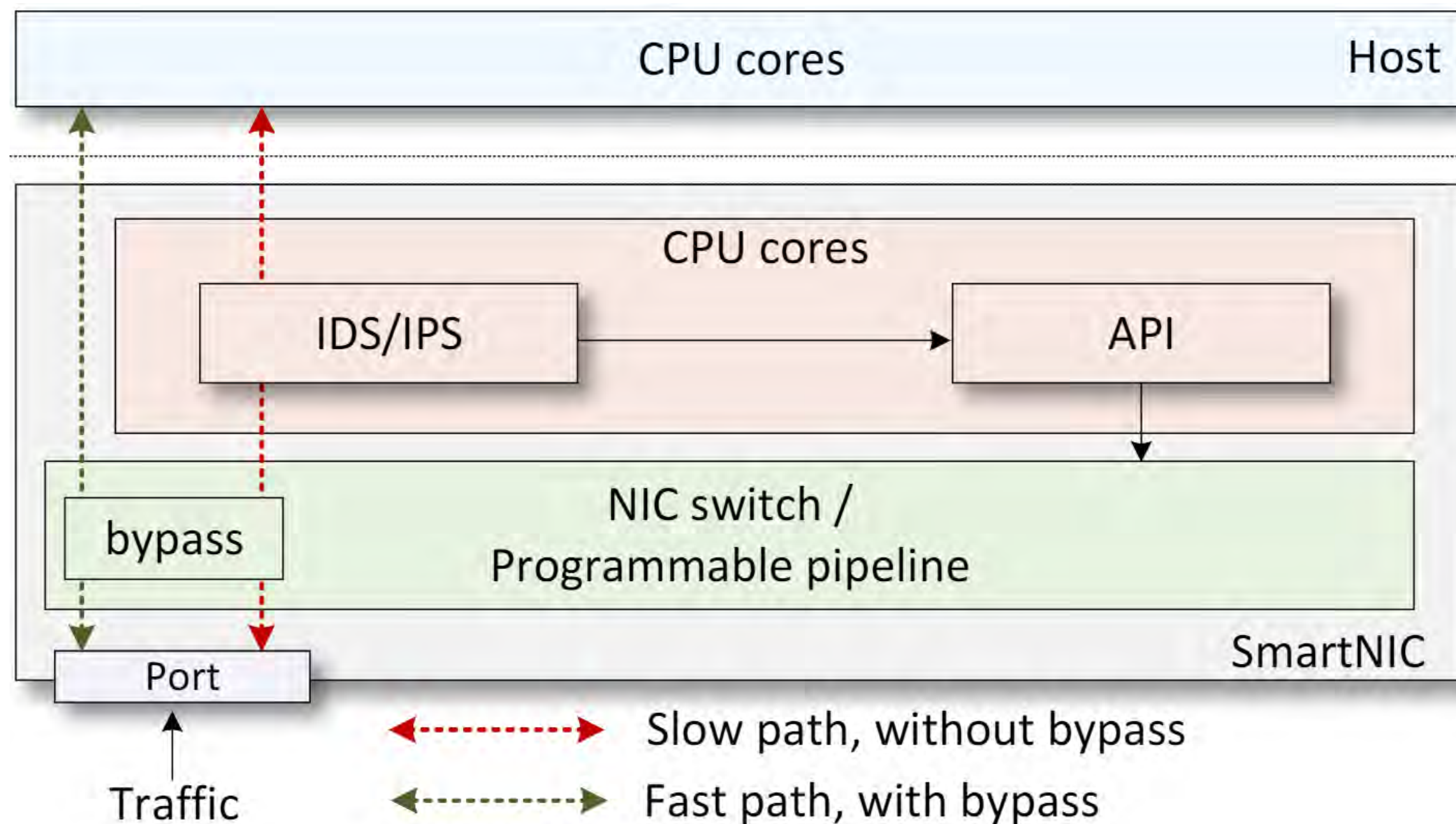
DPUs have been used to offload a variety of applications and infrastructure functions



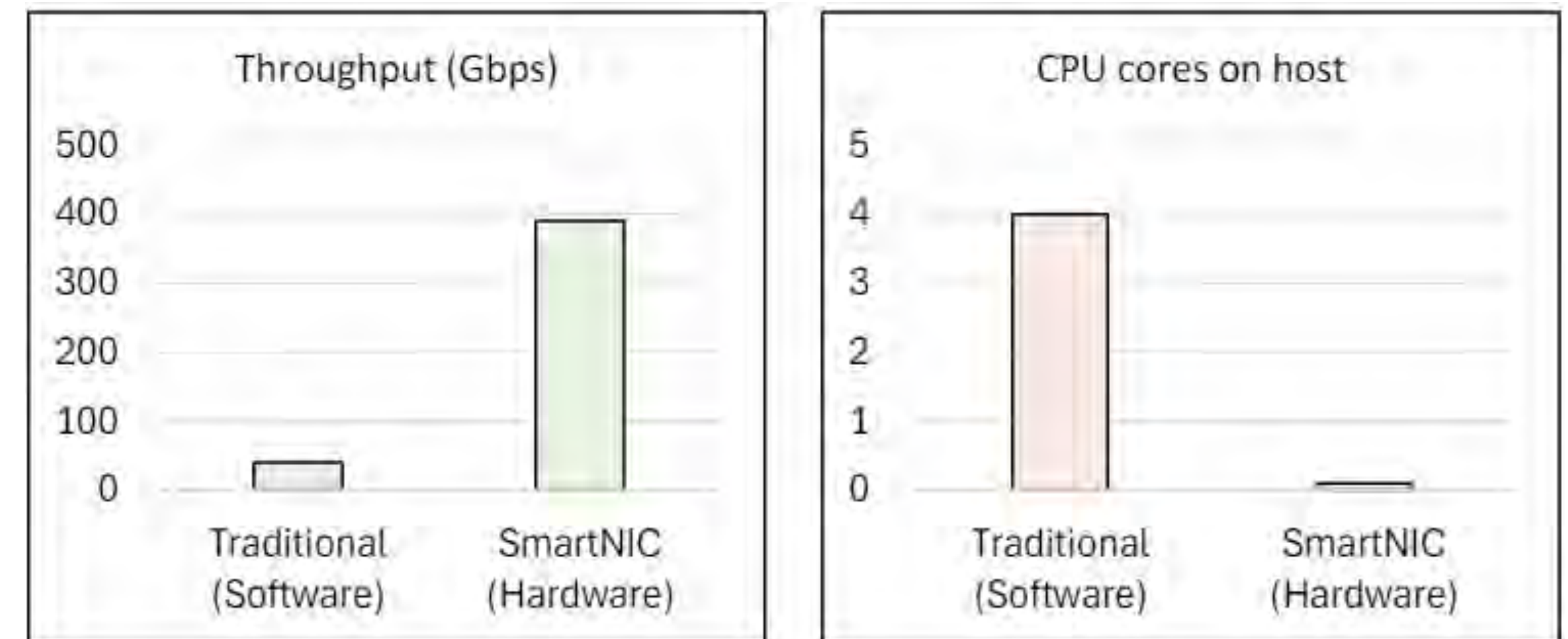
Accelerating IDS/IPS Functions

Intrusion Detection/Prevention System (IDS/IPS) functions can be offloaded to DPUs

- Traffic bypass, Deep Packet Inspection (DPI), signature matching, etc.

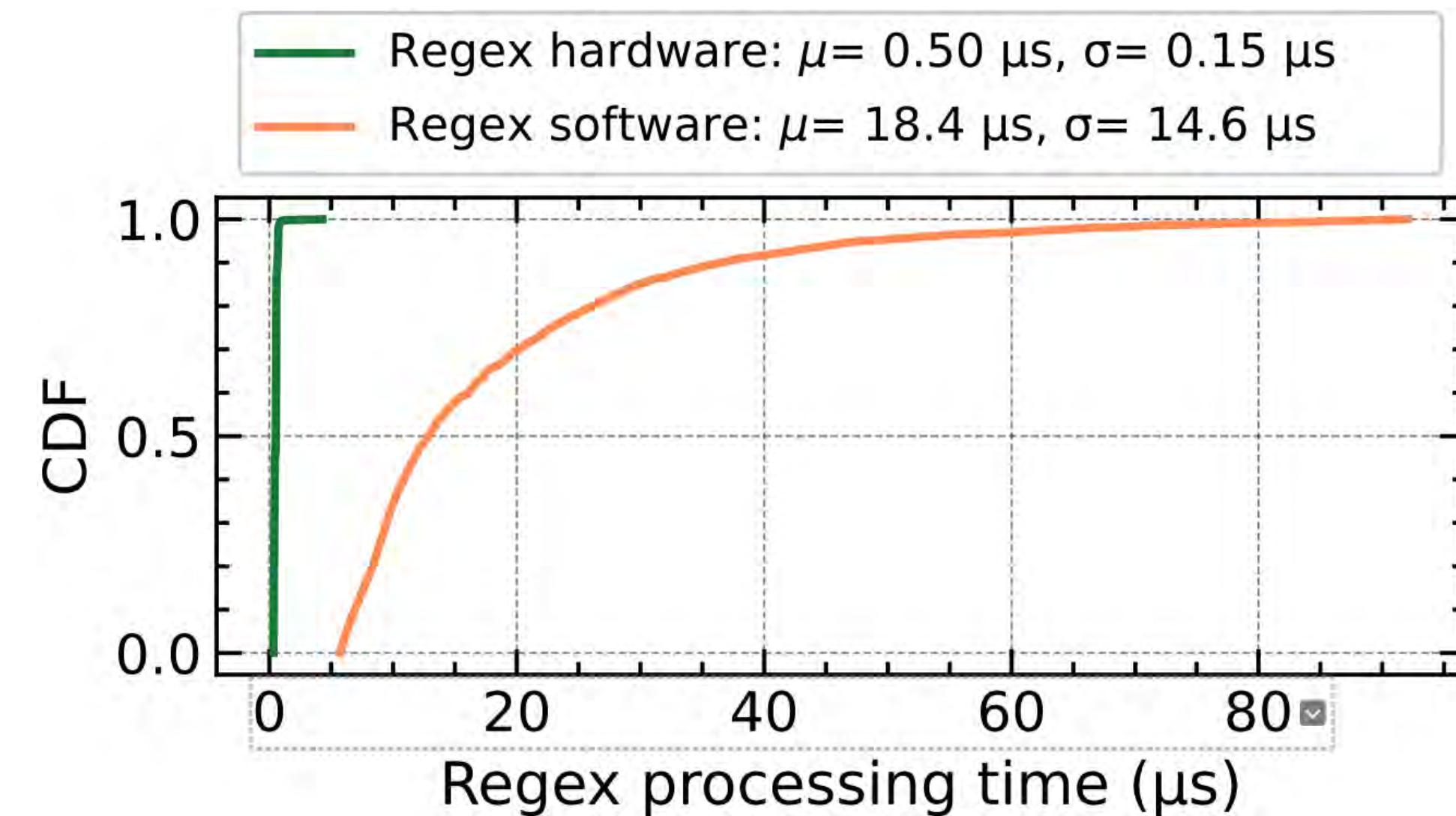
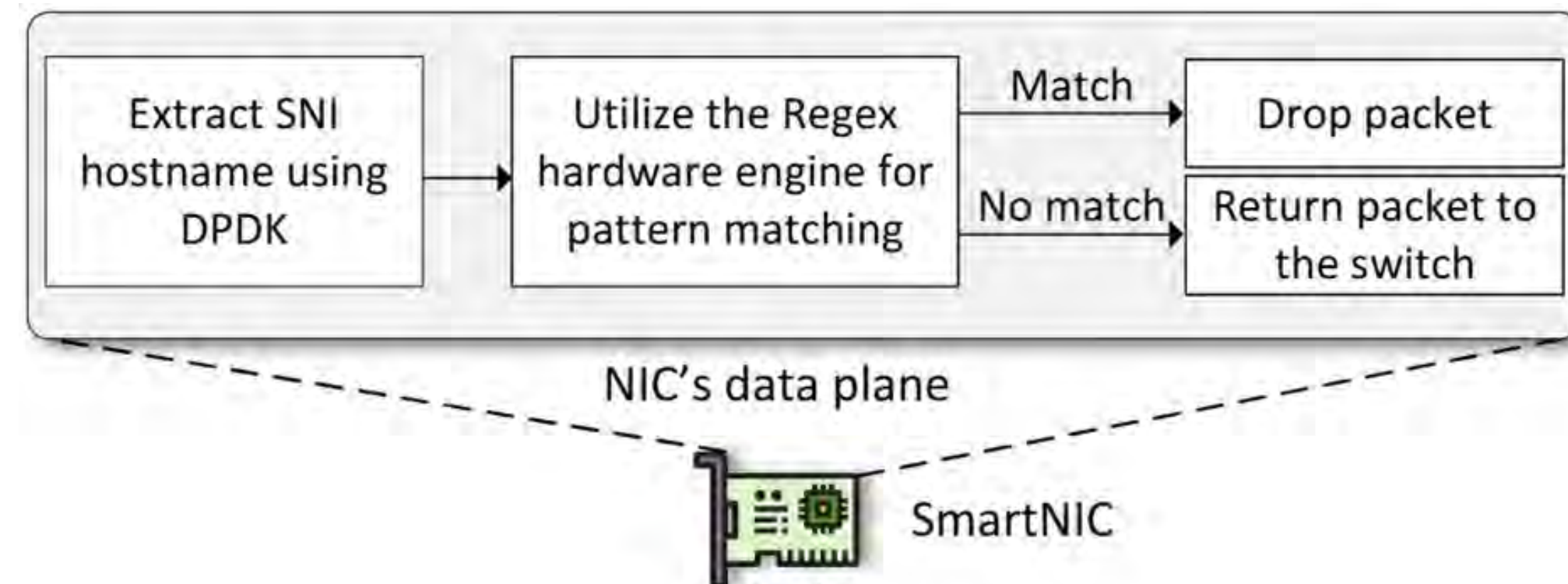


Suricata bypass function



Deep Packet Inspection Offload

DPU's are accelerating pattern matching through regular expression DSAs

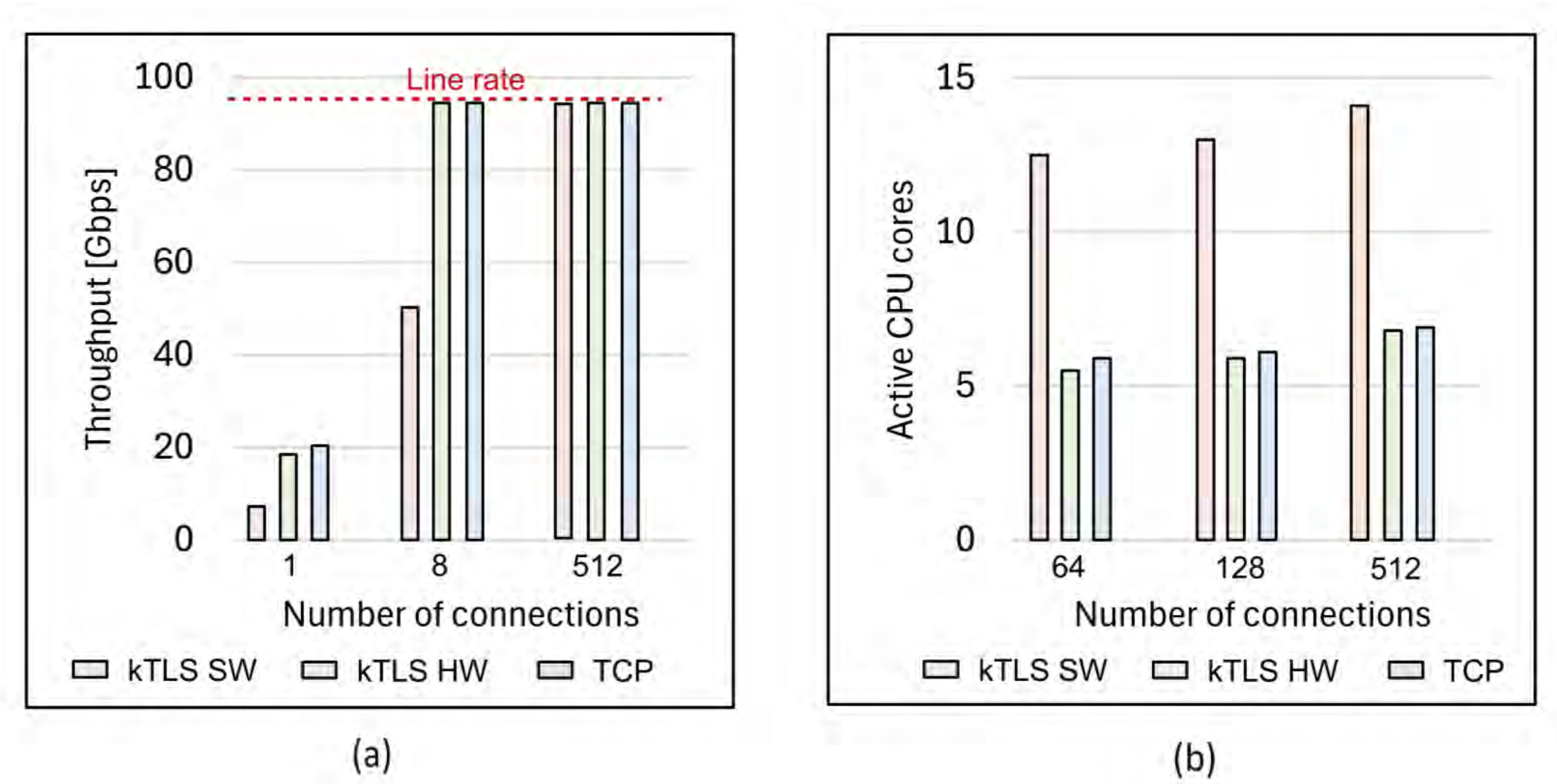


Cryptographic Functions Offload (1/2)

SmartNICs support hardware accelerators for offloading cryptographic operations:

- Symmetric encryption
- Asymmetric encryption
- True Random Number Generator (TRNG)

TLS hardware offload

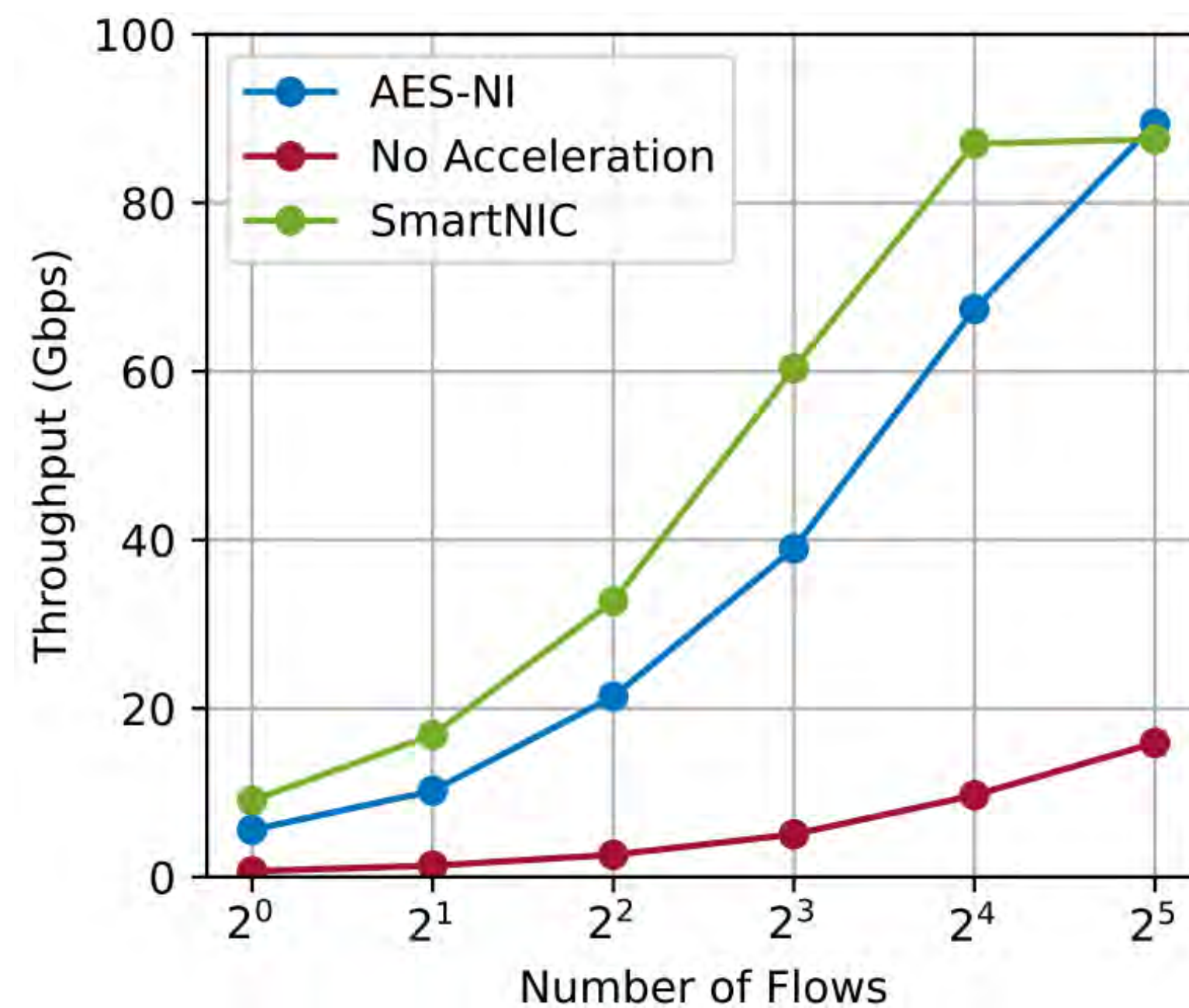


Cryptographic Functions Offload (2/2)

SmartNICs support hardware accelerators for offloading cryptographic operations:

- Symmetric encryption
- Asymmetric encryption
- True Random Number Generator (TRNG)
- Etc.

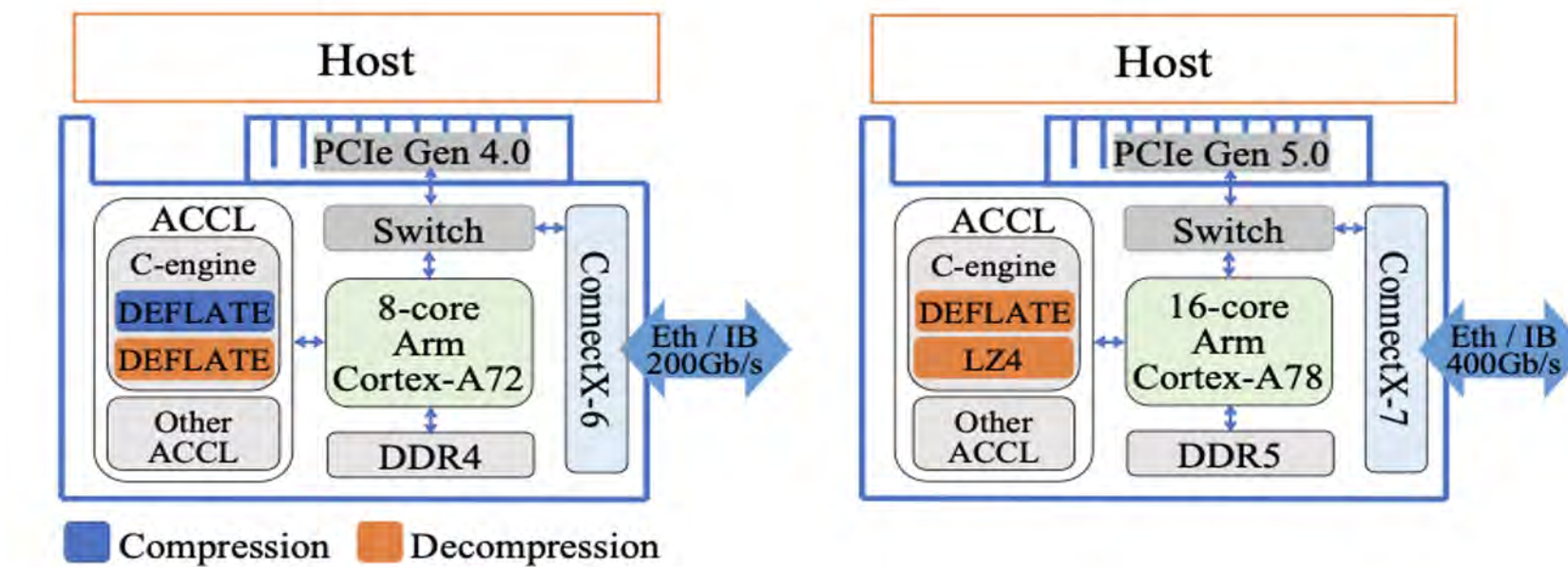
Large data transfers encryption (TLS offload)



(De)Compression Offload

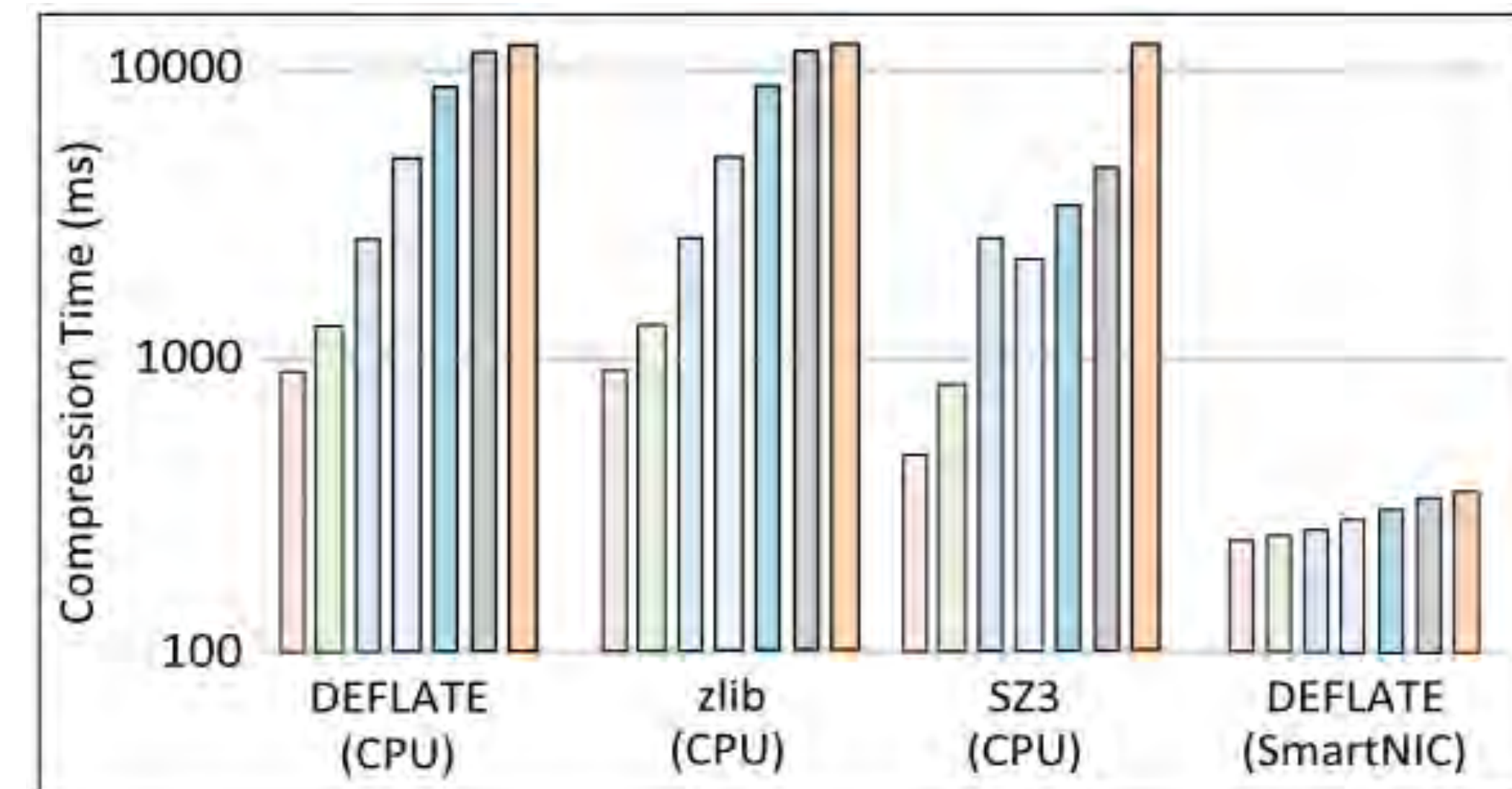
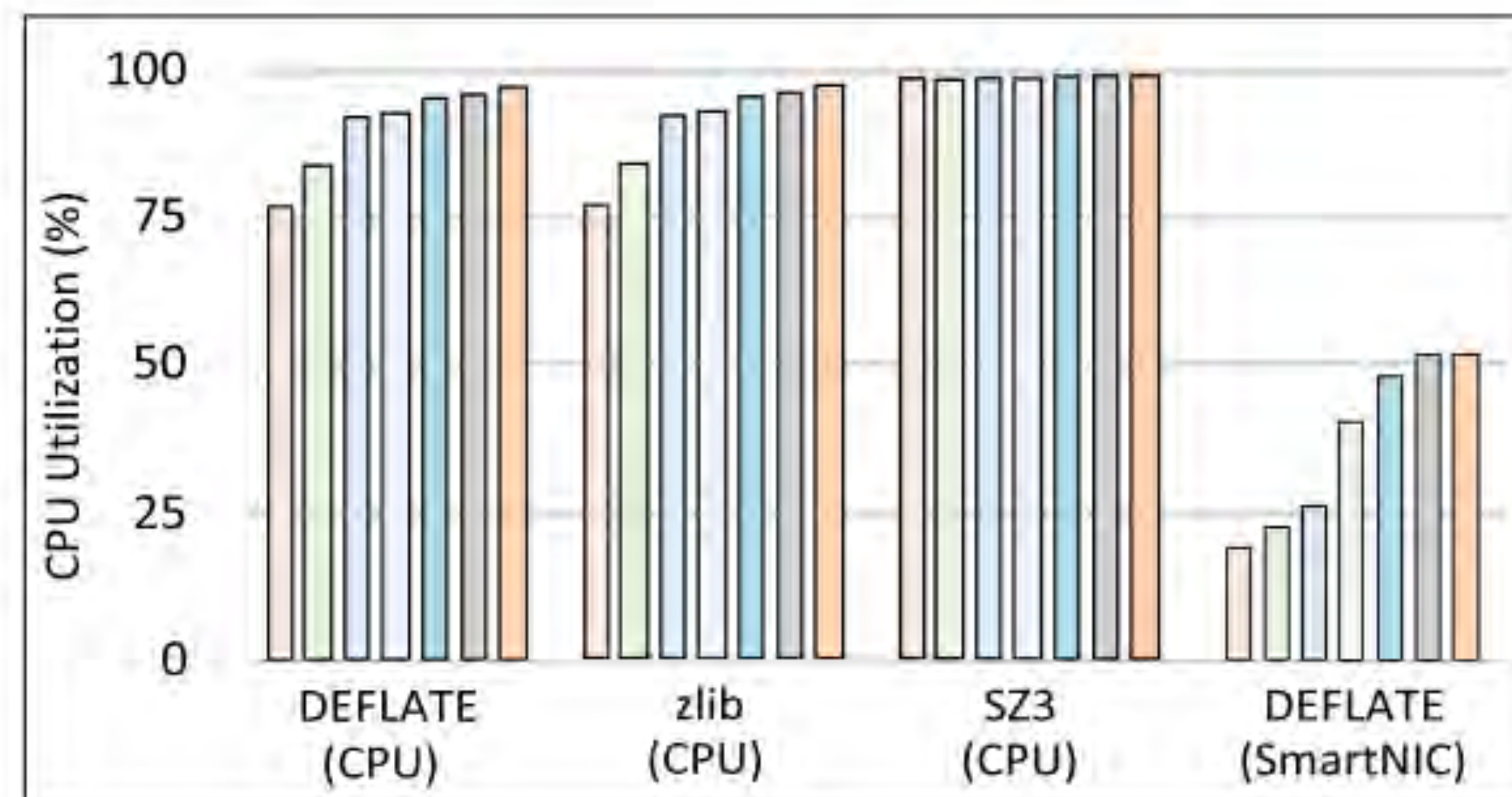
SmartNICs can be used to accelerate file compression and decompression operations

- The experiment shows results for compression algorithms over seven datasets
- Speedups in this work made use of DSAs on the BlueField platform



(a) BlueField-2 DPU Arch.

(b) BlueField-3 DPU Arch.

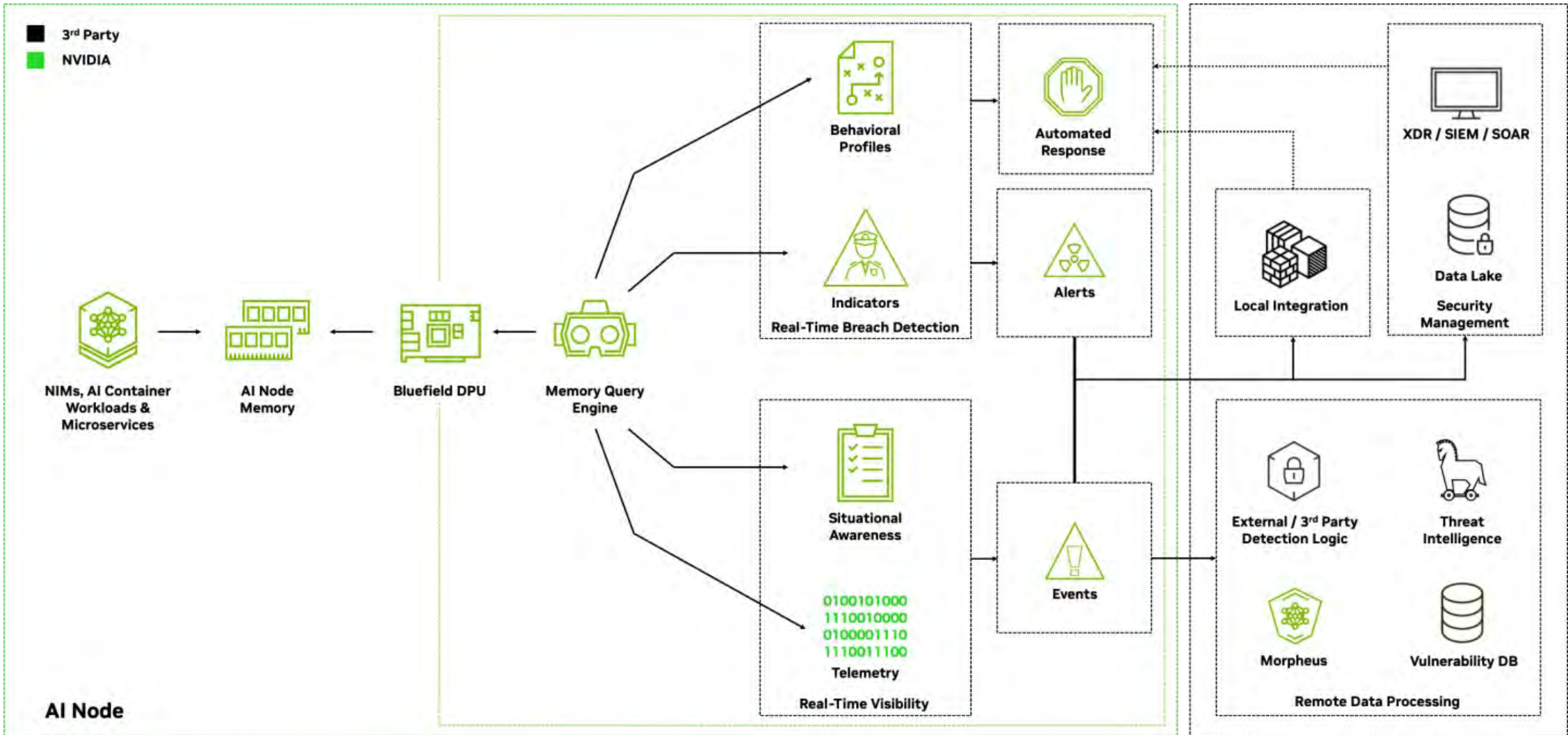


DPU Security Application: Host Memory Introspection

- DOCA Argus framework provides real-time situational awareness and runtime threat detection by inspecting host memory using advanced memory forensics
- Live machine introspection is performed at the hardware level using Direct Memory Access (DMA)
- Monitors threats in real time without impacting system performance
- Unlike conventional tools, Argus runs independently of the host, requiring no agents, integration, or reliance on host-based resources
- By operating outside the host, isolated in its own trust domain, DOCA Argus remains invisible to attackers—even if the system is compromised

Creation or Modification of System Processes

Type	Severity	Activity Name	Remarks
Event	Info	Process Created	A new process has been detected.
Event	Info	Process Terminated	A process has been terminated.
Event	Warning	Process Zombie	Detects a process in a zombie state.
Alert	High	Process Hidden	Detects a process in a hidden state.



DPU Security Application: Host Memory Introspection

Network Connections

Type	Severity	Activity Name	Remarks
Event	Info	Network Connection Created	A new TCP network connection has been created.
Event	Info	Network Connection Terminated	A TCP network connection has been terminated.
Alert	Low	TCP Connection Excessive Data	Monitors a TCP connection's incoming or outgoing data volume that exceeds a configurable threshold (separate thresholds for incoming and outgoing traffic).
Alert	Low	TCP Long-Lasting Connection	Monitors a TCP connection whose total duration exceeds a configurable time threshold.

DPU Security Application: Host Memory Introspection

Executed Binaries and Loaded Libraries (Software Bill of Materials/Process Attestation)

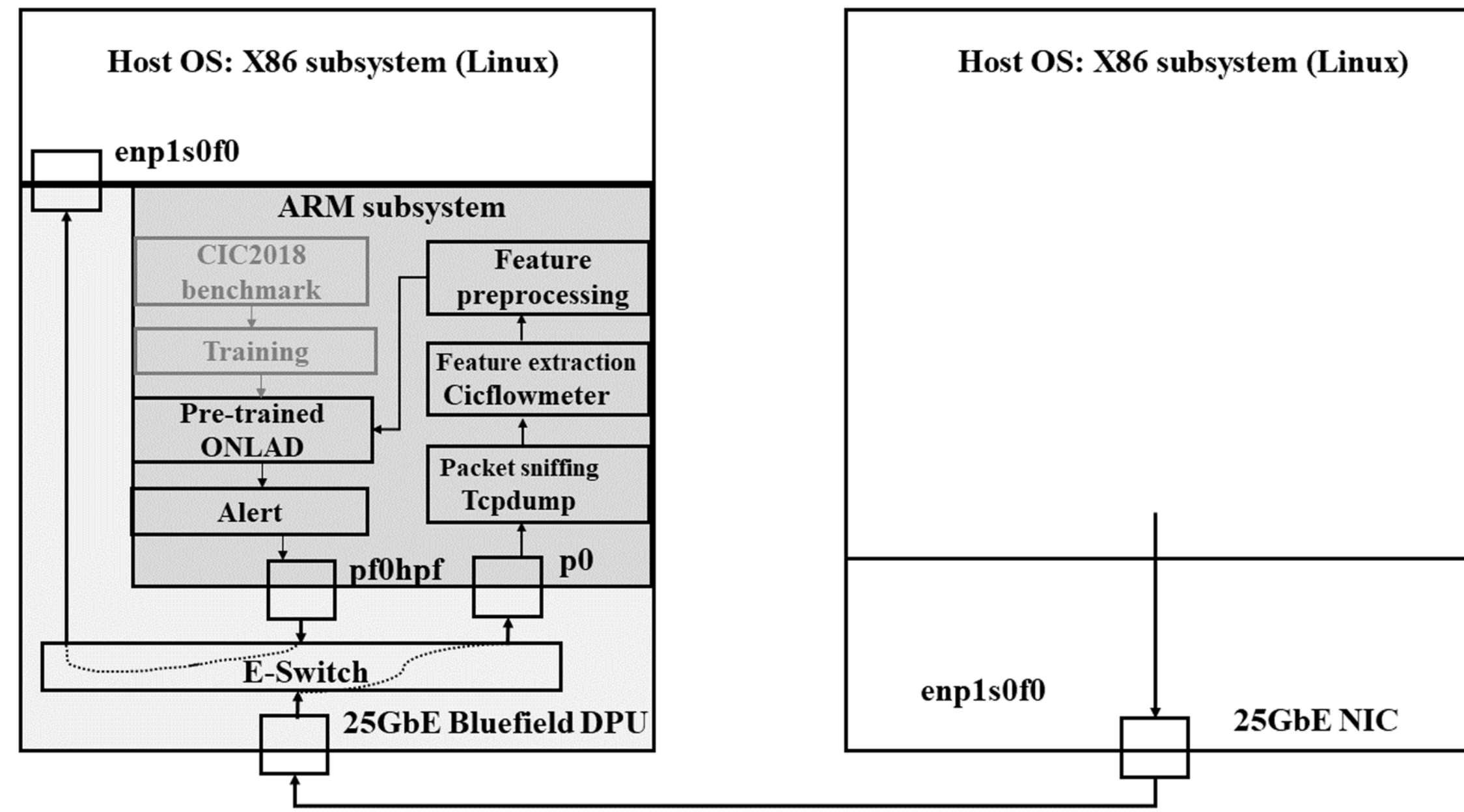
Type	Severity	Activity Name	Remarks
Alert	High	Foreign Binary Executed	Detects execution of a binary not included in the original container image or modified from it. May indicate that an attacker has control of the workload and is executing arbitrary commands.
Alert	High	Binary Executed Not as Intended	Detects execution of a binary from the original container image with command-line arguments and/or from a folder path not matching those in the original container image.
Alert	High	Foreign Binary Executed – File Size Mismatch	Detects execution of a binary whose reported file size differs from the file size of the corresponding binary in the original container image.

DPU Security Application: Host Memory Introspection

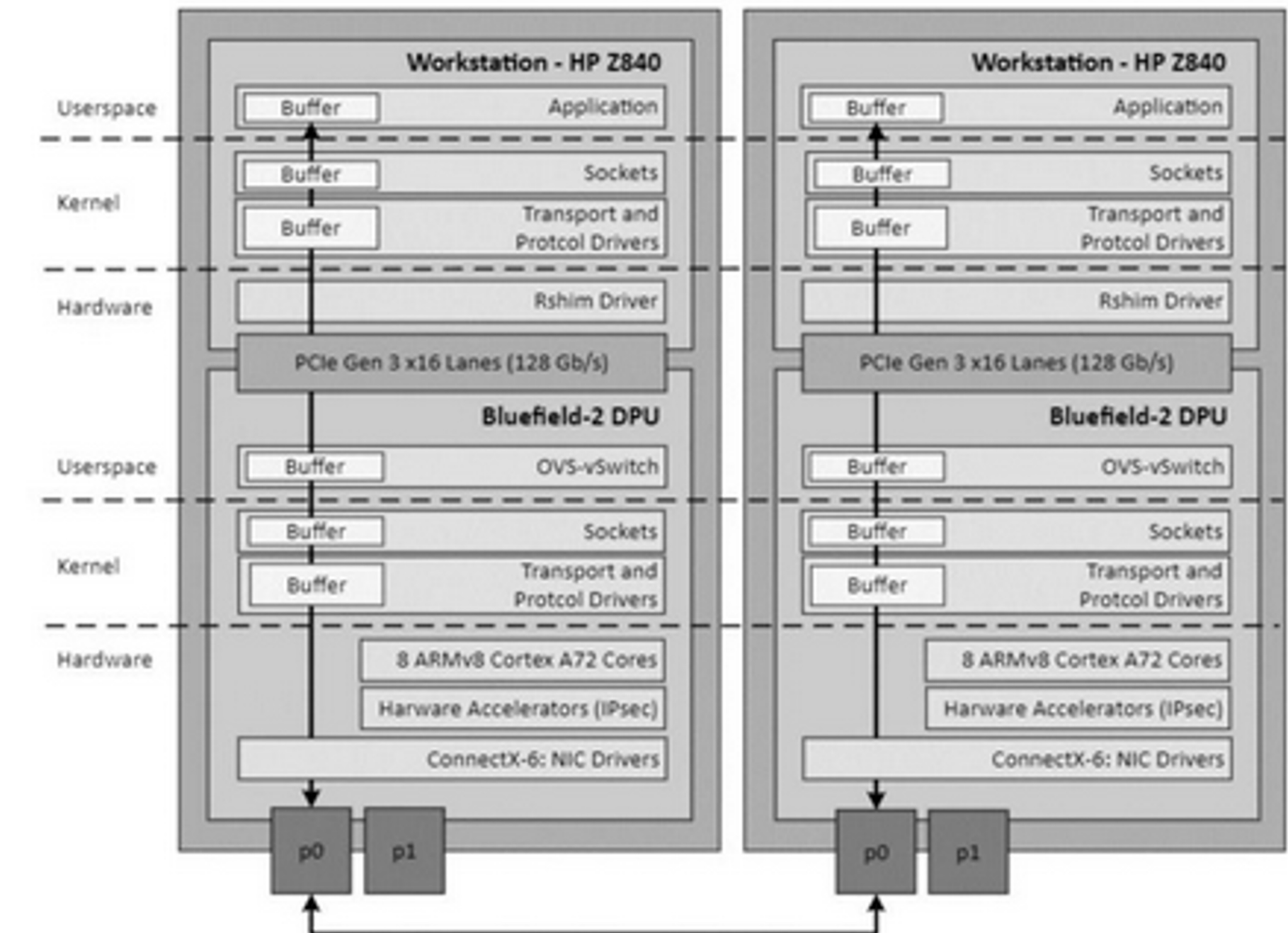
Process Memory

Type	Severity	Activity Name	Remarks
Event	Info	Process Memory Created	A new virtual memory area (e.g., heap, stack, executable) has been created. Default: off.
Event	Info	Process Memory Terminated	A virtual memory area is no longer visible (terminated). Default: off.
Event	Warning	New Executable Anonymous Memory Mapped	An executable anonymous memory area has been mapped.
Alert	Medium	Executable Permissions Added	Executable permissions have been added to a memory area.

Security Related Research on DPUs



Man Wu, Hiroki Matsutani, and Masaaki Kondo. "ONLAD-IDS: ONLAD-Based Intrusion Detection System Using SmartNIC." In *2022 IEEE 24th Int Conf on High Performance Computing & Communications; (HPCC/DSS/SmartCity/DependSys)*, pp. 546-553. IEEE, 2022.

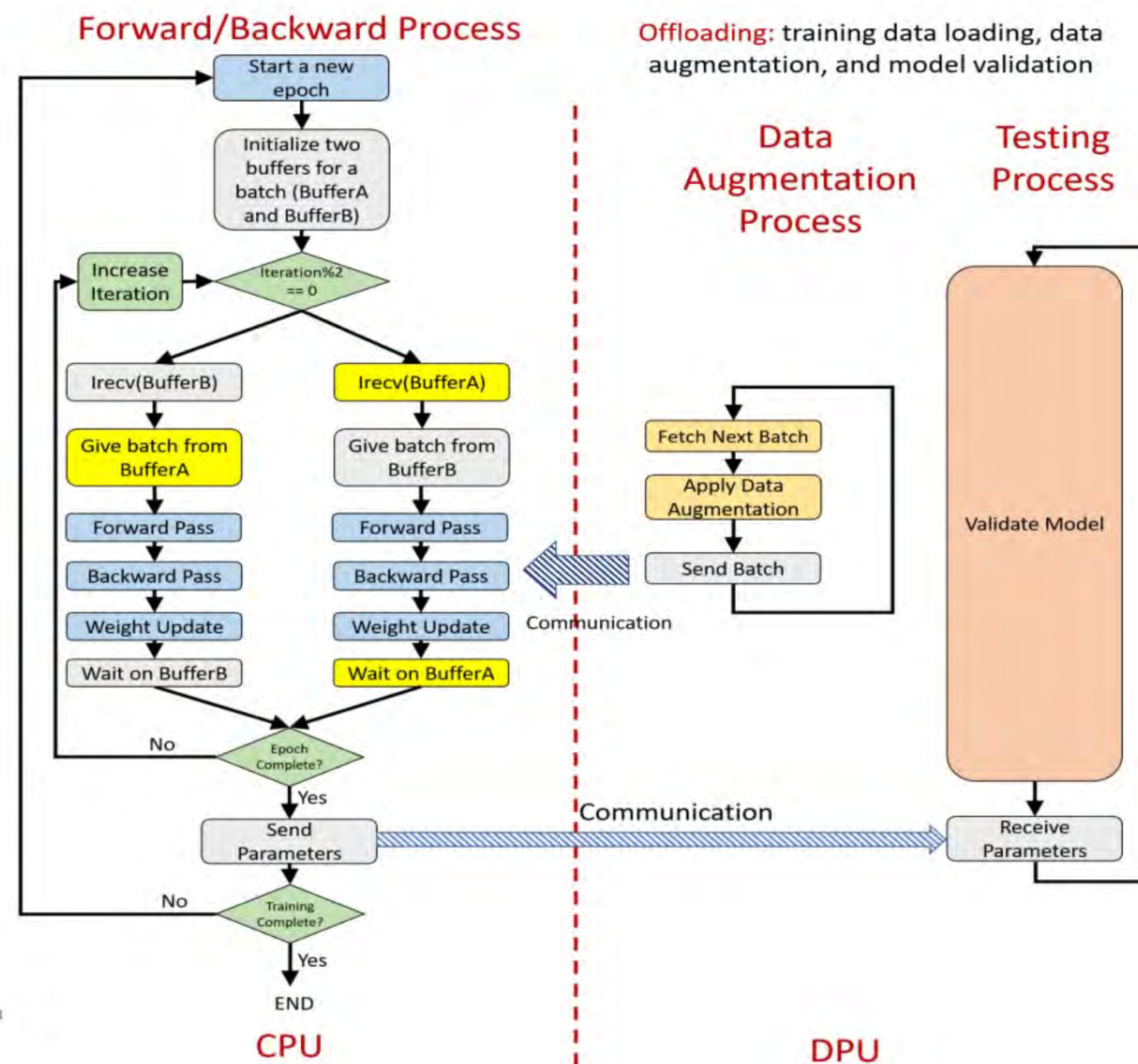


From Noah Diamon Scott Graham, and Gilbert Clark. "Securing InfiniBand networks with the Bluefield-2 data processing unit." In *International Conference on Cyber Warfare and Security*, vol. 17, no. 1; 2022

Recent work has looked at using frameworks like DOCA to implement zero trust architectures, but the addition of processing cores and a full OS stack to the DPU allows for the design of sophisticated IDS and secure network stacks

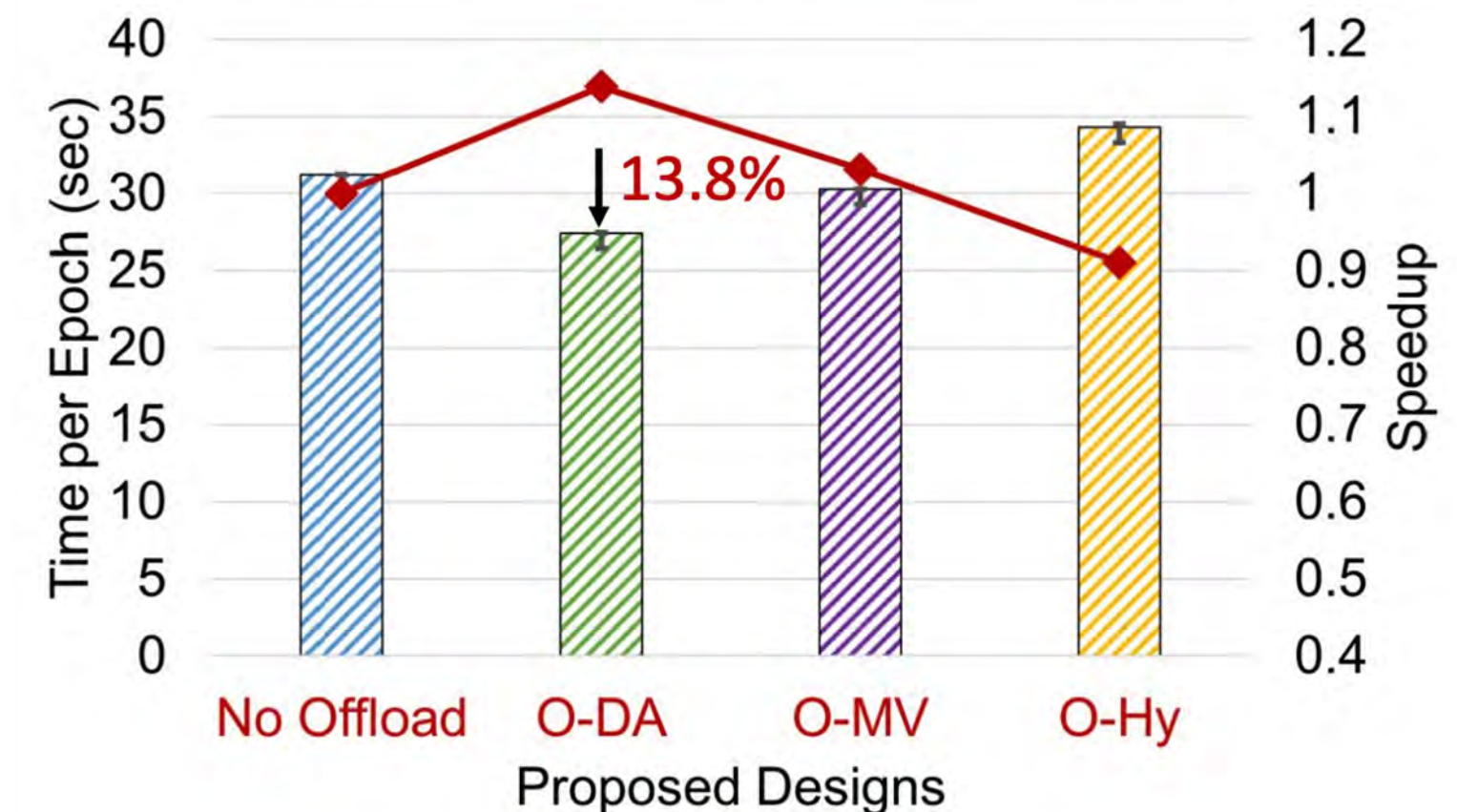
Accelerating MPI and Deep Learning Applications with the DPU Technology

Nick Sarkauskas, Arpan Jain, Nawras Alnaasan, Tu Tran, Bharat Ramesh, Aamir Shafi, Hari Subramoni, and **Dhableswar K. (DK) Panda**



Speedup

- Single node: O-DA (13.8%) and O-MV (3.1%)
- Multi-node: Achieves average 13.9% speedup on 1-16 nodes



Single Node Experiments

DPU is used to accelerate ML model training using the MVAPICH framework, achieving 14% speedup on 1-16 node tests

- Training processes on CPU
- Data augmentation processes (asynchronous, on DPU)
- Testing and Validation (synchronous, on DPU)

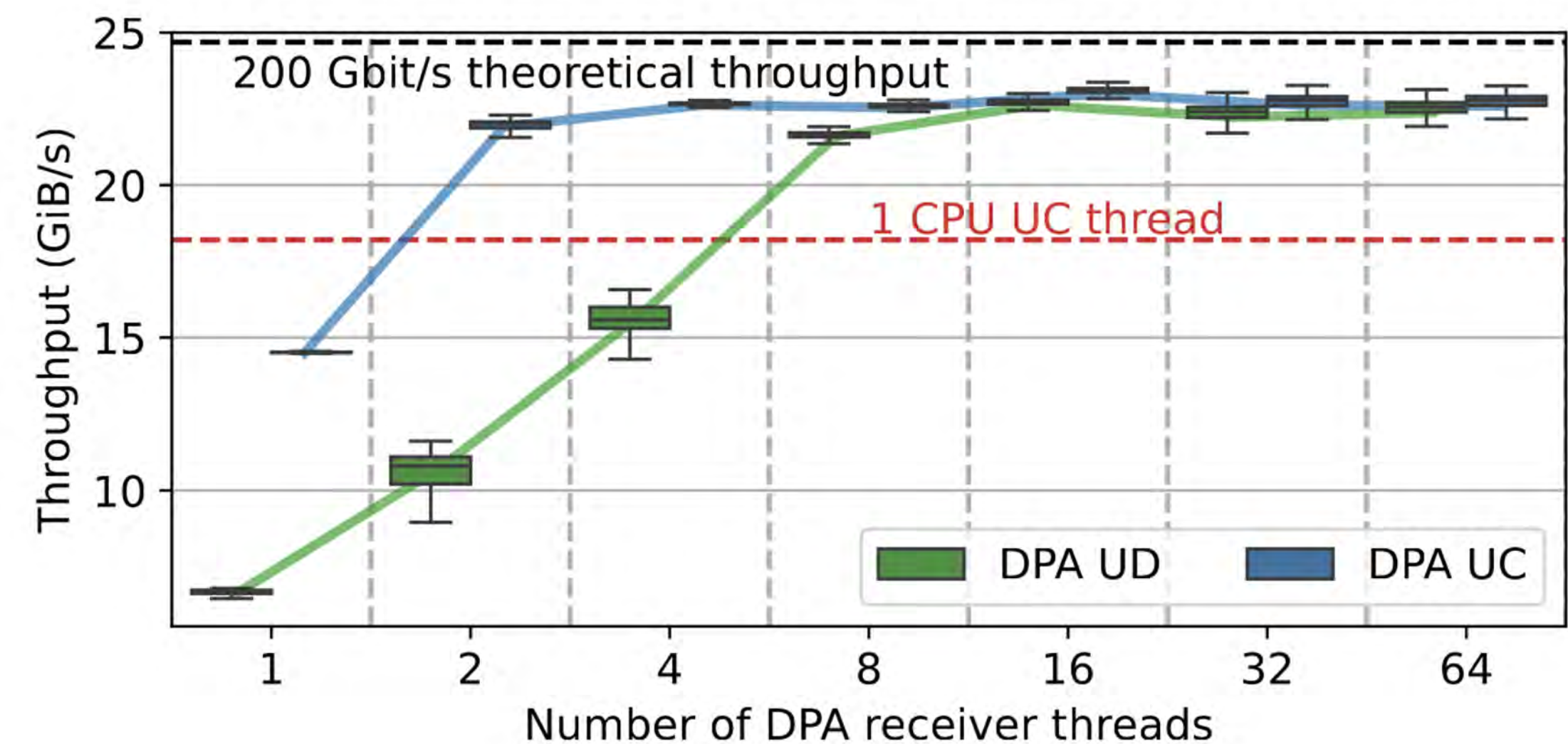
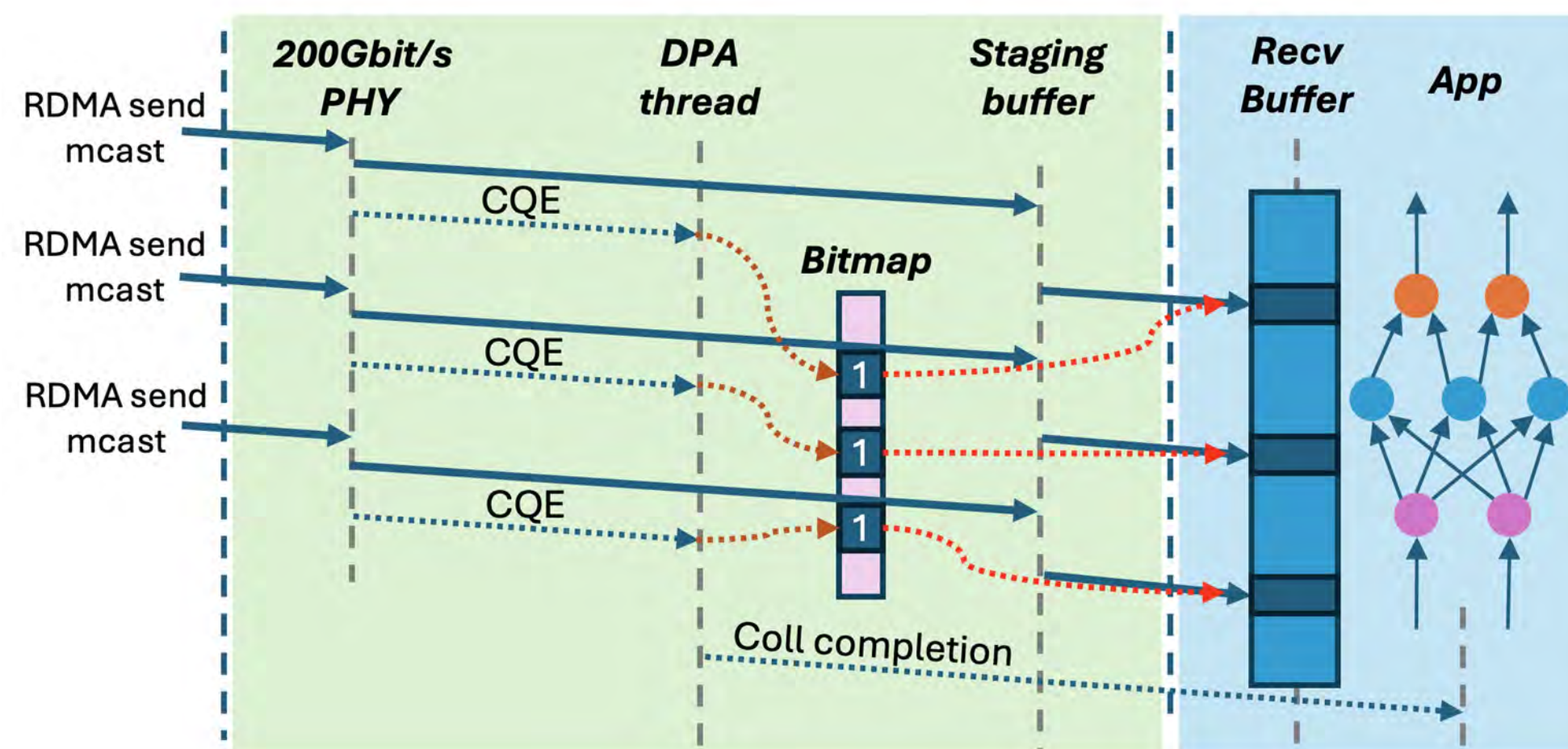
DPA use case: AI Collectives Acceleration

Fully Sharded Data Parallel (FSDP) training used for LLAMA 3 training

FSDP relies on Allgather that can be accelerated with hardware RDMA multicast

Multicast 4KB datagram processing introduces high CPU overheads at ≥ 200 Gbit/s

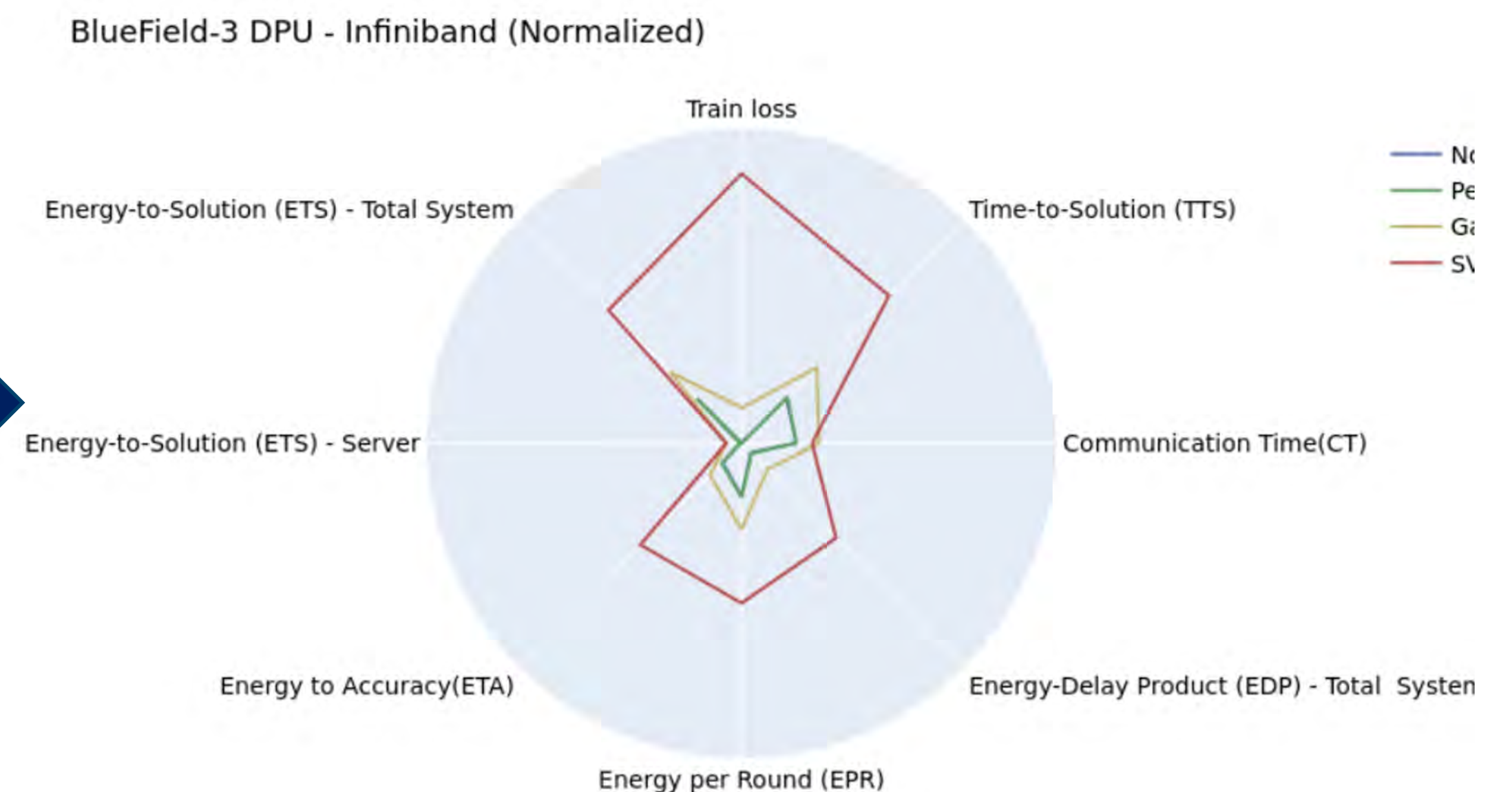
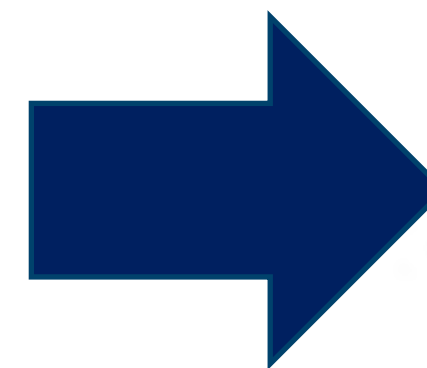
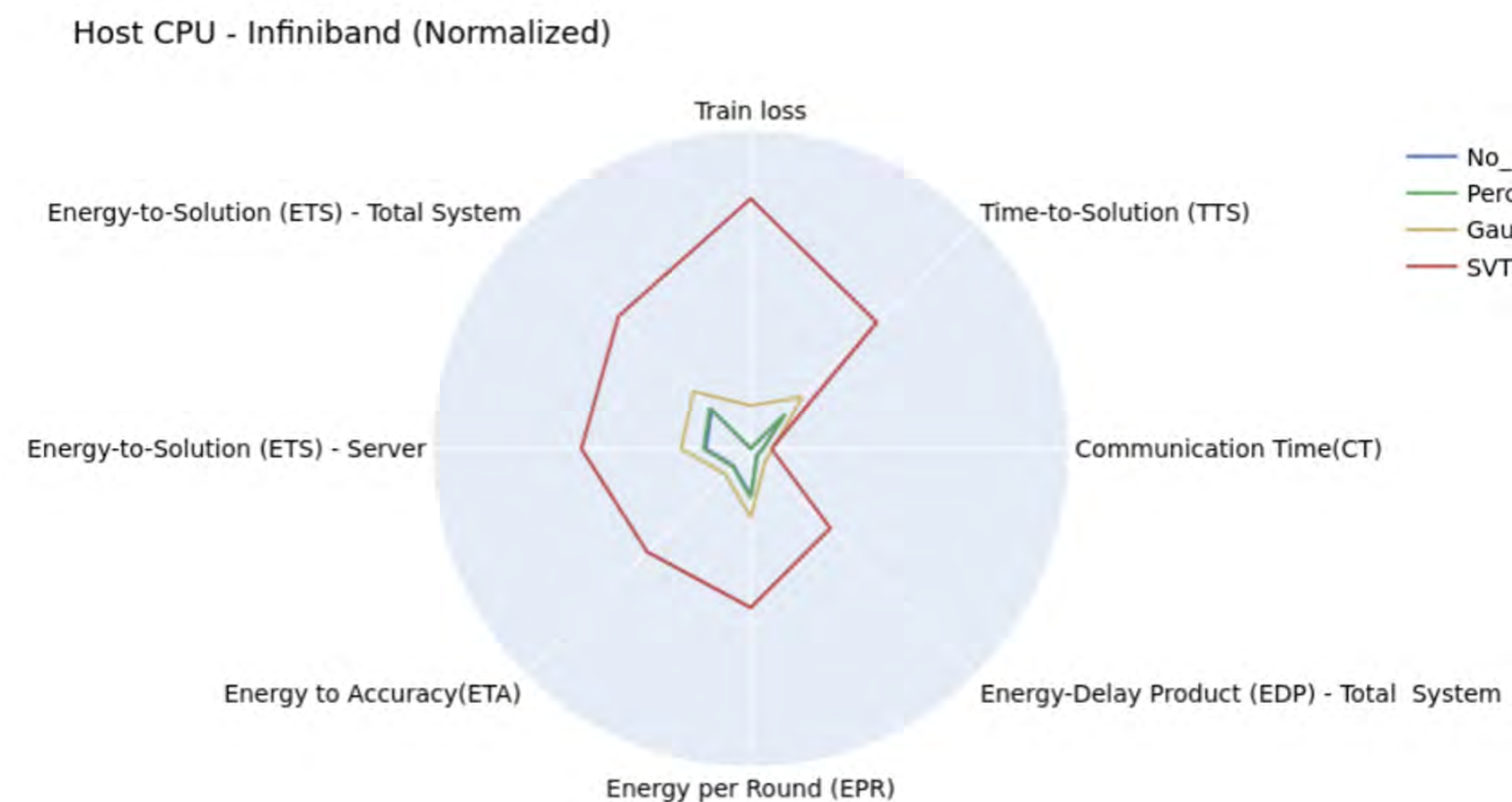
DPA is an ideal offloading Multicast Send/Receive



Mikhail Khalilov, Salvatore Di Girolamo, Marcin Chrapek, Rami Nudelman, Gil Bloch, and Torsten Hoefler,
“*Bandwidth-optimal, Fully-Offloaded Collectives*”, Supercomputing 2024.

SmartNIC-Accelerated Federated Learning Energy & Privacy Trade-offs

- BF3 reduces server-side energy use by up to 4.5× for Gaussian and Percentile methods.
- DPU favors lightweight privacy methods in energy-constrained settings.
- Setup:
 - FL Framework: NVFLARE
 - 5 clients – GH200
 - 1 server - BF-3
- Future work: accelerate compression and security

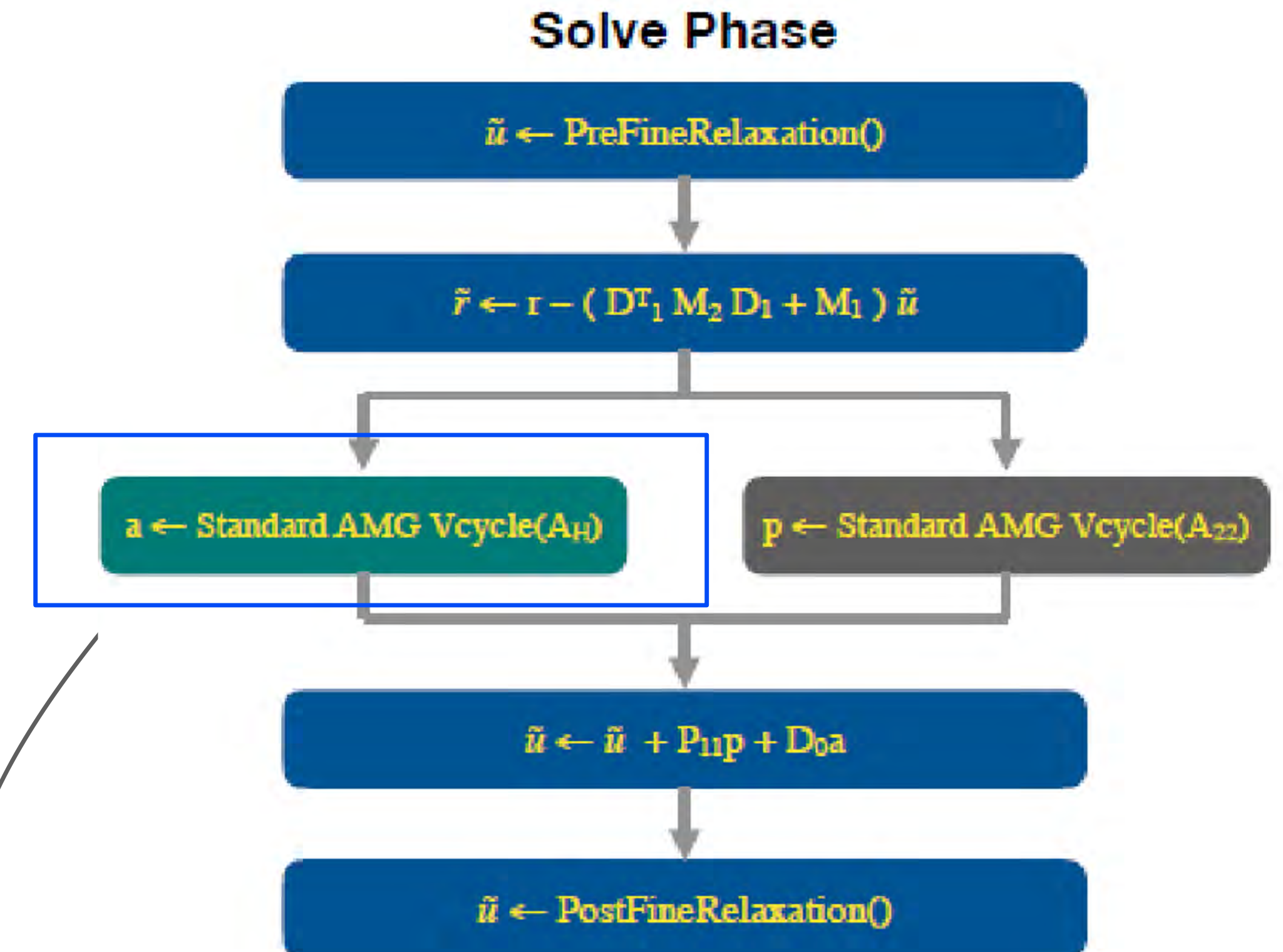
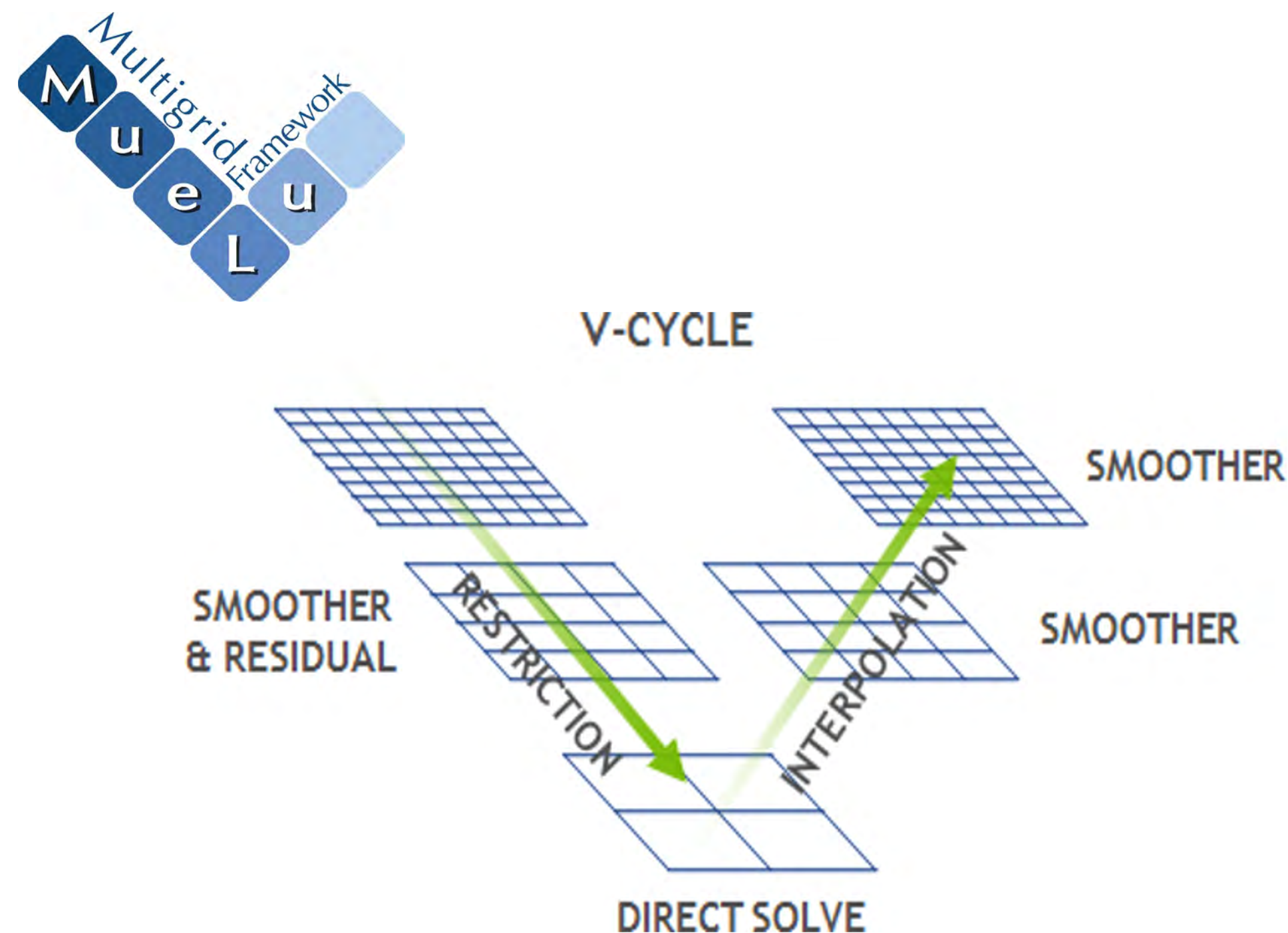


DPU Offload Work for HPC Libraries

HPC applications such as asymmetric multigrid can also potentially be offloaded to the DPU using MPI and the onboard Arm cores.

For example, Maxwell Equation Solvers as implemented in the MueLu library represents a compelling distributed and shared memory parallel algorithm. These implementations can be formulated using adaptive multigrid approaches and possible also parallelized to DPUs!

Results show that this offload can result in both a small speedup and greenup (efficiency gain) for a DPU offload enabled algorithm.



Possible Offload Candidate

Bochev, Pavel B., et al. "An algebraic multigrid approach based on a compatible gauge reformulation of Maxwell's equations." *SIAM Journal on Scientific Computing* 31.1 (2008)

Karamati, et al. *work under submission*.

More Information - SmartNIC Research Directions

SIAM PP 2024 mini-symposium at <https://crnch.gatech.edu/siam-pp-2024/>

ISC'2023 DPU Workshop <https://dpu.ornl.gov>

SmartNIC surveys like Kfoury, et al. "A comprehensive survey on SmartNICs: Architectures, development models, applications, and research directions." IEEE Access (2024).

AGENDA

Time	Topic	Details
9:00-9:10	Introduction	Attendee Survey; intro to Hands-On
9:10-9:25	Communication Offloading	SmartNIC overview DPU examples DPU programming models
9:25-9:40	SmartNIC Use Cases	Packet processing, cyber-security, AI/HPC
9:40-9:55	Infrastructure SW – DOCA and P4 Introduction	Brief introduction to DOCA/P4
10:00-10:30	HPC Programming – Part 1	MPI Offload; DPA Programming
10:30-11:00	HPC Programming – Part 2	OpenMP Offload
11:00-11:30	BREAK	
11:30-11:45	Hands-on DOCA/P4/DPA Demo	Attendees follow along in compiling and running test examples
11:45-12:15	Hands-on HPC Demo – OpenMP/MPI Offload	
12:15-12:30	Storage Acceleration	Discussion of use case for SmartNICs with storage systems for AI
12:30-12:35	Tutorial Survey	
12:35-1:00	Continued Hands-on Activities	Time to work with the instructors on the presented demos and test cases

See the updated agenda at <https://github.com/gt-crnc-hrg/smartnic-tutorial-isc26>



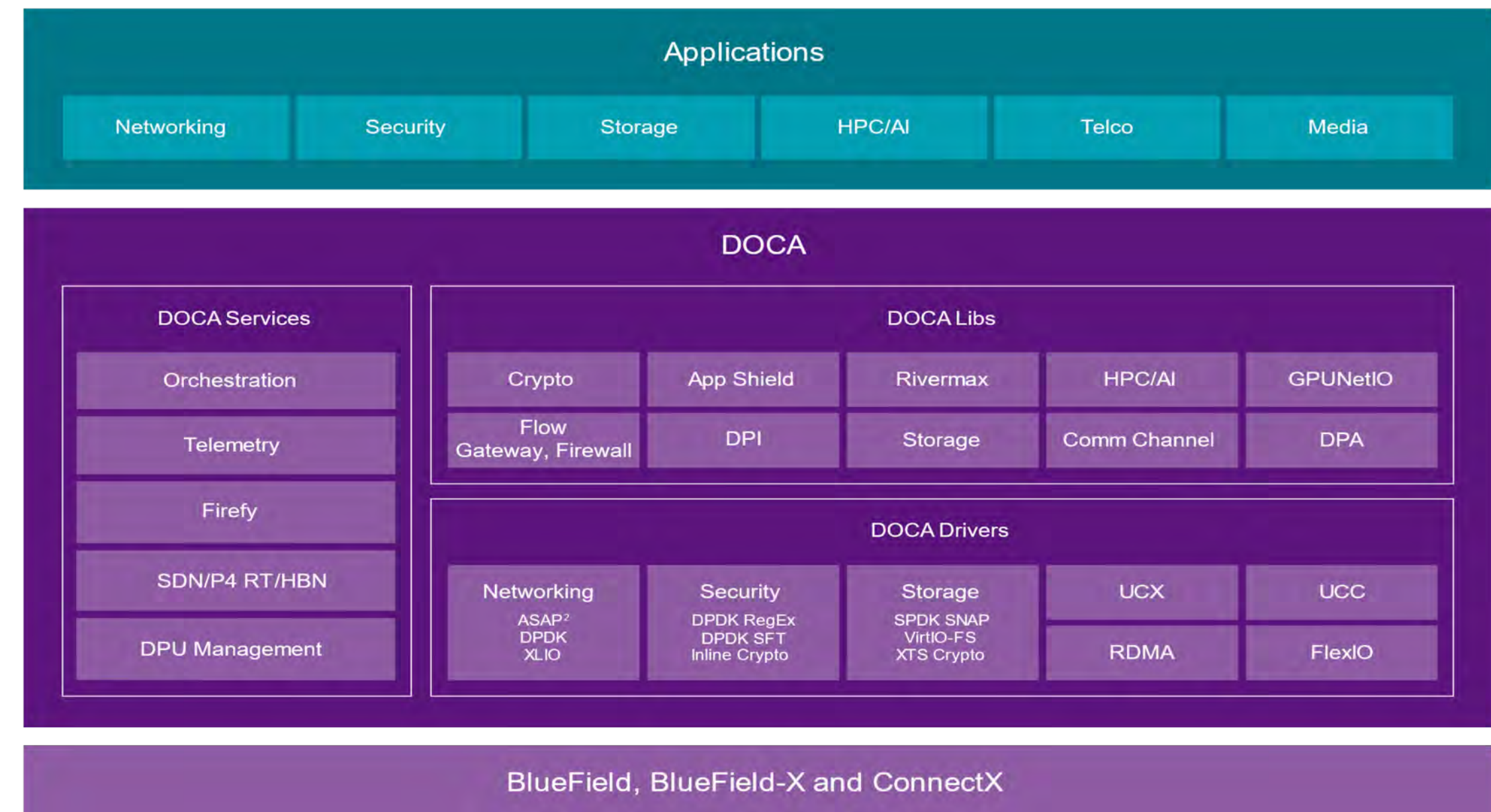
Please share your feedback in a short 4 question survey!



DOCA Programming

DOCA Programming

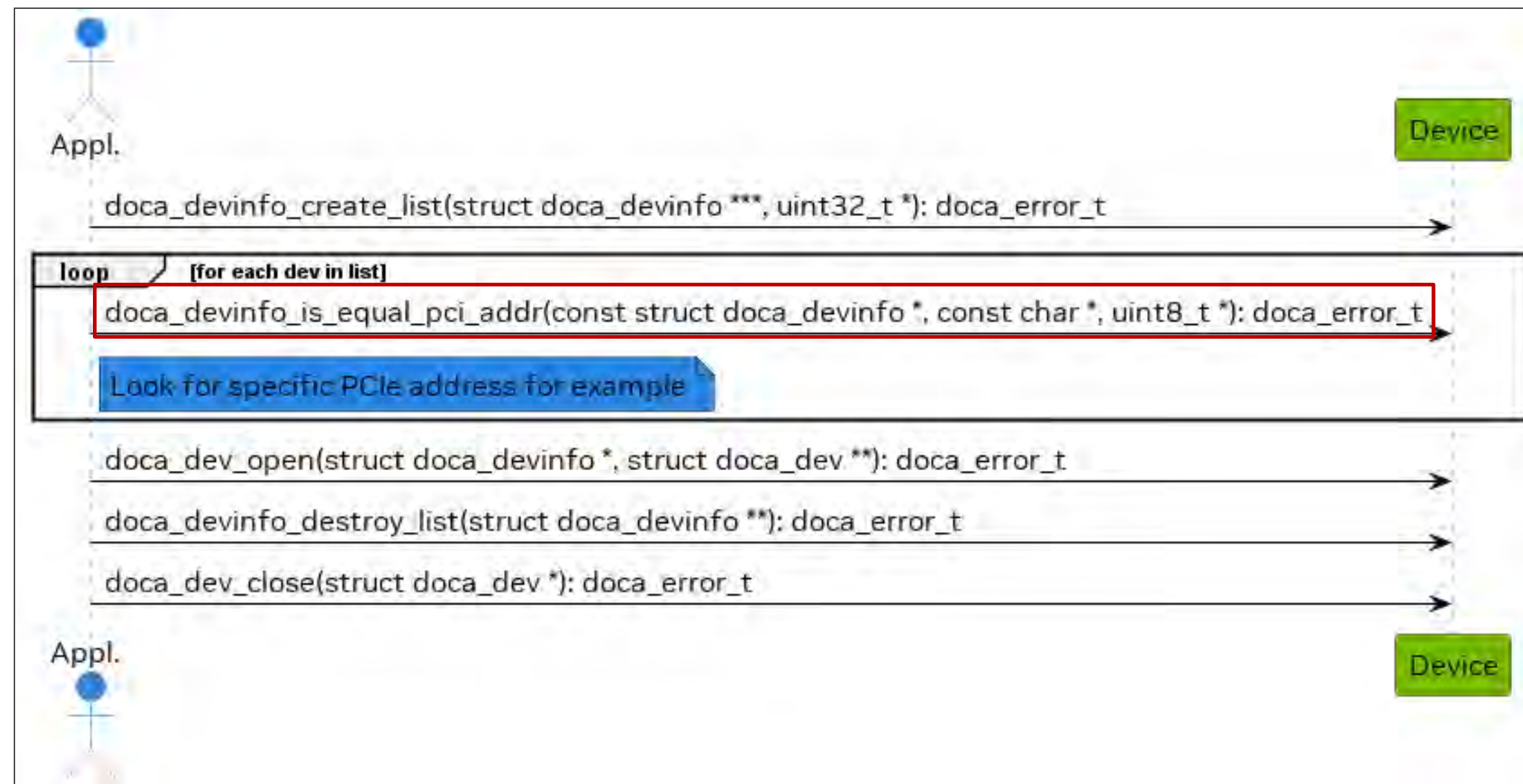
- DOCA exposes APIs to invoke the accelerators (e.g., decompression, direct memory access (DMA), crypto, etc.)
- The workflow consists of:
 1. Device discovery
 2. Create memory map (MMAP)
 3. Create & start a buffer inventory
 4. Create a Progress Engine (PE)
 5. Create and configure the accelerator instance
 6. Allocate and submit a task
 7. Drive progress & handle completion
 8. Cleanup



Device Discovery

To work with DOCA libraries, an application must open a device available on the DPU

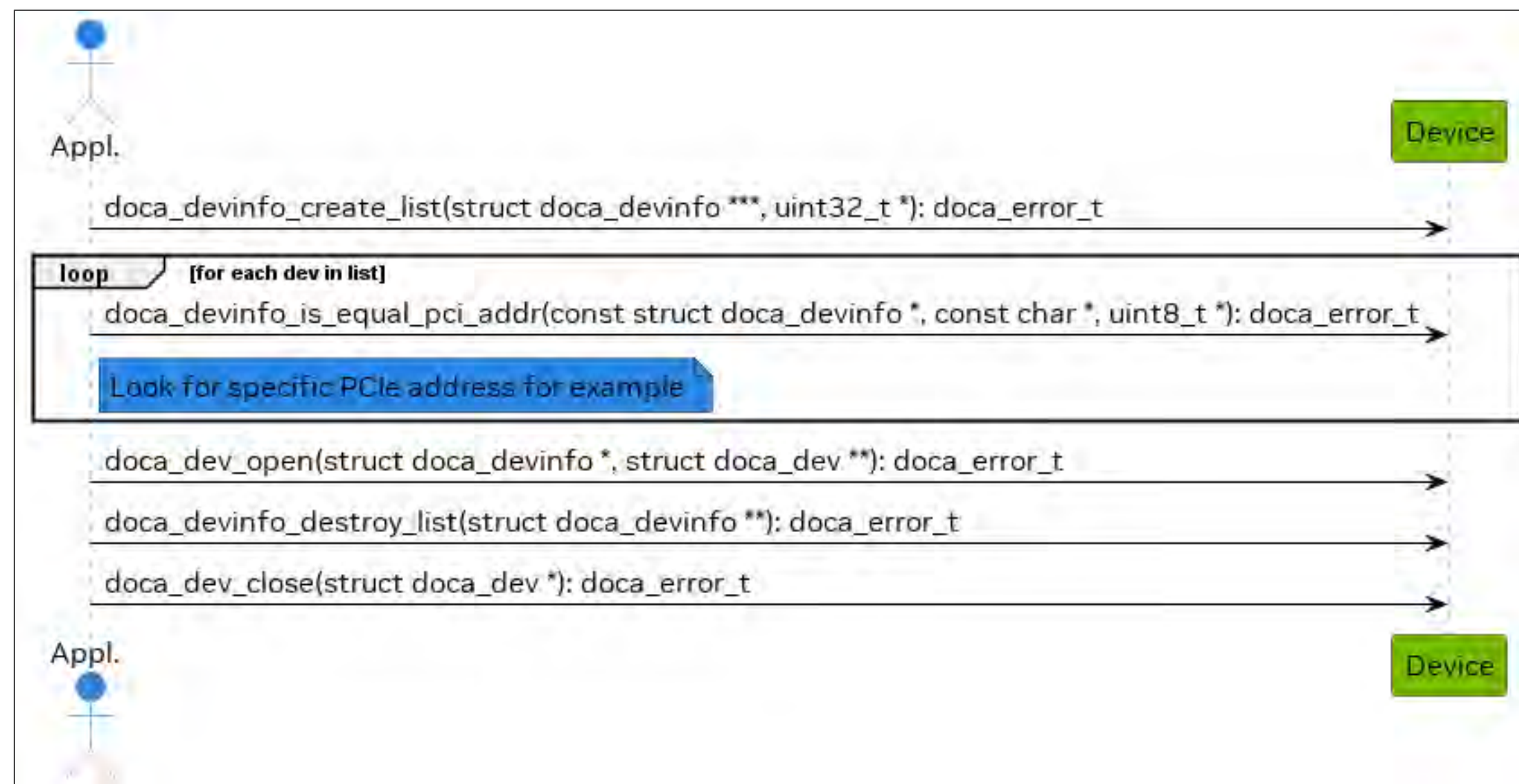
Open Device by PCI address



Device Discovery

To work with DOCA libraries, an application must open a device available on the DPU

Open Device by PCI address



Open Device by capability

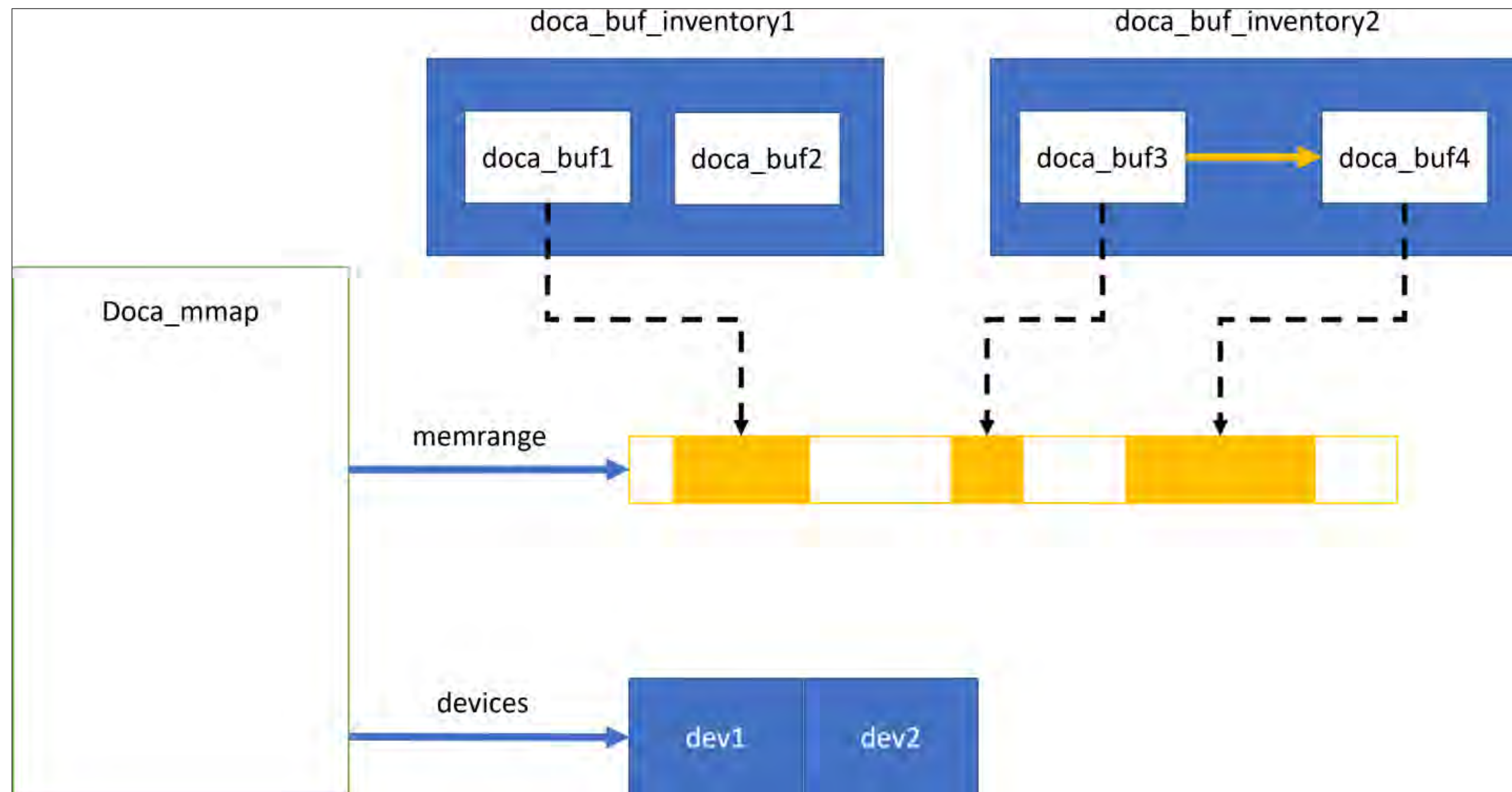
```

struct doca_devinfo **dev_list;
uint32_t nb_devs;
int res = doca_devinfo_create_list(&dev_list, &nb_devs);
if (res != DOCA_SUCCESS) {
    DOCA_LOG_ERR("Failed to load doca devices list: %s",
        doca_error_get_descr(res));
    return res;
}
for (int i = 0; i < nb_devs; i++) {
    res = doca_compress_cap_task_decompress_deflate_is_supported(dev_list[i]);
    if (res != DOCA_SUCCESS) {
        continue;
    }
    if (doca_dev_open(dev_list[i], &state->base.device) == DOCA_SUCCESS) {
        doca_devinfo_destroy_list(dev_list);
        break;
    }
}
  
```

Memory Map (Mmap)

doca_mmap serves as the memory pool for data buffers

The application provides a single memory region, as well as permissions for certain devices to access

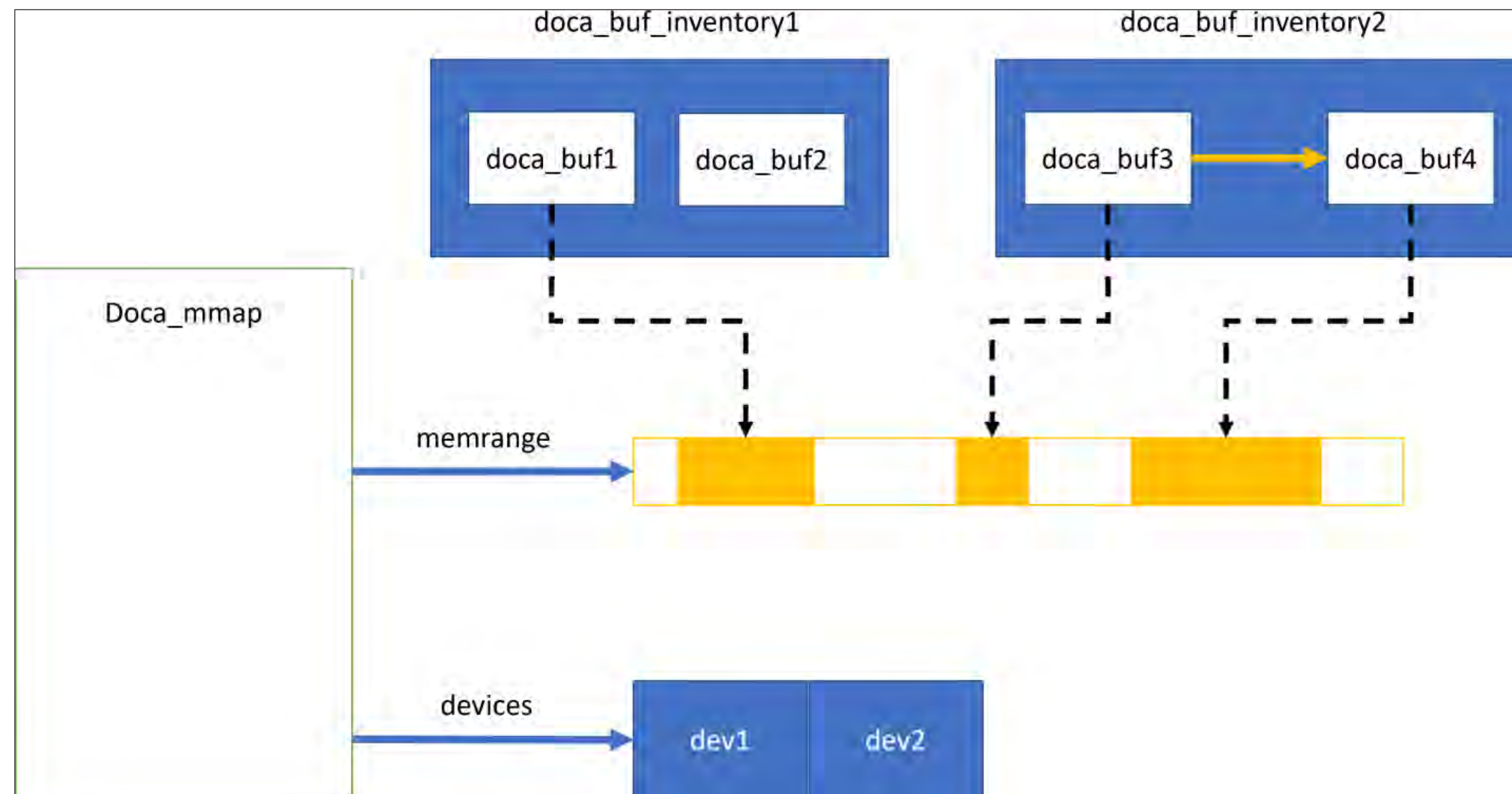


Buffer Inventory

doca_buf is the data buffer descriptor

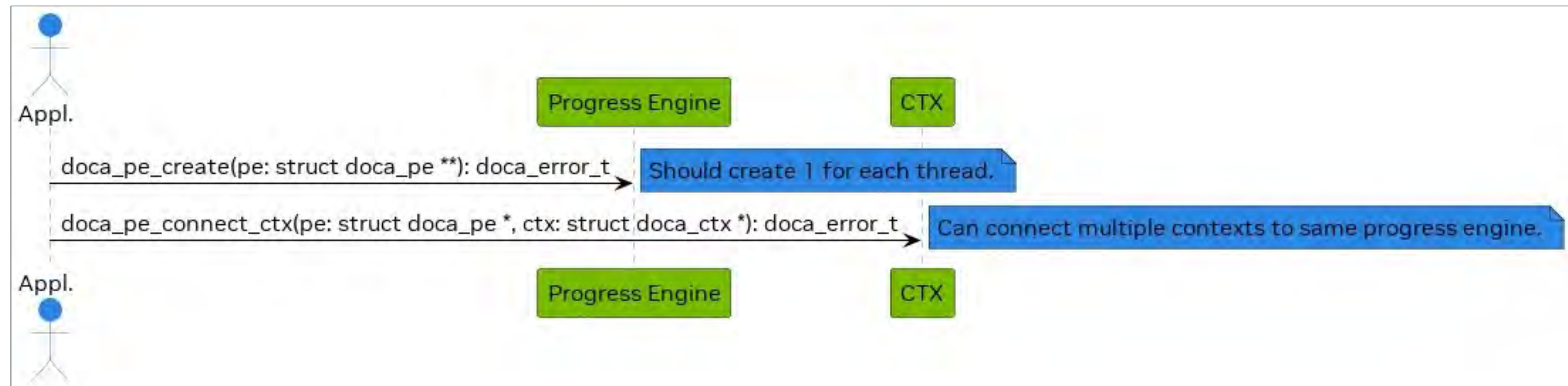
This is not the actual data buffer, rather, it is a descriptor that holds metadata on the "pointed" data buffer

doca_buf_inventory serves as a pool of doca_buf



Progress Engine (PE)

- The progress engine PE is responsible for scheduling workloads (i.e., picking the next workload to execute).
- PE enables asynchronous processing and handling of multiple tasks and events



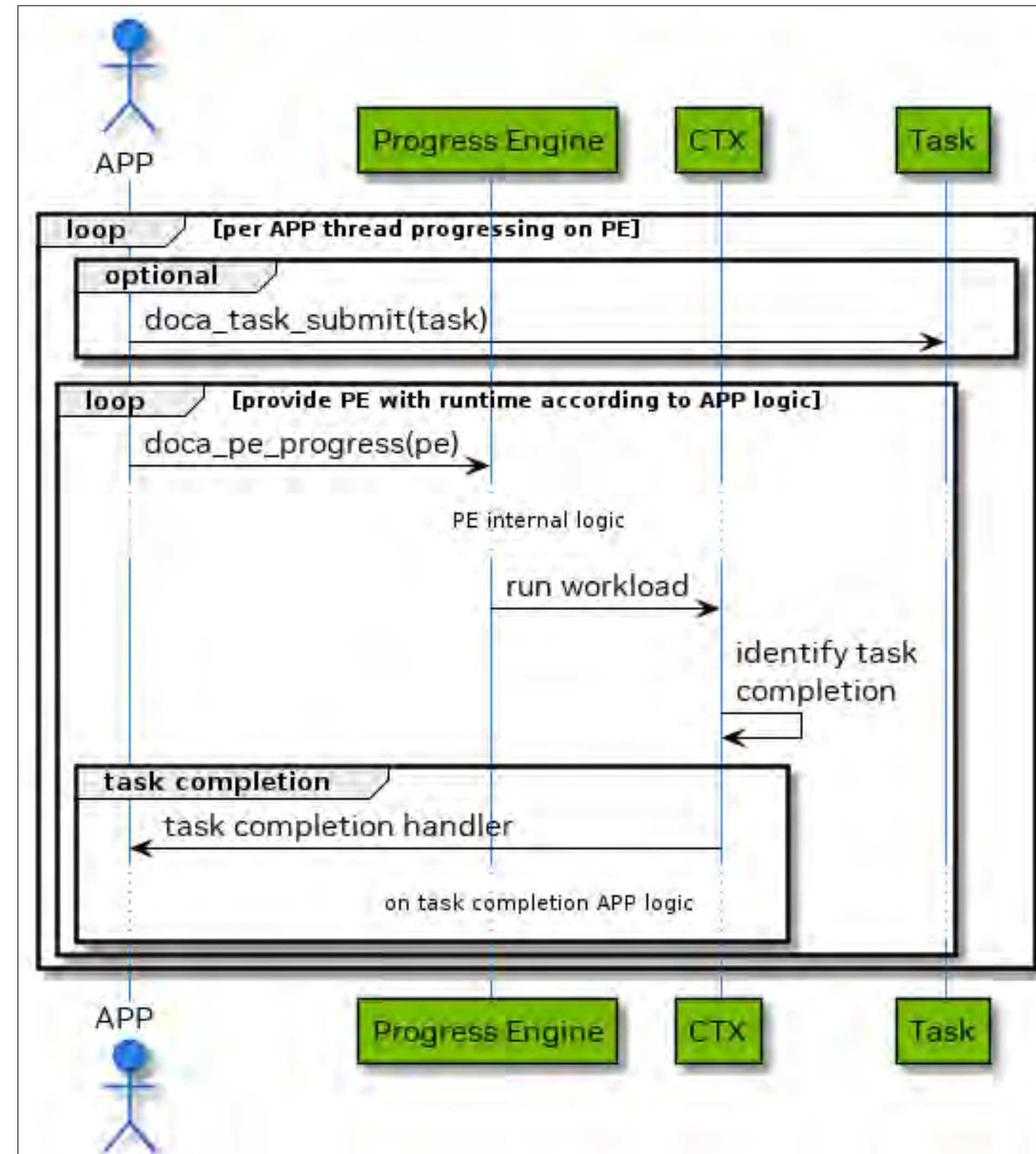
Accelerator Instance

- Create an instance of the accelerator
 - DOCA compress
 - DOCA SHA
 - DOCA RDMA
 - DOCA DMA
 - DOCA Ethernet
 - DOCA Comch
 - DOCA App Shield
 - ...

```
DOCA_LOG_INFO("Creating Compress");  
EXIT_ON_FAILURE(doca_compress_create(state->base.device, &state->compress));
```

Task Submission and Completion

Submit task and wait for completion



Cleanup

Gracefully release everything to avoid leaks and dangling mappings

Make sure all tasks are complete before stopping/destroying resources

DOCA Example on FABRIC Artifact Manager

- Compression acceleration example is available on the Artifact Manager

BlueField DOCA Example

(public) (TCP and P4 Programmable Data Plane Switches)

Back

Edit

Delete

This lab explores the fundamentals of invoking hardware accelerators using the DOCA Core library and the DOCA Execution Model on the NVIDIA BlueField DPU. The lab exercise focuses on configuring and utilizing the DOCA Compression Accelerator through the `doca_compress` API. The objective is to demonstrate how to create and submit compression tasks asynchronously and implement completion and error callbacks to handle task results. The lab provides hands-on experience with accelerator invocation via the DOCA Execution Model. A working knowledge of C programming is a recommended prerequisite.

👁 0

📄 0 (0)

📄 0

Oct. 13, 2025, 8:52 p.m.

education

example

tutorial

Add a new Version

Choose File

no file selected

Submit

Authors

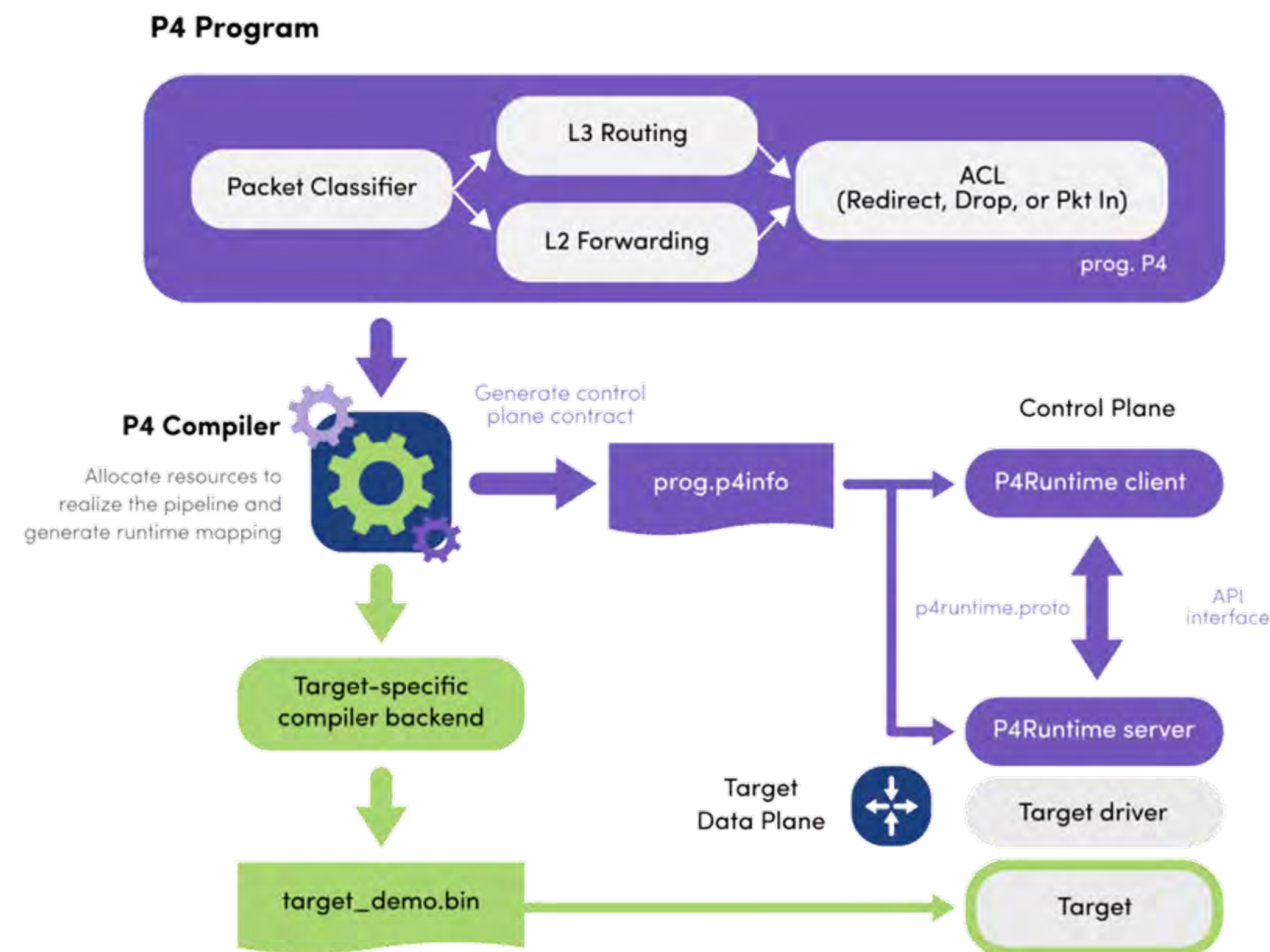
- Amith Gorthi Srinivasa Prabhakara Narasimha , University of South Carolina (Amithgspn@sc.edu)
- Elie Kfoury , University of South Carolina (ekfoury@email.sc.edu)



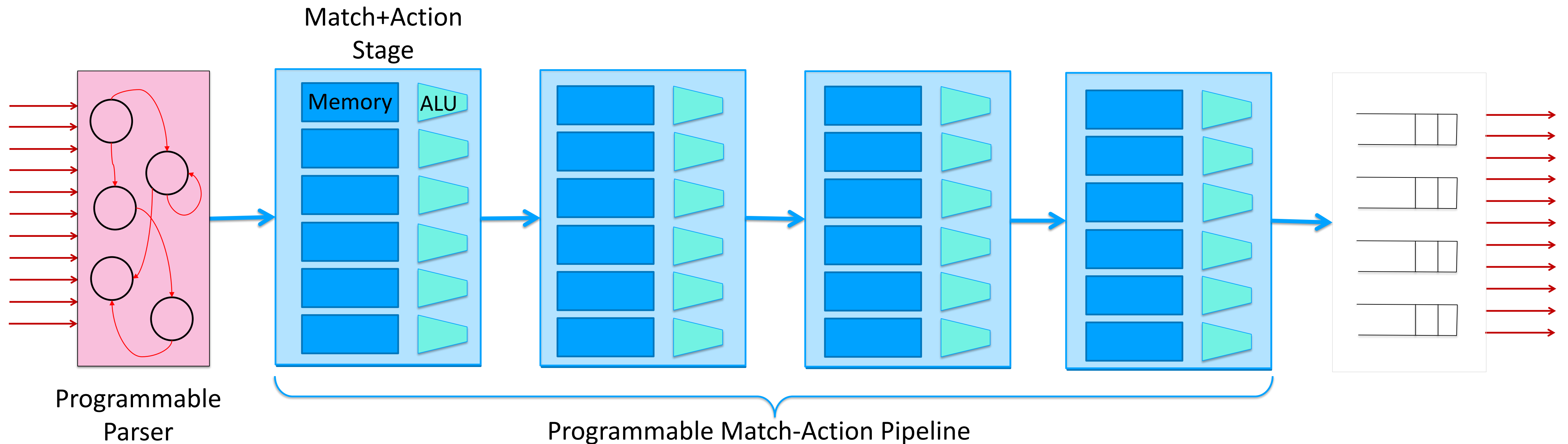
Programming the Accelerated Pipeline with P4

P4 Language

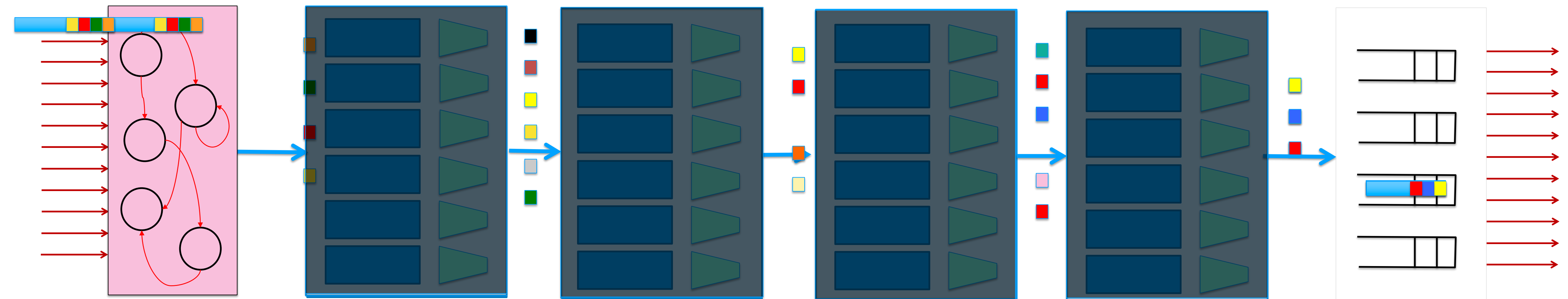
- P4 emerged as domain-specific language to define packet processing:
 - Define and parse new protocols
 - Customize packet processing functions



PISA: Protocol Independent Switch Architecture



PISA: Protocol Independent Switch Architecture



Example P4 Program

Parser Program

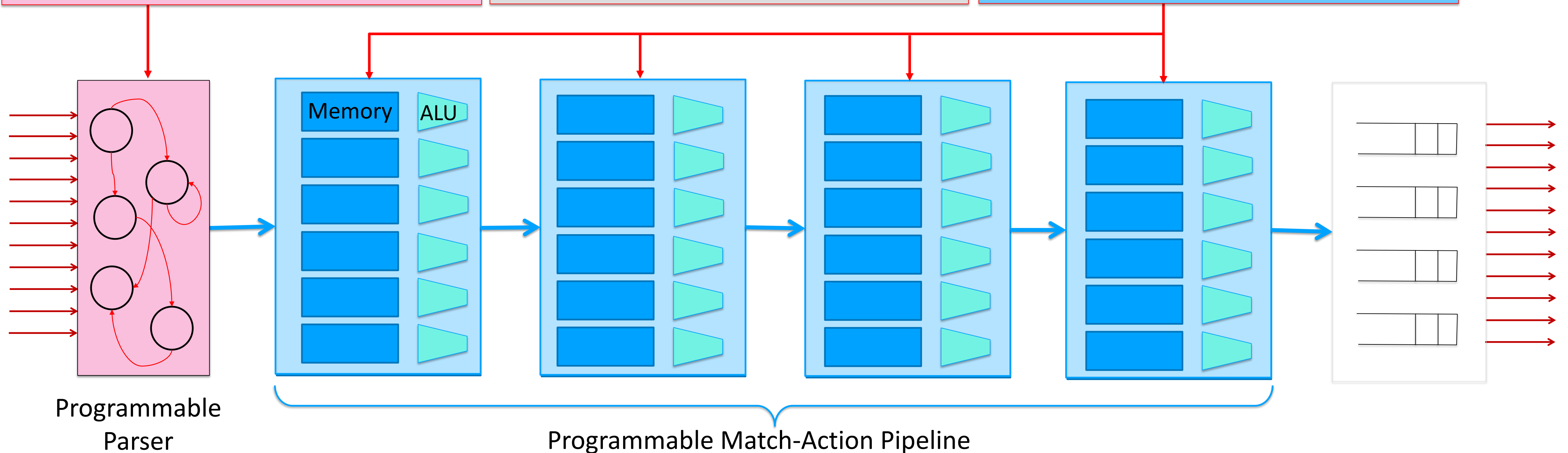
```
parser parse_ethernet {  
  extract(ethernet);  
  return switch(ethernet.ethertype) {  
    0x8100 : parse_vlan_tag;  
    0x0800 : parse_ipv4;  
    0x8847 : parse_mpls;  
    default: ingress;  
  }  
}
```

Header and Data Declarations

```
header_type ethernet_t { ... }  
header_type l2_metadata_t { ... }  
  
header ethernet_t ethernet;  
header vlan_tag_t  
vlan_tag[2];  
metadata l2_metadata_t l2_meta;
```

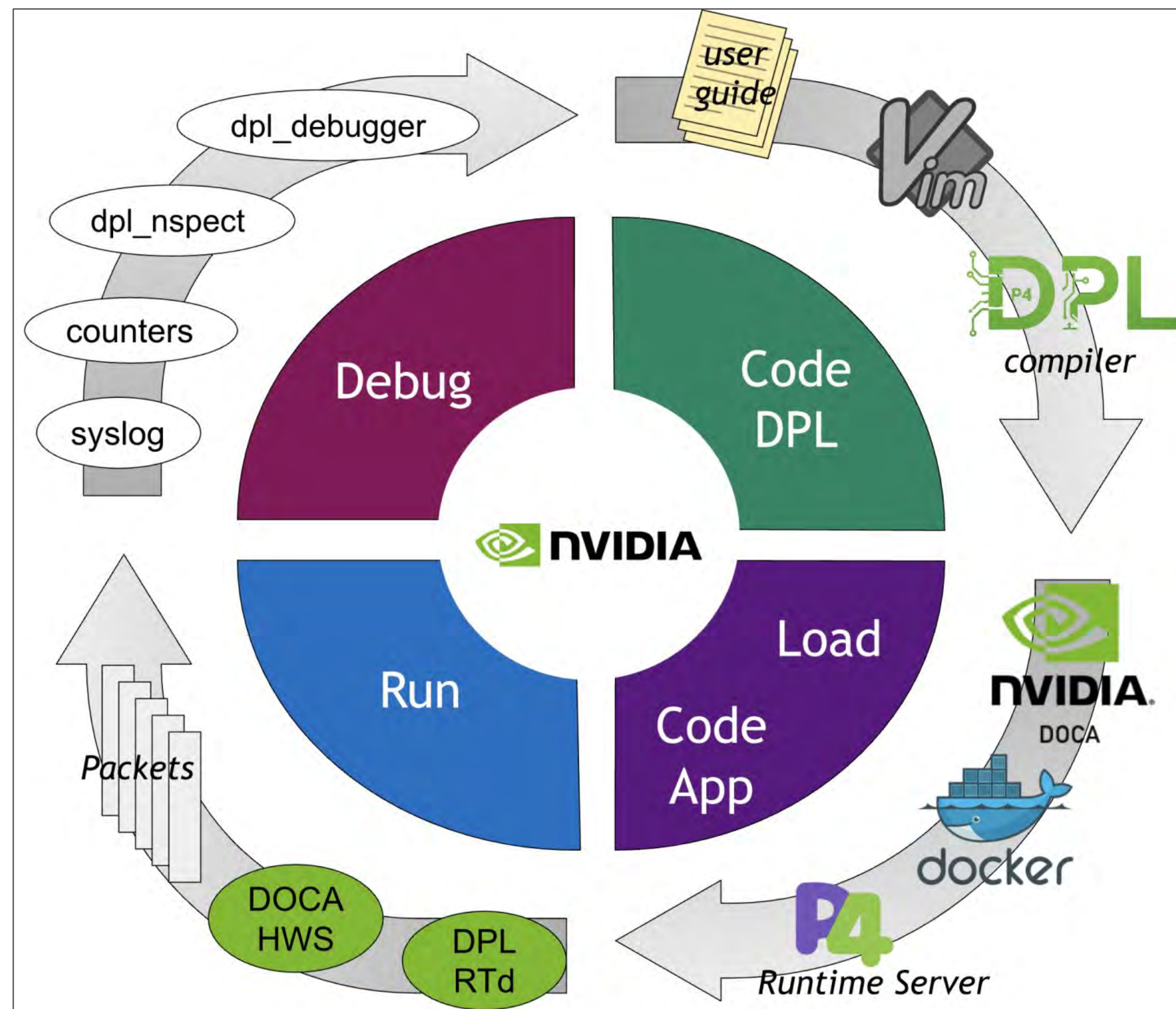
Tables and Control Flow

```
table port_table { ... }  
  
control ingress {  
  apply(port_table);  
  if (l2_meta.vlan_tags == 0) {  
    process_assign_vlan();  
  }  
}
```



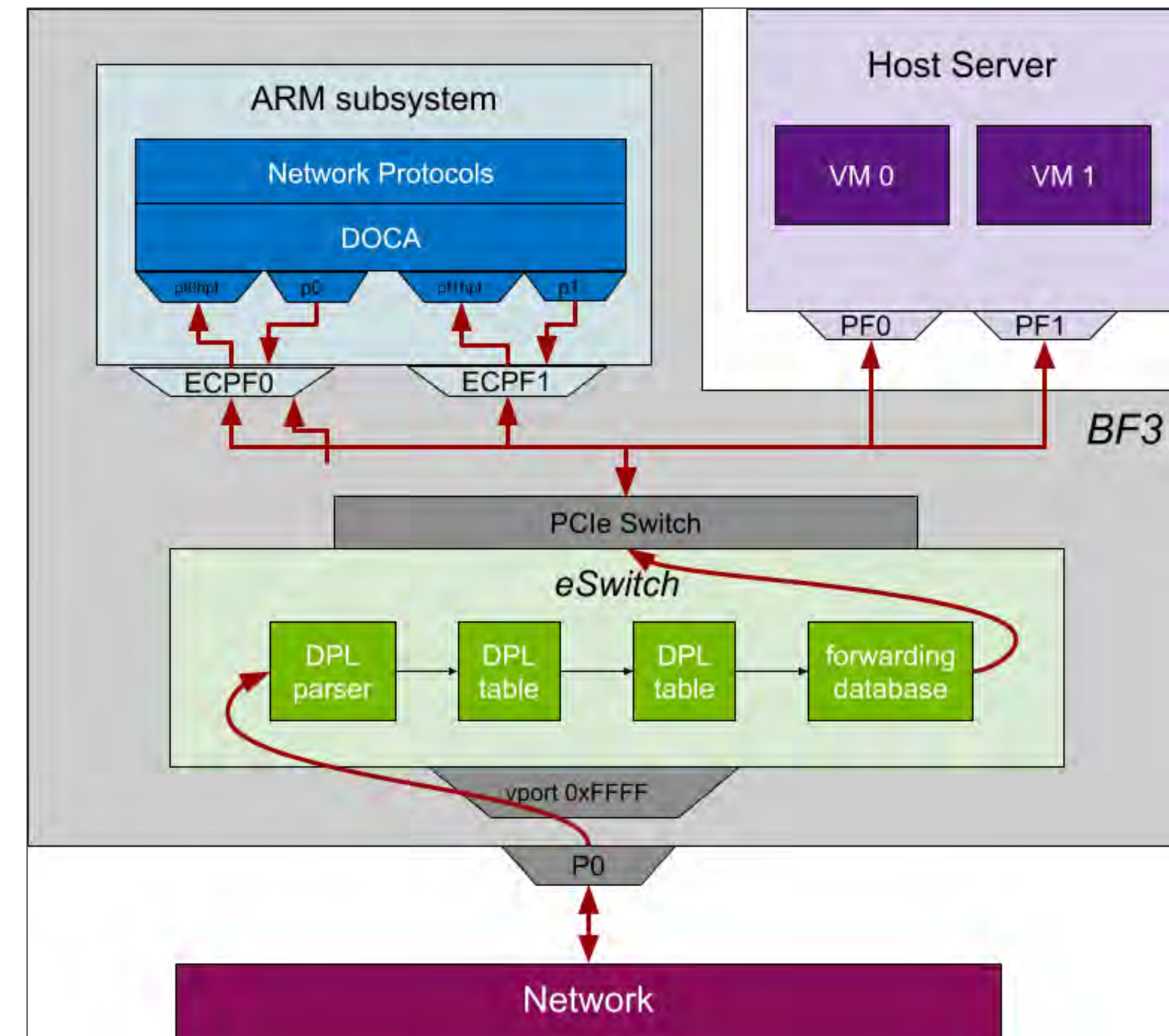
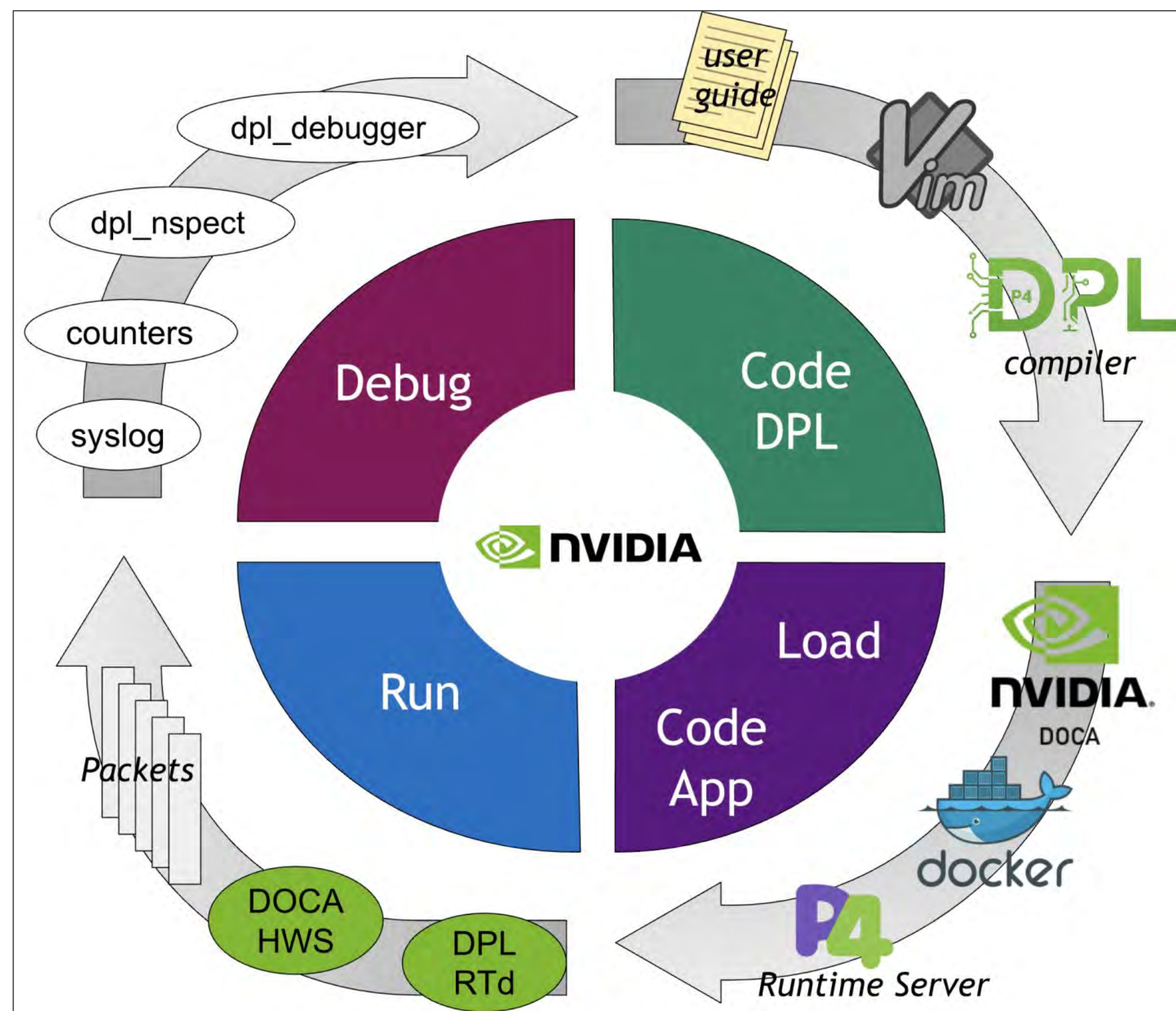
DPL Overview

- DOCA Pipeline Language (DPL) introduces a software development solution based on the P4 language
- DPL is derived from the P4-16 language specification



DPL Overview

- DOCA Pipeline Language (DPL) introduces a software development solution based on the P4 language
- DPL is derived from the P4-16 language specification



AGENDA

Time	Topic	Details
9:00-9:10	Introduction	Attendee Survey; intro to Hands-On
9:10-9:25	Communication Offloading	SmartNIC overview DPU examples DPU programming models
9:25-9:40	SmartNIC Use Cases	Packet processing, cyber-security, AI/HPC
9:40-9:55	Infrastructure SW – DOCA and P4 Introduction	Brief introduction to DOCA/P4
10:00-10:30	HPC Programming – Part 1	MPI Offload; DPA Programming
10:30-11:00	HPC Programming – Part 2	OpenMP Offload
11:00-11:30	BREAK	
11:30-11:45	Hands-on DOCA/P4/DPA Demo	Attendees follow along in compiling and running test examples
11:45-12:15	Hands-on HPC Demo – OpenMP/MPI Offload	
12:15-12:30	Storage Acceleration	Discussion of use case for SmartNICs with storage systems for AI
12:30-12:35	Tutorial Survey	
12:35-1:00	Continued Hands-on Activities	Time to work with the instructors on the presented demos and test cases

See the updated agenda at <https://github.com/gt-crnc-hrg/smartnic-tutorial-isc26>



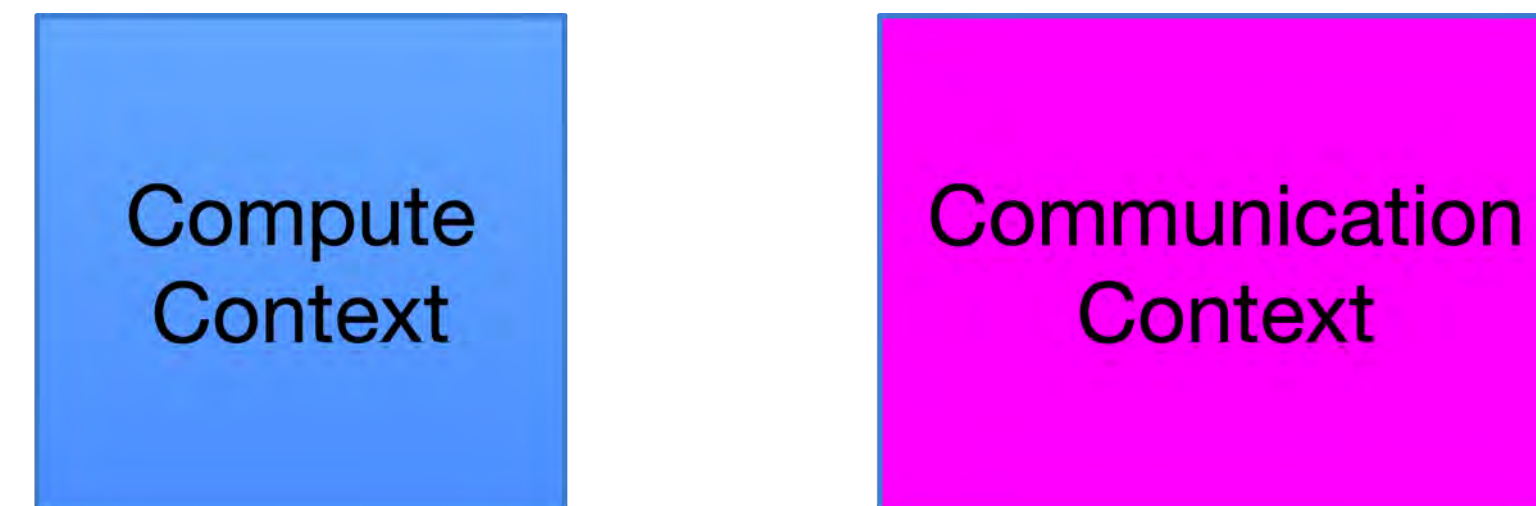
Please share your feedback in a short 4 question survey!



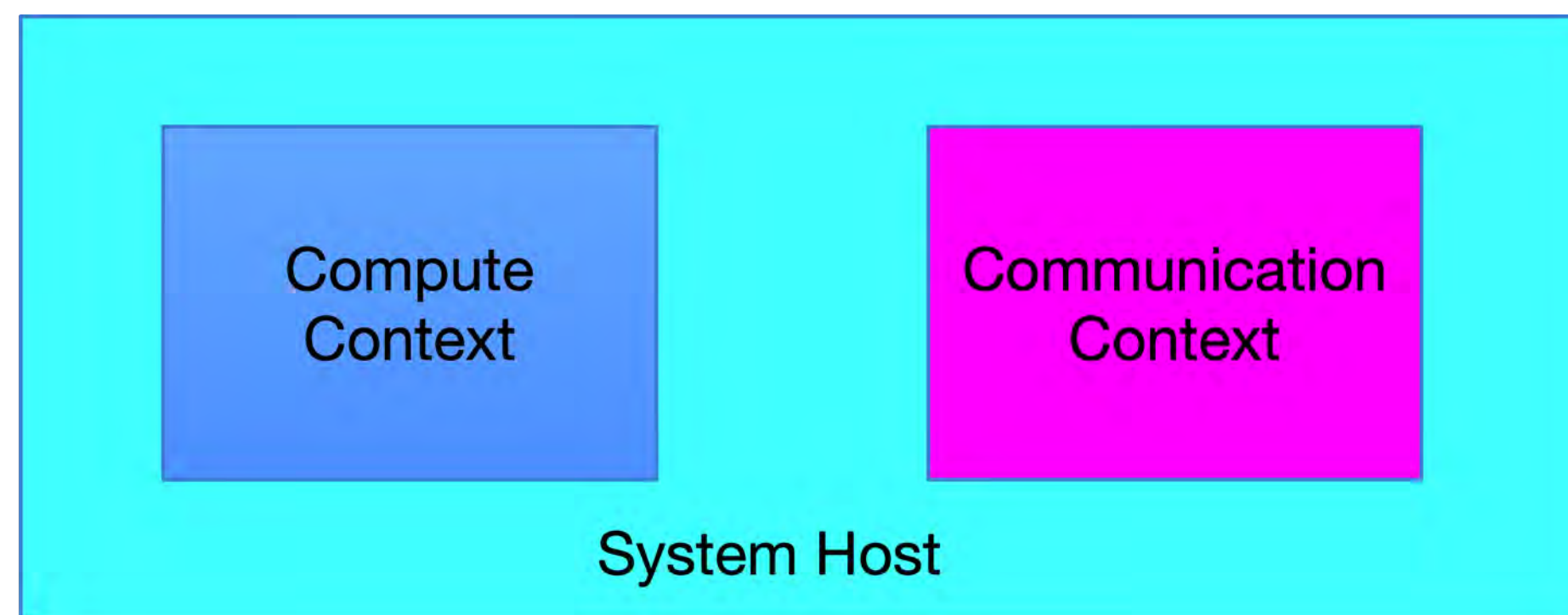
HPC/AI Programming – DPU/DPA and MPI Offload

Asynchronous Communication Model

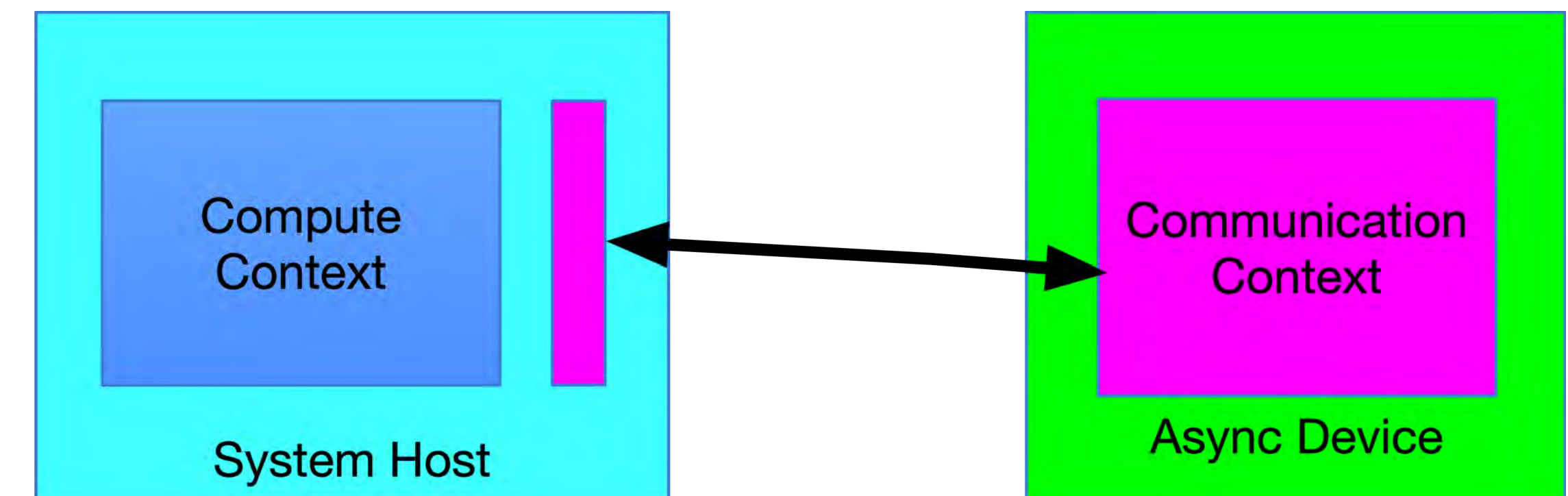
Application Components



Traditional System Design



Asynchronous System Design





NVIDIA Data Processing Unit (DPU)

DPU Software Components for HPC Programming

UCC

- Collective algorithms

Service Processes (SP)

- Run on the BlueField
- Single SP services several host-side processes

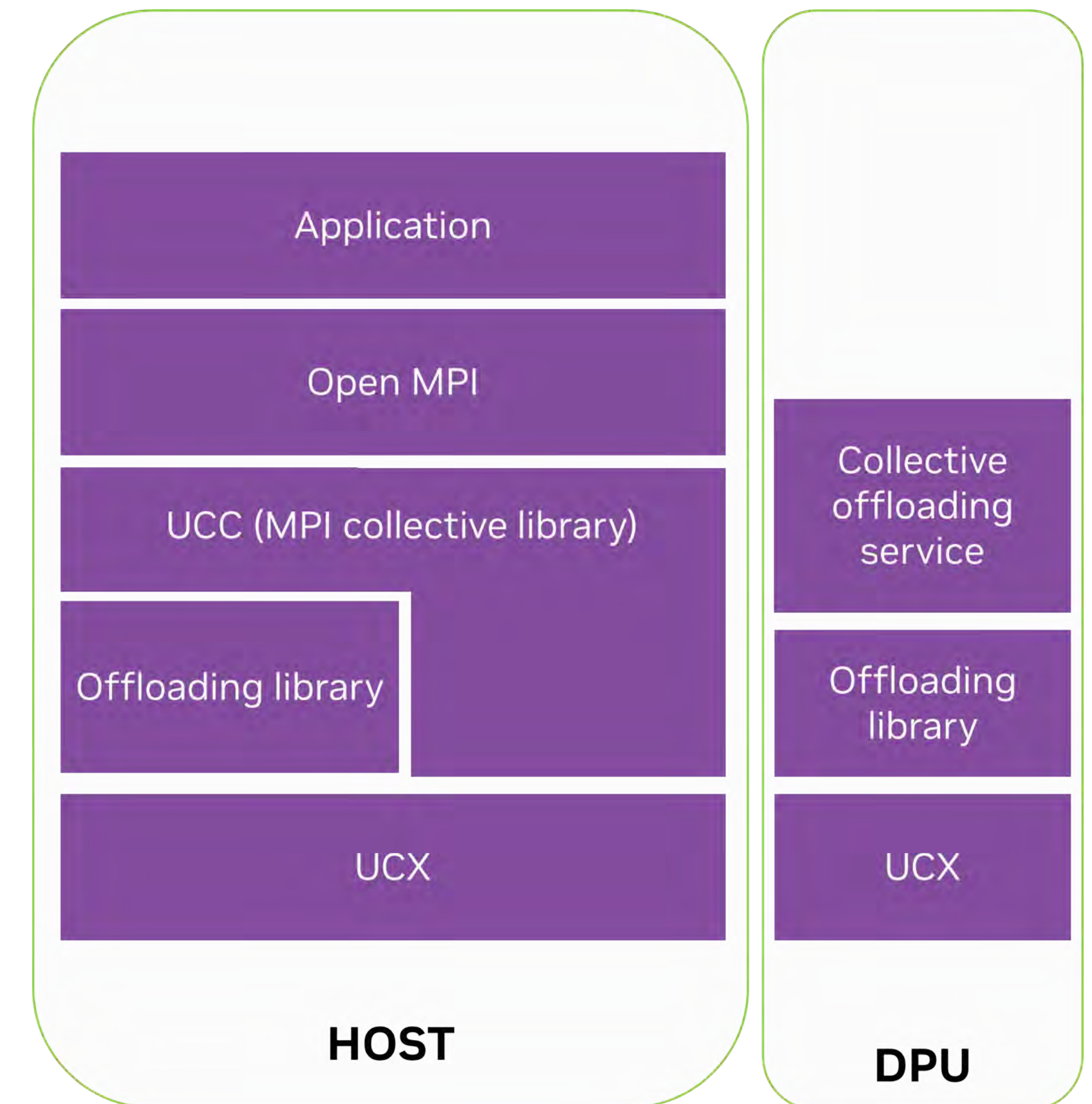
DOCA UROM – Run-time library used that serves the offloaded algorithms

- Control path – provides coordination services, active message based
- Services provided:
 - Offload engines
 - Offload service bootstrapping
 - Group and topology management
 - UCX endpoint cache
 - Execution context
 - Notifications

UCX

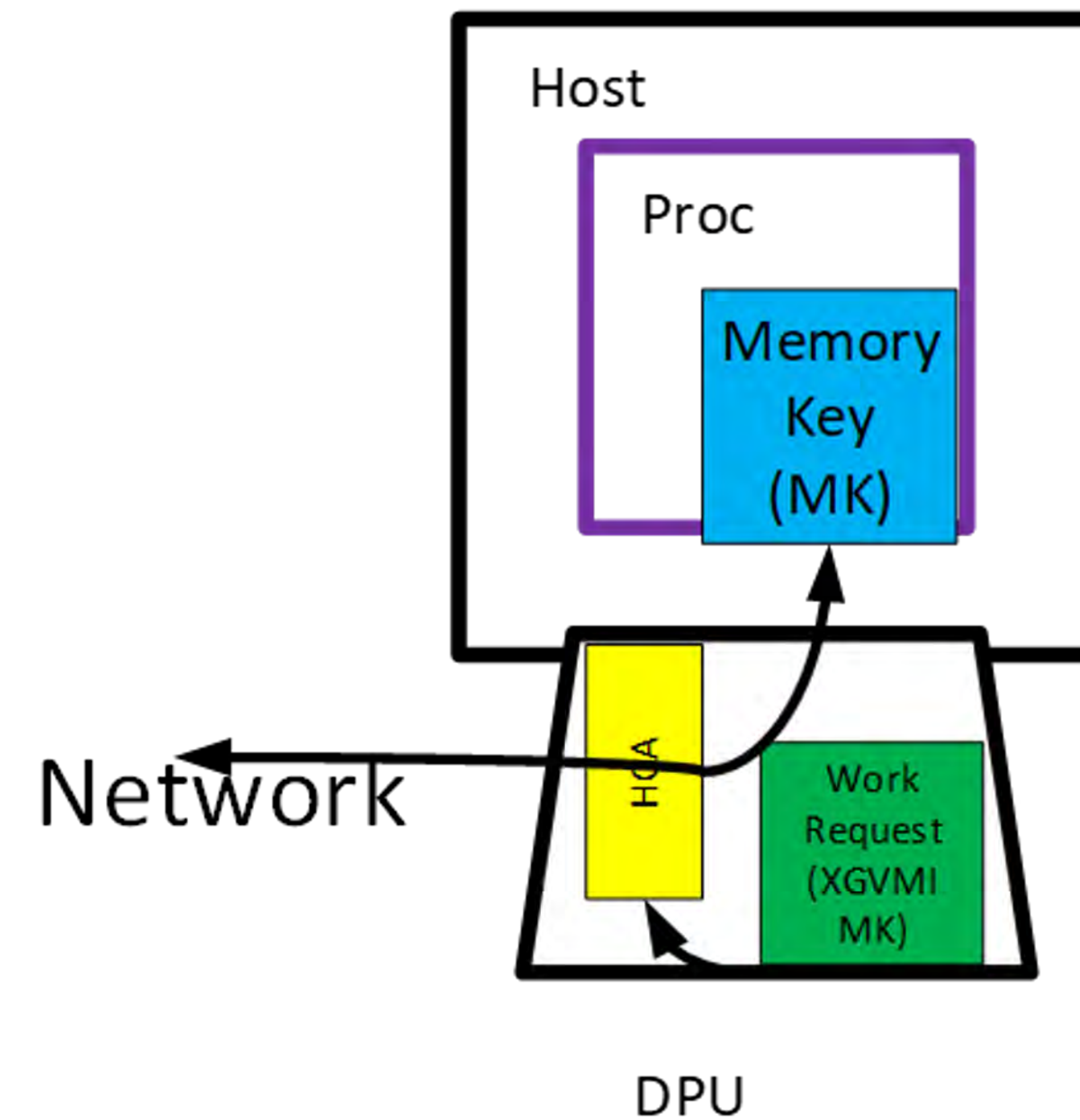
- Data-path services
- Extended to support the alias memory keys (XGVMI)

BlueField optimized collectives



Cross-GVMI Memory on DPUs

Memory keys that allow DPU based work-requests to reference host-side memory



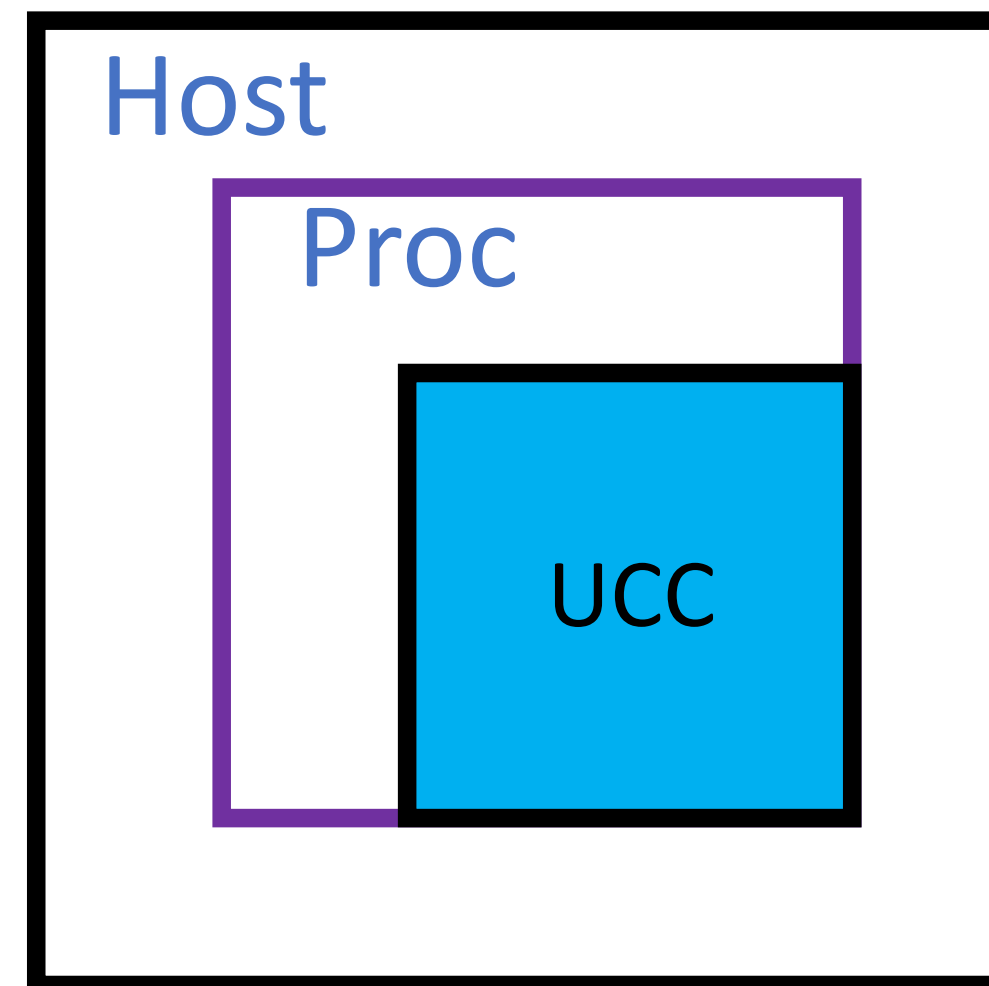
DPU can initiate data transfer on behalf of the host, with no host involvement

Do not need to move data to the DPU before the DPU can send it

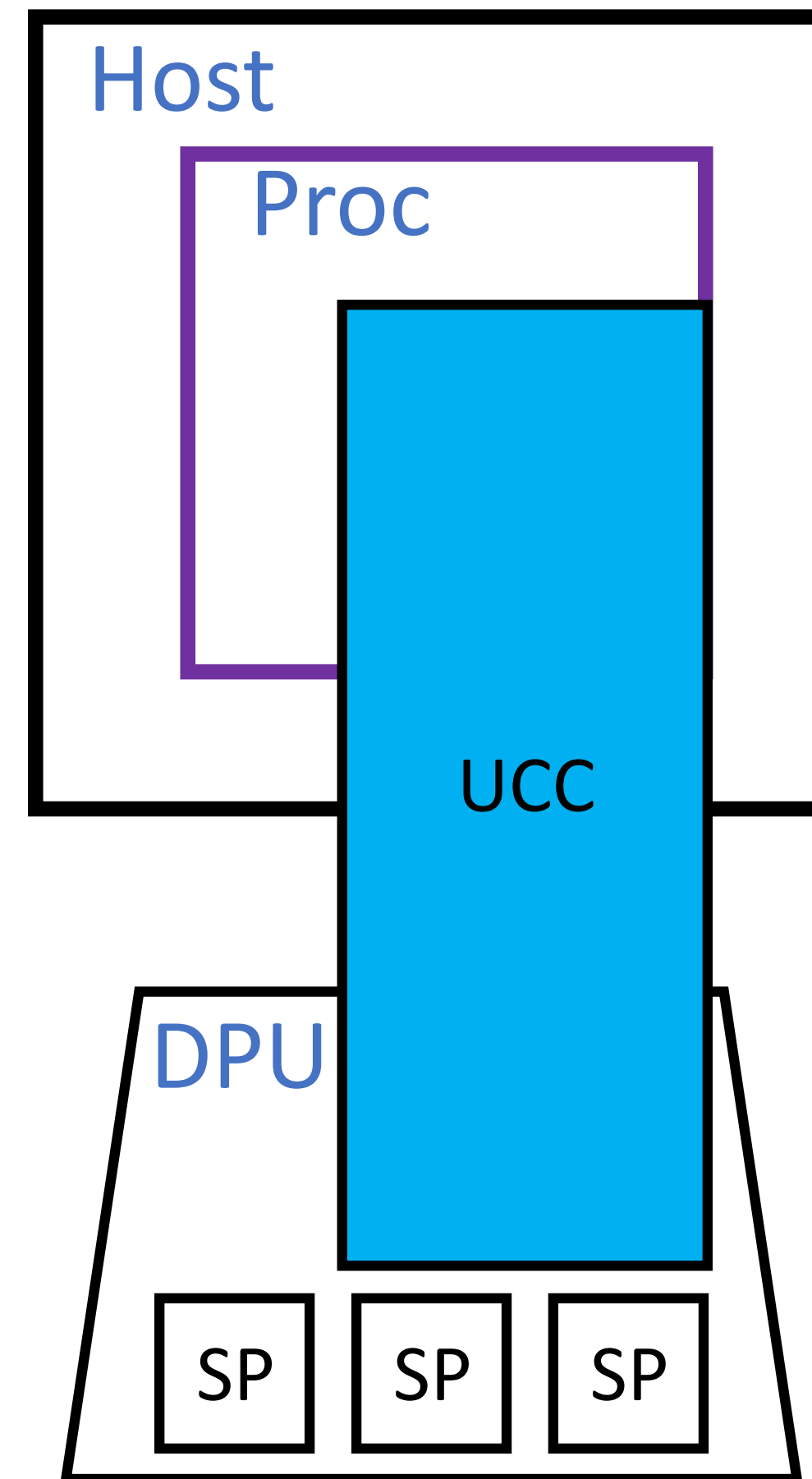
Can post receive work requests on behalf of memory that is host resident

UCC DPU Offload Model

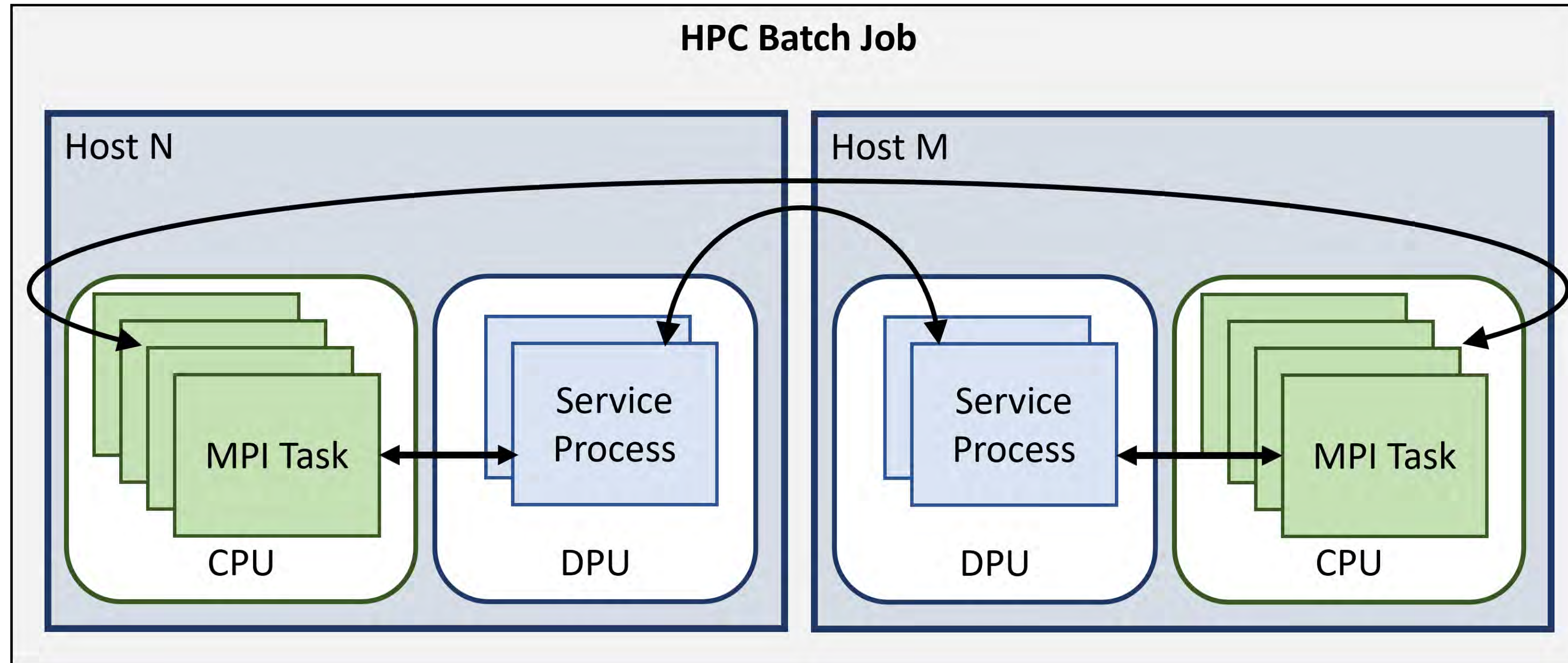
Host Only Model



DPU Offload

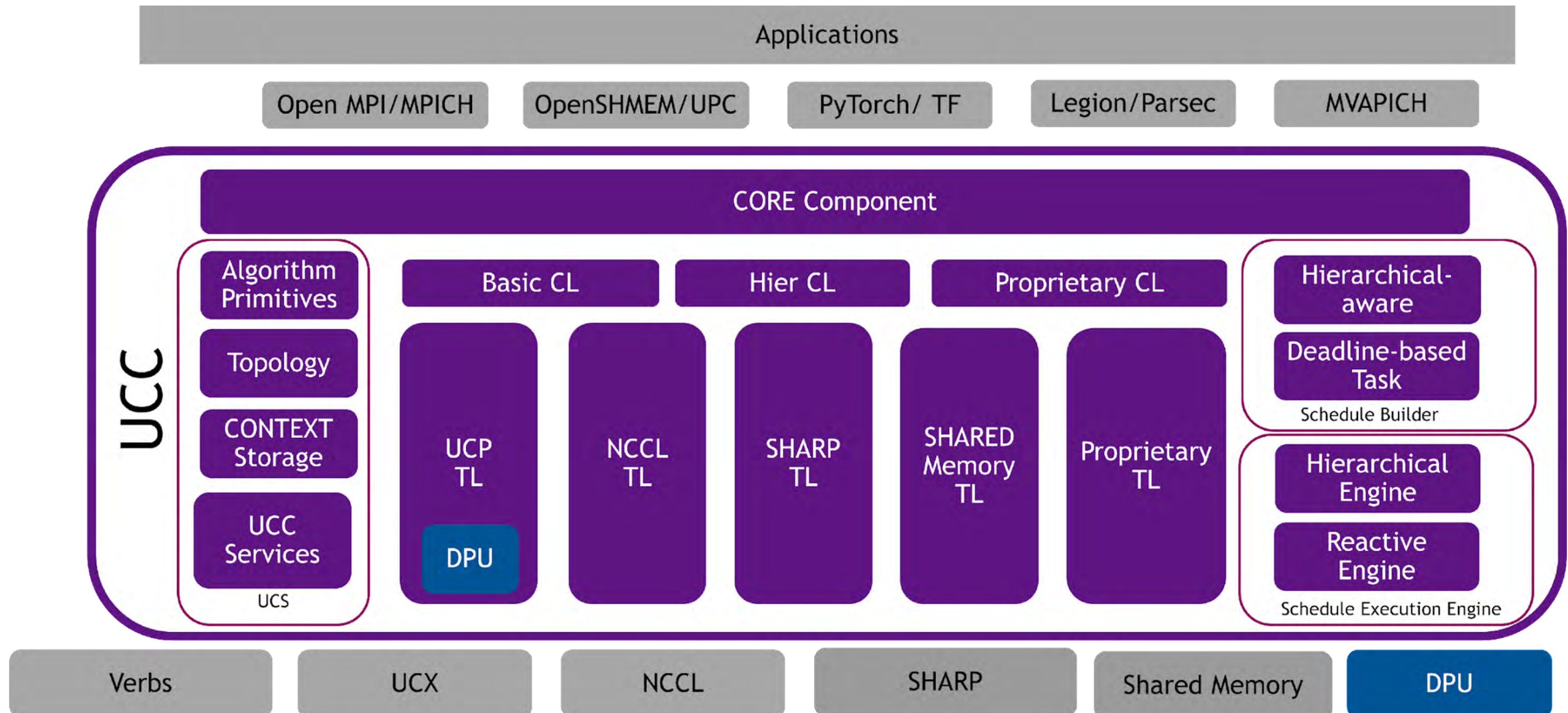


Job Process Layout

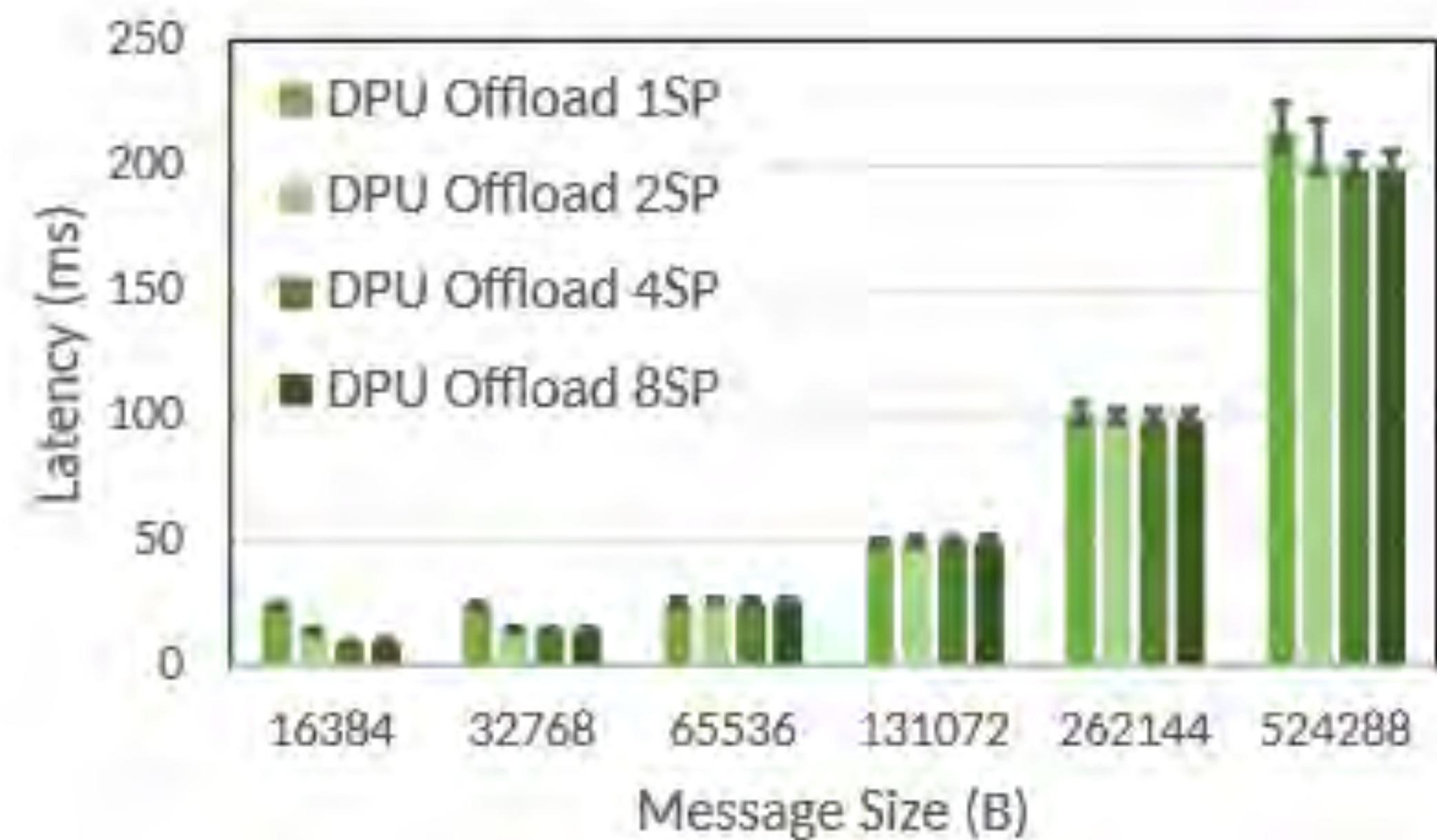


- Service Processes on the DPU enable seamless MPI jobs that make use of DPU and host CPU resources

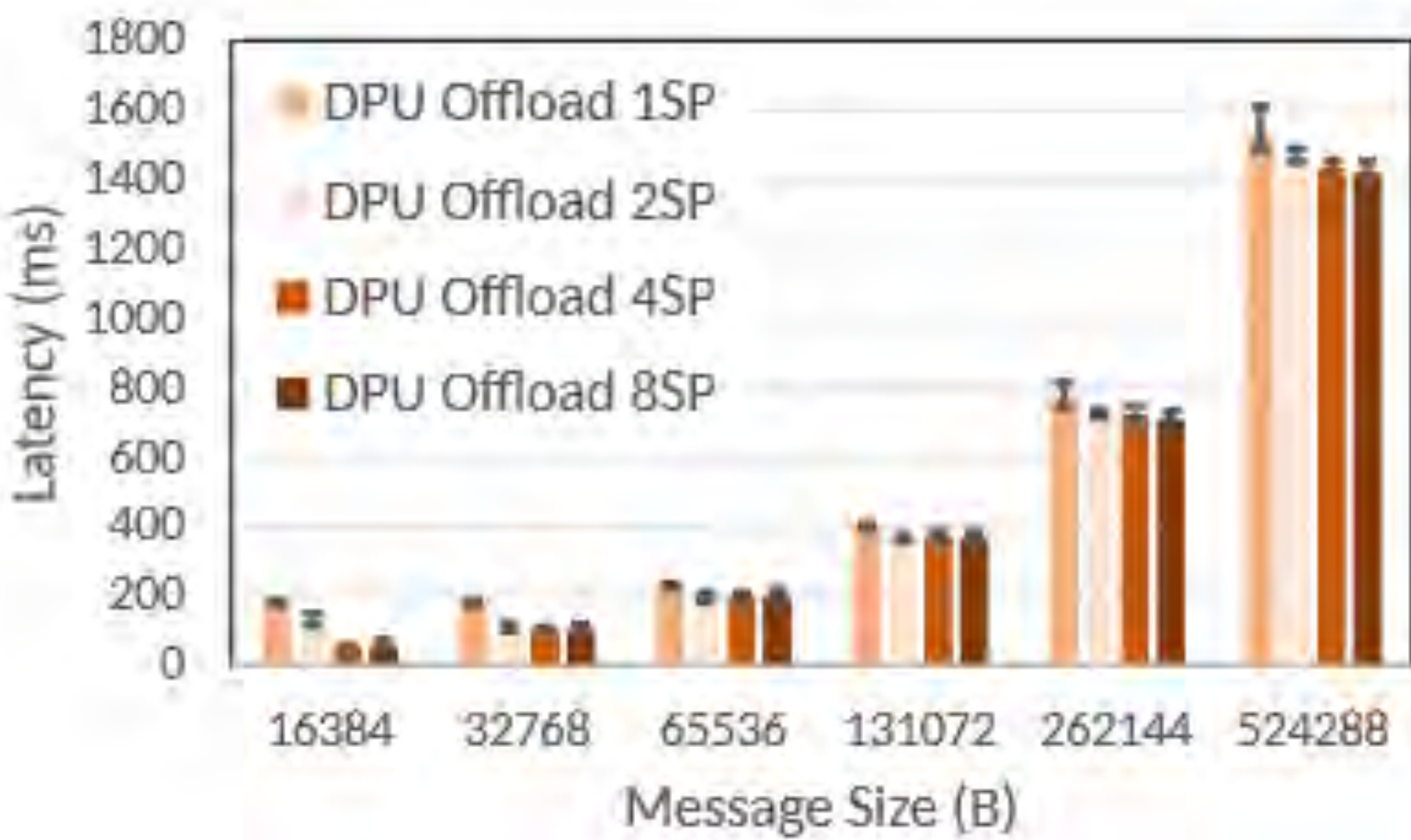
Unified Collective Communication (UCC) Architecture



Collective Latency (OSU) as a Function of Service Process Count

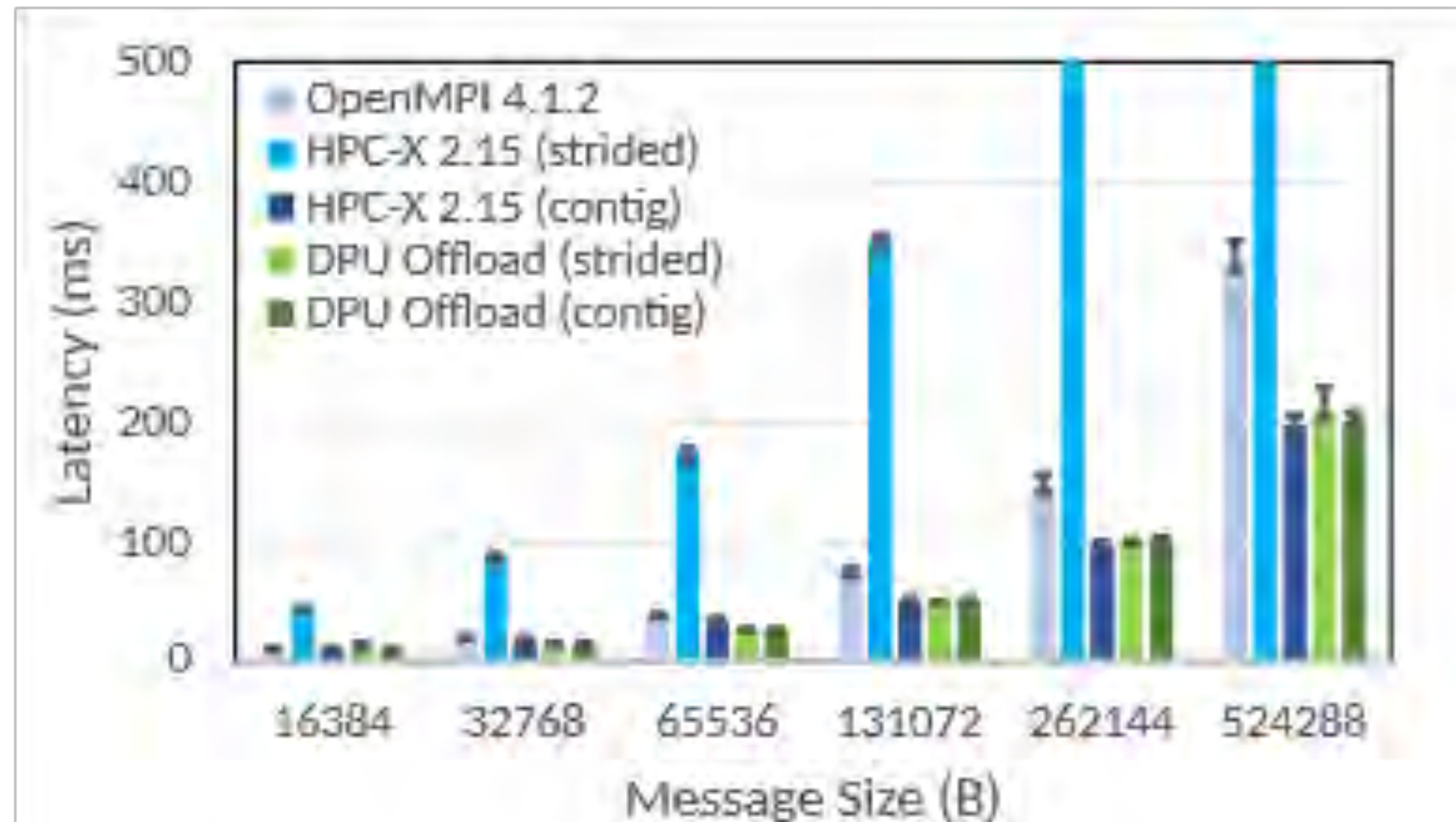


(a) MPI_Allgather

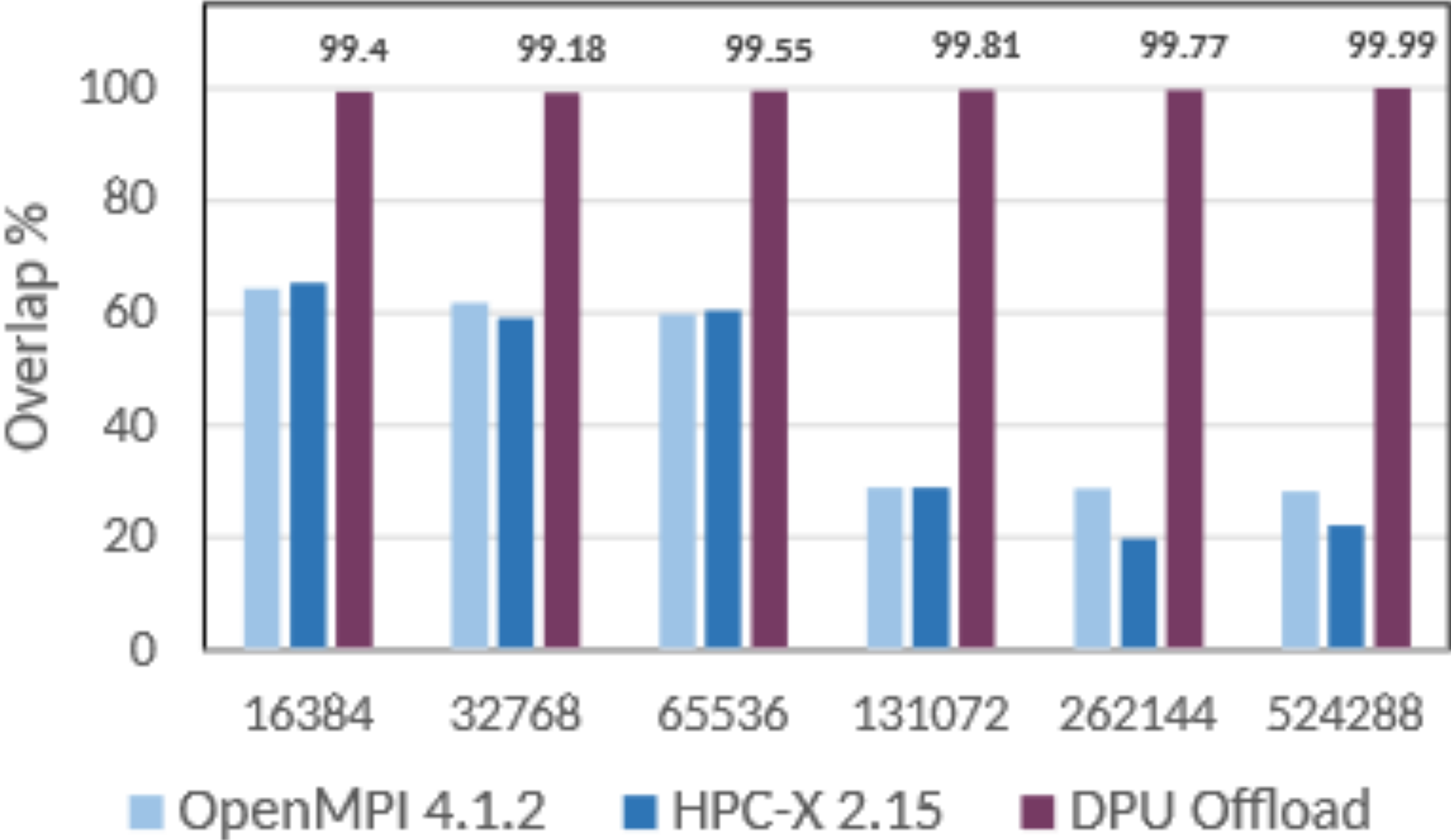


(b) MPI_Alltoallv

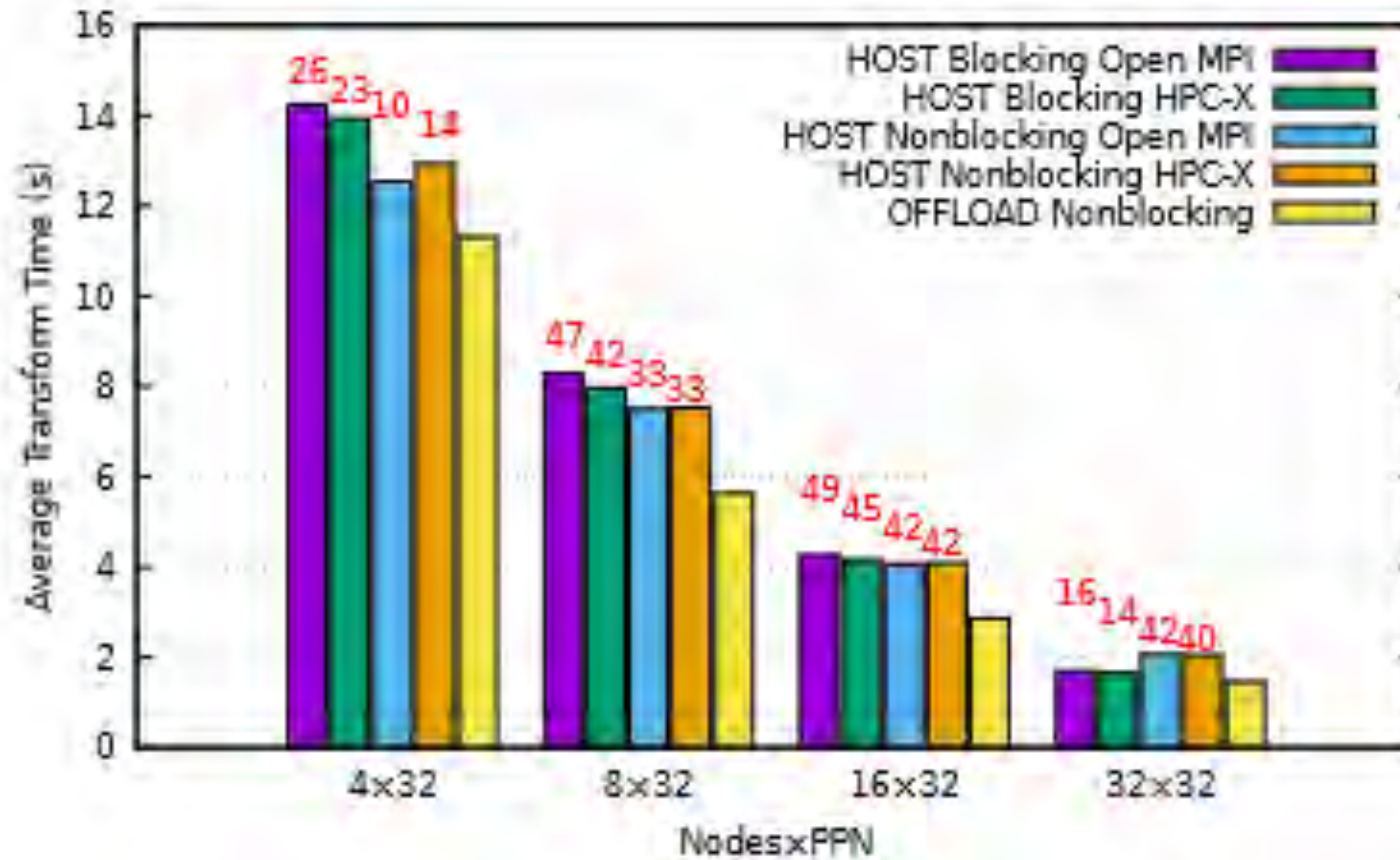
MPI_AllgatherV OSU Latency – Implementation Comparison



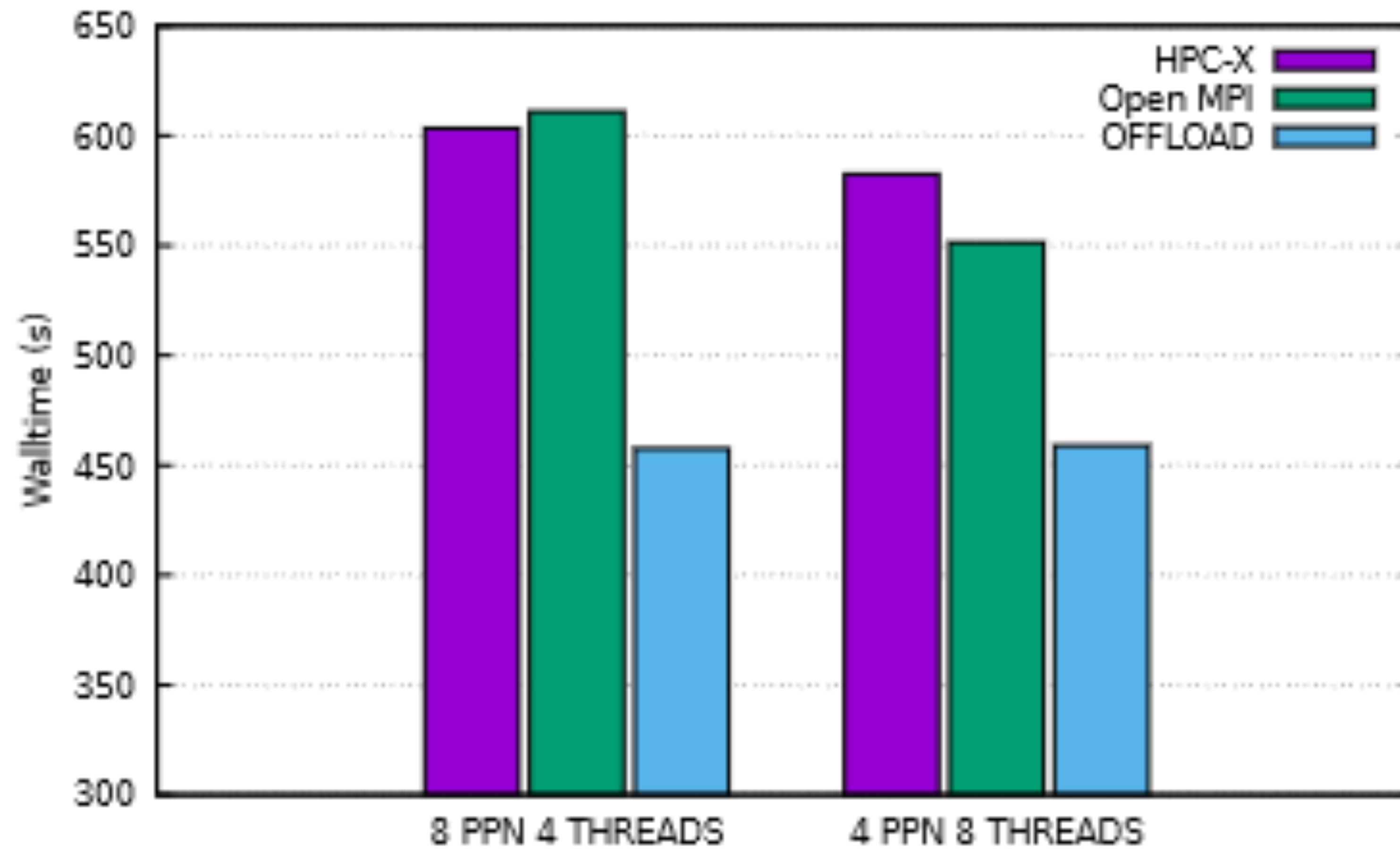
MPI_iAlltoallv OSU overlap as Function - Implementation comparison



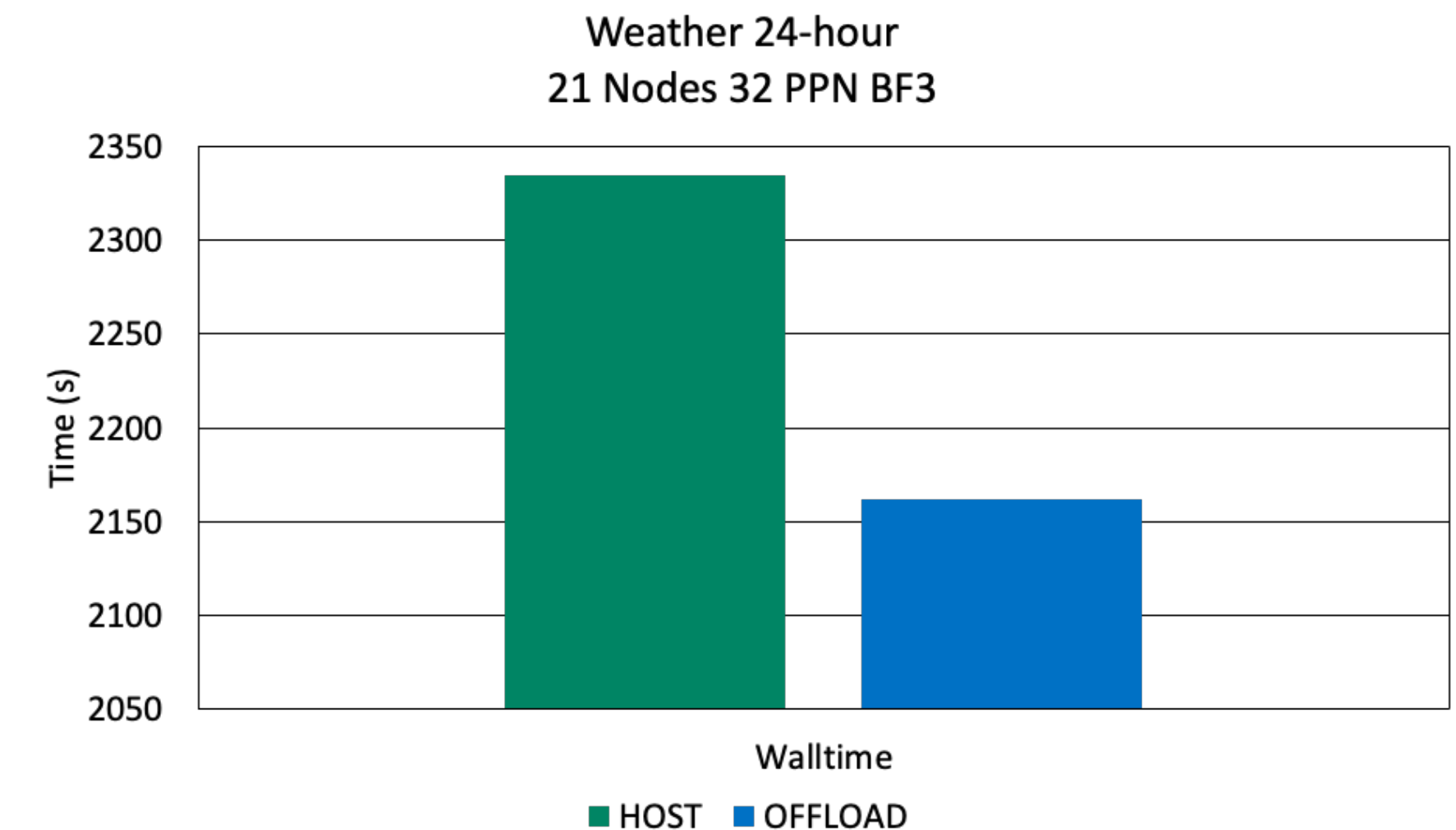
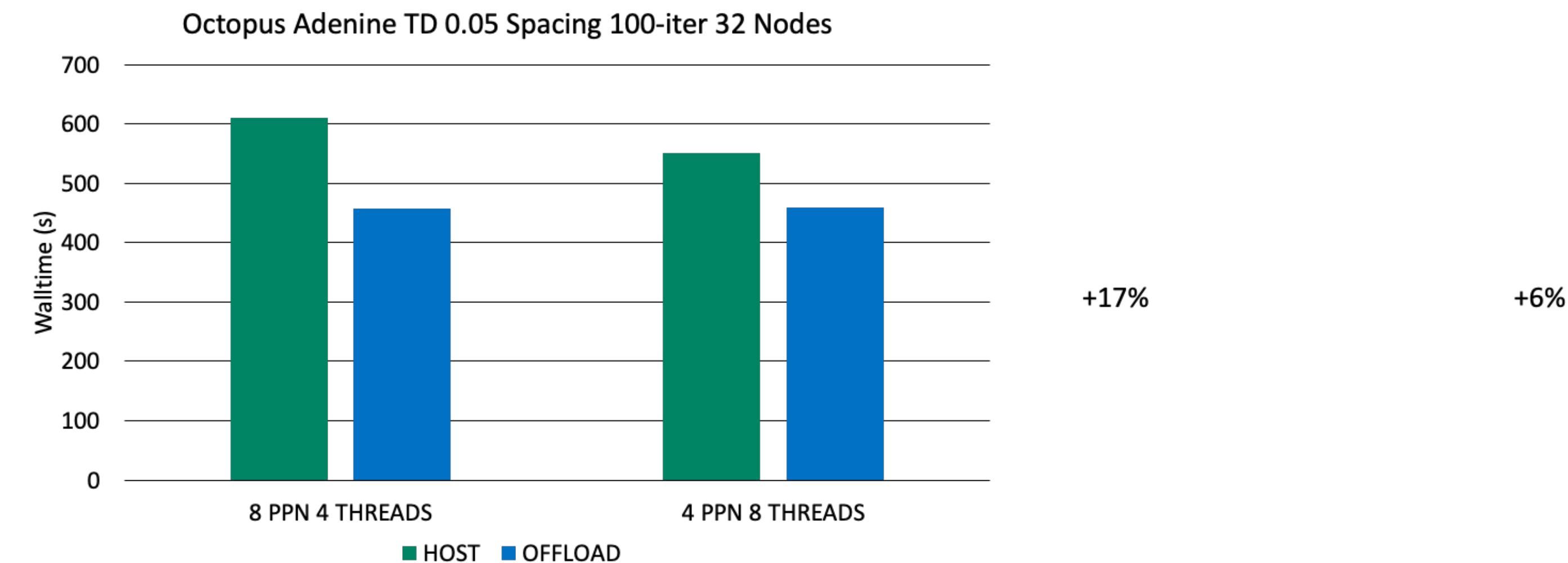
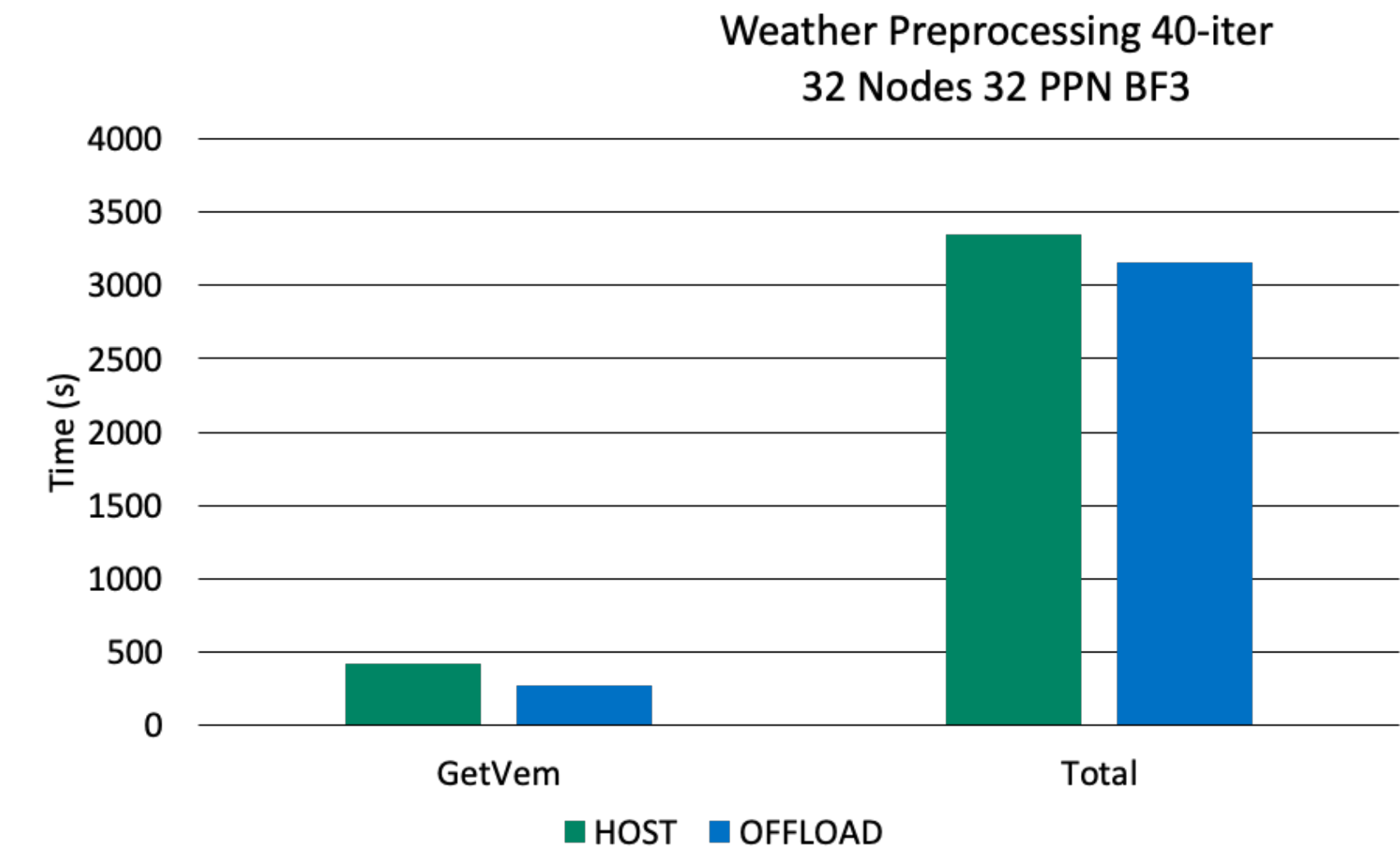
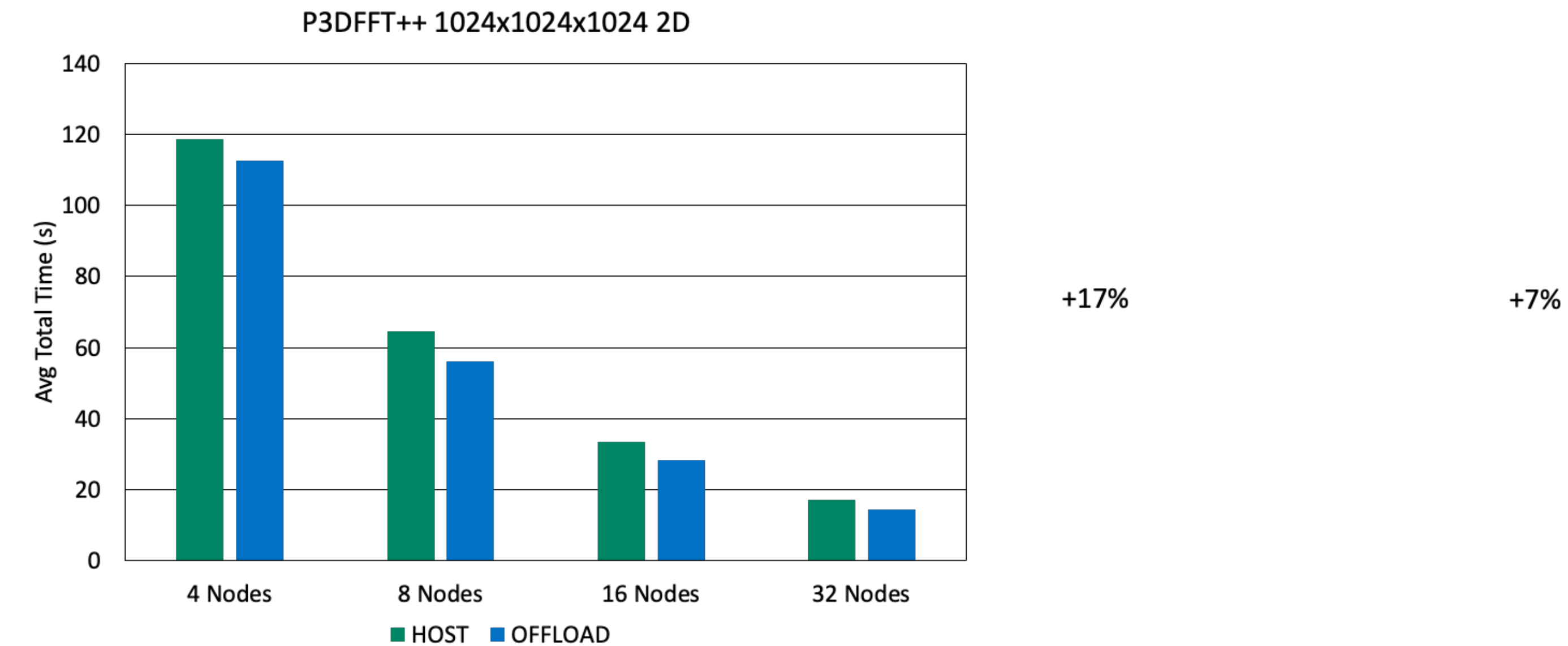
P3dFFT++ FFT Transform



Octopus Performance



BlueField-3 Application Acceleration





NVIDIA Data Path Accelerator (DPA)

SOME DPA USES

Customer Use Cases

- Custom device emulation (for either bare metal or VMs)
- Programmable Congestion Control
- Custom transport protocols
- SDN acceleration (connections-per-second boost)
- Custom flow actions (connection tracking, load balancer)

Out-of-the-Box DPA Based Usages

Examples of functionality implemented by NVIDIA

- Virtio-net
- Virtio-blk, Virtio-fs,
NVMe emulation,
NVMeoF-target-offload
- Congestion Control (CC)



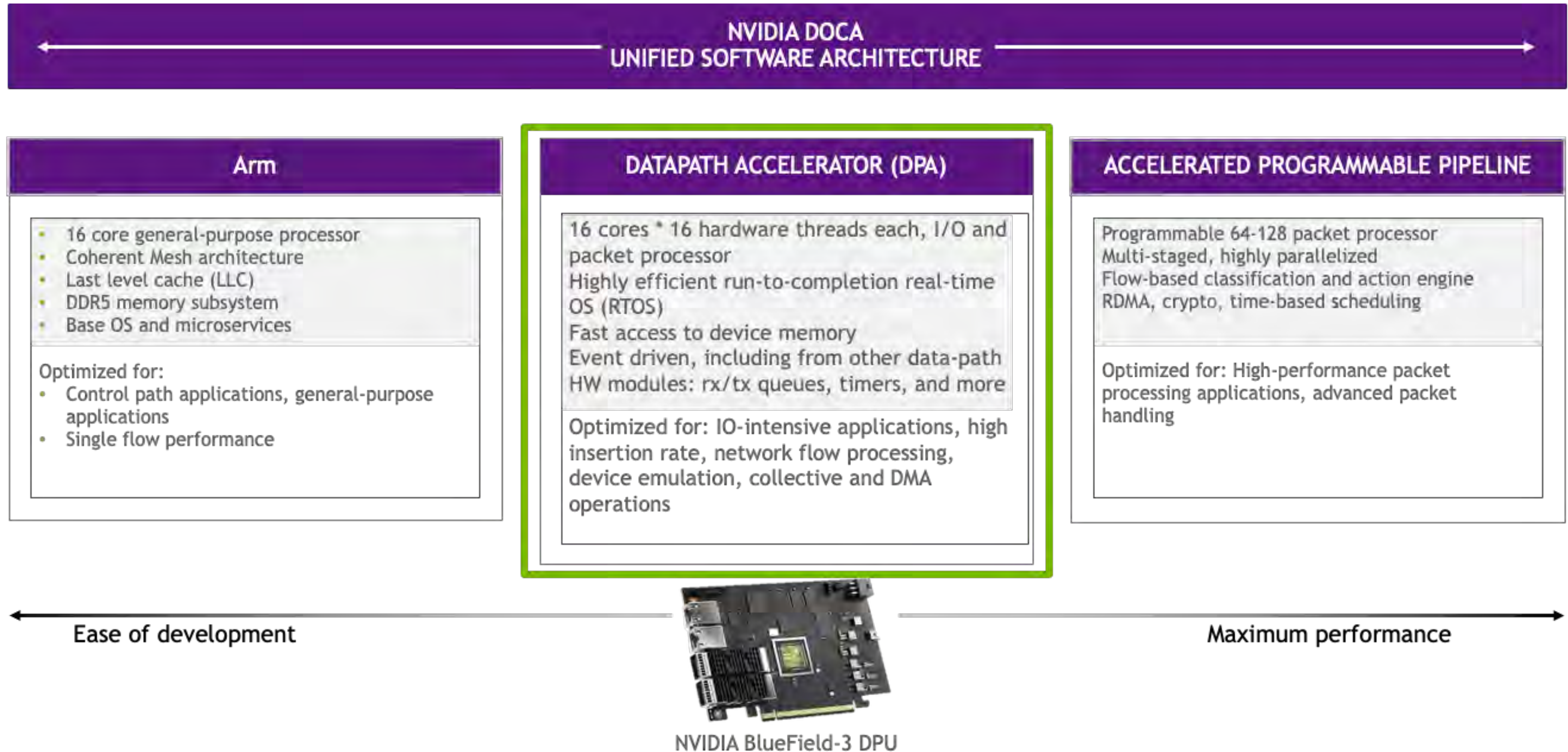
DESIGNED FOR PROGRAMMABILITY

- ▶ **Programming SDKs** - NVIDIA offers the following SDK layers to enable DPA programming:
 - ▶ DOCA-DPA SDK - for higher level abstraction - Customer's main programming tool
 - ▶ The SDK exposes DPA at the DOCA level and provides building blocks for Comms-SDK and PCC SDK
 - ▶ DPA process creation, thread creation / affinity, memory allocation, completion contexts
 - ▶ DOCA-DPA-Comms SDK - for higher level abstraction - Customer's main programming tool
 - ▶ The SDK implements the DPA data plane of DOCA libraries like DOCA-RDMA
 - ▶ DOCA-PCC SDK - specifically developed to enable PCC development
 - ▶ DOCA-FlexIO SDK - for low level I/O programmability (open-sourced for host code-development)
 - ▶ The SDK provides all the remaining functionality not offered by higher-level SDKs
- ▶ **Programming guide** with detailed APIs, explanations, and examples is available
- ▶ **Code examples** are shared as a part of the DOCA delivery
- ▶ **Toolchain** - code is compiled/built/uploaded as a part of the overall host/BlueField Arm image
- ▶ **Debugging** made possible through logs/printf and GDB-server
- ▶ **Perf analysis** supported by our supplied PS and Statistics tools

DEBUG AND PERF-ANALYSIS FOCUS

- ▶ **Tooling to support your debug and optimization efforts**
 - ▶ The DPA is a complex system, we offer multiple tools to help you get the best out of it
- ▶ **Logs** - the basic 'printf'-like functionality to gain visibility
- ▶ **GDB server** - enabling the connection of an external debugger
 - ▶ Including also dump-file analysis
- ▶ **PS** - monitor running DPA processes and threads
- ▶ **Statistics** - collect performance statistics for the running DPA processes and threads
- ▶ **For more details** - [NVIDIA DPA Tools - NVIDIA Docs](#)

DPA – HIGH PERFORMANCE AND FLEXIBLE DATAPATH



Data Path Accelerator (DPA)

RV64IMAC(B) + NVIDIA extensions (e.g. arithmetic ops)
Part of the Connect-X network core

High BW Dedicated local cluster cache

Access to Host system memory

NIC direct access to DPA caches

NIC interrupts and doorbells

Full access to accelerators

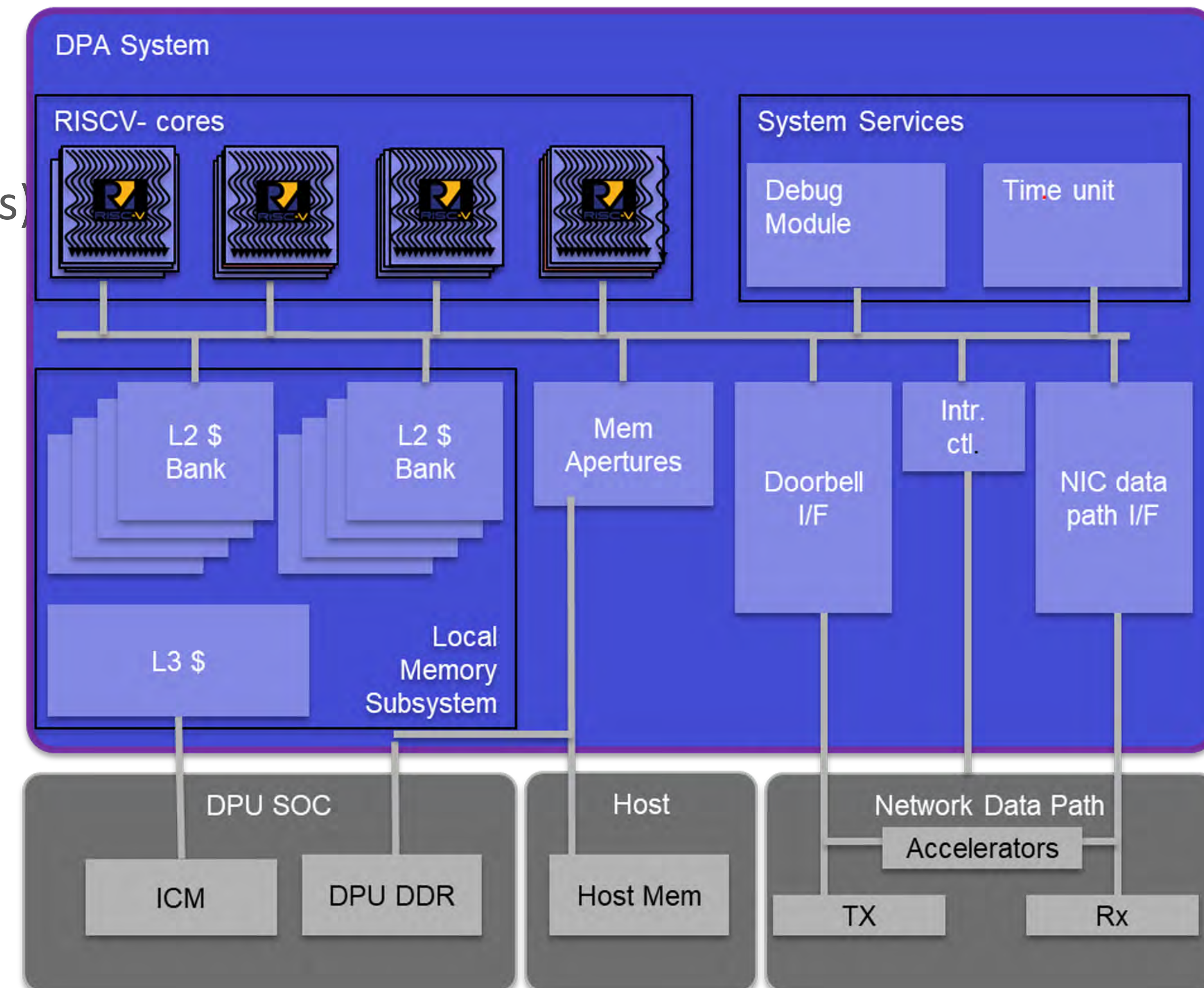
Standard Debug Module and timer unit

DPA-RTOS

Low memory footprint, highly threaded

Low latency scalable HW based scheduling

Process isolation enforced



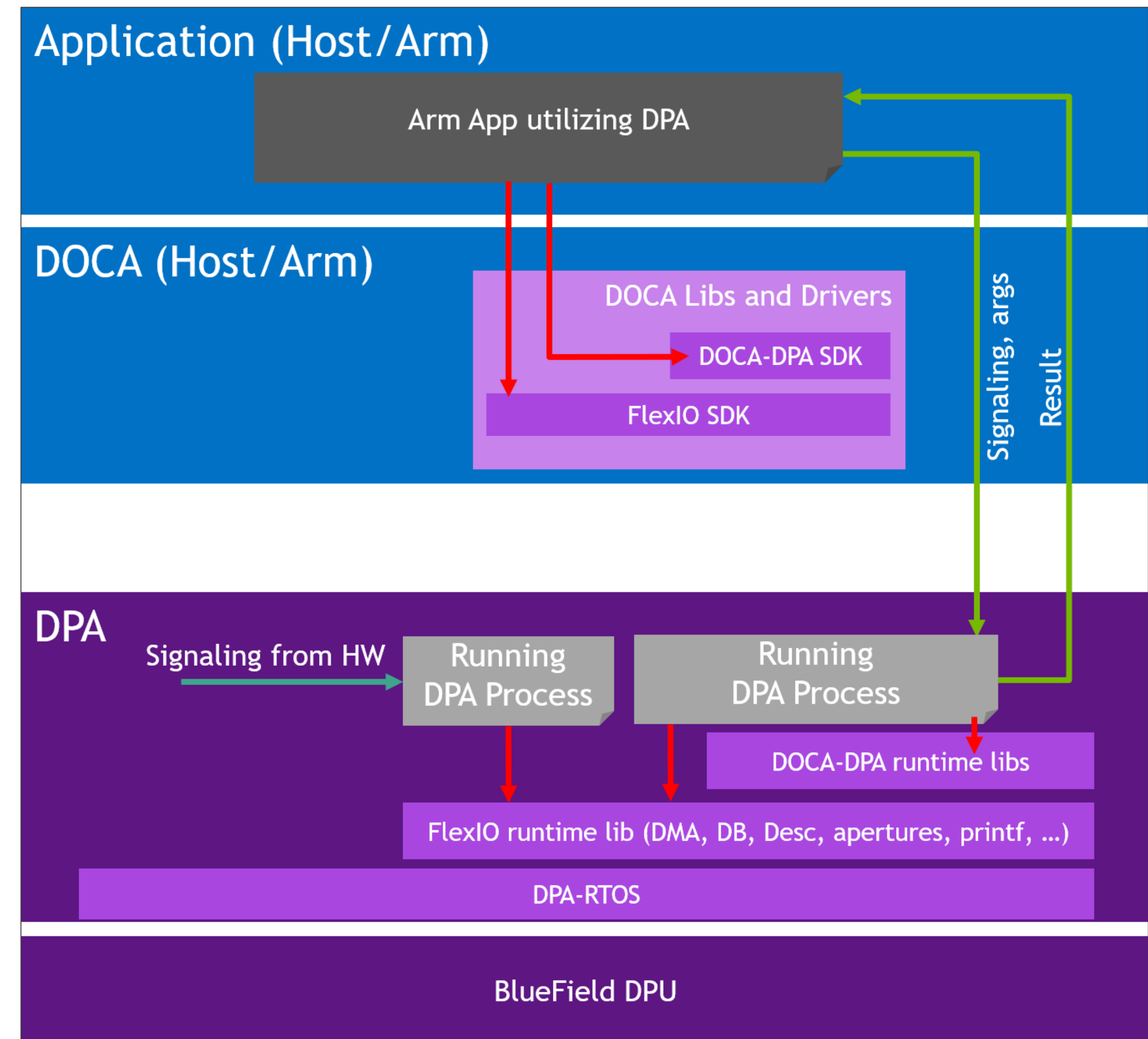
DPA highlights – Bluefield-3 (ConnectX-7 core)

DPA			
Architecture		Standard RISC-V RV64IMAC(B)-USM	
Pipeline Width		1 Instruction / Clock	
Threads per Core		16	
Number of Cores		16	
Frequency Target		Core 1.8 GHz / System 505 MHz	
Caches		Capacity	Access / Clock
	L1 Code / Core	8 KB (2-4K inst.)	1 (per Core)
	L1 Data / Thread	1 KB	1 (per Core)
	L2 / Cluster	1.5 MB	8
	L3 / Cluster	3 MB	2
	ICMC / NIC	4 MB	2

DPAs – More Details

- DPAs have a limited amount of memory
- DPA threads cannot run for more than a few seconds
- Custom micro-kernel
- No libc
 - No dynamic memory allocation from device
 - No standard mechanisms that most users rely on

- FlexIO is the lowest-level library for programming DPAs
 - Processes on the host are responsible for allocating device memory
 - Processes on the host start kernels (via DOCA FlexIO)
- It is possible to activate DPA threads on demand



DPA Execution Model

Collective execution initiated on host (MPI ranks) but executed on the DPAs

Collective algorithms consist of control messages and data transfers

- Control messages: coordination between ranks running on the hosts and DPA threads required by the algorithms
- Data transfer: move the application's data

Short-lived DPA threads

- Send & receive completions trigger the execution of a thread on the DPA
- Rely on persistent data accessible on/from the DPA to track & manage the state of the local collective operations
- When messages generated by the implementation of the collective algorithms complete, the local memory is updated to track the state of the said collective
- When the collective completes, a completion is generated for all the ranks

Programming DPAs

Two software packages are available

FlexIO: low-level library supporting both the host and device programming

<https://docs.nvidia.com/doca/api/3.0.0/flexio-sdk-api/index.html>

DOCA DPA: based on FlexIO but provides useful abstractions for the implementation of new collective algorithms

<https://docs.nvidia.com/doca/sdk/doca+dpa/index.html>

Similar to GPU programming in some ways:

- A kernel is started on the GPU

- The kernel code must be static (e.g., no dynamic memory allocation)

- The kernel code is “compiled in” the host-code using the dpacc compiler and linking all binaries together

- FlexIO handles the bootstrapping and execution of kernel codes

DPA specificities

- Kernels can be implemented to activate threads only upon completion of network operations

- The limited amount of memory imposes limitations on designing collective algorithms

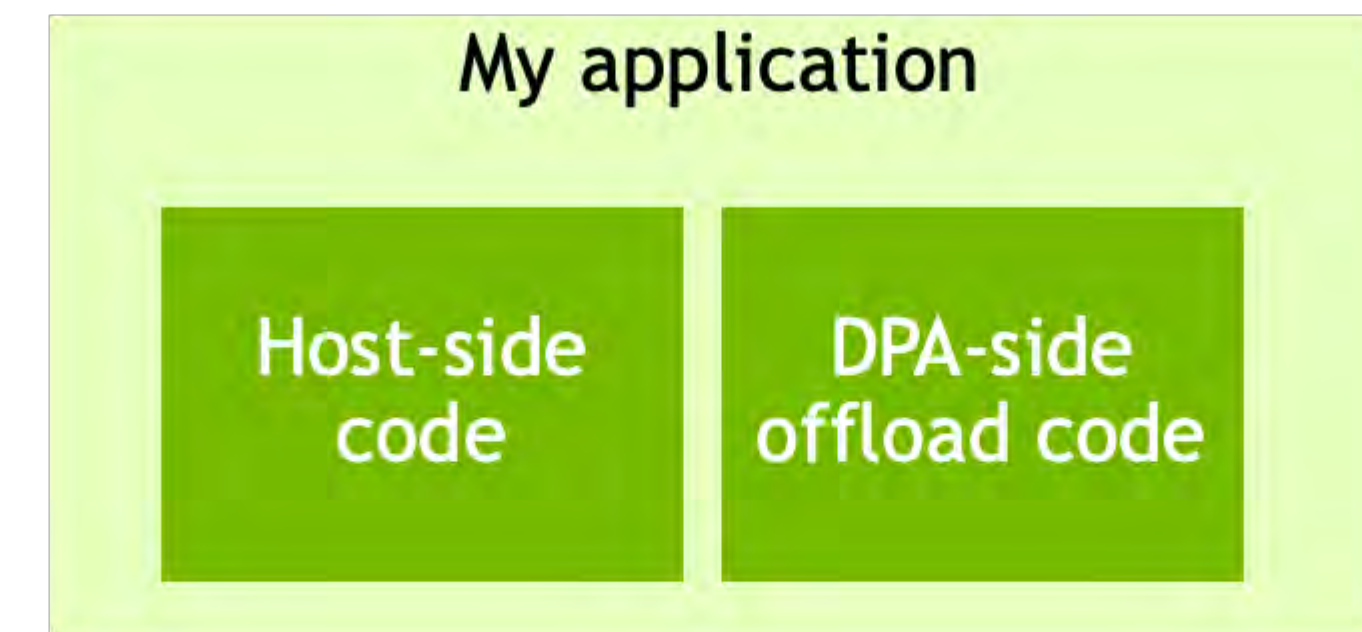


Figure 1: illustration of software DPA offloading



Figure 2: example of how the offload code is compiled and packaged in a single binary



HPC/AI Programming – OpenMP Offload



OpenMP Offload to DPUs

Brought to you by the AccelCom Group at BSC
M. Usman, S. Iserte, **A. J. Peña** - accelcom@bsc.es



Outline

Nvidia BlueField DPU Programming

OpenMP DPU Offloading Support (ODOS)

- Demo #1

ODOS MPI Support

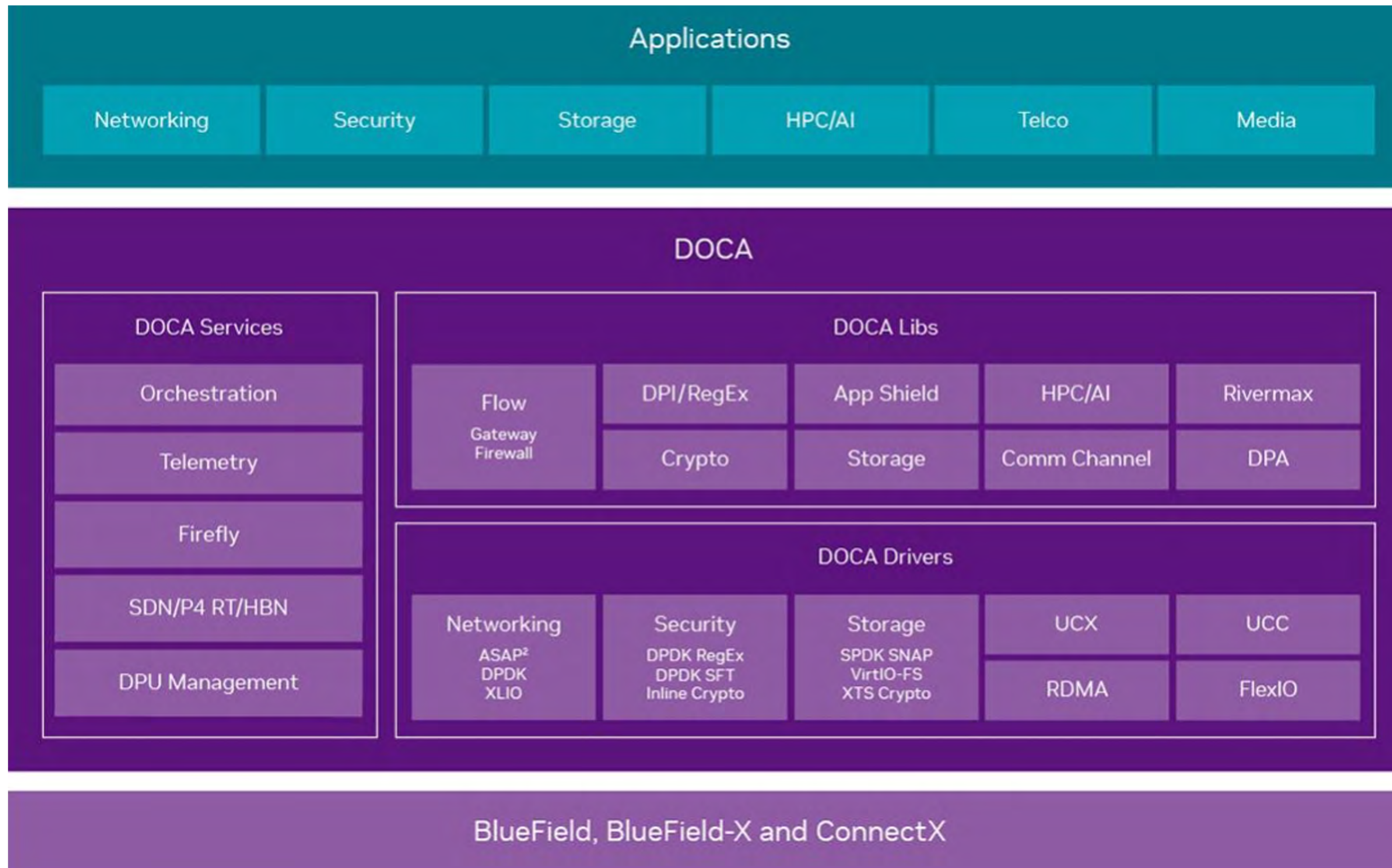
- Demo #2

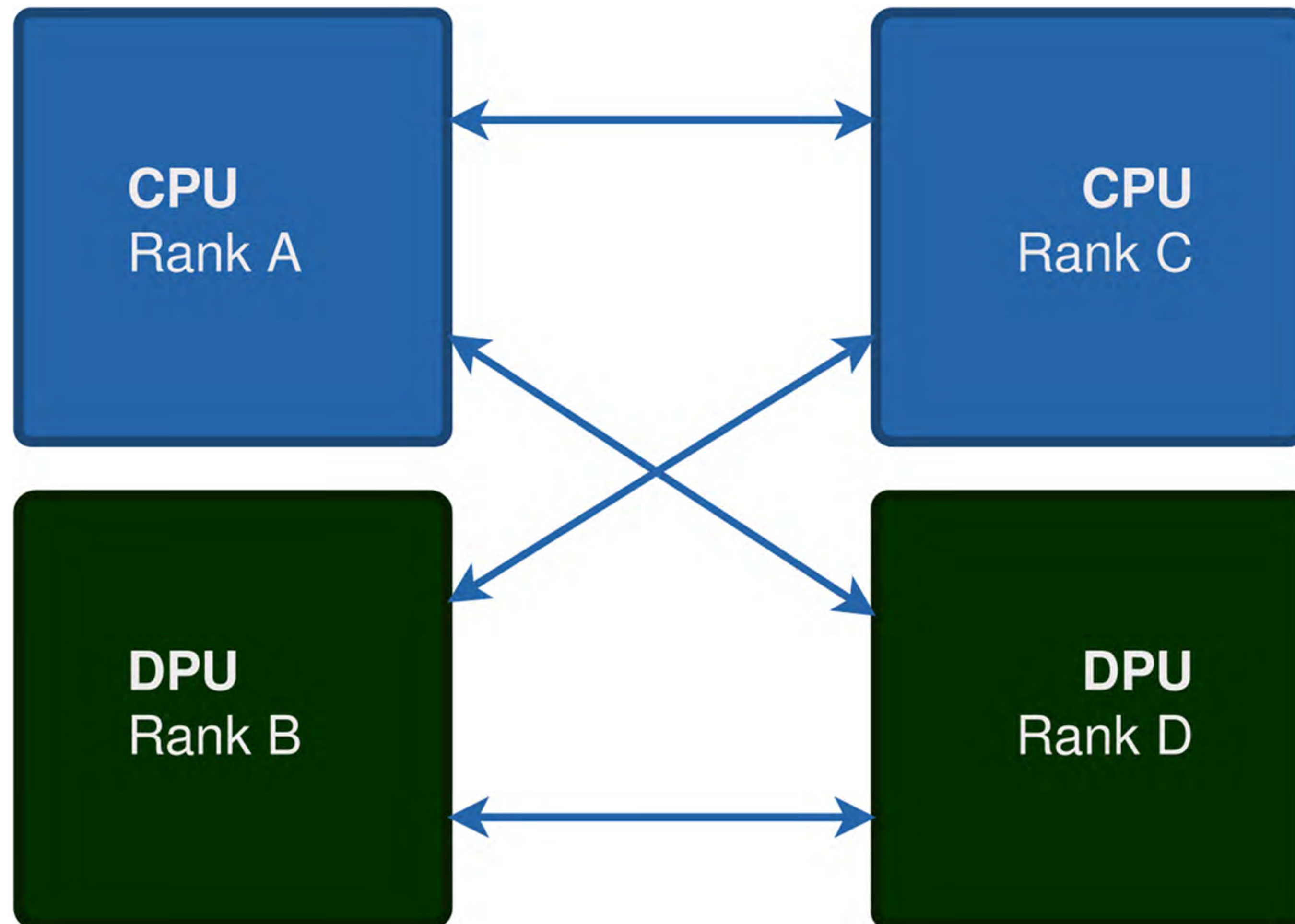
Use case: Halo Exchange

Performance

BlueField DPU Programming Low Level API

- DOCA is to DPUs what CUDA is to GPUs
- DOCA is not an offloading API
- Too low level for domain scientists





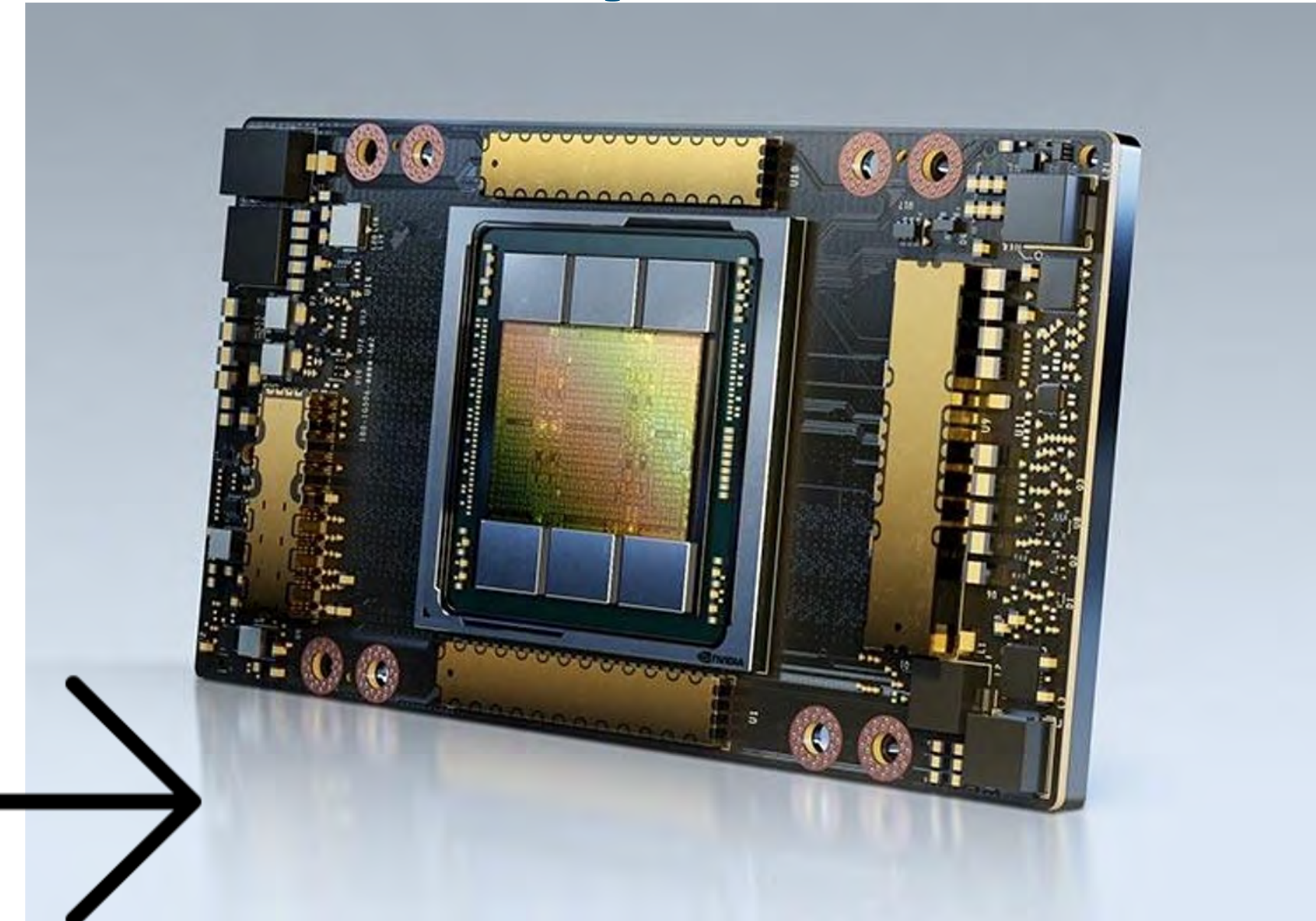
BlueField DPU MPMD Programming

- Multiple program multiple data execution model in MPI
- Poor productivity because the processors are widely different

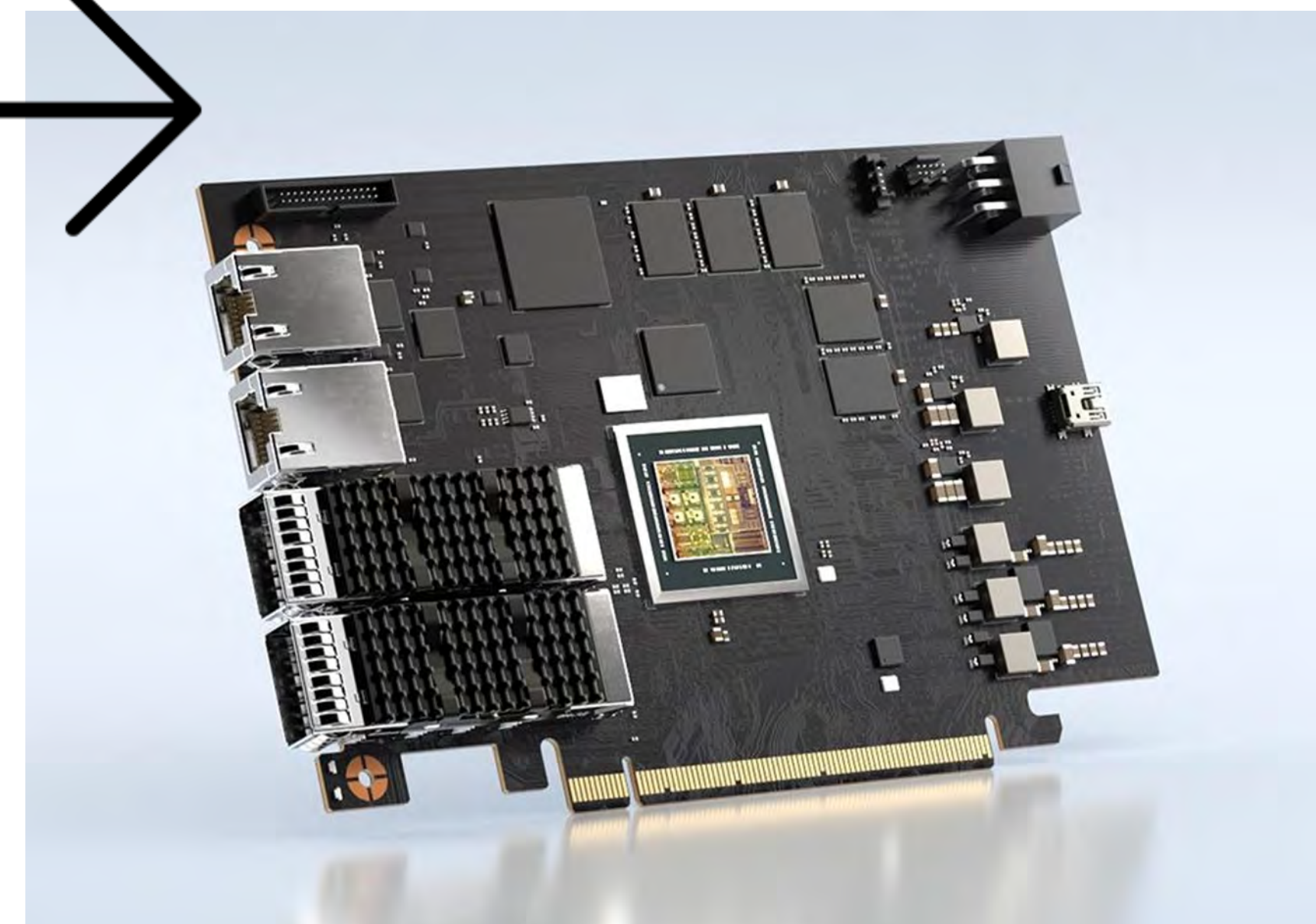
DPU Offloading Programming with standard OpenMP

```
int main ()  
{  
    /***/  
    /* exec in host */  
    /***/  
  
    #pragma omp target device ( )  
    {  
        /***/  
        /* exec in target */  
        /***/  
    }  
}
```

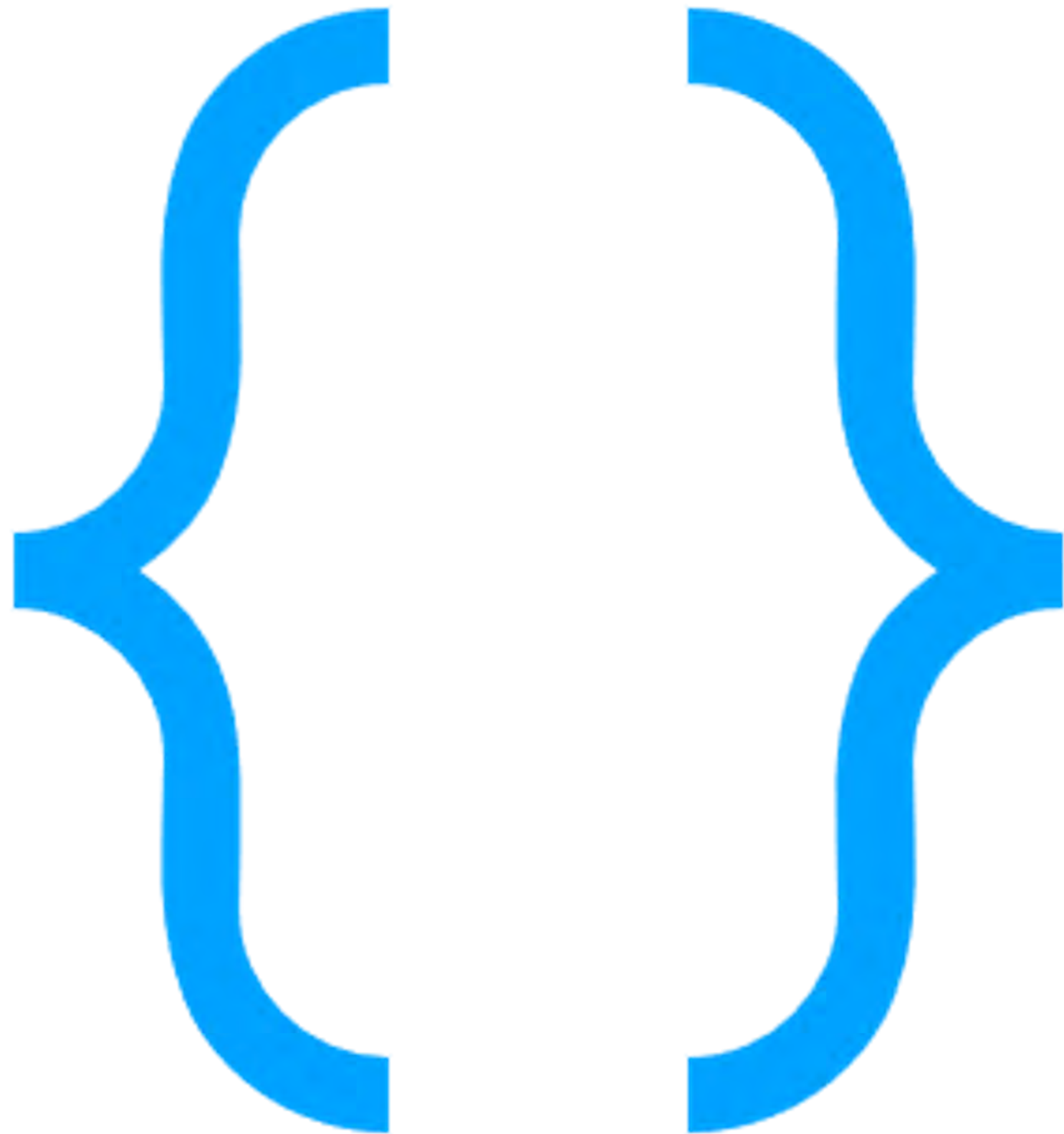
GPU



DPU



Basic Syntax



```
// exec in host
```

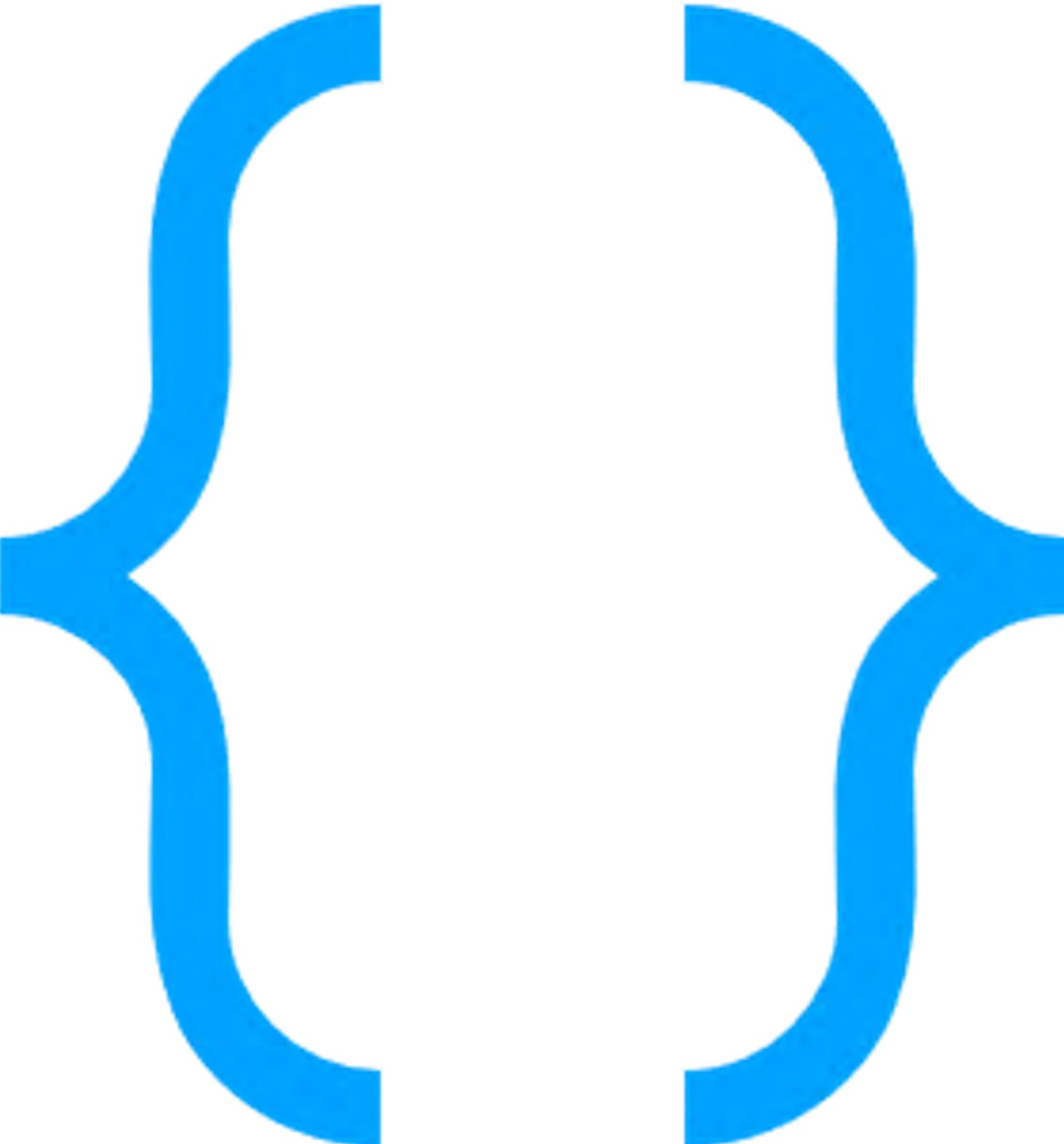
```
#pragma omp target
```

```
{
```

```
    // exec in target
```

```
}
```

How do I Know the Device Number?

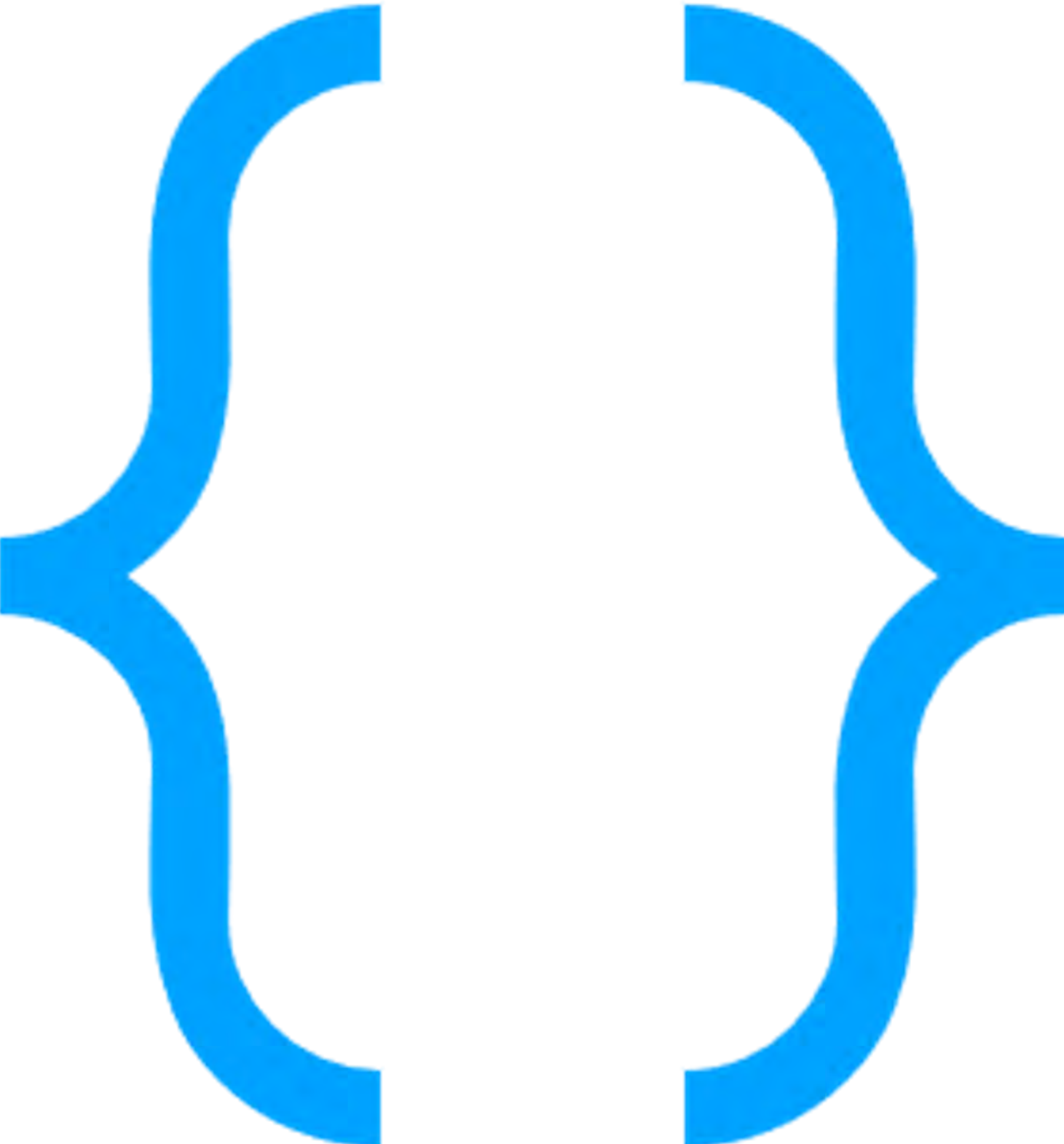


```
$ llvm-omp-device-info
```

```
Device (4):
```

```
  CUDA Driver Version:      10020
  CUDA Device Number:      0
  Device Name:              Tesla V100-SXM2-16GB
  Global Memory Size:      16911433728 bytes
  Number of Multiprocessors: 80
  Concurrent Copy and Execution: Yes
  Total Constant Memory:    65536 bytes
  Max Shared Memory per Block: 49152 bytes
  Registers per Block:      65536
  ...
  Compute Capabilities:     70
```


How do I Know the Device Number?



```
$ llvm-omp-device-info
```

```
...
```

```
Device (7):
```

```
BlueField DPU device
```

```
iface      : ib0
```

```
ib dev     : mlx5_2
```

```
doca id    : 2
```

```
pci addr   : 42:00:0
```

```
comm channel:
```

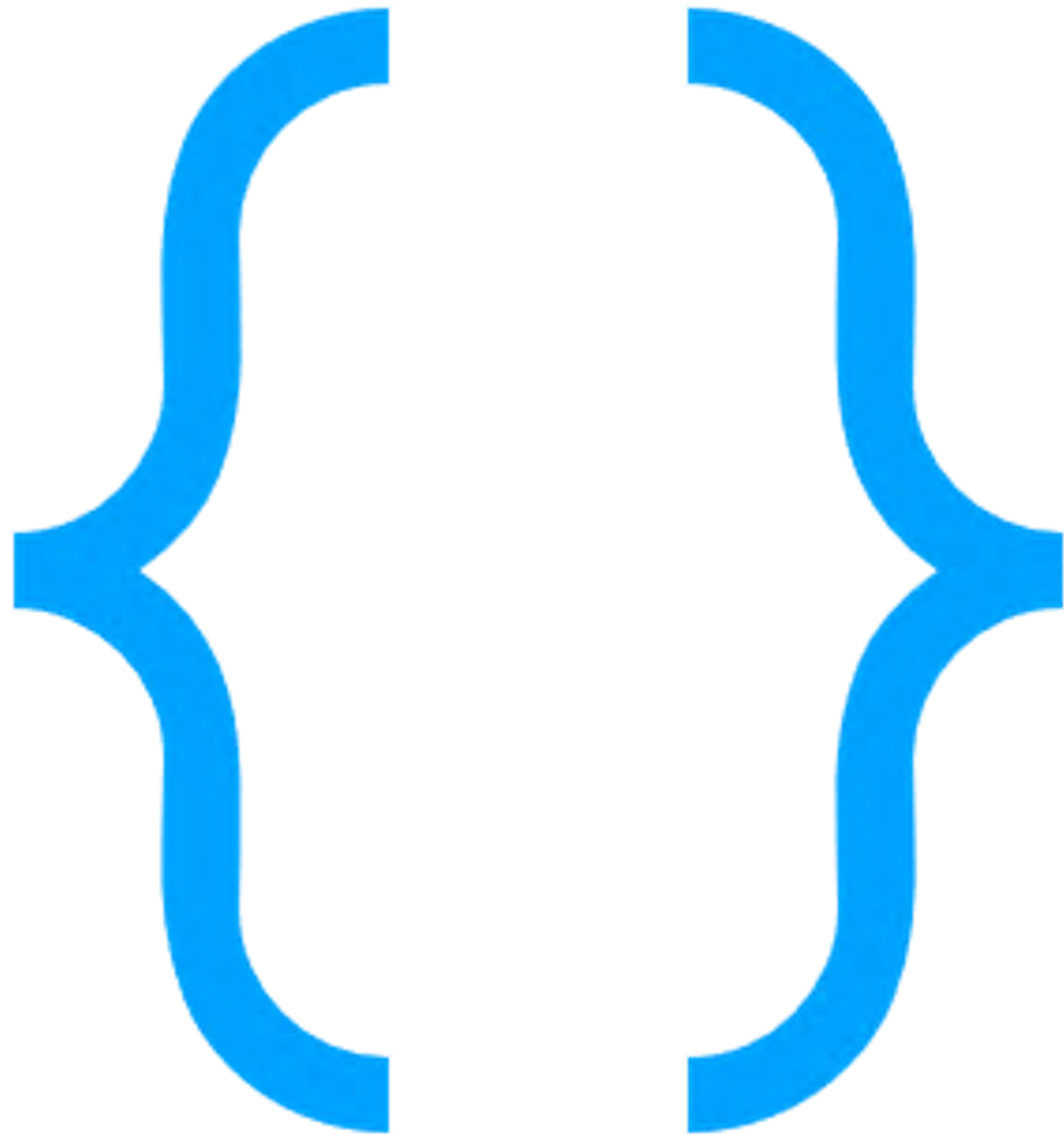
```
max msg size      : 4080
```

```
max send queue size : 8192
```

```
max receive queue size : 8192
```

```
...
```

How to Choose a DPU?



```
// exec in host  
#define __DPUID__ 7  
#pragma omp target device(__DPUID__)  
{  
    // exec in target  
}
```



```

void main( int argc, char* argv[] ) {
    // Data initialization
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &dpurank );
    if ( dpurank ) // DPU rank receives data
        MPI_Recv( &buf, SIZE, MPI_INT, 0, ... );
    else // Host rank sends data
        MPI_Send( &buf, SIZE, MPI_INT, dpurank, ... );
    for( int t = 1; t <= TIMESTEPS; t++ ) {
        if ( dpurank ); // Offloaded compute
        else; // Host compute
    }
    if ( dpurank ) // DPU rank send data
        MPI_Send( &buf, SIZE, MPI_INT, 0, ... );
    else // Host rank receive data
        MPI_Recv( &buf, SIZE, MPI_INT, dpurank, ... );
    MPI_Finalize();
}

```

```

void main( int argc, char* argv[] ) {
    // Data initialization
    #pragma omp target
        data map(tofrom:buf[0:SIZE]))
    {
        for( int t = 1; t <= TIMESTEPS; t++ ) {
            #pragma omp target
            {
                // Offloaded compute
            }
            // Host compute
        }
    }
}

```

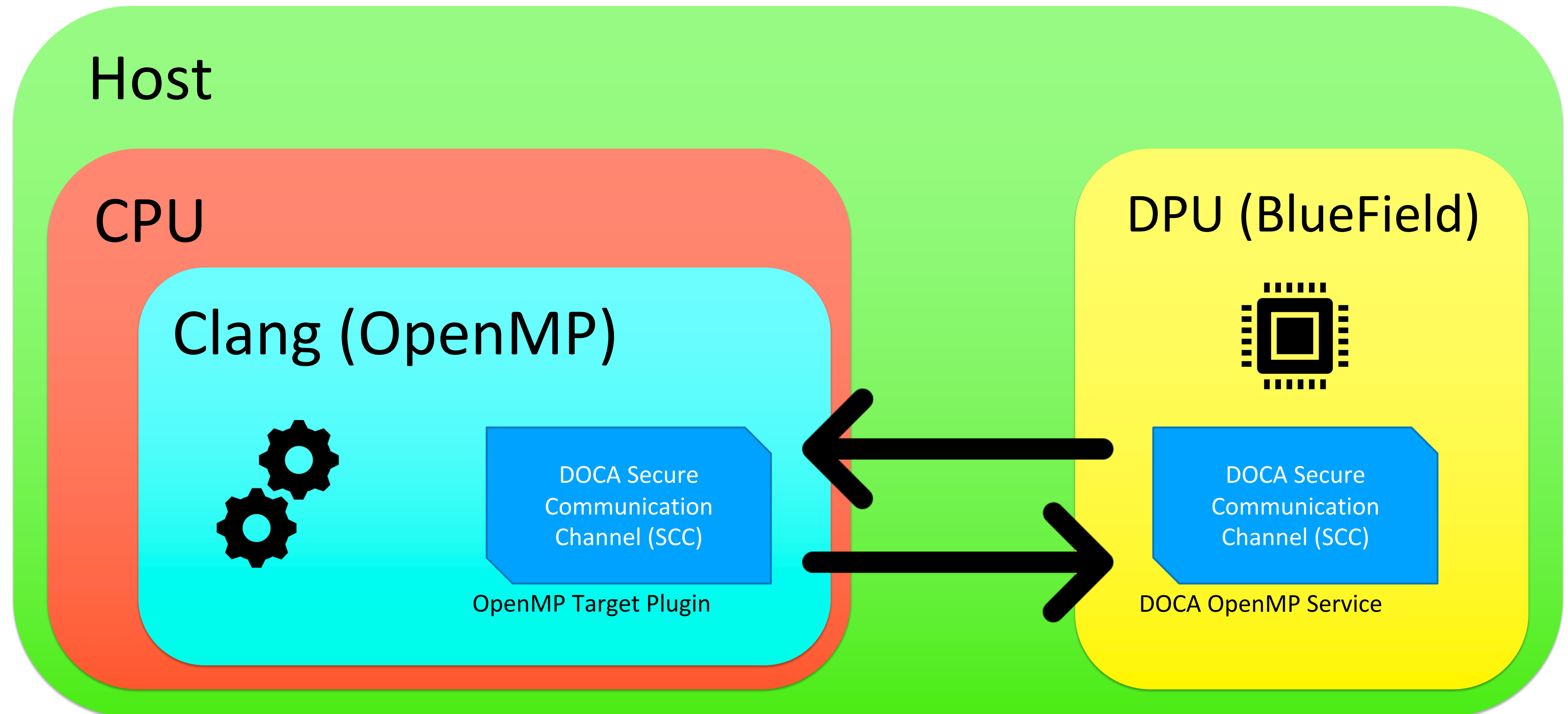




OpenMP DPU Offloading Support



ODOS Architecture



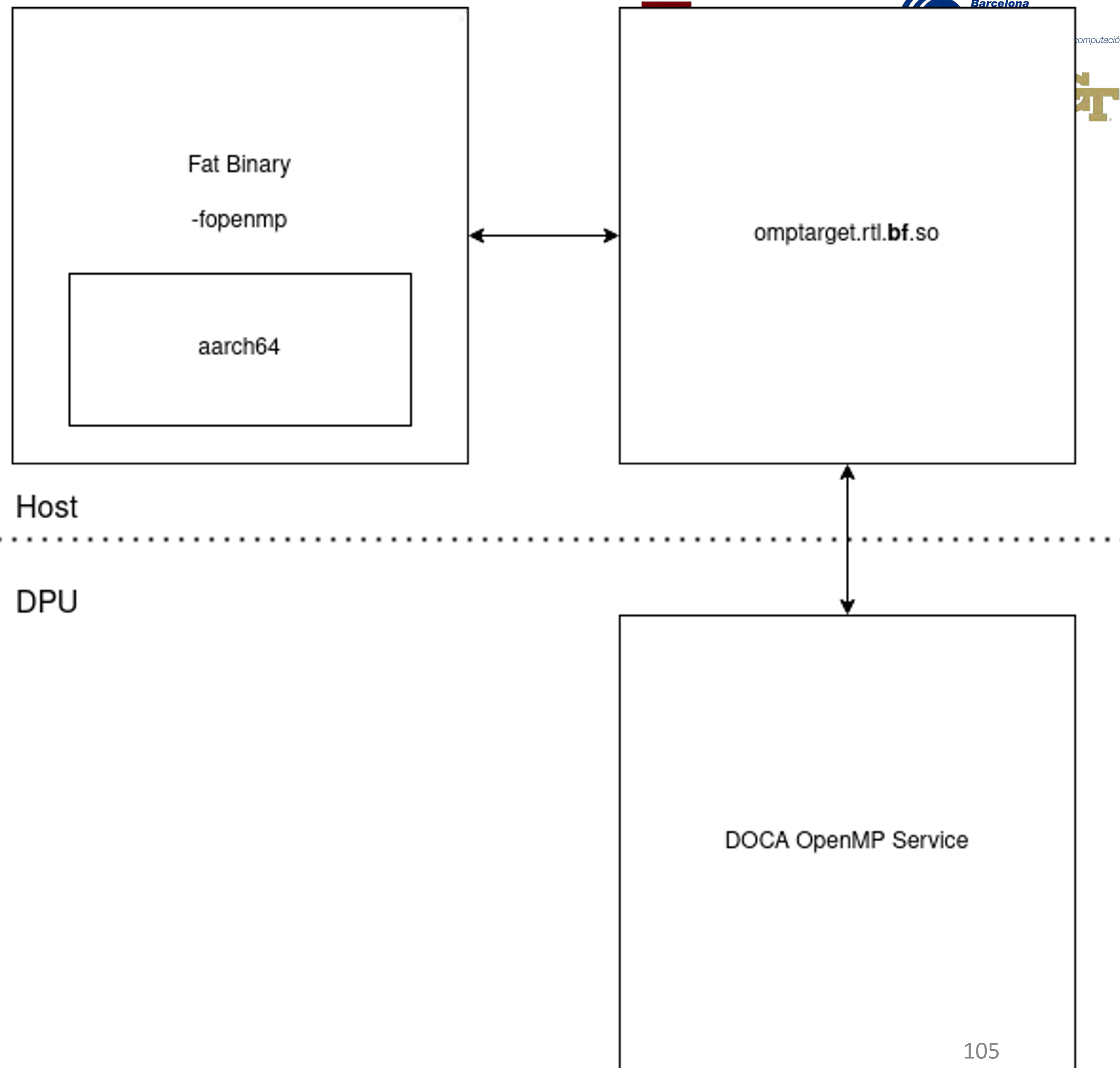
M. Usman, S. Iserte, R. Ferrer, and A. J. Peña, "DPU Offloading Programming with the OpenMP API." *LLVM-HPC23 co-located with SC23*, <https://dl.acm.org/doi/10.1145/3624062.3624165>

ODOS Modules

Fat Binary Generation

OpenMP Runtime (host)

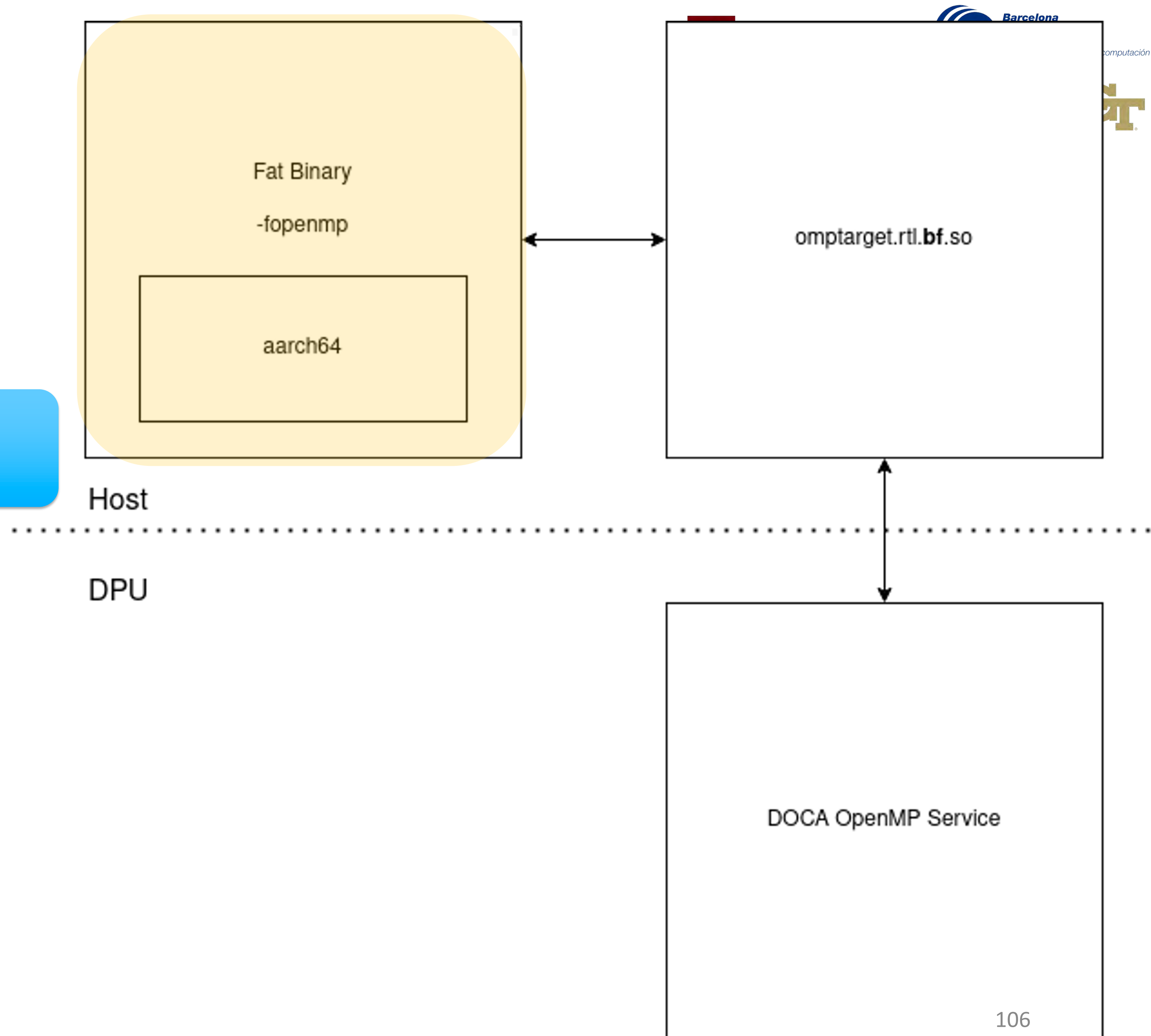
OpenMP Service (device)



Fat Binary Generation

Cross-compilation

- I.e., x86-64 & aarch64
- Default Linux GNU GCC compiler

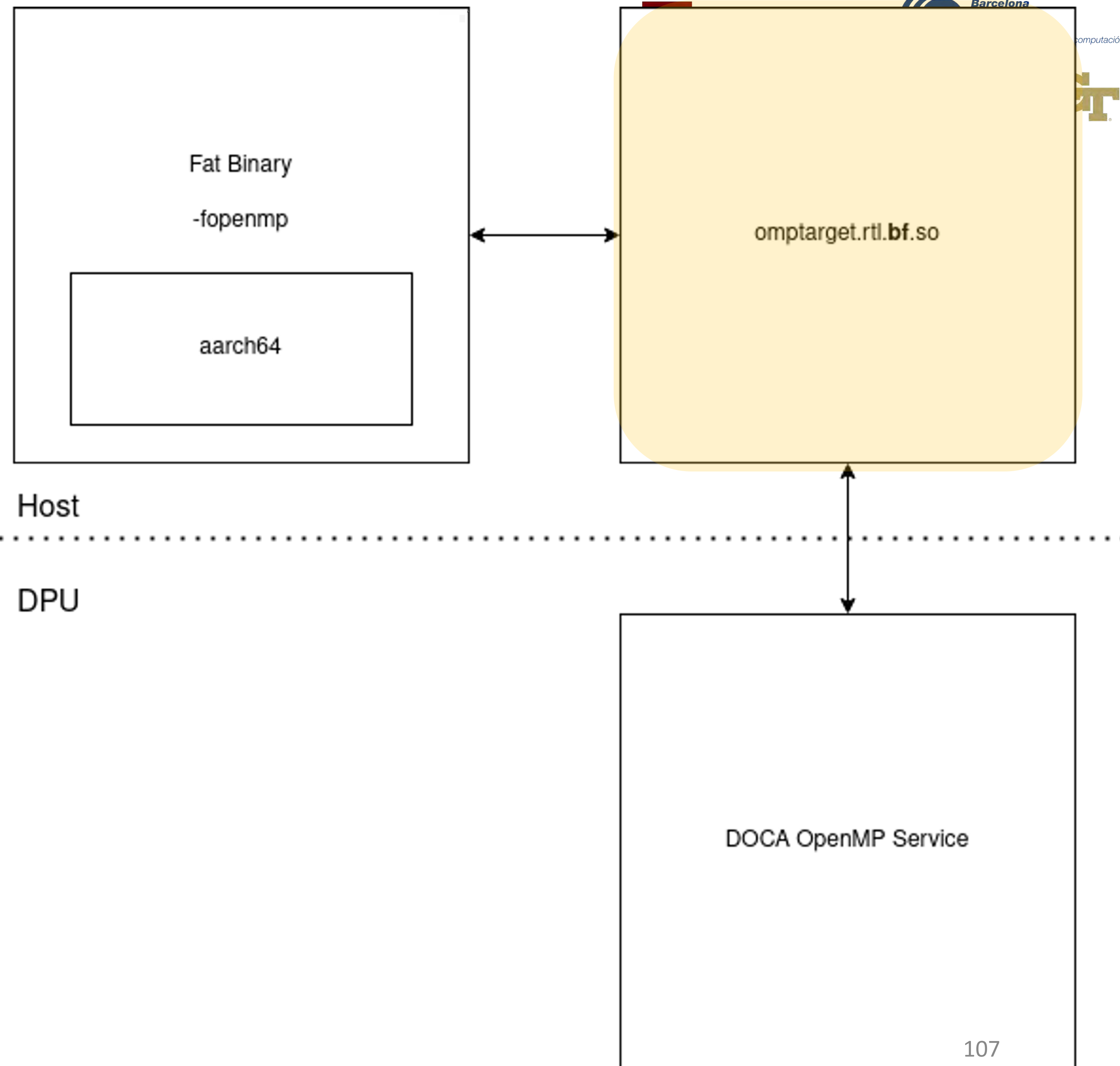


Target Plugin

Communication with the DPU using DOCA SCC

Send commands to

- Load binary
- Allocate and exchange data
- Execute target blocks



DOCA OpenMP Service

Receive commands using DOCA SCC

Load binary

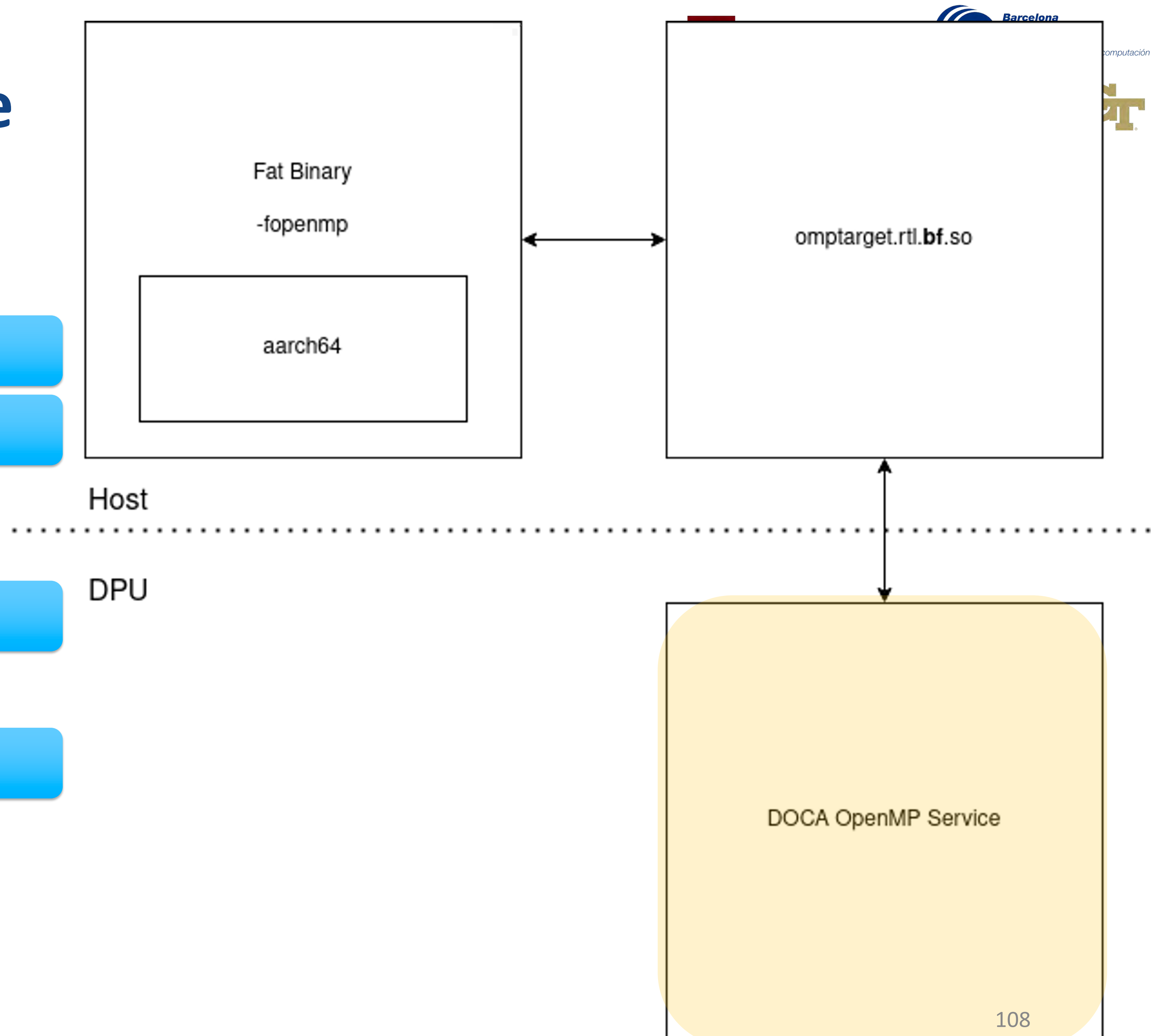
- Save image as a shared object
- Open the image [dlopen]
- Send back handles of symbols to host [dlsym]

Data operations

- Allocate/Delete [malloc/free]
- Send/Retrieve [DOCA SCC]

Target block execution

- Run target handle [ffi_prep_cif, ffi_call]





DEMO #1

- How to compile ODOS-enabled applications
- Basic example
- OpenMP "Parallel" feature
- Shared libraries
- Asynchronous TCP

How to Compile ODOS-enabled Applications

ODOS-enabled LLVM

Enable OpenMP and
specify offloading targets

```
$ clang -fopenmp -fopenmp-targets=aarch64-unknown-linux app.c -o app
```

```
#include <omp.h>
#include <stdio.h>

int main()
{
    #pragma omp target
    puts("Hi Folks!");

    return 0;
}
```

Host

uthmanhere@thor013:~/omp_exp/labs_sc23/task_a/build\$

DPU

uthmanhere@thorbf3a013:~\$


```
#include <omp.h>
#include <stdio.h>
```

```
int main()
{
    #pragma omp target
    #pragma omp parallel
        puts("Hey! OpenMP even work in DPU...");

    return 0;
}
```

Host

uthmanhere@thor013:~/omp_exp/labs_sc23/task_b/build\$

DPU

uthmanhere@thorbf3a013:~

Shared Libraries

```
$ #Compile shared object
```

```
$ LLVM/bin/clang log.c -shared -o liblog_x86.so
```

```
$ #Cross-compile shared object
```

```
$ LLVM/bin/clang log.c -shared -target aarch64-unknown-linux -o liblog_aarch64.so
```

```
$ #Link the objects and generate the fat binary
```

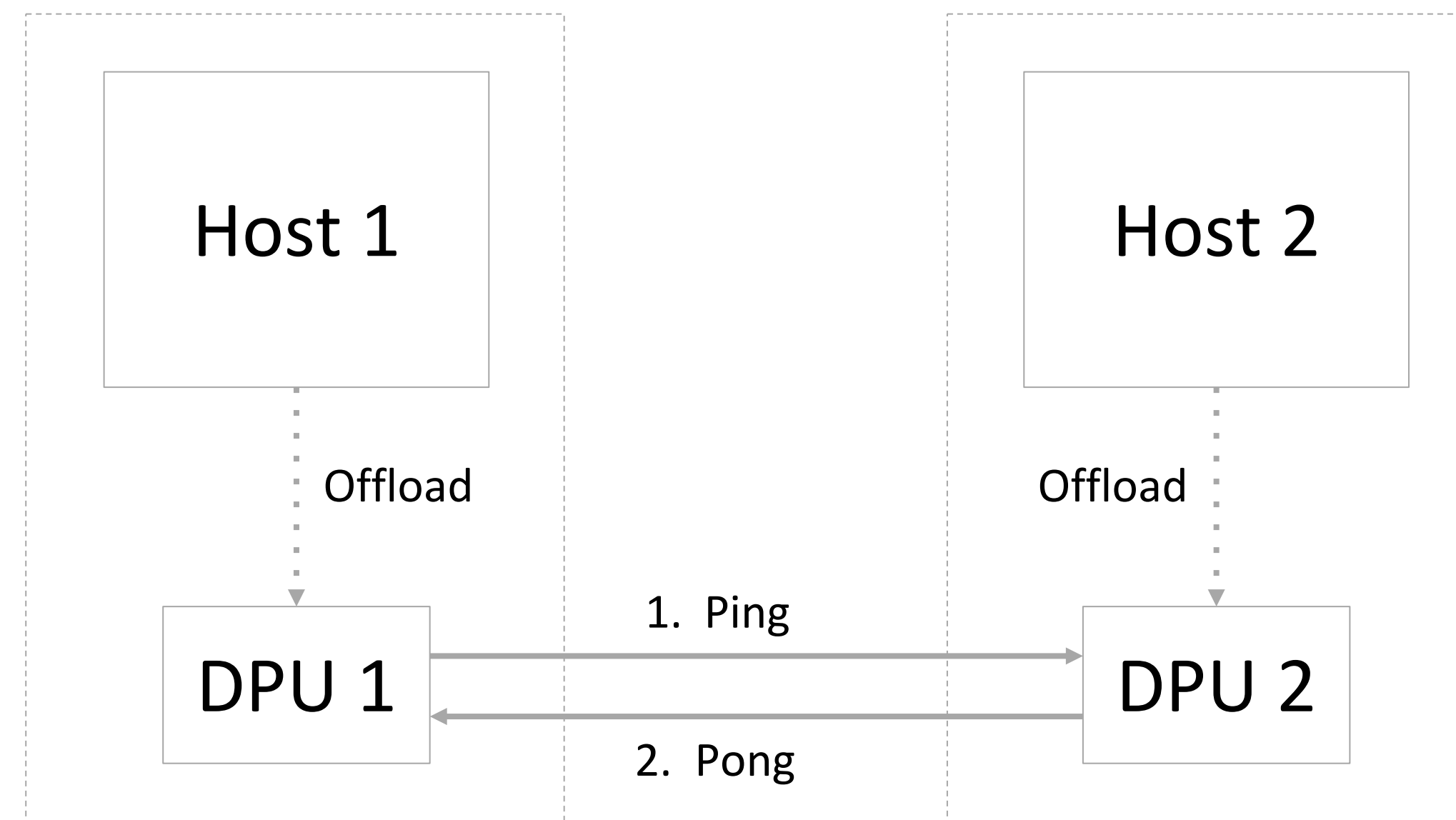
```
$ LLVM/bin/clang -fopenmp -fopenmp-targets=aarch64-unknown-linux code.c -o code -L.  
-llog_x86 -Wl,--device-linker=-L.,--device-linker=-llog_aarch64
```

Asynchronous TCP I/O in DPU using OpenMP (1/2)

```
#pragma omp target nowait
{
    // initialize tcp sockets, listen, bind, connect, and accept.
    fd = tcp_init(mode);

    // compute and communicate asynchronously on DPU
    ping_pong(fd, mode);

    close(fd);
}
#pragma omp taskwait
```



Asynchronous TCP I/O in DPU using OpenMP (2/2)

```
void ping_pong(int fd, char mode) {  
    int i, m = 0;  
    for (i = 0; i < _ITERATIONS_; ++i) {  
        if (mode == 'c') {  
            printf("to server > %02d\n", m);  
            send(fd, &m, sizeof(m), 0);  
            recv(fd, &m, sizeof(m), 0);  
            ++m;  
        } else {  
            recv(fd, &m, sizeof(m), 0);  
            ++m;  
            printf("to client > %02d\n", m);  
            send(fd, &m, sizeof(m), 0);  
        }  
    }  
}
```

```
uthmanhere@thor013:~/omp_exp/labs_sc23/task_d/build$  
uthmanhere@thor013:~/omp_exp/labs_sc23/task_d/build$
```

```
uthmanhere@thorbf3a013:~$ ./doca_openmp_service  
connection established.
```

1

Server (Pong)

```
uthmanhere@thor014:~/omp_exp/labs_sc23/task_d/build$  
uthmanhere@thor014:~/omp_exp/labs_sc23/task_d/build$
```

```
uthmanhere@thorbf3a014:~$ ./doca_openmp_service  
connection established.
```

2

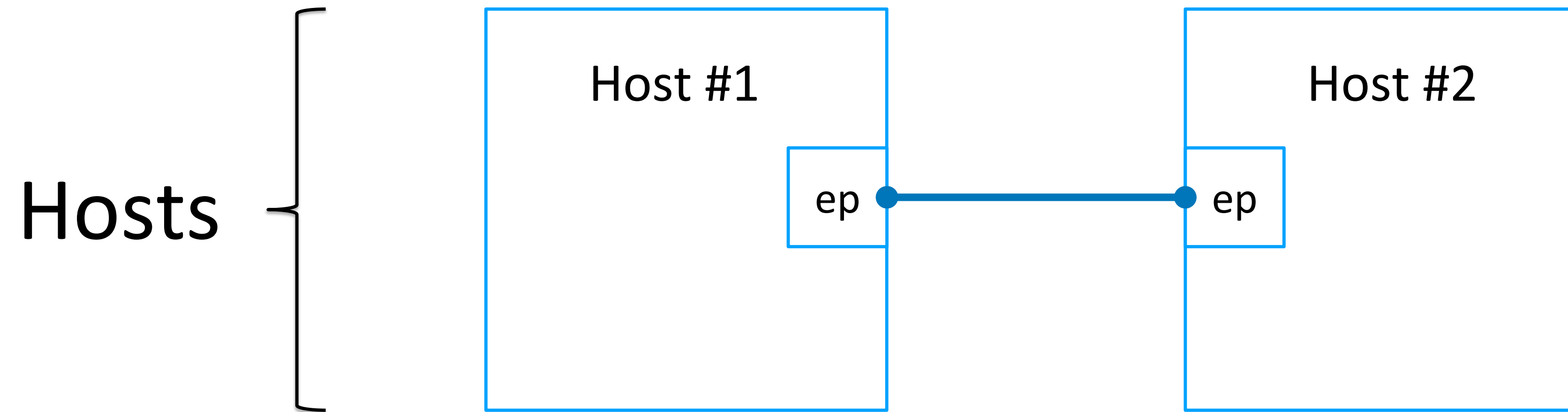
Client (Ping)



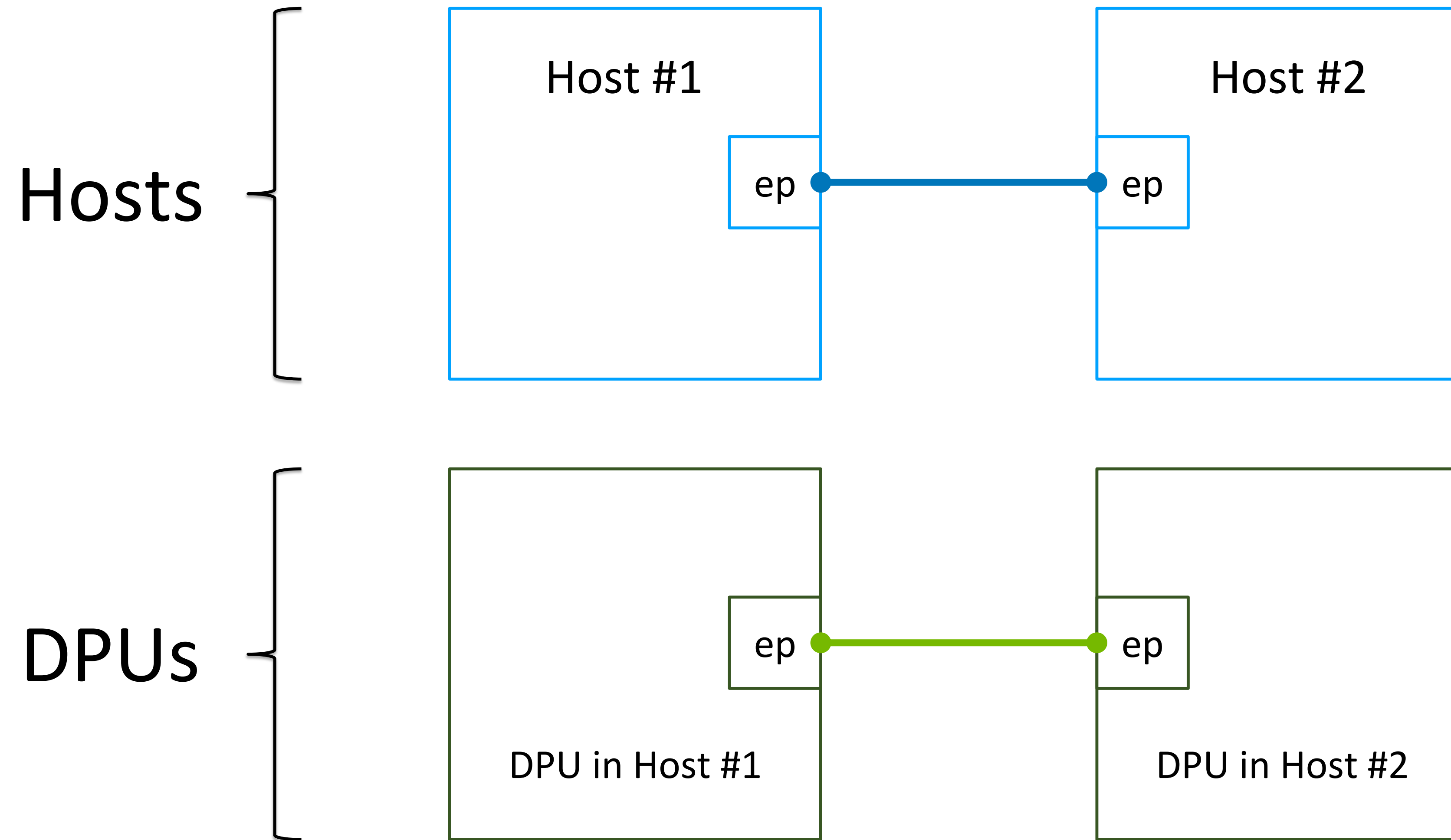
ODOS MPI

M. Usman, M. Benito, S. Iserte, and A. J. Peña, "ODOS-MPI: HPC-Friendly SmartNIC Offloading of Computation/Communication Kernels", in *SC25*

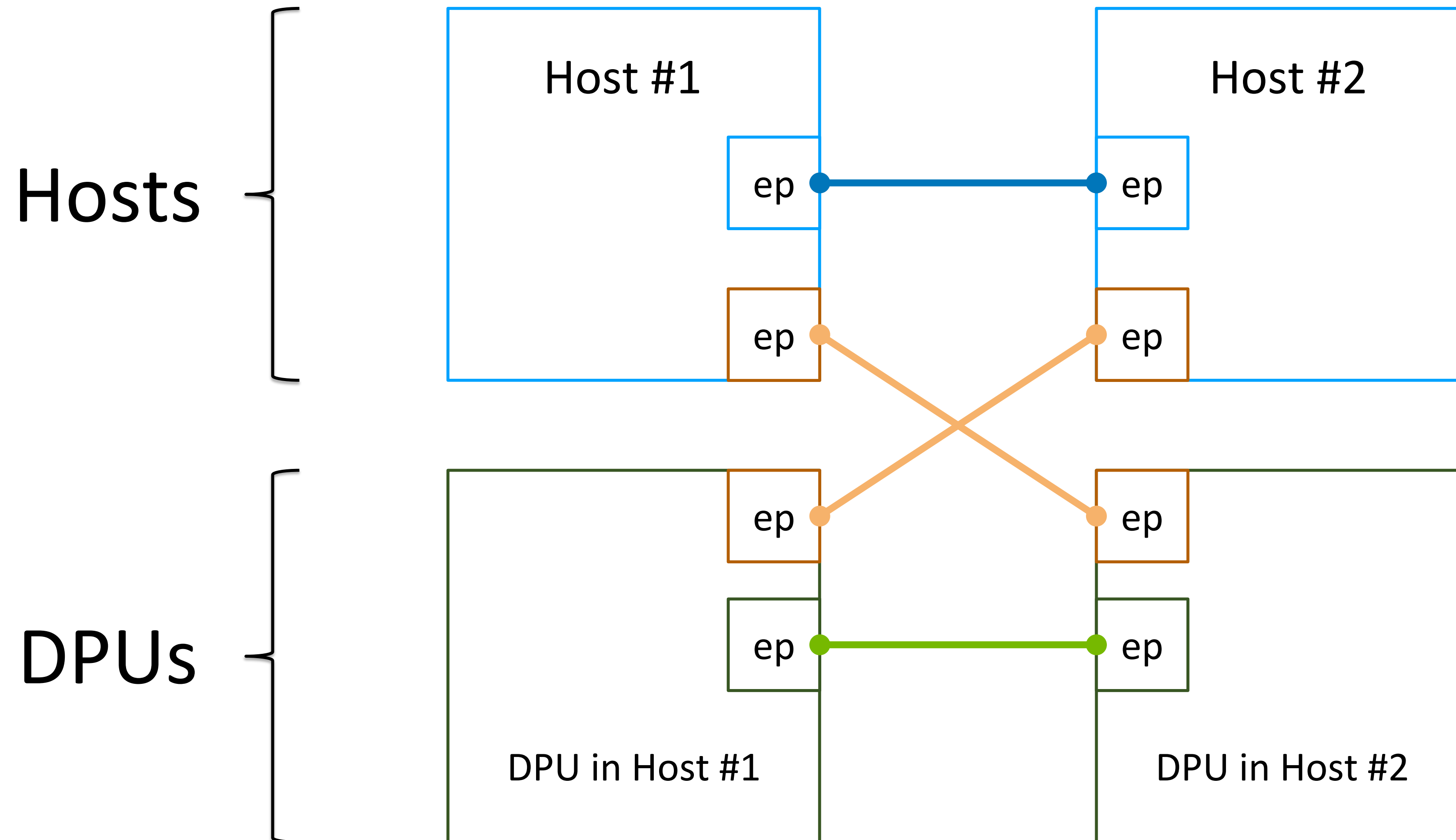
End-Points Links (1/3)



End-Points Links (2/3)



End-Points Links (3/3)



DPU as a local co-processor

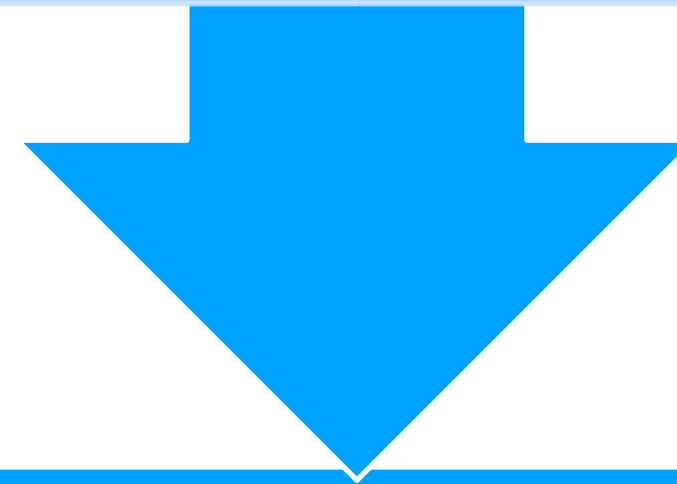
- DPUs don't have their own MPI rank number 😊

Open MPI for ODOs

Initialization

PML UCX INIT

Create endpoints



Communicate

PML UCX SEND

Send over endpoint



DEMO #2

- MPI P2P
- MPI RMA

- **2 Nodes**
- **1 rank per node**
- **Ranks can execute commands in the host and in the DPU**

MPI P2P DPU-to-DPU

```
#pragma omp target
{
    if ( rank == 0 ) {
        MPI_Send( &buf, 1, MPI_INT, dst, 0, MPI_COMM_WORLD );
    } else {
        MPI_Recv( &recv_data, 1, MPI_INT, src, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE );
    }
}
```


MPI P2P DPU-to-Host

```
if ( rank == 0 ) {  
    #pragma omp target  
    {  
        MPI_Info info;  
        MPI_Info_create( &info );  
        MPI_Info_set( info, "REMOTE_DEST_HETERO", "1" );  
        MPI_Comm_set_info( MPI_COMM_WORLD, info );  
        MPI_Send( &buf, 1, MPI_INT, dst, 0, MPI_COMM_WORLD );  
        MPI_Info_delete( info, "REMOTE_DEST_HETERO" );  
        MPI_Info_free( &info );  
    }  
} else {  
    MPI_Recv( &recv_data, 1, MPI_INT, src, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE );  
}
```

MPI P2P Example

Host to
host:
"3310"

Device to
device:
"3311"

Device to
host:
"3312"

Host to
device:
"3313"

```

athens@harphe11a0802:~$ pwd /bin/mpirun -n 2 -p 2 \
--mca hellos002,hellos012 \
--mca OCK_LOG_LEVEL=ERROR \
--mca /app_pt2pt
client mode
client mode
going to connect...
going to connect...
connected
connected
host to host transfer
-- rank 1 recv 3310 from rank 0
-- rank 0 sent 3310 to rank 1
dpu to dpu transfer
dpu to host transfer
-- rank 1 recv 3312 from rank 0
host to dpu transfer
-- rank 0 sent 3313 to rank 1
athens@harphe11a0802:~$

athens@harphe11a0802:~$ pwd /bin/mpirun -n 2 -p 2 \
--mca hellos002,hellos012 \
--mca OCK_LOG_LEVEL=ERROR \
--mca /dca-amp-service
server mode
dpu is listening...
server mode
dpu is listening...
-- rank 0 sent 3311 to rank 1
-- rank 1 recv 3311 from rank 0
-- rank 0 sent 3312 to rank 1
-- rank 1 recv 3313 from rank 0
athens@harphe11a0802:~$

```



```
uthmanhere@helios002:$ `pwd`/bin/mpirun -q -np 2 \  
-H helios002,helios012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/app_pt2pt  
client mode  
client mode  
going to connect....  
going to connect....  
connected  
connected  
host to host transfer  
-- rank 1 recv 3310 from rank 0  
-- rank 0 sent 3310 to rank 1  
dpu to dpu transfer  
dpu to host transfer  
-- rank 1 recv 3312 from rank 0  
host to dpu transfer  
-- rank 0 sent 3313 to rank 1  
uthmanhere@helios002:$
```

```
uthmanhere@heliosbf2a002:$ `pwd`/bin/mpirun -q -np 2 \  
-H heliosbf2a002,heliosbf2a012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/doca-omp-service  
server mode  
dpu is listening...  
server mode  
dpu is listening...  
-- rank 0 sent 3311 to rank 1  
-- rank 1 recv 3311 from rank 0  
-- rank 0 sent 3312 to rank 1  
-- rank 1 recv 3313 from rank 0  
^C^Cuthmanhere@heliosbf2a002:$
```


MPI Collectives across DPUs

```
#pragma omp target
{
    MPI_Alltoall(send_buffer, 1, MPI_INT, recv_buffer, 1, MPI_INT, MPI_COMM_WORLD);
}
```

MPI RMA Host-to-DPU

```
#pragma omp target nowait
{
    MPI_Win_create(&window_data_dpu, sizeof(int), sizeof(int), MPI_INFO_NULL, MPI_COMM_WORLD, &win);
    MPI_Win_fence(0, win);
    MPI_Win_free(&win);
}

MPI_Info info;
MPI_Info_create(&info);
MPI_Win_create(&window_data, sizeof(int), sizeof(int), info, MPI_COMM_WORLD, &win);

if (rank == src) {
    MPI_Info_set(info, "REMOTE_DEST_HETERO", "1");
    MPI_Win_set_info(win, info);
    MPI_Put(&data, 1, MPI_INT, dst, 0, 1, MPI_INT, win);
    MPI_Info_delete(info, "REMOTE_DEST_HETERO");
}

MPI_Win_fence(0, win);
MPI_Win_free(&win);
MPI_Info_free(&info);
```

Host-DPU Simultaneous RMA Communication (1/4)

Simultaneous one-sided cross-communication:

Rank#0_Host to Rank#1_DPU

Rank#0_DPU to Rank#1_Host

```
uthmanhere@helios002:$  
  
uthmanhere@heliosbf2a002:$ `pwd`/bin/mpirun -q -np 2 \  
-H heliosbf2a002,heliosbf2a012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/doca-omp-service  
  
[3] 0:srun 1:ssh* 2:bash 3:bash 4:vim- 5:vim 6:bash "login01.hpcadvisoryco" 15:49 14-Aug-24
```


Host-DPU Simultaneous RMA Communication (2/4)

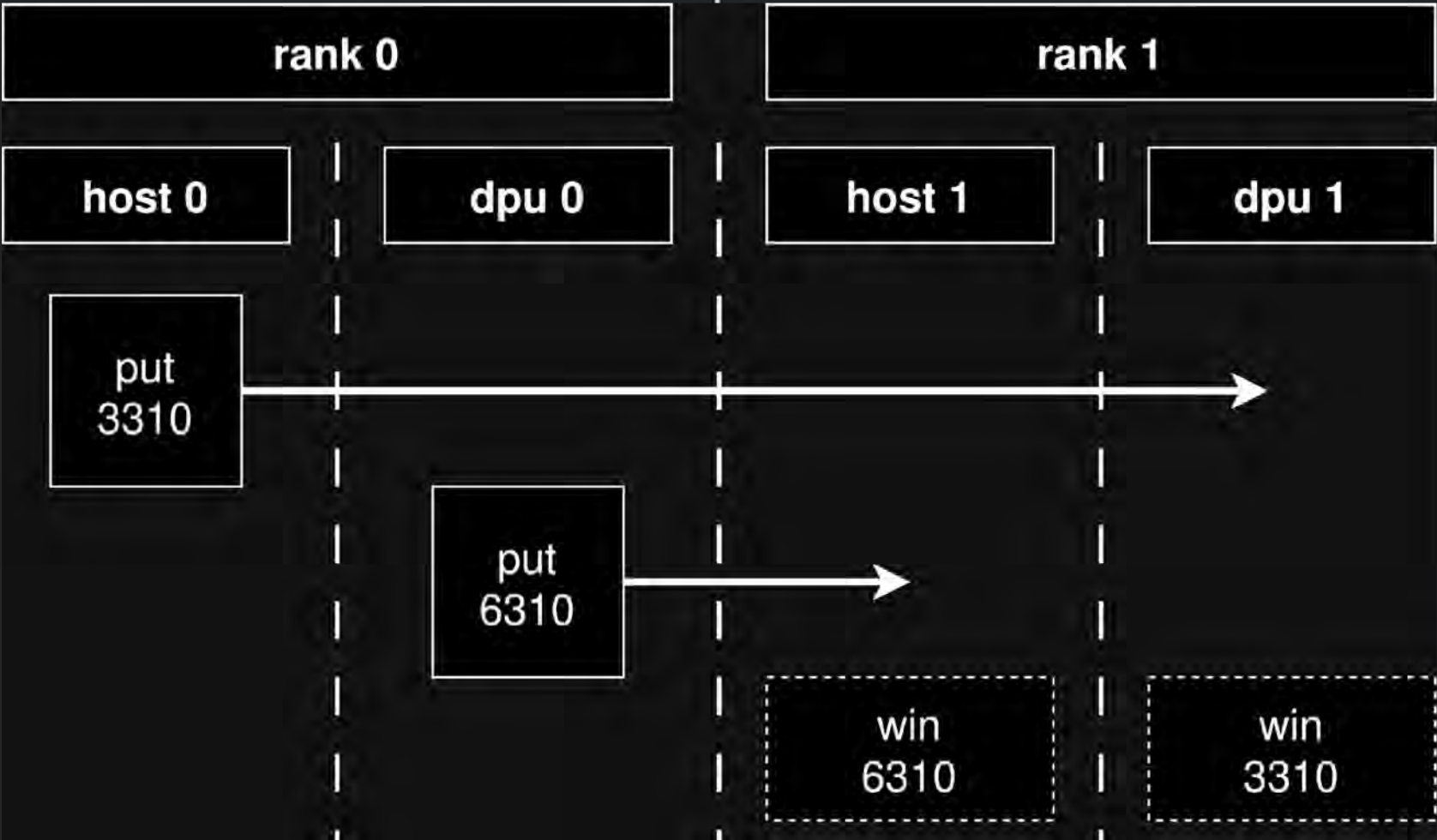
```
uthmanhere@helios002:$ `pwd`/bin/mpirun -q -np 2 \  
    -H helios002,helios012 \  
    -x UCX_LOG_LEVEL=ERROR \  
    `pwd`/app_osc  
client mode  
going to connect....  
client mode  
going to connect....  
connected  
connected
```

```
uthmanhere@heliosbf2a002:$ `pwd`/bin/mpirun -q -np 2 \  
    -H heliosbf2a002,heliosbf2a012 \  
    -x UCX_LOG_LEVEL=ERROR \  
    `pwd`/doca-omp-service  
server mode  
dpu is listening...  
server mode  
dpu is listening...  
█
```

Host-DPU Simultaneous RMA Communication (3/4)

```
uthmanhere@helios002:$ `pwd`/bin/mpirun -q -np 2 \  
-H helios002,helios012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/app_osc  
client mode  
going to connect....  
client mode  
going to connect....  
connected  
connected  
>>> process 1 got 6310 from rank 0 by put
```

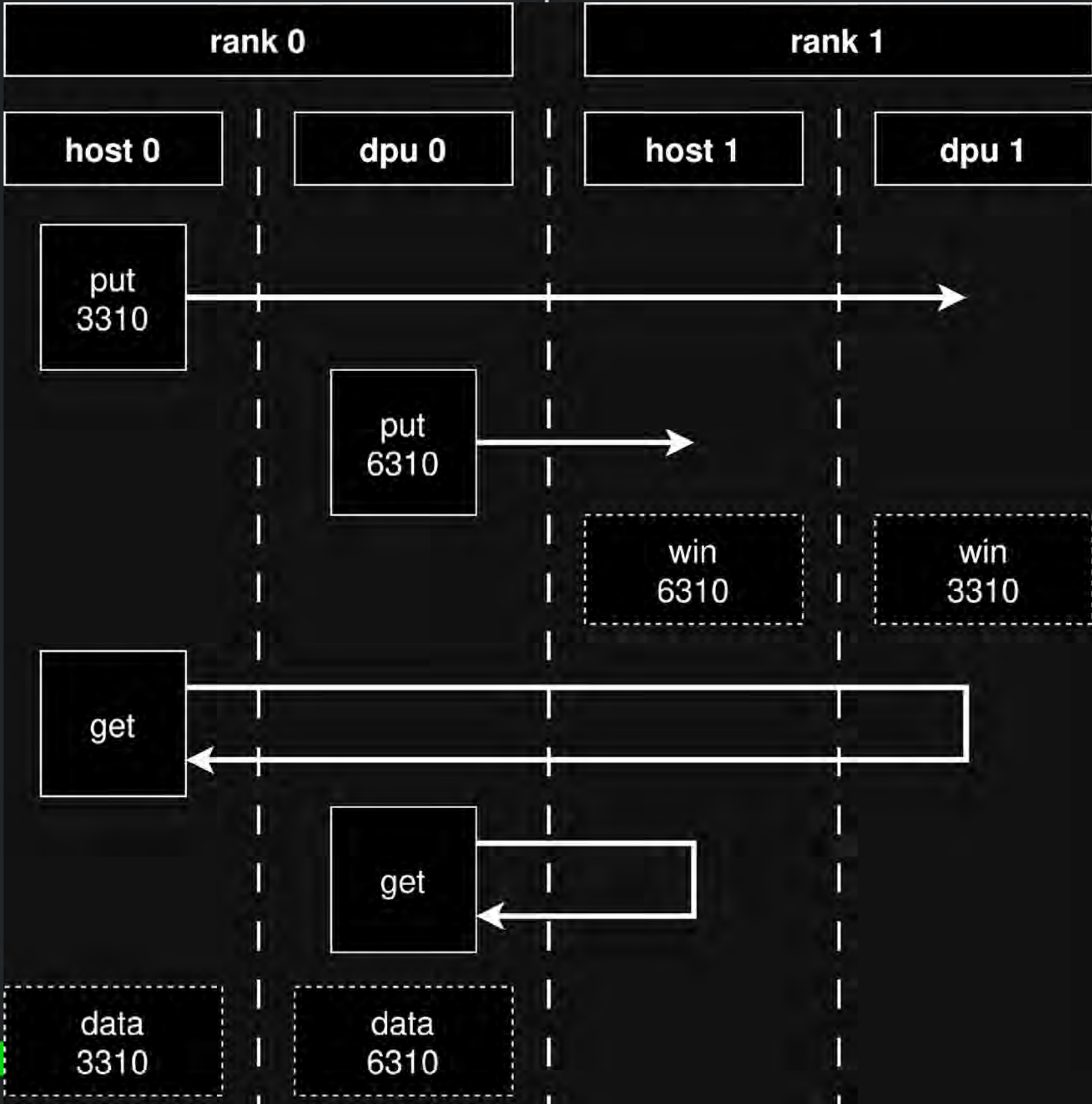
```
uthmanhere@heliosbf2a002:$ `pwd`/bin/mpirun -q -np 2 \  
-H heliosbf2a002,heliosbf2a012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/doca-omp-service  
server mode  
dpu is listening...  
server mode  
dpu is listening...  
>>> process 1 got 3310 from rank 0 by put
```



Host-DPU Simultaneous RMA Communication (4/4)

```
uthmanhere@helios002:$ `pwd`/bin/mpirun -q -np 2 \  
-H helios002,helios012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/app_osc  
client mode  
going to connect....  
client mode  
going to connect....  
connected  
connected  
>>> process 1 got 6310 from rank 0 by put  
>>> process 0 got 3310 from rank 1 by get  
uthmanhere@helios002:$
```

```
uthmanhere@heliosbf2a002:$ `pwd`/bin/mpirun -q -np 2 \  
-H heliosbf2a002,heliosbf2a012 \  
-x UCX_LOG_LEVEL=ERROR \  
`pwd`/doca-omp-service  
server mode  
dpu is listening...  
server mode  
dpu is listening...  
>>> process 1 got 3310 from rank 0 by put  
>>> process 0 got 6310 from rank 1 by get  
█
```

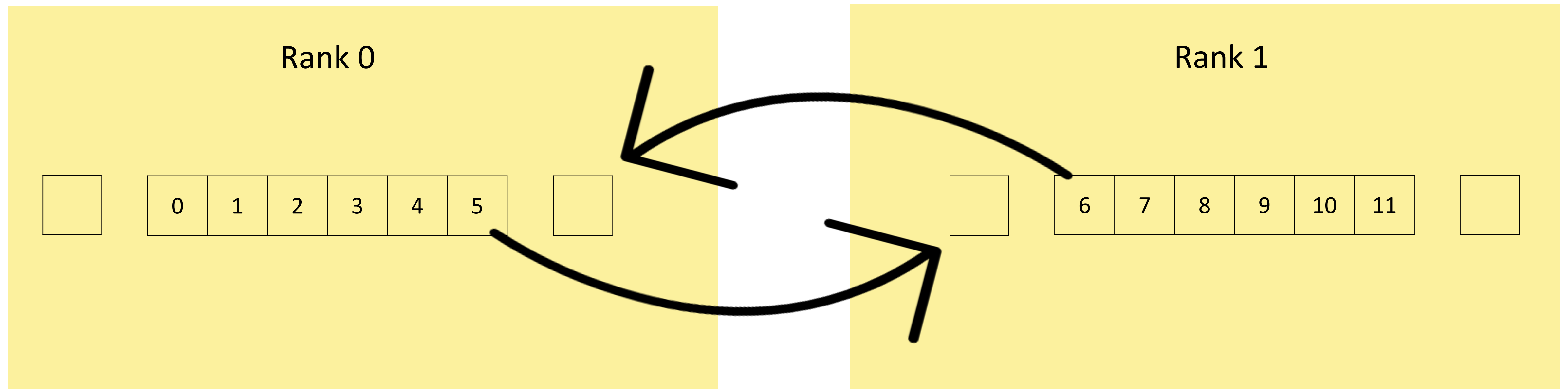
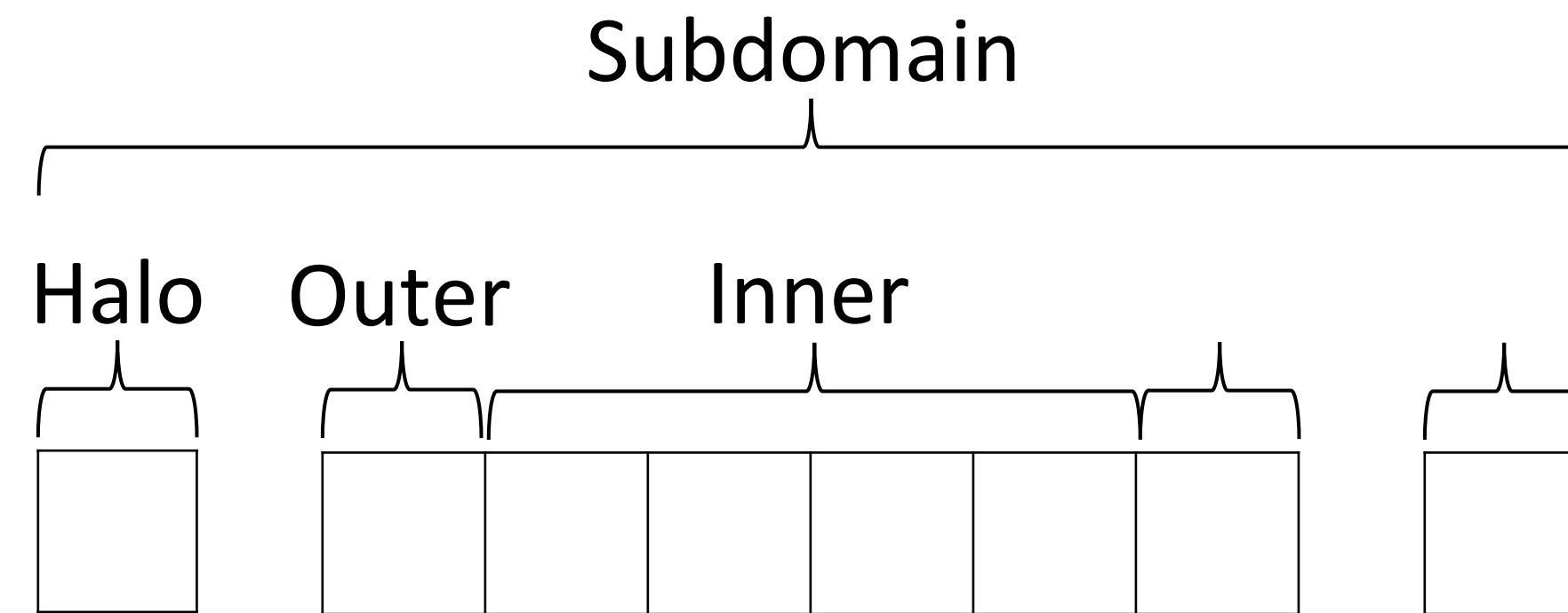




Example: Halo Exchange

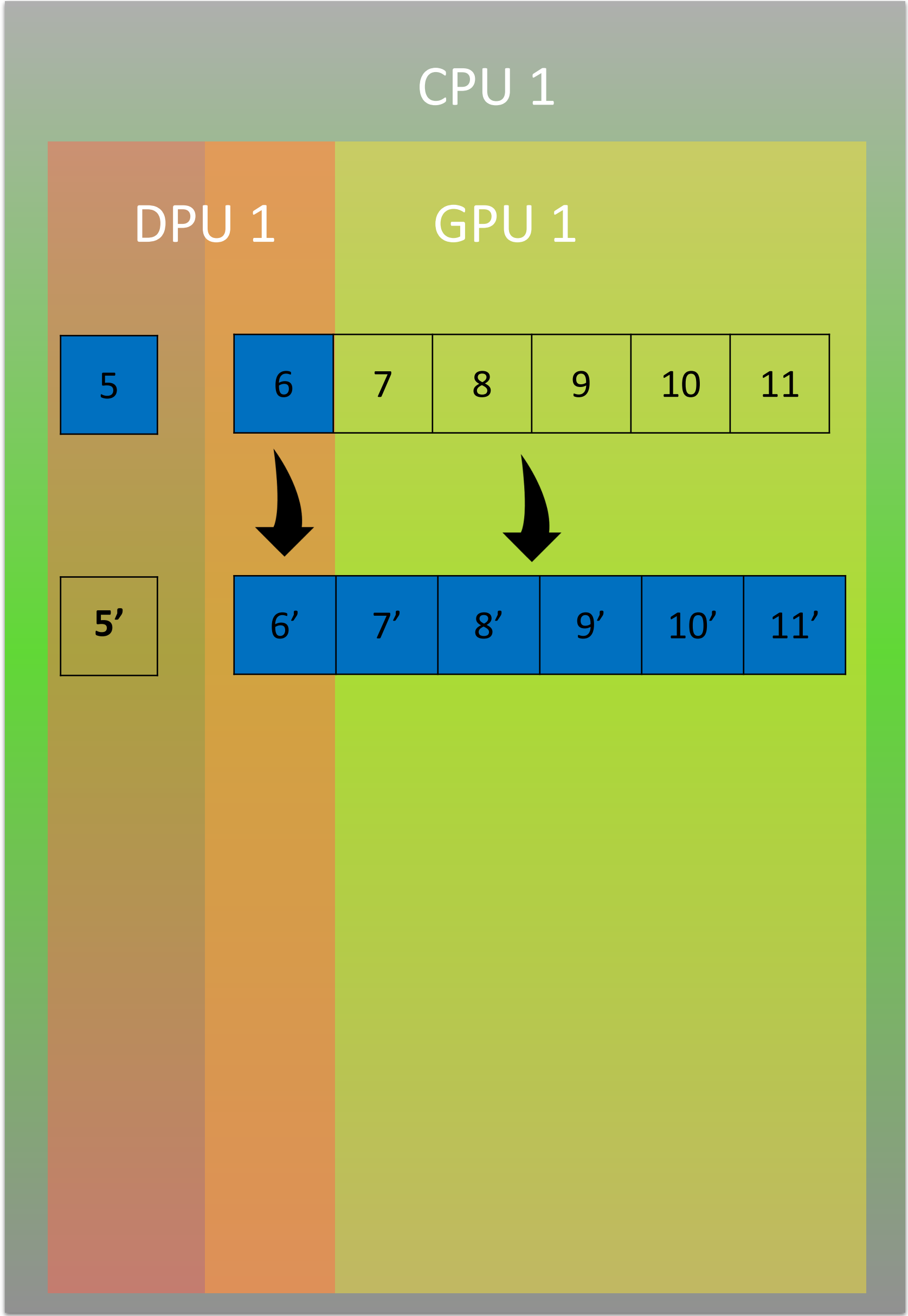
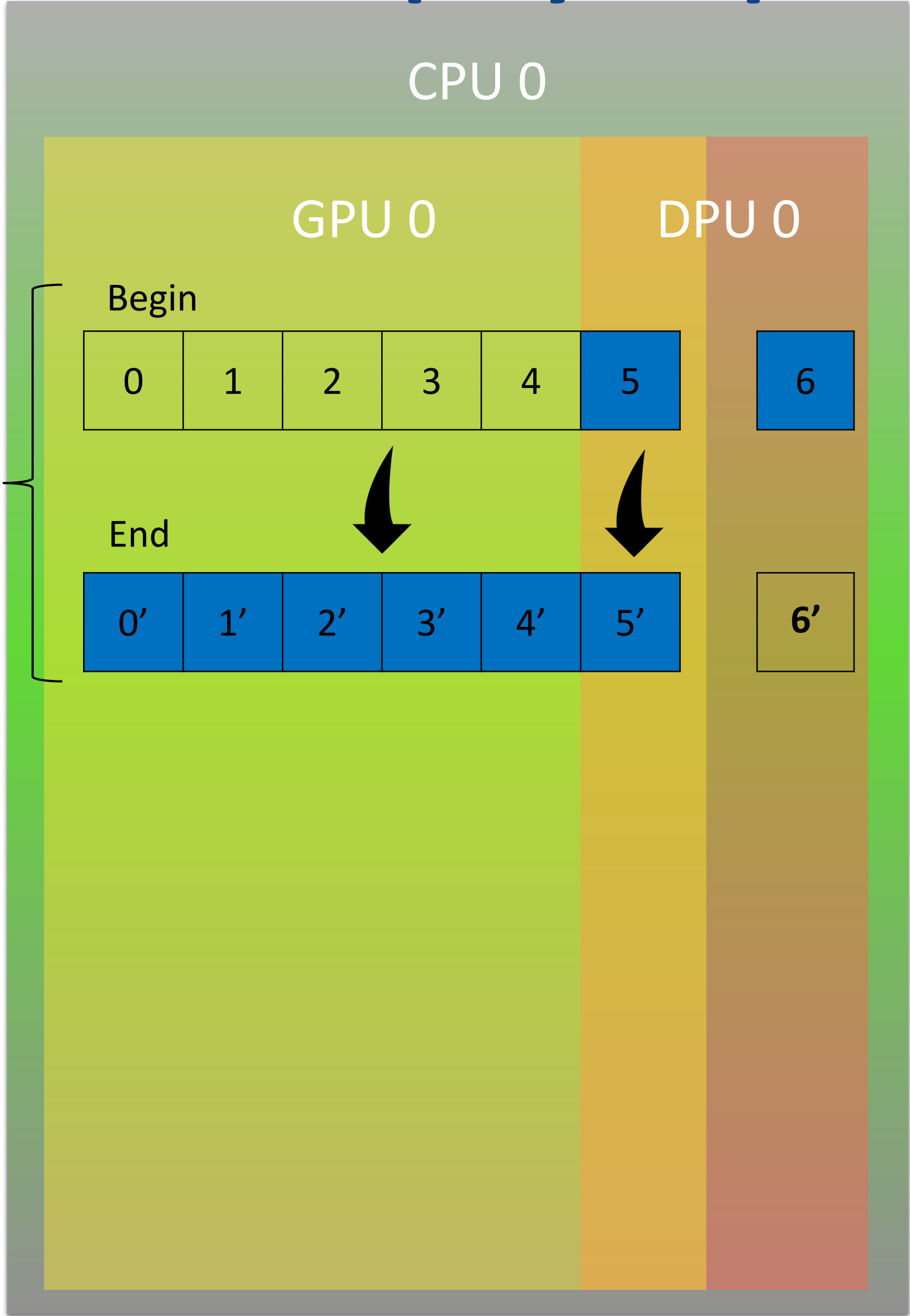
Halo Exchange Offloading

SIZE = 8
THICKNESS = 1

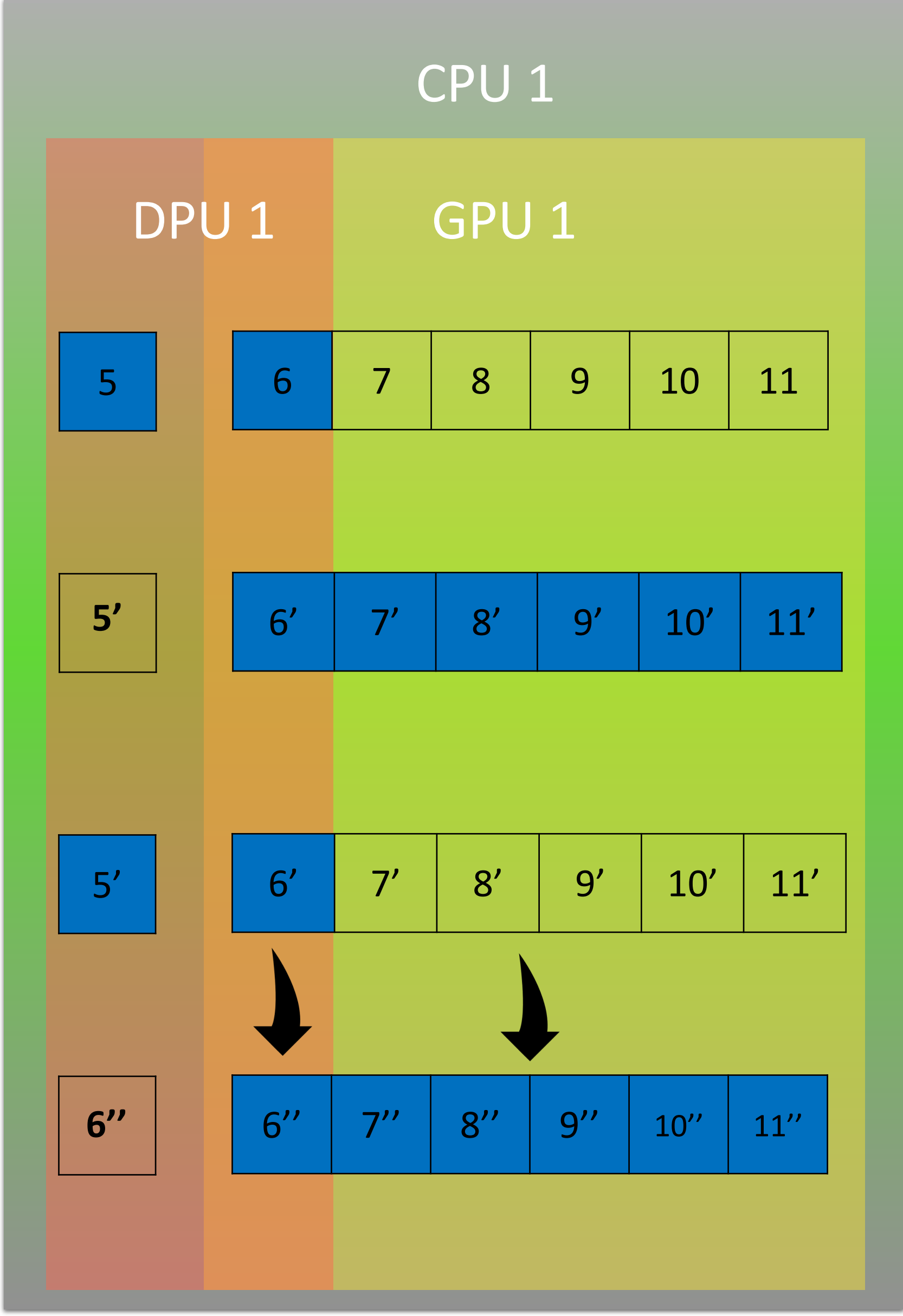
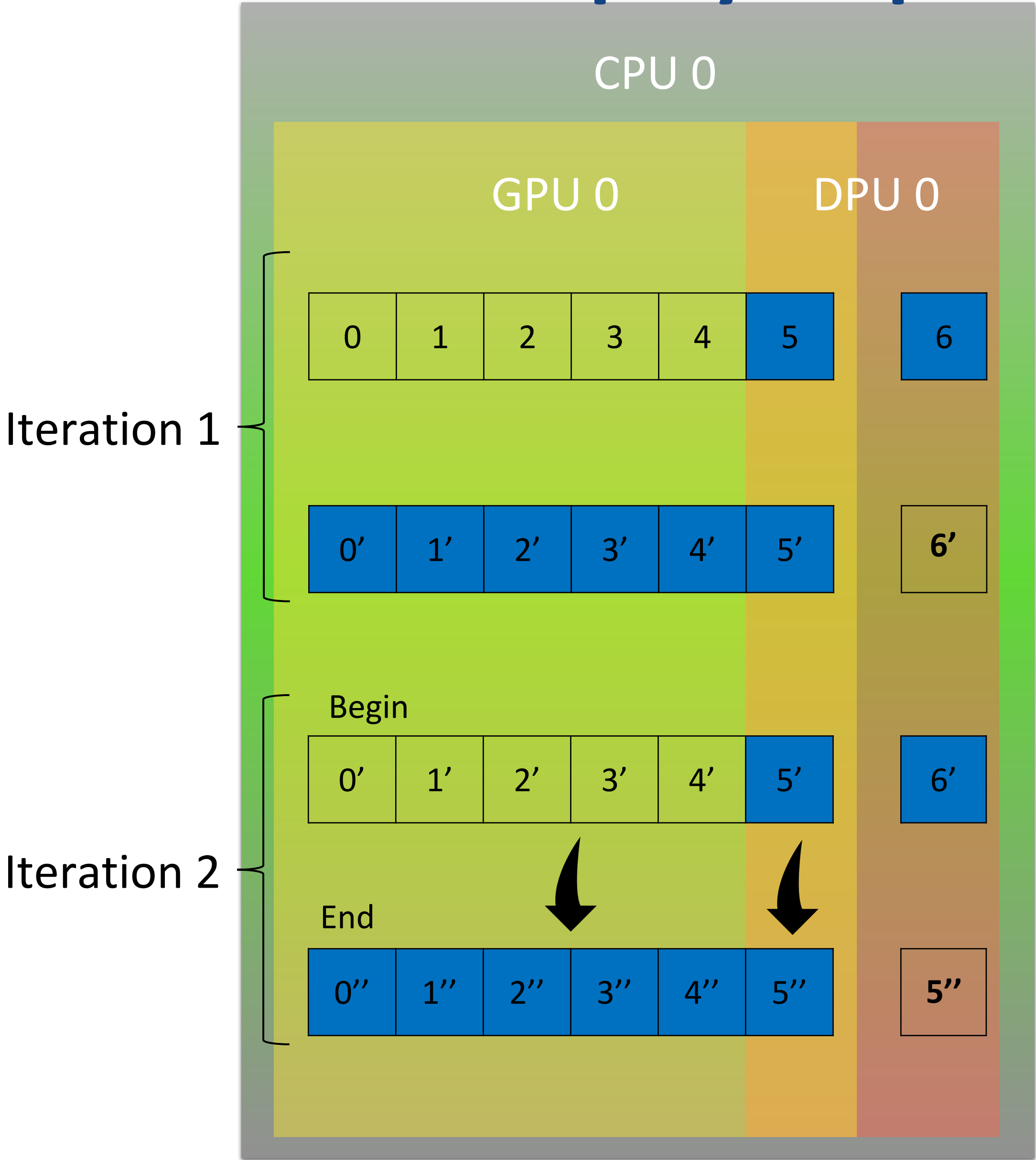


Execution step by step

Iteration 1



Execution step by step



GPU

```

int curr_subdomain[SIZE + THICKNESS * 2];
int next_subdomain[SIZE + THICKNESS * 2];
#pragma omp target data map(to: curr_subdomain[THICKNESS:SIZE-THICKNESS])
                        map(from: next_subdomain[THICKNESS:SIZE-THICKNESS])
                        device (GPU_ID) nowait
{
    #pragma omp target teams num_teams(2)
    {
        #pragma omp parallel
        {
            next_subdomain[THICKNESS] = compute_inner(curr_subdomain[THICKNESS:SIZE-THICKNESS])
        }
    }
}

#pragma omp target data map(to: curr_subdomain[0:THICKNESS*2], curr_subdomain[SIZE-THICKNESS*2:SIZE])
                        map(from: next_subdomain[0:THICKNESS*2], next_subdomain[SIZE-THICKNESS*2:SIZE])
                        device (DPU_ID) nowait
{
    #pragma omp parallel
    #pragma omp single
    {
        #pragma omp task
        {
            next_subdomain[THICKNESS] = compute_outer(curr_subdomain[0:THICKNESS*2], left);
        }
        #pragma omp task
        {
            next_subdomain[SIZE-THICKNESS*2] = compute_outer(curr_subdomain[SIZE-THICKNESS*2:SIZE], right);
        }
    }
}

#pragma omp barrier
curr_subdomain = copy_domain(next_subdomain);

```

DPU

Offloaded Function to the DPU

```
int* compute_outer(int* data, int rank_id)
{
    int halo[THICKNESS], outer_domain[THICKNESS], tag_id = 0;
    MPI_Request request;
    MPI_Status status;

    MPI_Irecv(halo, THICKNESS, MPI_INT, rank_id, tag_id, MPI_COMM_WORLD, &request);
    outer_domain = compute(data);
    MPI_Send(outer_domain, THICKNESS, MPI_INT, rank_id, tag_id, MPI_COMM_WORLD);
    MPI_Wait(&request, &status);
    return outer_domain;
}
```


Performance Evaluation



Testbed



Jupiter

Intel Xeon 10-core E5-2680, 64GB DD3

NVIDIA BlueField-2 HDR 100Gbps: 8 ARMv8 A72 cores, 16GB DDR4

Thor

2x Intel Xeon 16-core E5-2697, 256GB DD4

NVIDIA BlueField-3 NDR 200Gbps: 16 ARMv8.2 A78 cores, 16GB DDR5



Software Stack

OpenMP 5.0

LLVM 14.0.6

DOCA 2.0.2

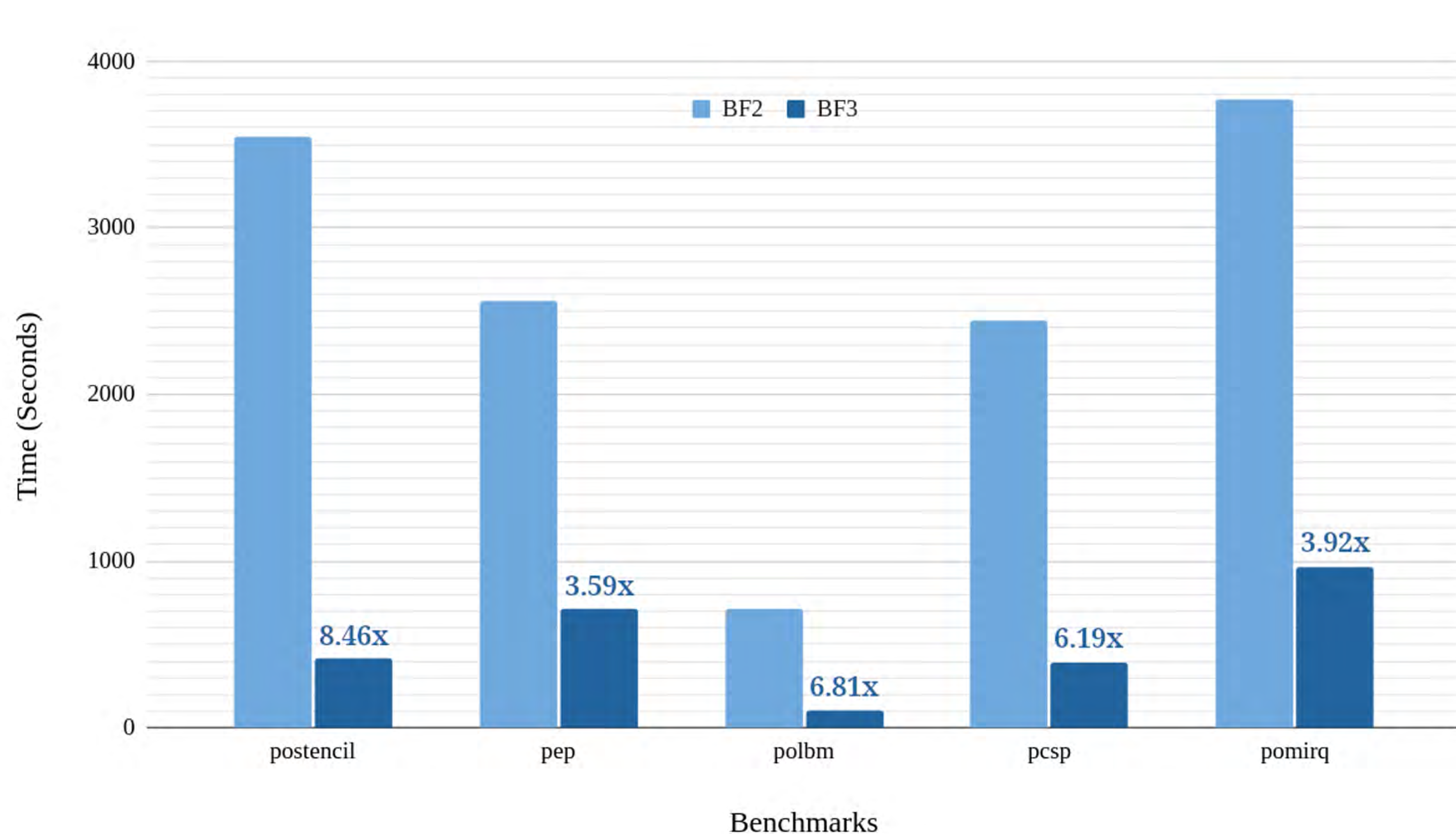
OpenMPI 4.1.6

SPEC ACCEL Benchmark Suite (OpenMP target Offloading)

- Removed OpenMP “distribute” construct for GPUs
- Added “simd” construct for enabling SIMD instructions

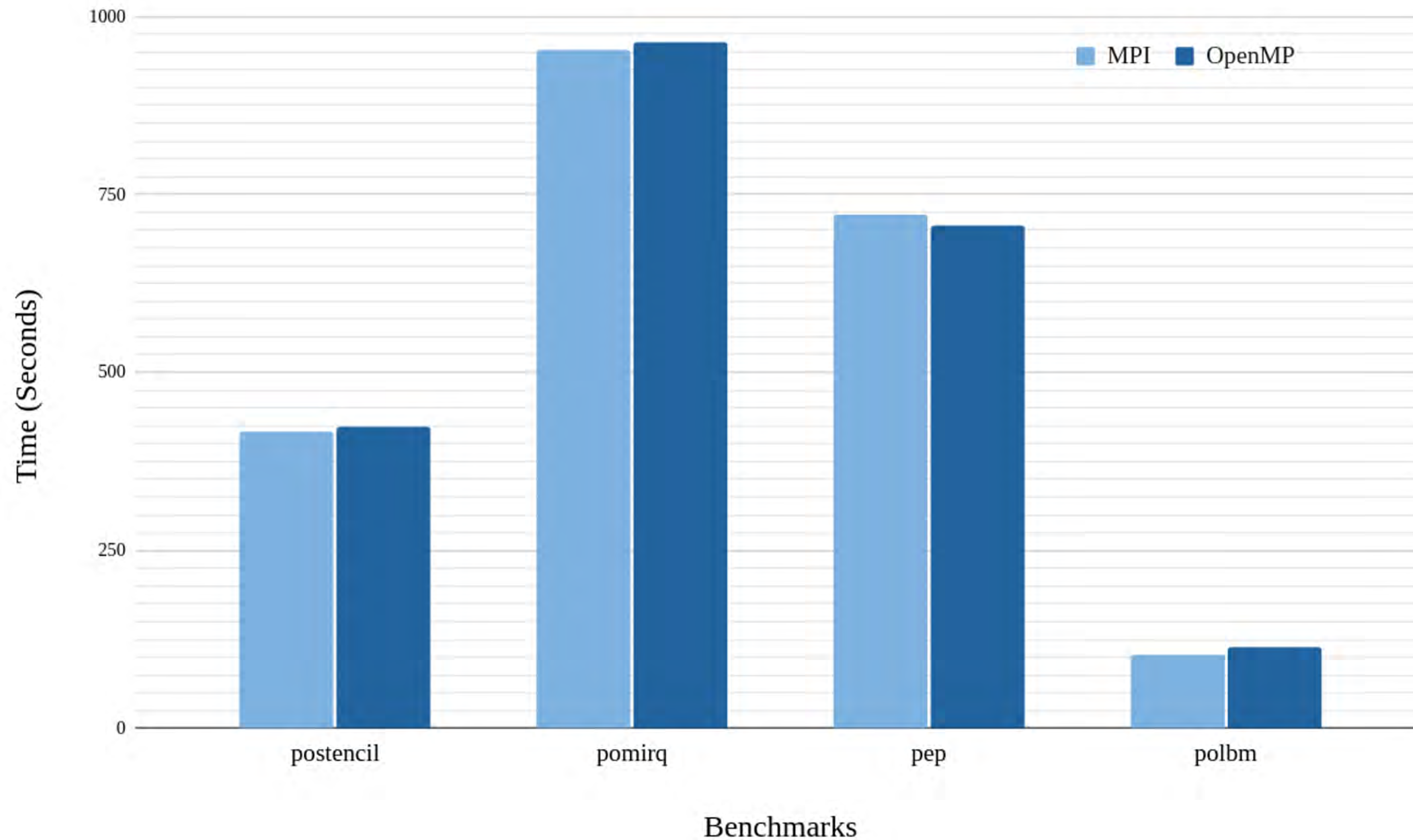
OSU Microbenchmarks (variation supporting DOCA)

BlueField-2 and BlueField-3 Performance Comparison



M. Usman, S. Iserte, R. Ferrer, and A. J. Peña, "DPU Offloading Programming with the OpenMP API." *LLVM-HPC23 co-located with SC23*, <https://dl.acm.org/doi/10.1145/3624062.3624165>

OpenMP and MPI Performance Comparison



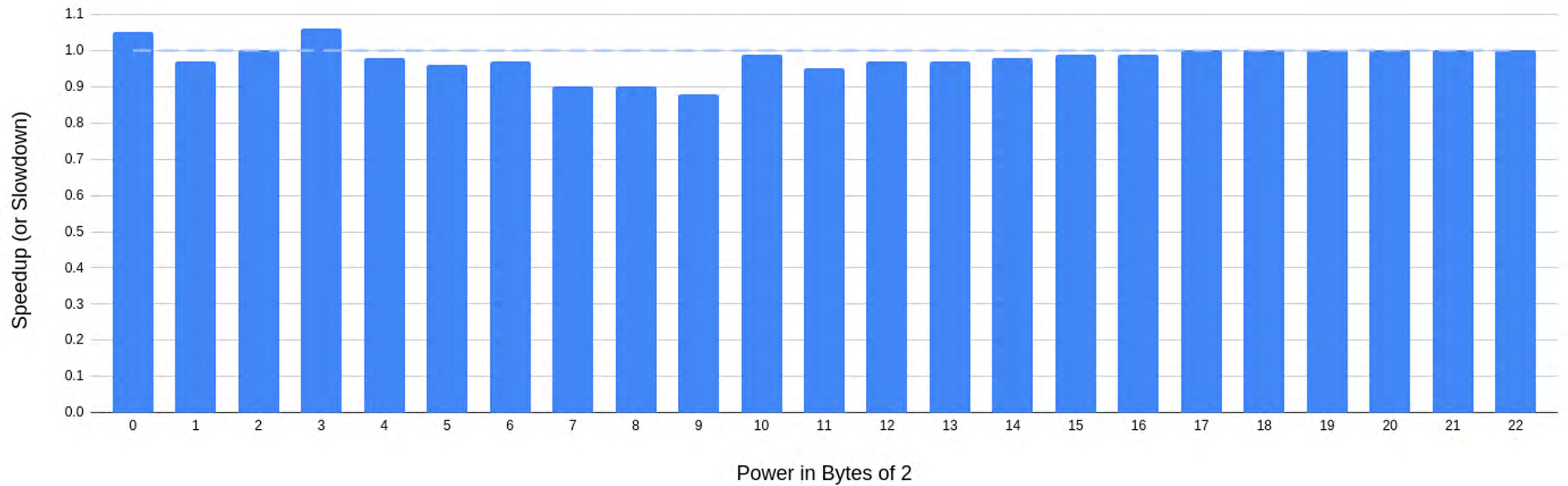
M. Usman, S. Iserte, R. Ferrer, and A. J. Peña, "DPU Offloading Programming with the OpenMP API." *LLVM-HPC23 co-located with SC23*, <https://doi.org/10.1145/3624062.3624165>

MPI vs ODOS MPI Offloading

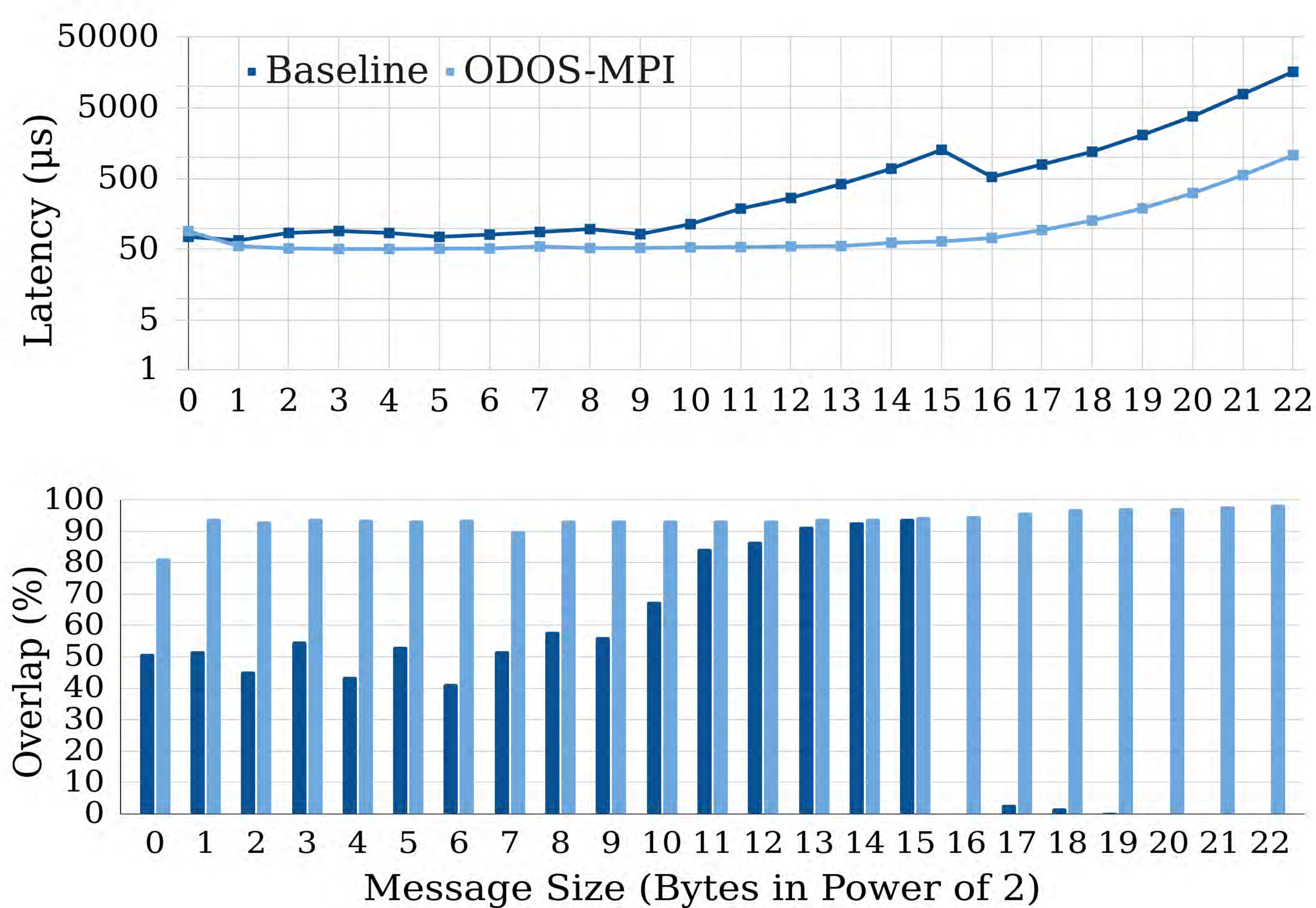
Custom version of OSU to offload MPI to DPU

MPI Bandwidth
between host and bf dpu

odos bw / ref bw same perf



OSU MB: Computation-Communication Overlap



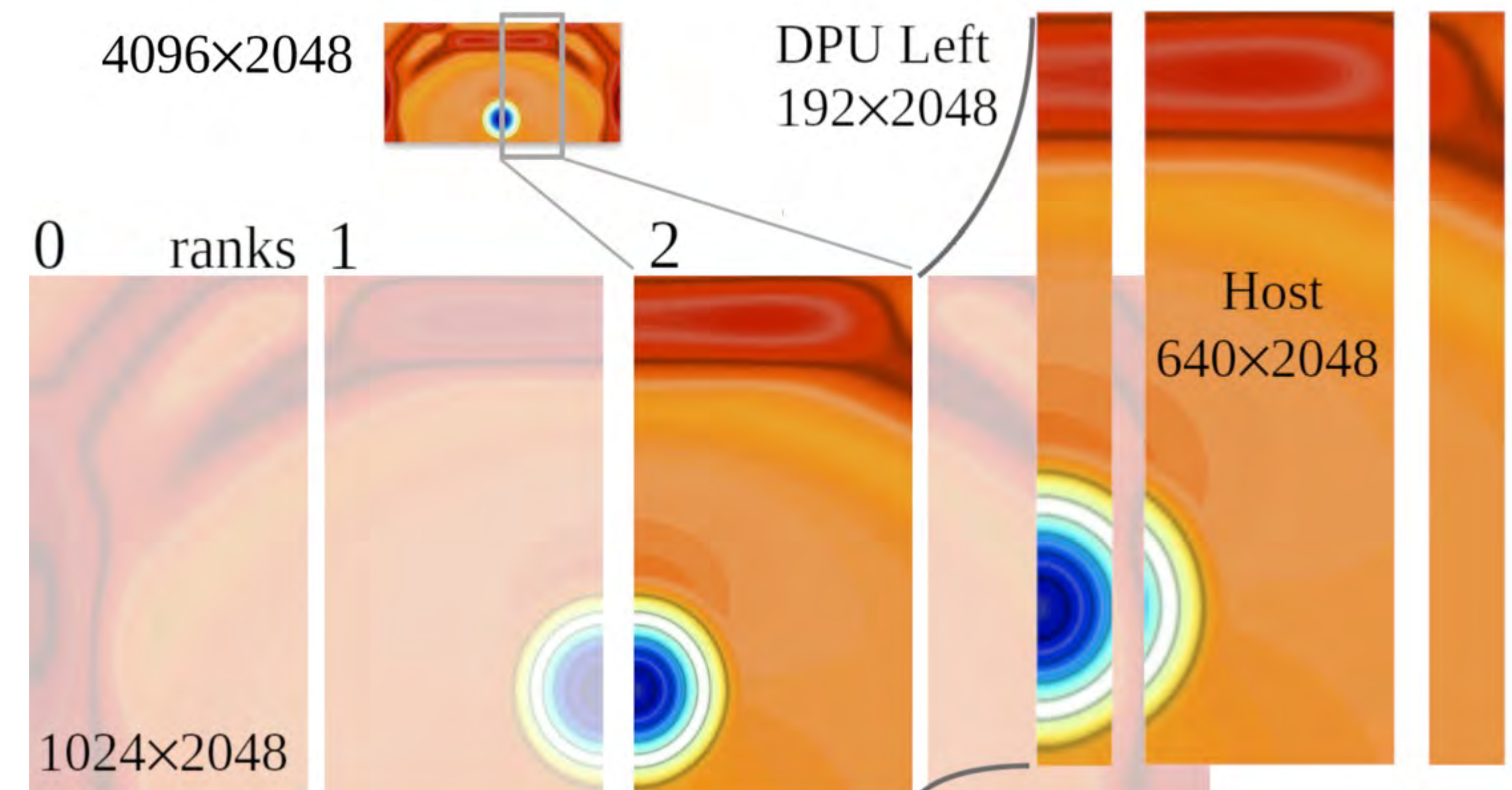
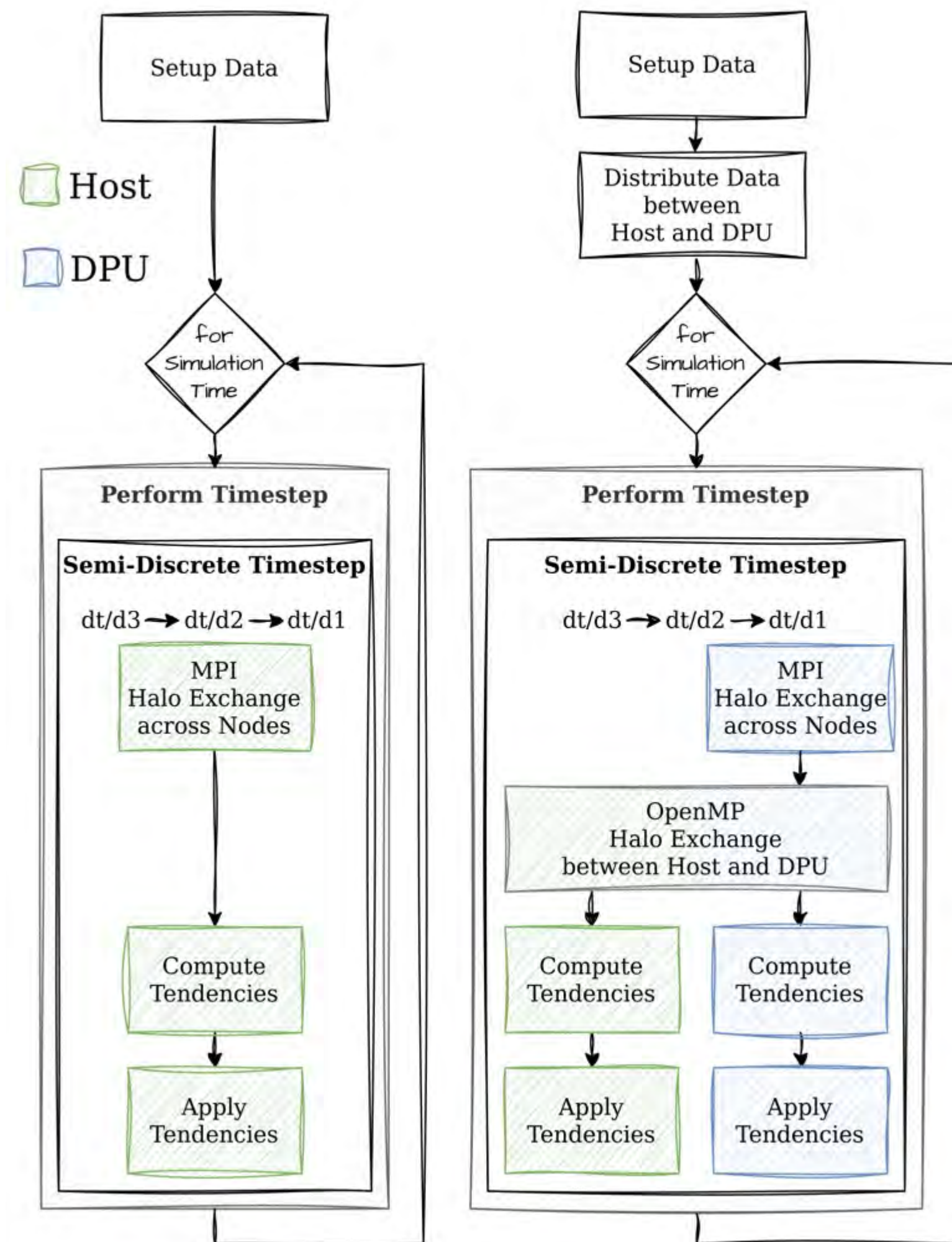


Demo: miniWeather Code Walkthrough

miniWeather Application

Simulates 2-D inviscid Euler equations for stratified fluid dynamics

Using Runge-Kutta Time Integration method



miniWeather Code Walkthrough

```
main()  
  setup: init() and distribute()  
  while simulation time does not reach:  
    perform_timestep() for both x and z directions  
      semi_discrete_timestep for dt/d3  
      semi_discrete_timestep for dt/d2  
      semi_discrete_timestep for dt/d1  
    if timestep to output? output()  
  finalize()
```

miniWeather Code Walkthrough

```
main()  
  setup: init() and distribute()  
  while simulation time does not reach:  
    perform_timestep() for both x and z directions  
    semi_discrete_timestep for dt/d3, dt/d2 and dt/d1  
    if timestep to output? output()  
  finalize()
```


miniWeather Code Walkthrough – Simulation Loop

```
double t_start = MPI_Wtime();
while (etime < sim_time) {
    //If the time step leads to exceeding the simulation time, shorten it for the last step
    if (etime + dt > sim_time) { dt = sim_time - etime; }
    //Perform a single time step
    perform_timestep(&stateh, &state_tmph, &fluxh, &tendh,
                    &statedl, &state_tmpl, &fluxdl, &tenddl,
                    &statedr, &state_tmpr, &fluxdr, &tenddr, dt);

    //Update the elapsed time and output counter
    etime = etime + dt;
    output_counter = output_counter + dt;
    //If it's time for output, reset the counter, and do output

    if (output_freq >= 0 && output_counter >= output_freq) {
#pragma omp target update from(statedl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target update from(statedr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
        gather(state, statedl, stateh, statedr);

        output_counter = output_counter - output_freq;
        output(state, etime);
    }
}
```


miniWeather Code Walkthrough



Changes Required

- Setup data allocation, transfer and deallocation
- Setup compute and data updates

miniWeather Code Walkthrough – Data Allocations



```
#pragma omp target enter data map(to:statedl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target enter data map(to:statedr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target enter data map(to:state_tmpdl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target enter data map(to:state_tmpdr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target enter data map(to:fluxdl[0:NUM_VARS*(nz+1)*(nxdl+1)])
#pragma omp target enter data map(to:fluxdr[0:NUM_VARS*(nz+1)*(nxdr+1)])
#pragma omp target enter data map(to:tenddl[0:NUM_VARS*(nz)*(nxdl)])
#pragma omp target enter data map(to:tenddr[0:NUM_VARS*(nz)*(nxdr)])
#pragma omp target enter data map(to:hy_dens_cell[0:nz+2*hs])
#pragma omp target enter data map(to:hy_dens_theta_cell[0:nz+2*hs])
#pragma omp target enter data map(to:hy_dens_int[0:nz+1])
#pragma omp target enter data map(to:hy_dens_theta_int[0:nz+1])
#pragma omp target enter data map(to:hy_pressure_int[0:nz+1])
#pragma omp target enter data map(to:sendbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target enter data map(to:sendbuf_r[0:NUM_VARS*hs*nz])
#pragma omp target enter data map(to:recvbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target enter data map(to:recvbuf_r[0:NUM_VARS*hs*nz])
#pragma omp target enter data map(to:sendbuf_local[0:NUM_VARS*2*hs*(nz+2*hs)])
#pragma omp target enter data map(to:recvbuf_local[0:NUM_VARS*2*hs*(nz+2*hs)])
```

miniWeather Code Walkthrough – Data Transfer



```
#pragma omp target update to (statedl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target update to (statedr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target update to (state_tmpdl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target update to (state_tmpdr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target update to (fluxdl[0:NUM_VARS*(nz+1)*(nxdl+1)])
#pragma omp target update to (fluxdr[0:NUM_VARS*(nz+1)*(nxdr+1)])
#pragma omp target update to (tenddl[0:NUM_VARS*(nz)*(nxdl)])
#pragma omp target update to (tenddr[0:NUM_VARS*(nz)*(nxdr)])
#pragma omp target update to (hy_dens_cell[0:nz+2*hs])
#pragma omp target update to (hy_dens_theta_cell[0:nz+2*hs])
#pragma omp target update to (hy_dens_int[0:nz+1])
#pragma omp target update to (hy_dens_theta_int[0:nz+1])
#pragma omp target update to (hy_pressure_int[0:nz+1])
#pragma omp target update to (sendbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target update to (sendbuf_r[0:NUM_VARS*hs*nz])
#pragma omp target update to (recvbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target update to (recvbuf_r[0:NUM_VARS*hs*nz])
```


miniWeather Code Walkthrough – Simulation

```
semi_discrete_step()  
  set_halo_values()  
  compute_tendencies()  
  apply_tendencies()
```

miniWeather Code Walkthrough – Halo Exchange

The Halo Exchange is divided in 2 Exchanges:

Halo Exchange between nodes

- MPI-based
- Performed in DPU

Halo Exchange between host and DPU

- OpenMP-based
- Performed between host and DPU

miniWeather Code Walkthrough – Halo Exchange

The Halo Exchange between nodes happens in DPU

```
#pragma omp target map(to:nxd1,nxdr,nz,hs,nranks,left_rank,right_rank,myrank)
{
    setup buffers
    mulitple MPI_Irecv
    setup buffers
    mulitple MPI_Isend
}
```


miniWeather Code Walkthrough – Halo Exchange

```
//Set the halo values for this MPI task's fluid state in the x-direction
if (mainproc) t_tmp = MPI_Wtime();

#pragma omp target map(to:nxdl,nxdr,nz,hs,nranks,left_rank,right_rank,myrank)
set_halo_values_x(state_forcingdl_, state_forcingdr_, nxdl, nxdr, nz, hs, nranks, left_rank, right_rank,
                  sendbuf_l_, sendbuf_r_, recvbuf_l_, recvbuf_r_,myrank);

if (mainproc) {
    prof.halo_x_mpi += MPI_Wtime() - t_tmp;
    t_tmp = MPI_Wtime();
}
```

miniWeather Code Walkthrough – Halo Exchange

The Halo Exchange between host and DPU

```
// pack buffers...
#pragma omp target nowait
{
    pack DPU buffers
}
    pack host buffers
#pragma omp taskwait

// exchange...
#pragma omp target update to( sendbuf_local[0:2*hs*(nz+2*hs)*NUM_VARS] )
#pragma omp target update from(recvbuf_local[0:2*hs*(nz+2*hs)*NUM_VARS] )

// unpack buffers...
#pragma omp target nowait
{
    unpack DPU buffers
}
    unpack host buffers
#pragma omp taskwait
```


miniWeather Code Walkthrough – Pack & Exchange

```
#pragma omp target nowait
{
    for (int var = 0; var < NUM_VARS; ++var) {
        for (int i = 0; i < nz+2*hs; ++i) {
            for (int j = 0; j < hs; ++j) {
                int to_, from_;
                to_ = var*(nz+2*hs)*(2*hs) + i*(2*hs) + j;
                from_ = var*(nxdl+2*hs)*(nz+2*hs) + i*(nxdl+2*hs) + nxdl + j;
                recvbuf_local[to_] = state_forcingdl[from_];
                to_ = var*(nz+2*hs)*(2*hs) + i*(2*hs) + hs + j;
                from_ = var*(nxdr+2*hs)*(nz+2*hs) + i*(nxdr+2*hs) + hs + j;
                recvbuf_local[to_] = state_forcingdr[from_];
            }
        }
    }

    for (int var = 0; var < NUM_VARS; ++var) {
        for (int i = 0; i < nz+2*hs; ++i) {
            for (int j = 0; j < hs; ++j) {
                int to_, from_;
                to_ = var*(nz+2*hs)*(2*hs) + i*(2*hs) + j;
                from_ = var*(nxh +2*hs)*(nz+2*hs) + i*(nxh +2*hs) + hs + j;
                sendbuf_local[to_] = (*state_forcingh)[from_];
                to_ = var*(nz+2*hs)*(2*hs) + i*(2*hs) + hs + j;
                from_ = var*(nxh +2*hs)*(nz+2*hs) + i*(nxh +2*hs) + nxh + j;
                sendbuf_local[to_] = (*state_forcingh)[from_];
            }
        }
    }
}

#pragma omp taskwait
#pragma omp target update to( sendbuf_local[0:2*hs*(nz+2*hs)*NUM_VARS])
#pragma omp target update from(recvbuf_local[0:2*hs*(nz+2*hs)*NUM_VARS])
```


miniWeather Code Walkthrough – Compute Tendencies

Both host and DPU compute tendencies for their own parts

```
// setup buffers...  
#pragma omp target nowait  
{  
    compute tendencies for left  halo  
    compute tendencies for right halo  
}  
    compute tendencies for inner data  
#pramga omp taskwait
```

miniWeather Code Walkthrough – Compute Tendencies

```
//Compute the time tendencies for the fluid state in the x-direction
#pragma omp target nowait
{
    compute_tendencies_x(state_forcingdl_,fluxdl_,tenddl_,dt,nxdl, nz, hs, hy_dens_cell_, hy_dens_theta_cell_);
    compute_tendencies_x(state_forcingdr_,fluxdr_,tenddr_,dt,nxdr, nz, hs, hy_dens_cell_, hy_dens_theta_cell_);
}

if (mainproc) {
    prof.tend_x_launch += MPI_Wtime() - t_tmp;
    t_tmp_ = MPI_Wtime();
}
compute_tendencies_x(*state_forcingh, *fluxh, *tendh, dt,nxh, nz, hs, hy_dens_cell_, hy_dens_theta_cell_ );
if (mainproc) {
    prof.tend_x_host += MPI_Wtime() - t_tmp_;
    t_tmp_ = MPI_Wtime();
}
#pragma omp taskwait
if (mainproc) {
    prof.tend_x_wait += MPI_Wtime() - t_tmp_;
    prof.tend_x      += MPI_Wtime() - t_tmp;
    t_tmp = MPI_Wtime();
}
```

miniWeather Code Walkthrough – Apply Tendencies

Both host and DPU apply tendencies for their own parts

```
// setup buffers...  
#pragma omp target nowait  
{  
    apply tendencies for left halo  
    apply tendencies for right halo  
}  
    apply tendencies for inner data  
#pramga omp taskwait
```


miniWeather Code Walkthrough – Apply Tendencies

DPU Target code

Same code for host
but for inner data

```
#pragma omp target nowait
{
#pragma omp parallel for private(inds,indt,x,z,x0,z0,xrad,zrad,amp,dist,wpert,indw) collapse(3)
    for (ll=0; ll<NUM_VARS; ll++) {
        for (k=0; k<nz; k++) {
            for (i=0; i<nxdl; i++) {
                if (data_spec_int == DATA_SPEC_GRAVITY_WAVES) {
                    puts("error!!! spec gravity is not supported yet!");
                    // there is stuff here... You just can't see it
                    break;
                }
                inds = ll*(nz+2*hs)*(nxdl+2*hs) + (k+hs)*(nxdl+2*hs) + i+hs;
                indt = ll*nz*nxdl + k*nxdl + i;
                state_outdl[inds] = state_initdl[inds] + dt * tenddl[indt];
            }
        }
    }
}

#pragma omp parallel for private(inds,indt,x,z,x0,z0,xrad,zrad,amp,dist,wpert,indw) collapse(3)
    for (ll=0; ll<NUM_VARS; ll++) {
        for (k=0; k<nz; k++) {
            for (i=0; i<nxdl; i++) {
                if (data_spec_int == DATA_SPEC_GRAVITY_WAVES) {
                    puts("error!!! spec gravity is not supported yet!");
                    // there is stuff here... You just can't see it
                }
                inds = ll*(nz+2*hs)*(nxdl+2*hs) + (k+hs)*(nxdl+2*hs) + i+hs;
                indt = ll*nz*nxdl + k*nxdl + i;
                state_outdl[inds] = state_initdl[inds] + dt * tenddl[indt];
            }
        }
    }
}
} /* omp target */
```


miniWeather Code Walkthrough – Gather Output

```
double t_start = MPI_Wtime();
while (etime < sim_time) {
    //If the time step leads to exceeding the simulation time, shorten it for the last step
    if (etime + dt > sim_time) { dt = sim_time - etime; }
    //Perform a single time step
    perform_timestep(&stateh, &state_tmph, &fluxh, &tendh,
                    &statedl, &state_tmpl, &fluxdl, &tenddl,
                    &statedr, &state_tmpr, &fluxdr, &tenddr, dt);

    //Update the elapsed time and output counter
    etime = etime + dt;
    output_counter = output_counter + dt;
    //If it's time for output, reset the counter, and do output

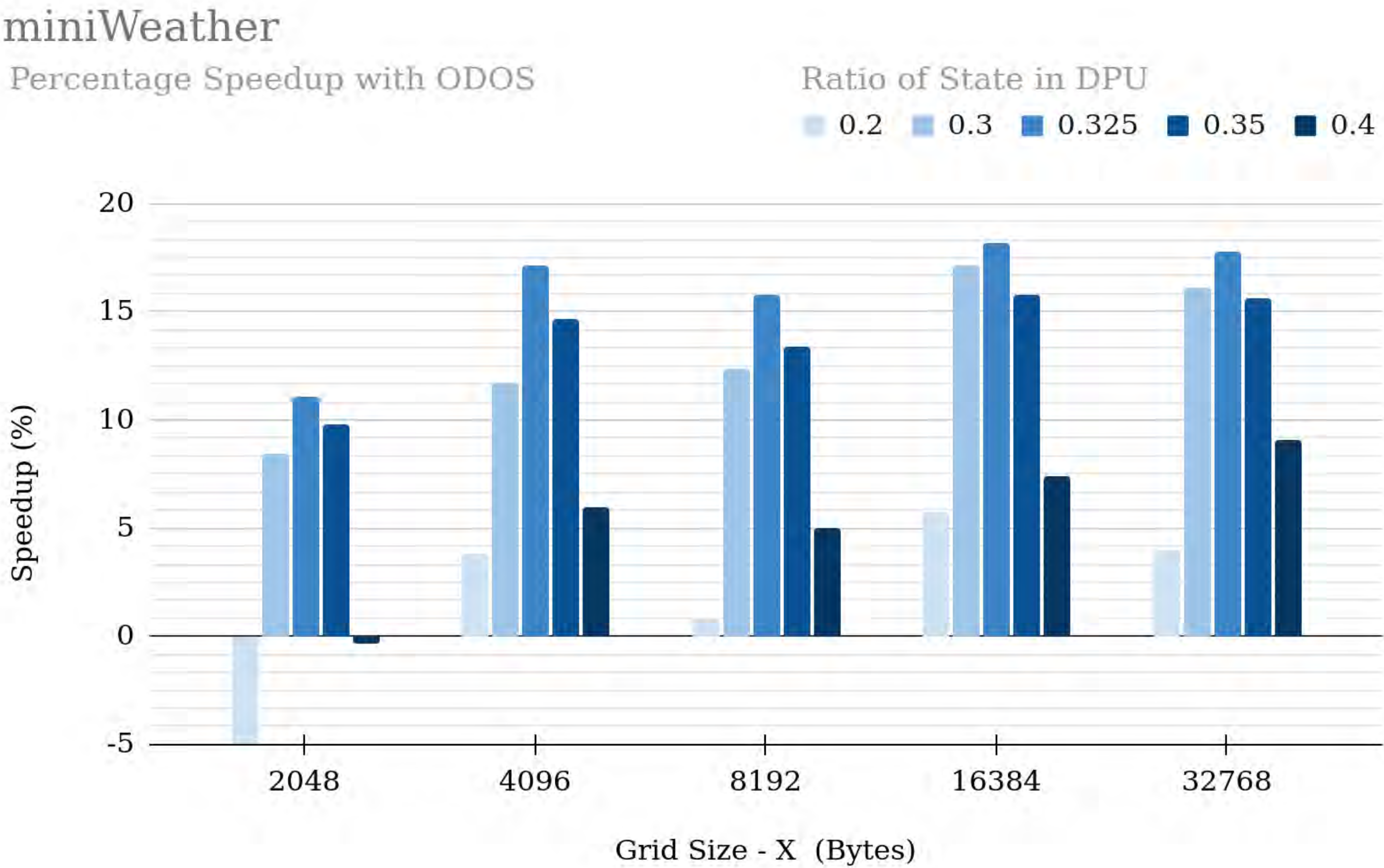
    if (output_freq >= 0 && output_counter >= output_freq) {
#pragma omp target update from(statedl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target update from(statedr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
        gather(state, statedl, stateh, statedr);

        output_counter = output_counter - output_freq;
        output(state, etime);
    }
}
```


miniWeather Code Walkthrough – Finalize

```
#pragma omp target exit data map(delete:statedl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target exit data map(delete:statedr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target exit data map(delete:state_tmpdl[0:NUM_VARS*(nz+2*hs)*(nxdl+2*hs)])
#pragma omp target exit data map(delete:state_tmpdr[0:NUM_VARS*(nz+2*hs)*(nxdr+2*hs)])
#pragma omp target exit data map(delete:fluxdl[0:NUM_VARS*(nz+1)*(nxdl+1)])
#pragma omp target exit data map(delete:fluxdr[0:NUM_VARS*(nz+1)*(nxdr+1)])
#pragma omp target exit data map(delete:tenddl[0:NUM_VARS*(nz)*(nxdl)])
#pragma omp target exit data map(delete:tenddr[0:NUM_VARS*(nz)*(nxdr)])
#pragma omp target exit data map(delete:hy_dens_cell[0:nz+2*hs])
#pragma omp target exit data map(delete:hy_dens_theta_cell[0:nz+2*hs])
#pragma omp target exit data map(delete:hy_dens_int[0:nz+1])
#pragma omp target exit data map(delete:hy_dens_theta_int[0:nz+1])
#pragma omp target exit data map(delete:hy_pressure_int[0:nz+1])
#pragma omp target exit data map(delete:sendbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target exit data map(delete:sendbuf_r[0:NUM_VARS*hs*nz])
#pragma omp target exit data map(delete:recvbuf_l[0:NUM_VARS*hs*nz])
#pragma omp target exit data map(delete:recvbuf_r[0:NUM_VARS*hs*nz])
#pragma omp target exit data map(delete:sendbuf_local[0:NUM_VARS*2*hs*(nz+2*hs)])
#pragma omp target exit data map(delete:recvbuf_local[0:NUM_VARS*2*hs*(nz+2*hs)])
```


miniWeather: Percentage Speedup with ODOS-MPI



M. Usman, M. Benito, S. Iserte, and A. J. Peña, "ODOS-MPI: HPC-Friendly SmartNIC Offloading of Computation/Communication Kernels", in SC25



BREAK

AGENDA

See the updated
agenda at

<https://github.com/gt-crnc-hrg/smartnic-tutorial-isc26>



Please share your
feedback in a short
4 question survey!

Time	Topic	Details
9:00-9:10	Introduction	Attendee Survey; intro to Hands-On
9:10-9:25	Communication Offloading	SmartNIC overview DPU examples DPU programming models
9:25-9:40	SmartNIC Use Cases	Packet processing, cyber-security, AI/HPC
9:40-9:55	Infrastructure SW – DOCA and P4 Introduction	Brief introduction to DOCA/P4
10:00-10:30	HPC Programming – Part 1	MPI Offload; DPA Programming
10:30-11:00	HPC Programming – Part 2	OpenMP Offload
11:00-11:30	BREAK	
11:30-11:45	Hands-on DOCA/P4/DPA Demo	Attendees follow along in compiling and running test examples
11:45-12:15	Hands-on HPC Demo – OpenMP/MPI Offload	
12:15-12:30	Storage Acceleration	Discussion of use case for SmartNICs with storage systems for AI
12:30-12:35	Tutorial Survey	
12:35-1:00	Continued Hands-on Activities	Time to work with the instructors on the presented demos and test cases



Hands-On Section

See the **HANDS_ON** updated documentation at <https://github.com/gt-crunch-rg/smartnic-tutorial-isc26>

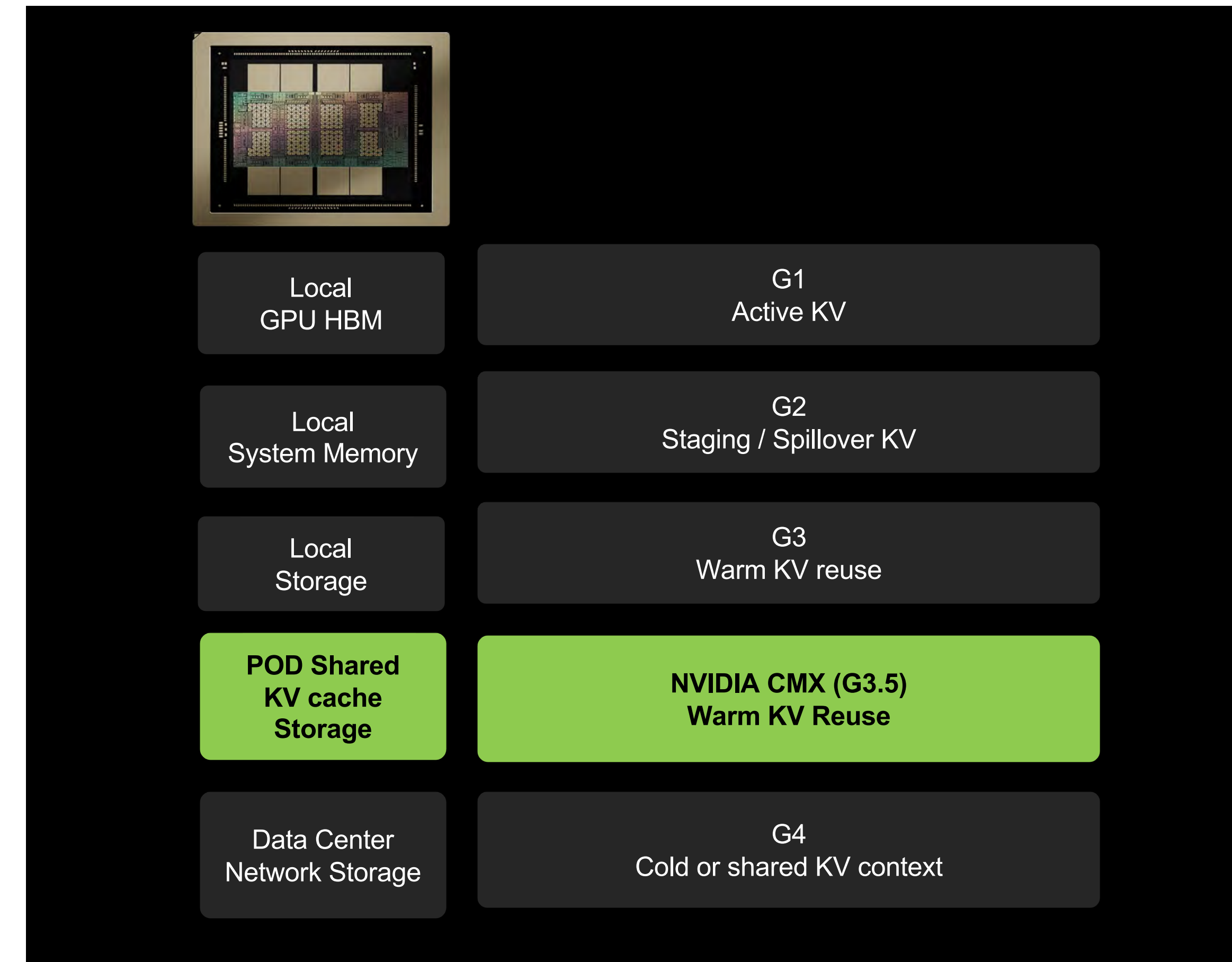


Storage Applications

Inference Context is the New Bottleneck

Introducing New Storage Tier, Purpose Built For KV Cache

- Inference context is large, immutable, and re-computable
- Performance depends on storing and reusing context
- System memory is inherently limited
- Scaling traditional storage increases cost and power
- Efficient context scaling demands a new architecture
- Need a new storage tier – G3.5, purpose built for KV cache

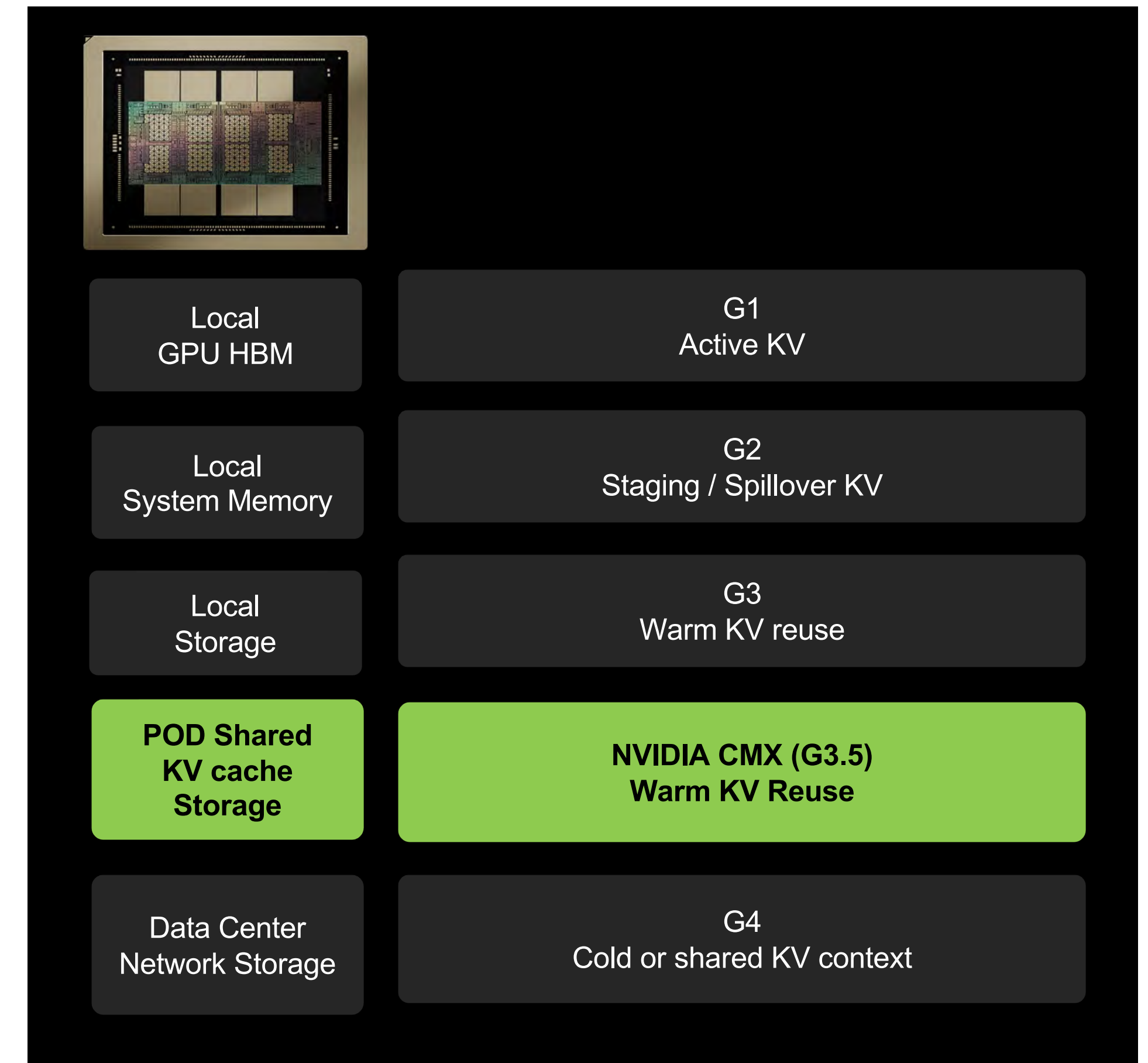


Context Memory Hierarchy

Inference Context is the New Bottleneck

Introducing New Storage Tier, Purpose Built For KV Cache

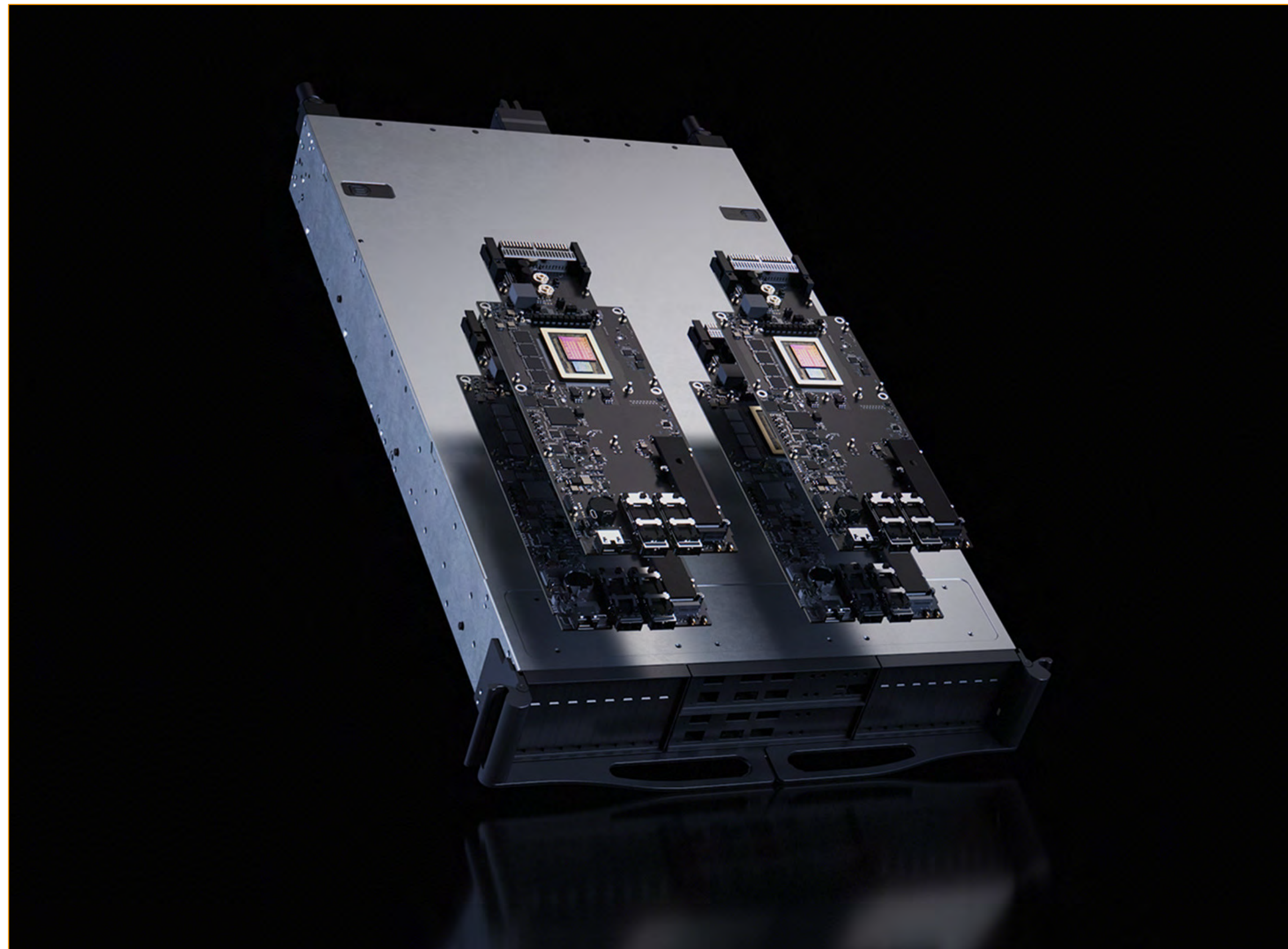
- Inference context is large, immutable, and re-computable
- Performance depends on storing and reusing context
- System memory is inherently limited
- Scaling traditional storage increases cost and power
- Efficient context scaling demands a new architecture
- Need a new storage tier – G3.5, purpose built for KV cache



Context Memory Hierarchy

NVIDIA CMX Context Memory Storage Platform

NVIDIA BlueField powered storage infrastructure for the next frontier of AI



NVIDIA CMX Platform

- AI-native context storage tier
- Shared across GPUs for pod-level context reuse
- Complements GPU memory, local, and networked storage
- 5X higher token throughput and 5X higher power efficiency
- Accelerated by NVIDIA BlueField-4, Spectrum-X Ethernet, DOCA, and Dynamo
- Designed by NVIDIA and built by storage industry leaders



NVIDIA DOCA 3.0 For Storage

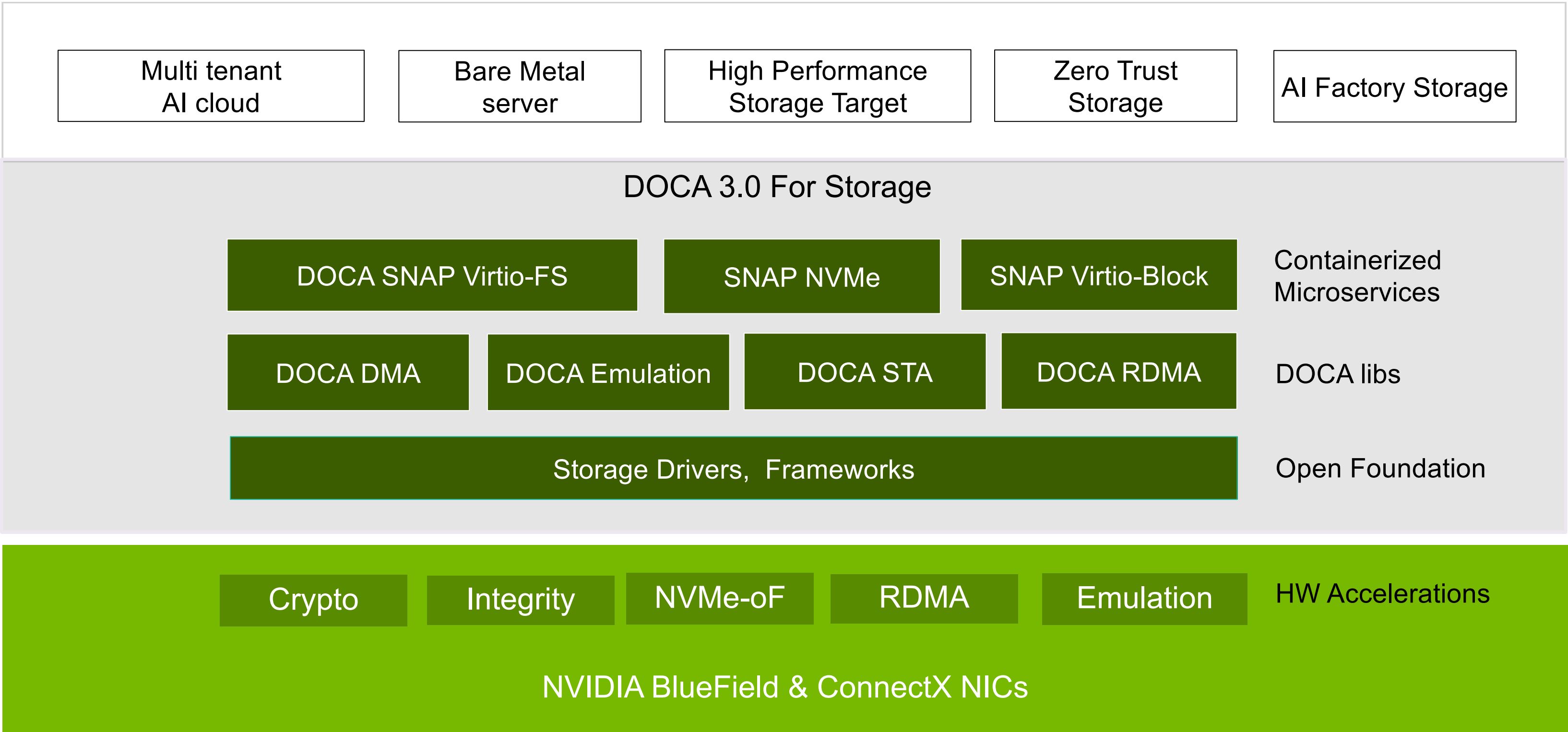
Powering Accelerated & Secured AI Storage Solutions

Disaggregated, virtualized storage at scale

HW accelerated data path

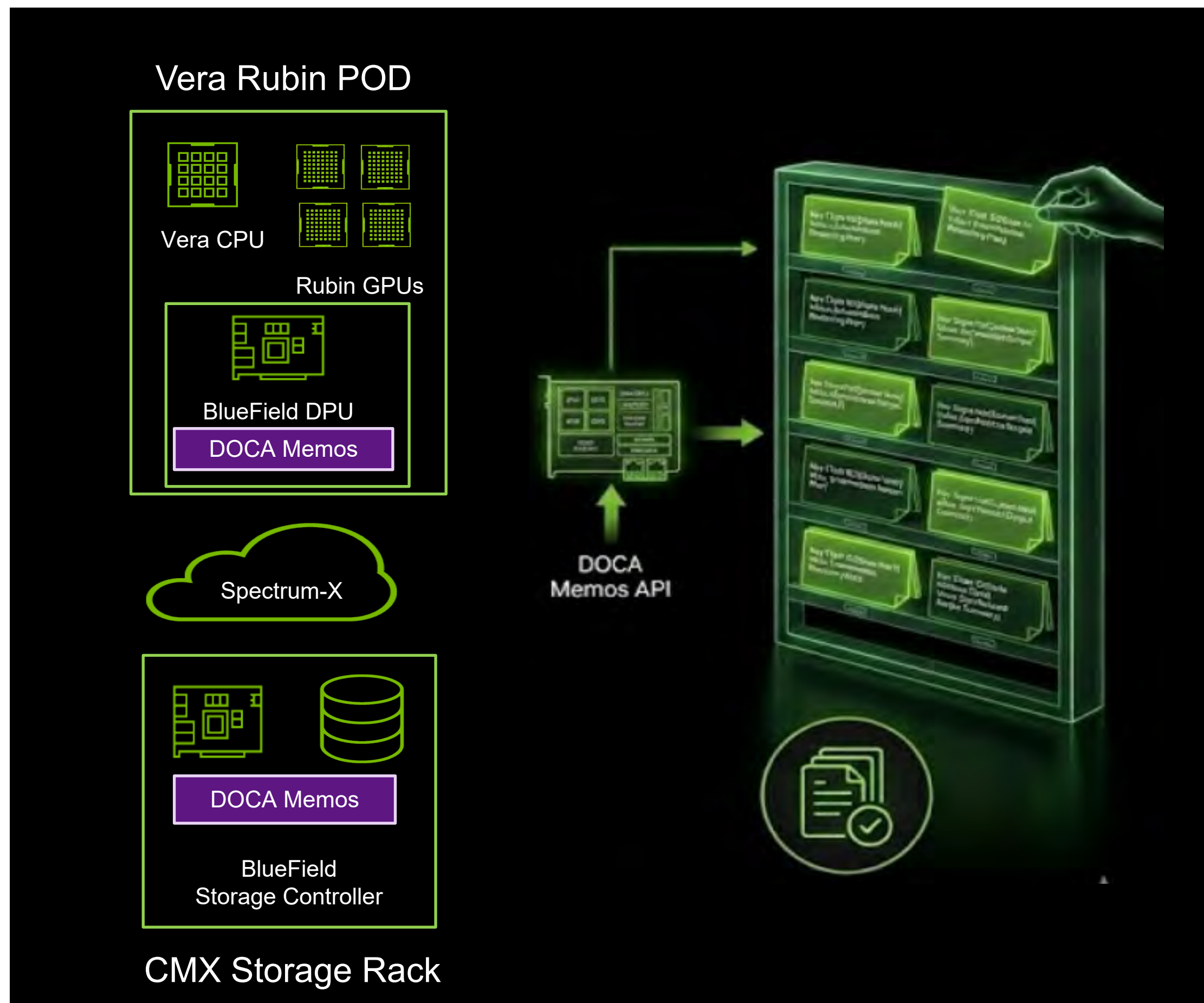
Secured, isolated, High performance storage access

Simple, flexible, operationally efficient storage infrastructure



Introducing DOCA Memos For The Agentic Era

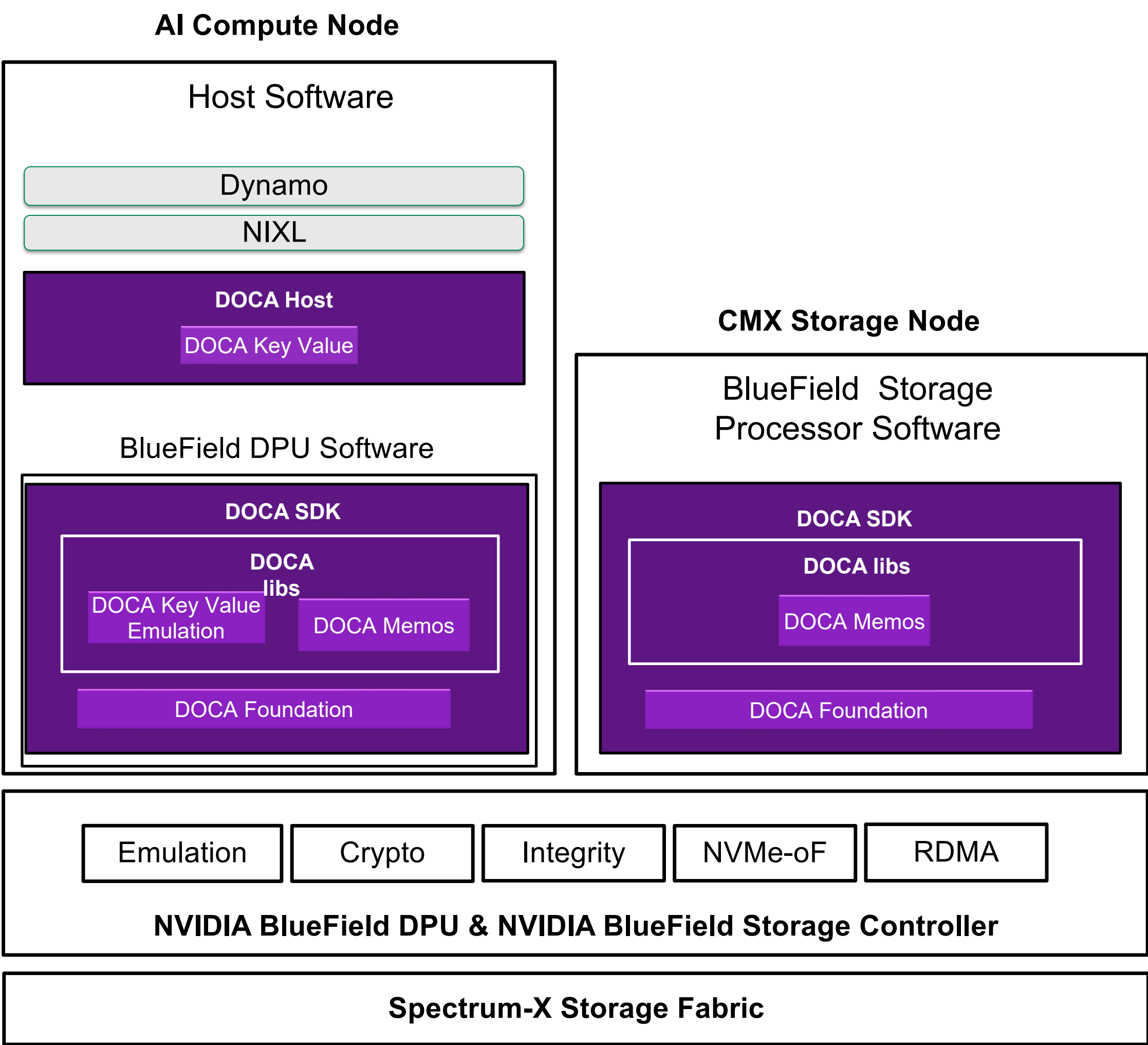
KV Cache Acceleration And Sharing At Pod Scale



- SDK for efficient KV cache data movement and sharing
- Runs on BlueField at compute and context memory nodes
- Purpose built for KV cache inference use case
- Key-value APIs, expose context directly to AI compute
- Developer-ready, extensible framework

DOCA Memos

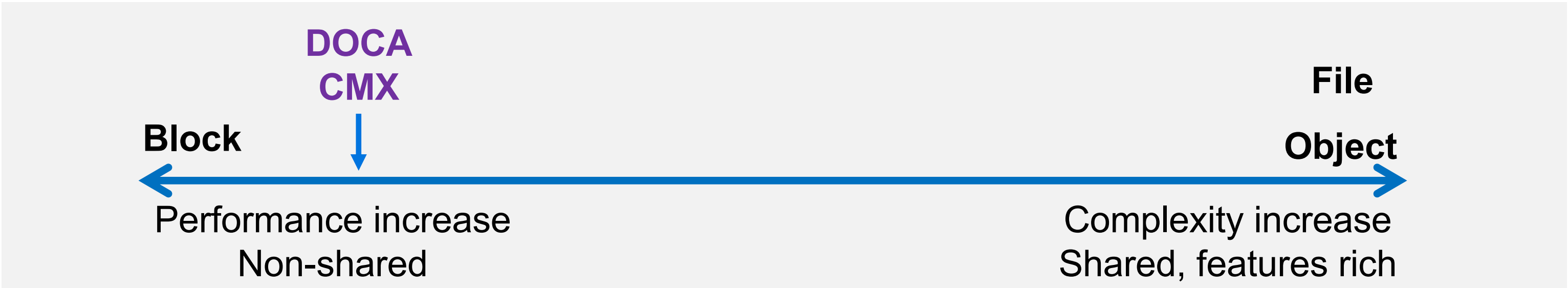
Unlocking Full Potential Of BlueField For CMX



- Tight integration with NVIDIA inference stack (Dynamo, NIXL)
- 800Gbps Spectrum-X, high effective bandwidth
- POD level scalable and power efficient
- Hardware-accelerated KV Cache integrity and encryption
- Elastic & secured KV cache volume
- Empower storage industry ecosystem

DOCA Memos – Purpose Built For KV Cache Use Case

Enabling new storage class, leveraging unique properties of KV cache blocks and Agentic AI

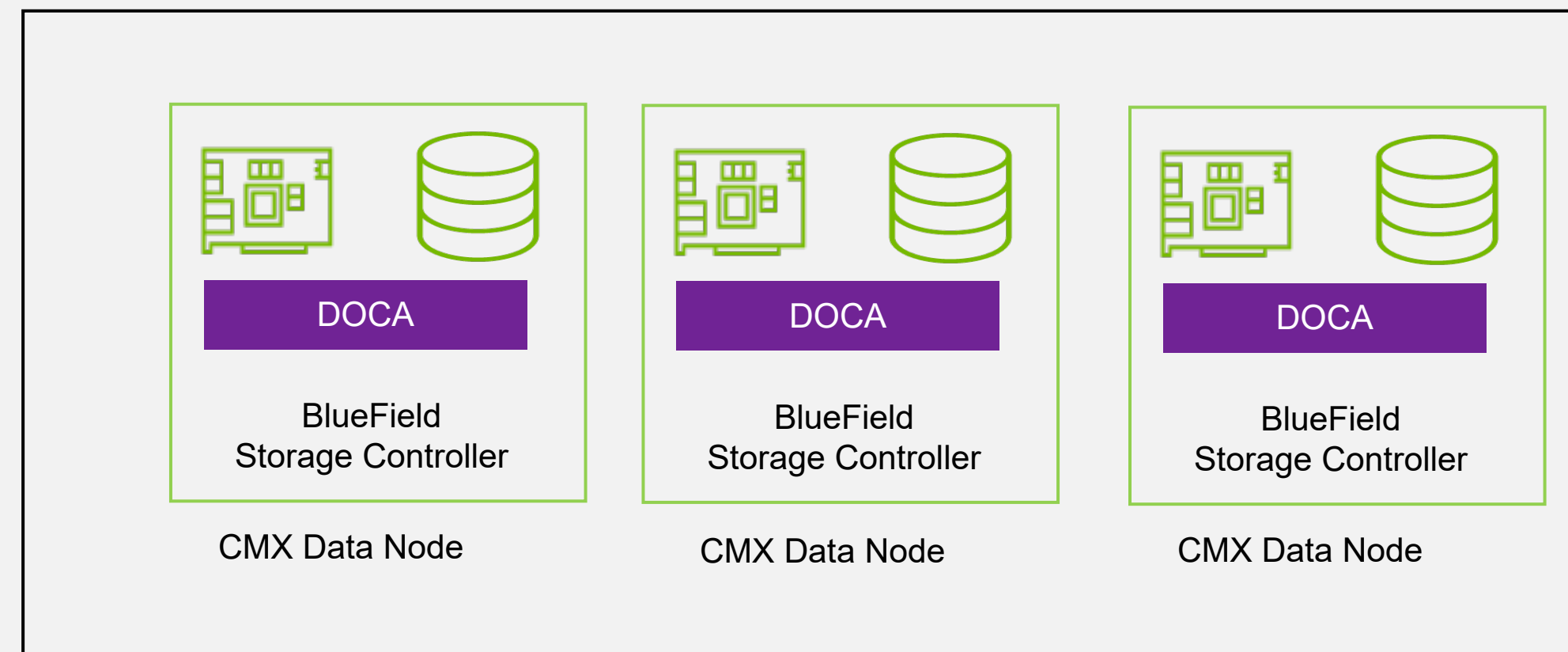
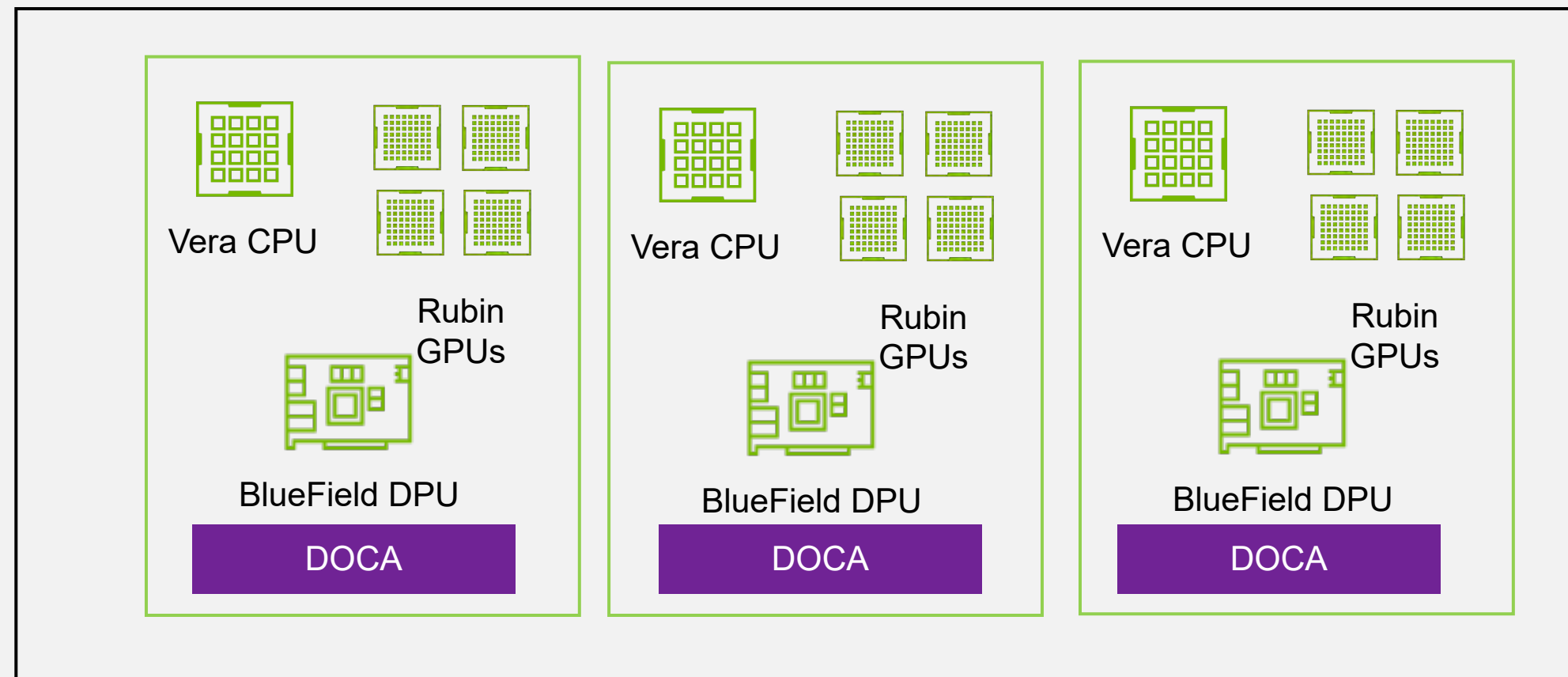


Feature	DOCA Memos	Legacy file/object storage
I/O Size	Large (>1MB), fixed, pre-configured	Flexible, from 1B to GBs
Mutability	Immutable	Updates anywhere, at any granularity
Durability	<100%	100% guaranteed
Metadata	Minimal	Filenames, folders, keys, lookups, free/busy lists
Write Amplification	Close to 1	High
SSD Lifetime	Large IOs in favor of SSD durability	Must be general purpose

CMX Co-Design Advantages For Accelerated KV Cache System

Distributed KV Cache: End-to-End BlueField Roles

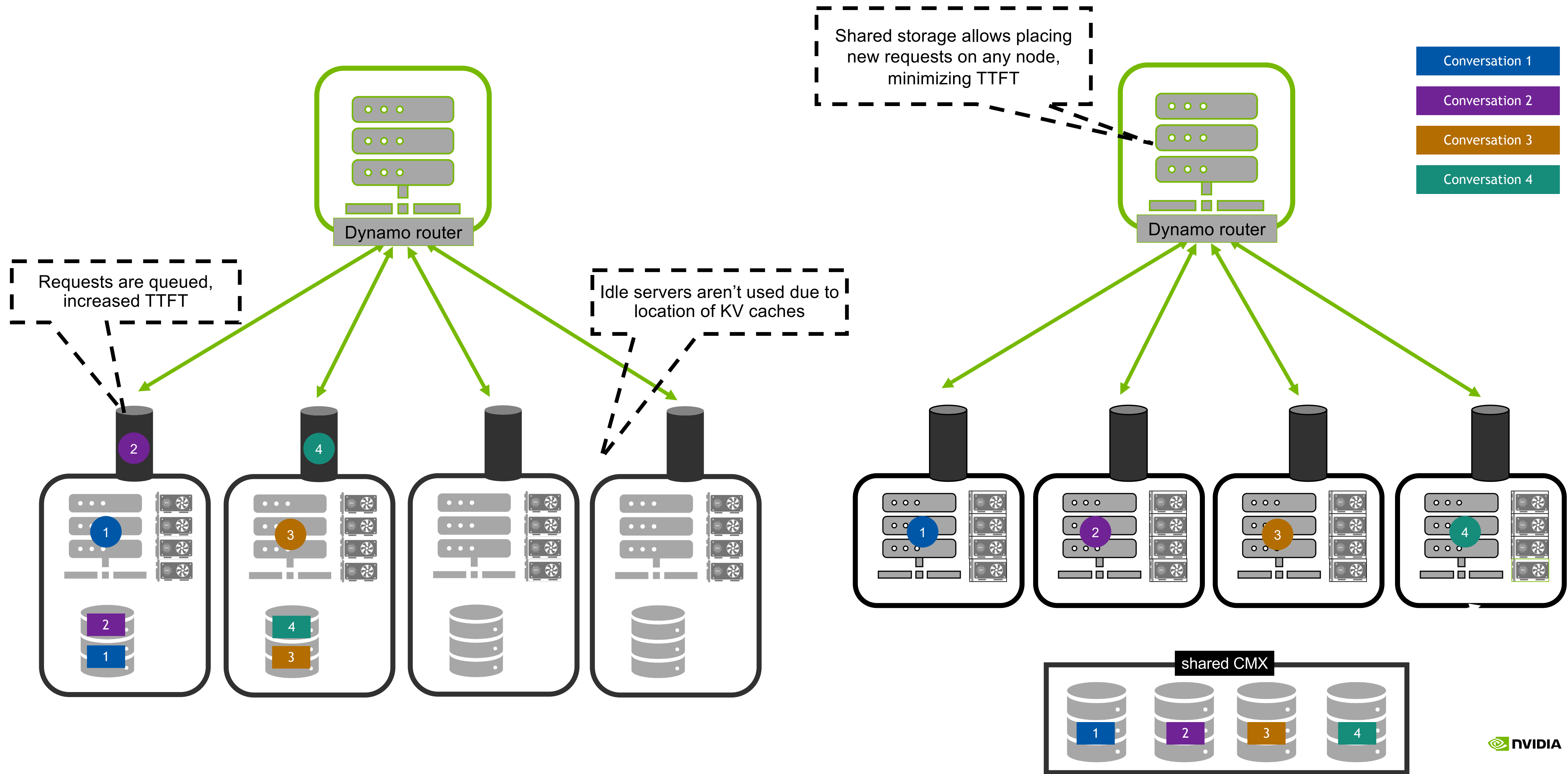
Vera Rubin POD



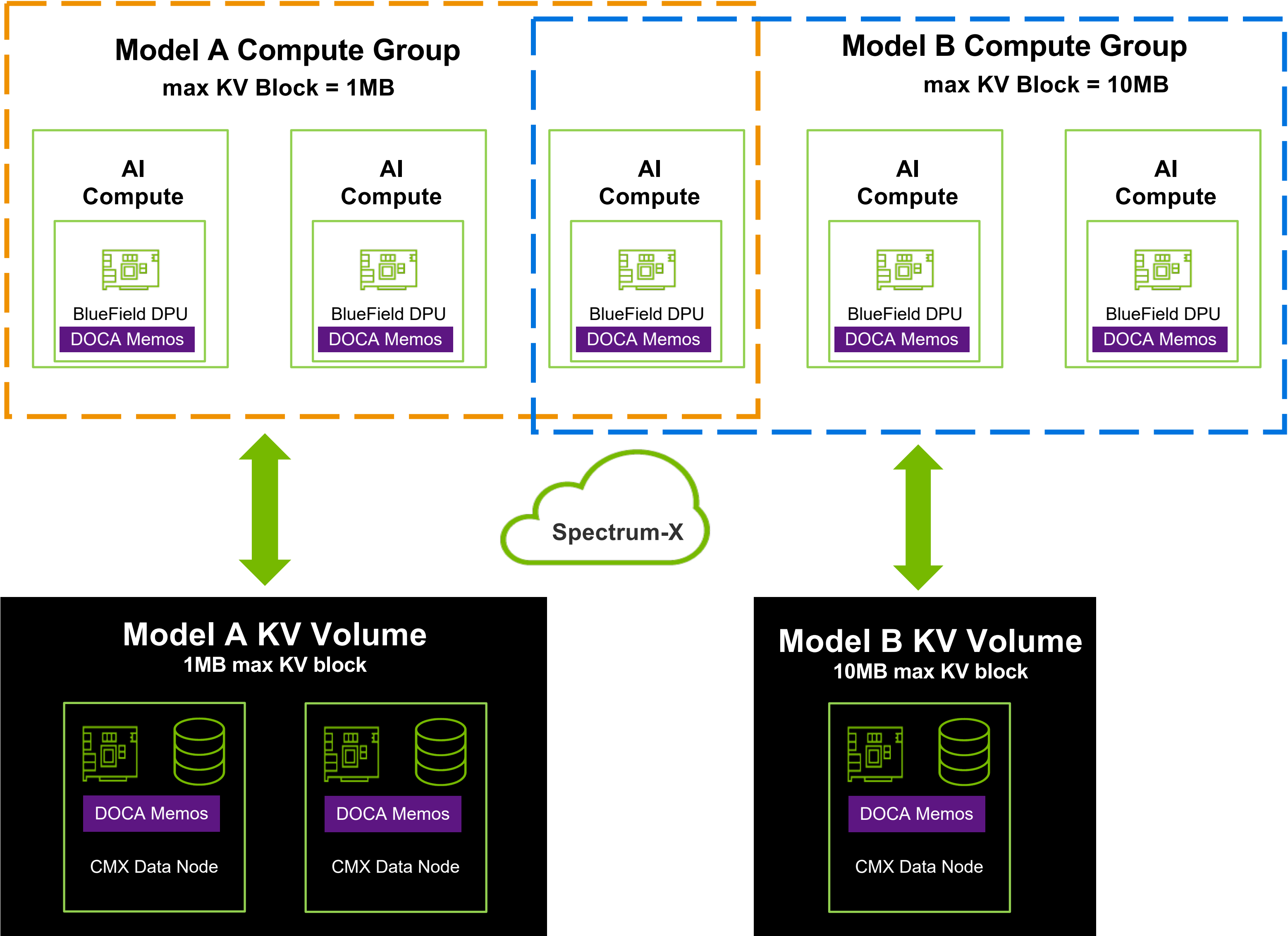
CMX Storage Rack

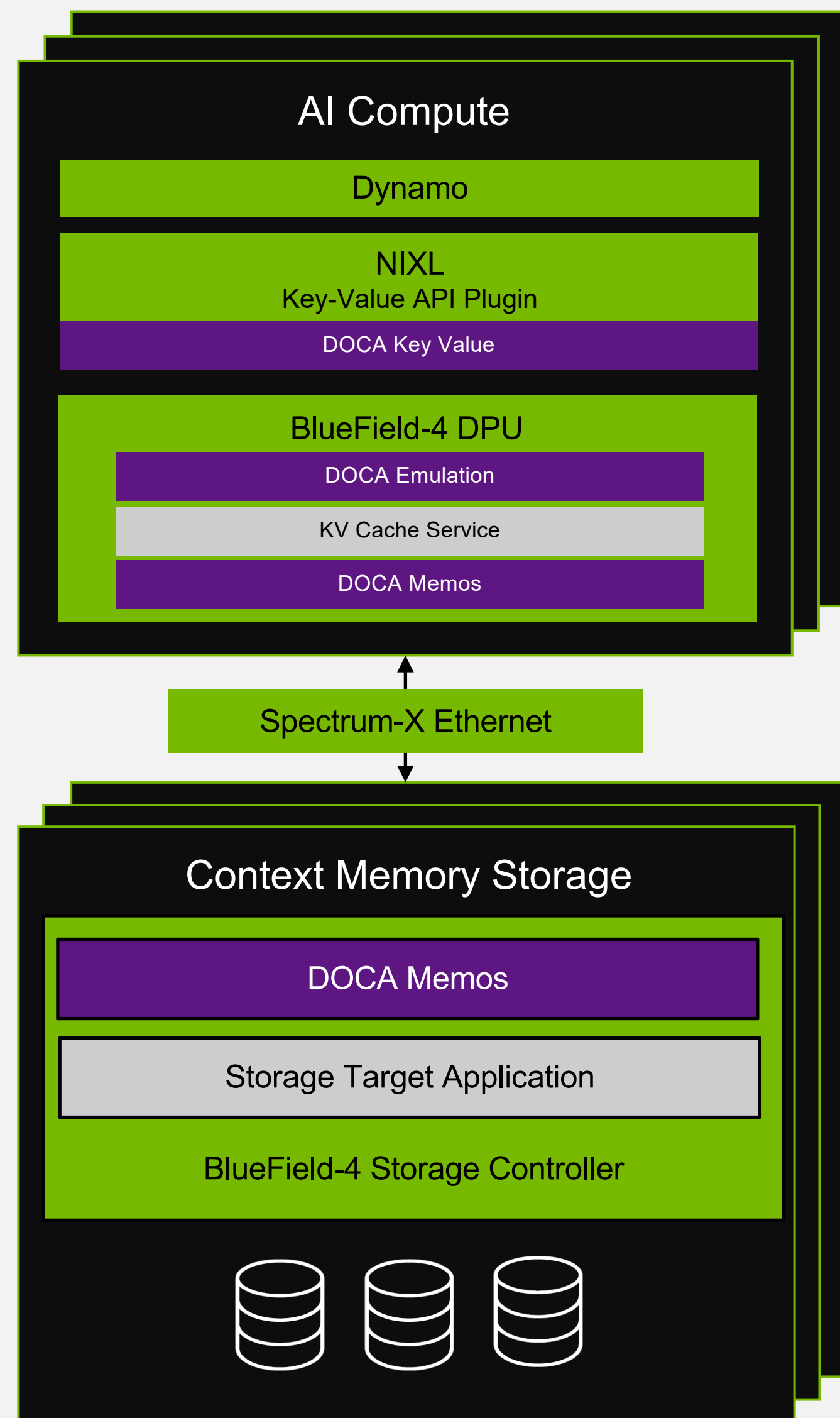
- KV cache pipeline is split for optimal latency, bandwidth, & sharing between
- **Compute-Side BlueField DPU: KV cache Mapping & Isolation**
 - Translates KV requests into KV-aware traffic with KV extensions & hints
 - Isolates KV logic and metadata execution environment with access control
- **CMX-Side BlueField Storage Processor: Placement & Sharing**
 - Data placement on shared volumes with system awareness
 - Manage SSD data placement to maximize endurance & ensuring data consistency
 - Near data decisions for optimal data path utilization

DOCA Memos – Sharing Data Across GPUs In Block Storage Performance



CMX Storage Rack Management With DOCA Memos





NVIDIA

NVIDIA DOCA

Partner

DOCA Memos Components

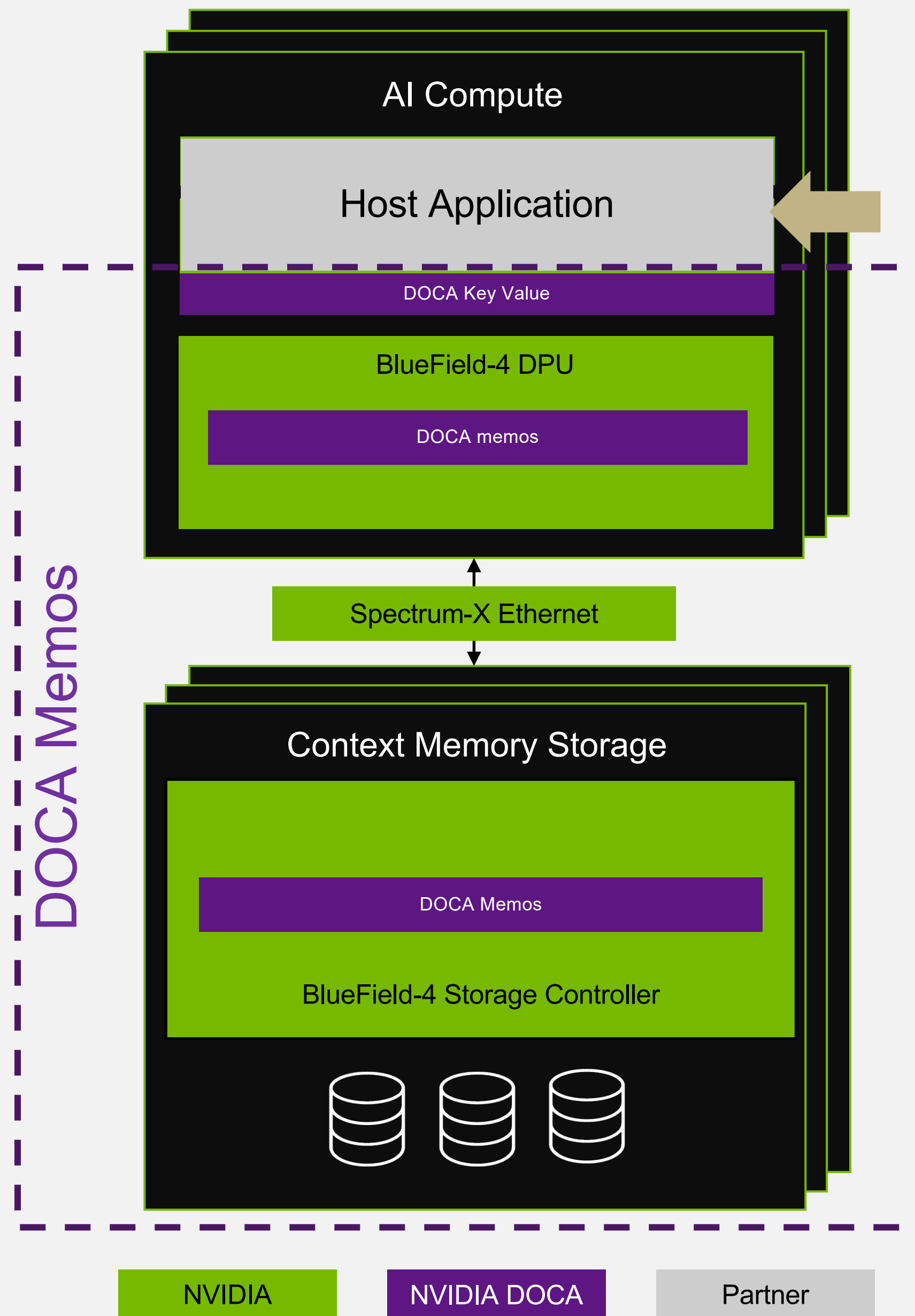
Secure, low-latency KV cache access at pod scale

- **Host:** DOCA Memos host driver
 - Provides C API for inferencing applications, generates KeyValue requests to DPU
 - `store(key, val)` // `key=prefix_hash, val=KV`
 - `retrieve(key)`
- **DPU:** DOCA Memos initiator library
 - Provides C API for DPU emulation service, applies Memos logic, generates network requests to CMX
 - `store(key, val)` // `key=prefix_hash, val=KV`
 - `retrieve(key)`
- **CMX:** DOCA Memos target library
 - Handles requests from network, applies additional Memos logic, provides C API for CMX Target software to read/write blocks from/to SSDs
 - `write_blocks_cb(data_ptr, disk, lba, len)`
 - `read_blocks_cb(data_ptr, disk, lba, len)`

Host Applications Integrating With DOCA Memos

Requirements and Best Practices

- Up to 128 bit keys
- Don't gather per-KV cache metadata
 - Expecting “infinite” amount of KV cache blocks
 - Save to CMX with key=prefix_hash, retrieve later using same prefix_hash
- Use huge pages - to reduce amount of page translations
- Avoid *exists()* before *retrieve()*
 - *retrieve()* serves as lookup - fails fast if KV cache isn't found
- Correctly handle failed *retrieve()*
 - Successful *exists()* is not a guarantee...
 - Search in next tier or recalculate
- Support calculation of context “holes”
 - Re-calculate missing KV blocks in the middle, keep using the rest
- Modify workload routing to utilize shared storage benefits



NVIDIA Context Memory Storage Platform

Pod-level architecture for sharing inference context at scale

KV Blocks Manager (Dynamo KVBM / LMCache)

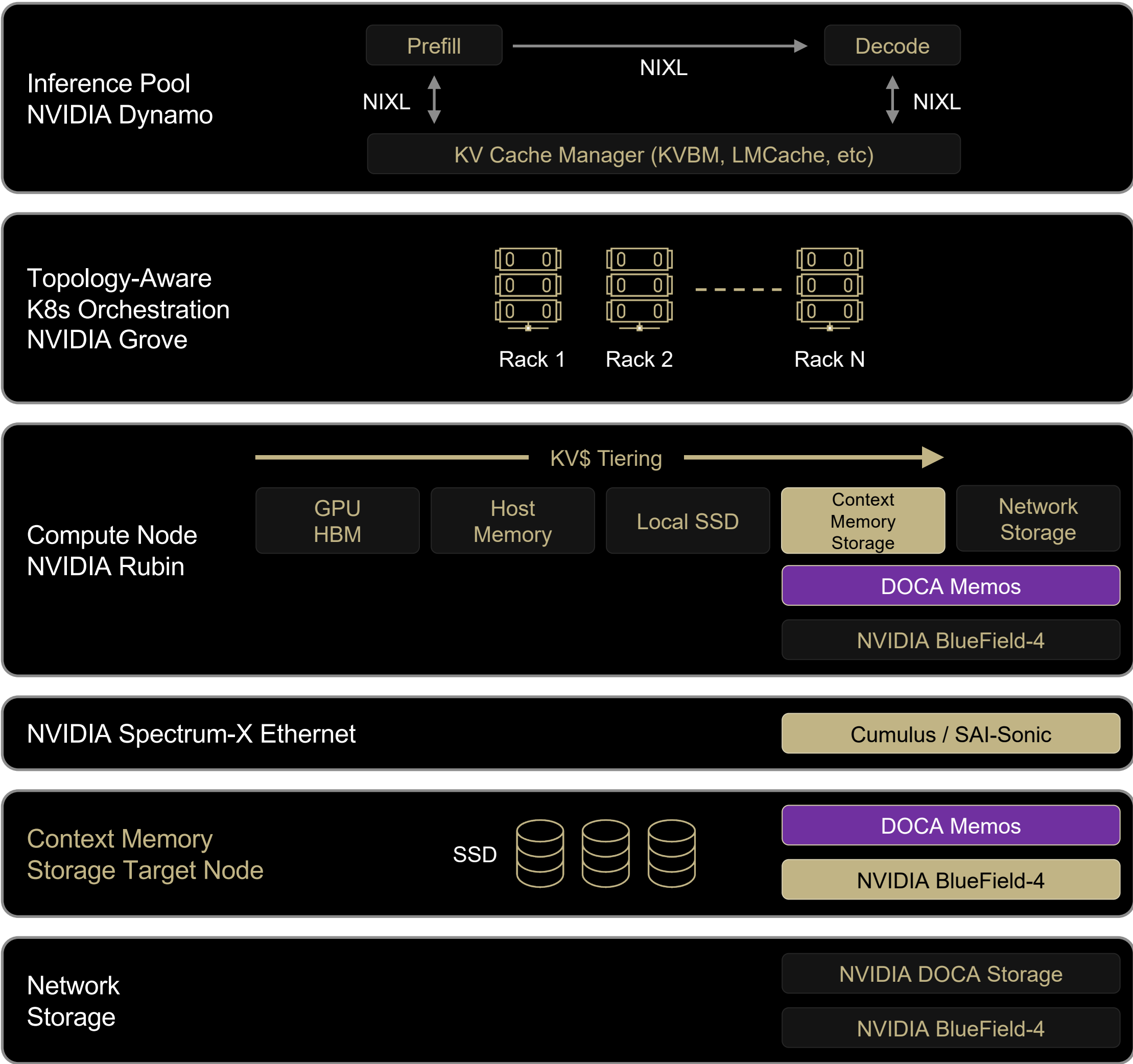
- NIXL support for DOCA KV and 128bit keys
- Huge pages support
- Avoid metadata tracking in host caching layer
- Support for fine grained error reporting for blocks

NIXL

- New NIXL *DOCA Memos* backend
- Batched store/retrieve/exists support
- Fault tolerance and fine-grained error reporting

Future work

- Partial KV retrieval (“holes”)
- Metrics publishing by DOCA Memos
- Advanced KV eviction and offload filtering policies
- KV hints & workload aware signals

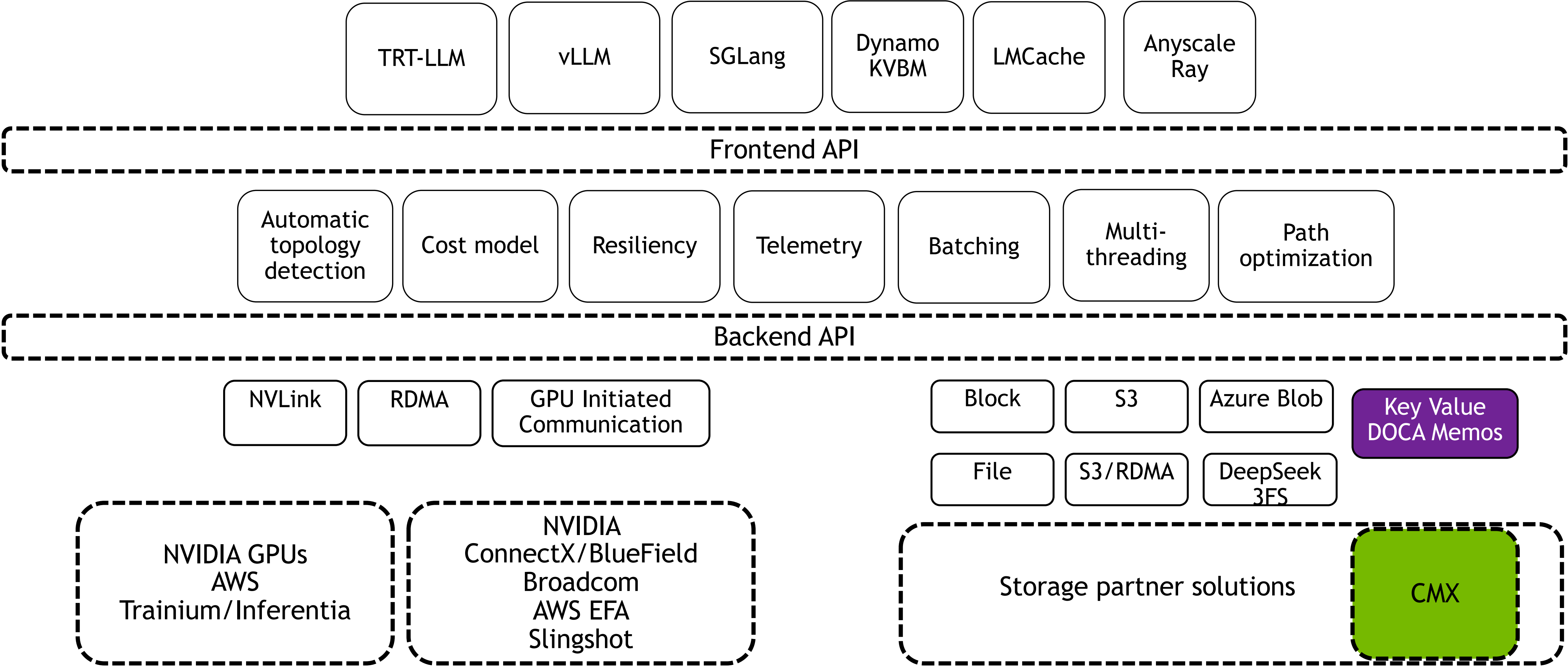


NIXL V1.0 is released!

pip install nixl

NVIDIA Inference Xfer Library (NIXL)

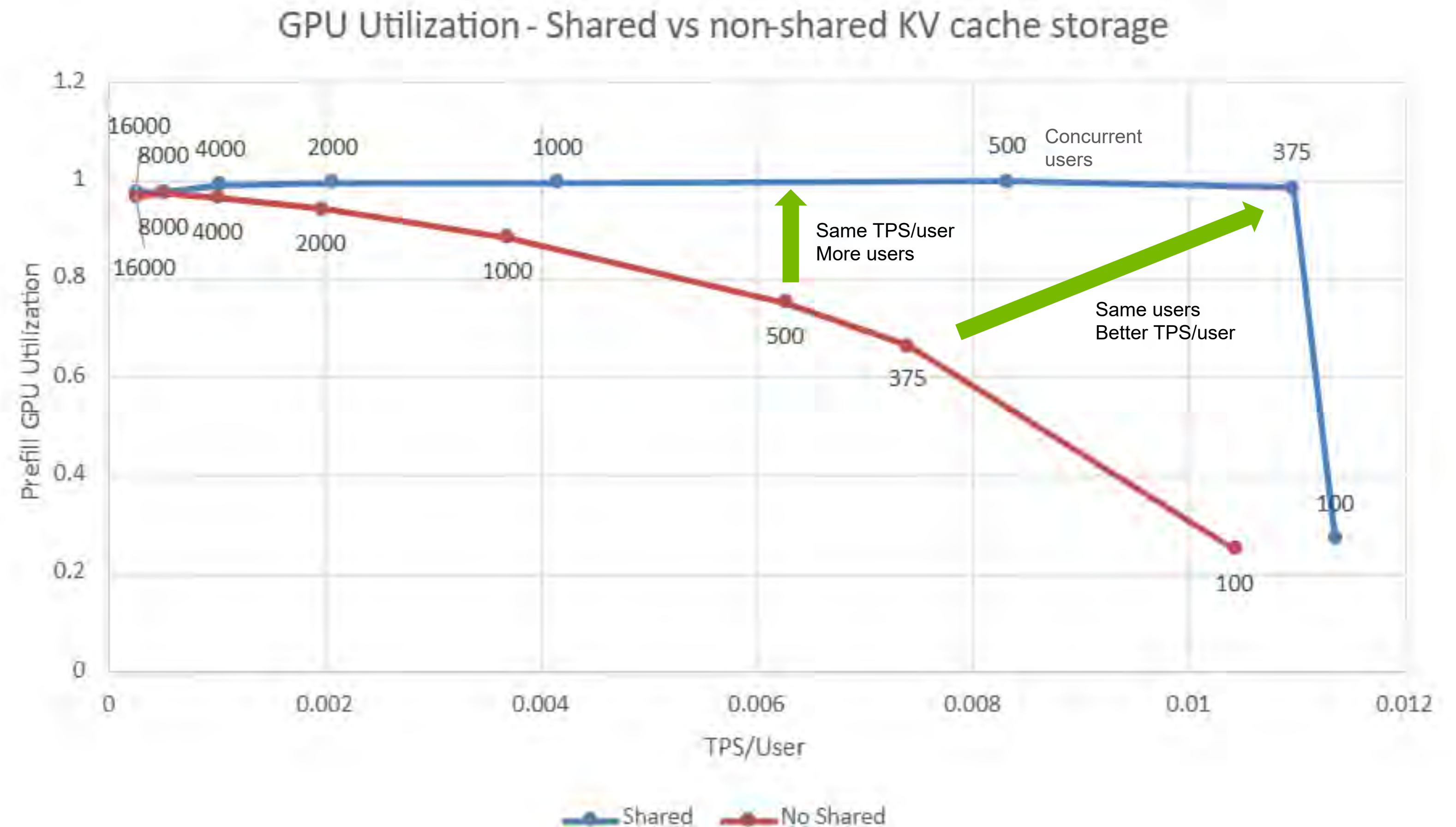
NIXL is a cross-vendor communication framework optimized for high-performance AI inference data transfers



Simulation

Sharing vs. no sharing

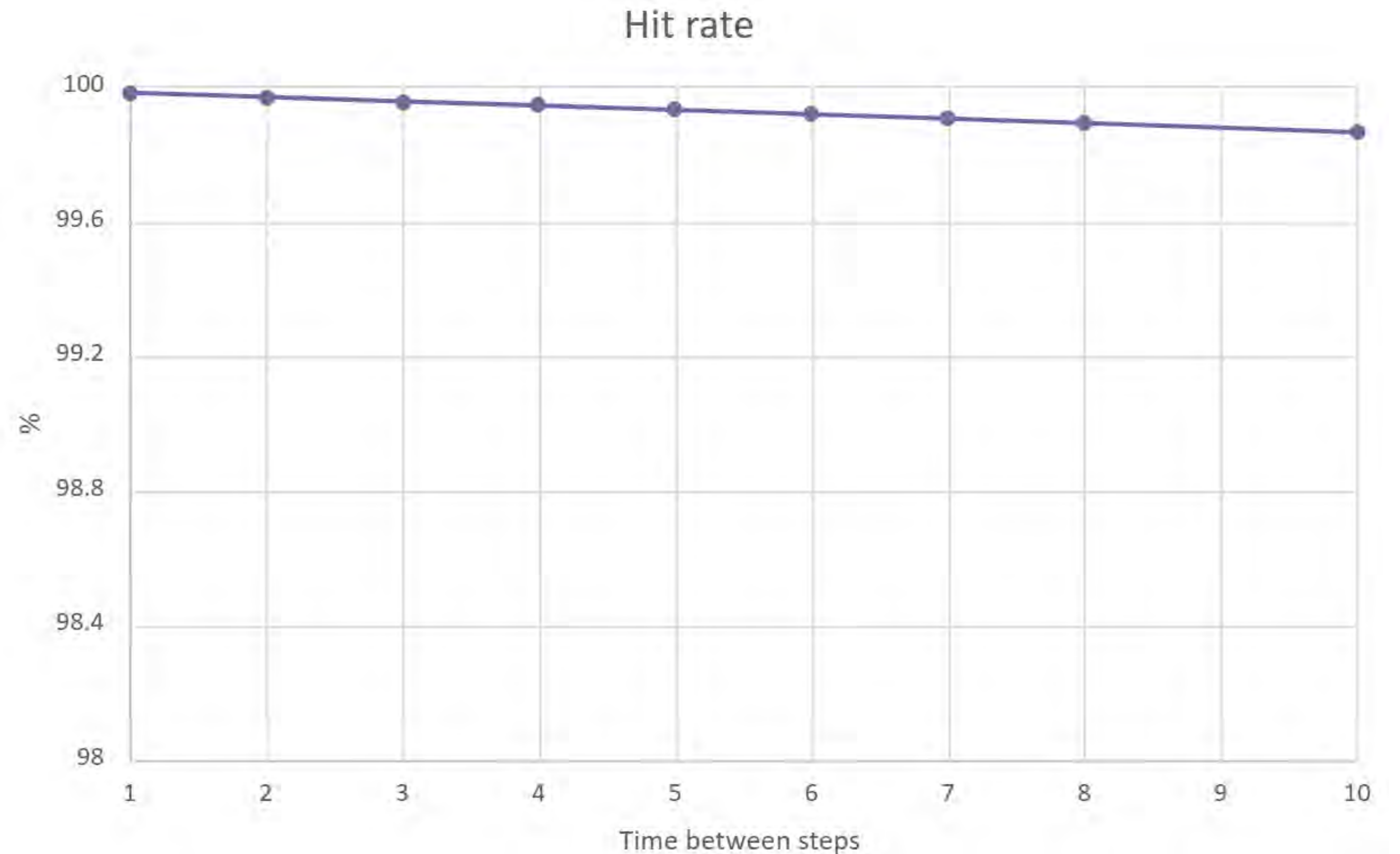
- Simulation parameters
 - 100 GPUs
 - 64 steps per agent
 - 4K tokens added per step as new ISL
 - 128K cached tokens
- Takeaways
 - Sharing KV cache blocks allows 100% GPU utilization across larger range of TPS/User
 - Leads to:
 - Keep specific TPS/User for more users, or,
 - Improve TPS/User for the same amount of users



Simulation

DOCA Memos Hit Rate

- Simulation parameters
 - GPT-OSS-20B, 1000 GPUs
 - Each agent 50 steps, 2K new tokens
 - Time in GPU 0.5s
 - 18 PB CMX storage
 - 700-5000 concurrent agents
 - Selecting ideal #agents to maximize GPU
 - Account for time in GPU for recalculation (miss)
 - Time in GPU 0.5s x number of recalculated steps
- Takeaways
 - Hit rate is above 99.8% with planned CMX
 - GPU “goodput” is above 96%





Wrap-Up

Summary

In this tutorial we have covered the following:

- Described how SmartNICs can be used as HPC/AI accelerators

- Described how users can benefit from acceleration engines in their current and future applications

- Described how one can use one family of SmartNICs to accelerate applications and communication libraries

- Shared demo experiences using SmartNICs as well as hands-on samples.

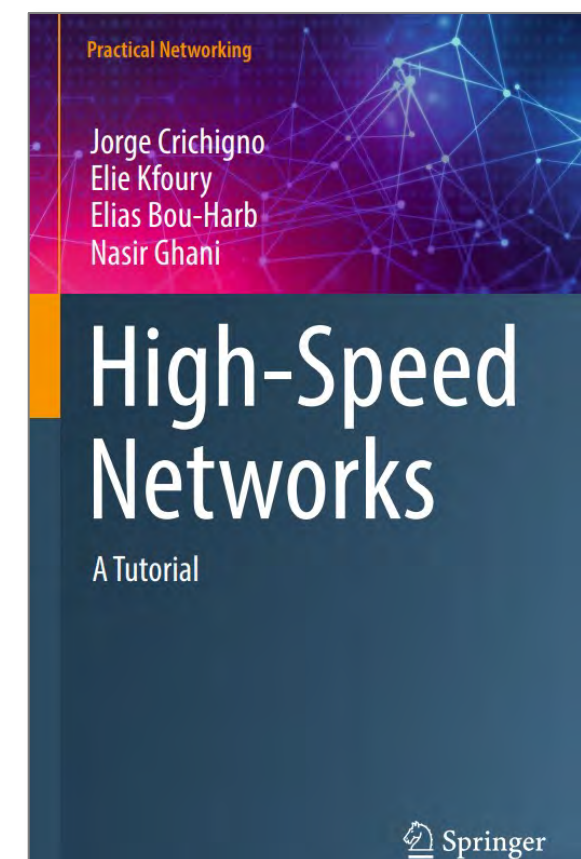
SmartNICs and DPUs for HPC/AI is a wide open area for future research and engineering!

Cybertraining Material (USC)

Information about lab libraries is available at: <https://research.cec.sc.edu/cyberinfra/cybertraining>

Programmable data plane devices

1. DPU Programming using P4
2. DPU Programming using DOCA
3. P4-DPDK Security
4. Introduction to P4-DPDK
5. Cybersecurity Apps with P4 Programmable Data Planes
6. P4 Programmable Data Planes: Applications, Stateful Elements, Custom Packet Processing
7. Intro to P4 Programmable Switches
8. Intro to P4 Programmable Switches with Intel's Tofino
9. P4 Monitoring Applications



Traditional Protocols

10. Software-defined Networking (SDN)
11. Open vSwitch
12. Network Tools and Protocols
13. Introduction to IPv6
14. Open Shortest Path First (OSPF)
15. Introduction to BGP
16. MPLS and Advanced BGP

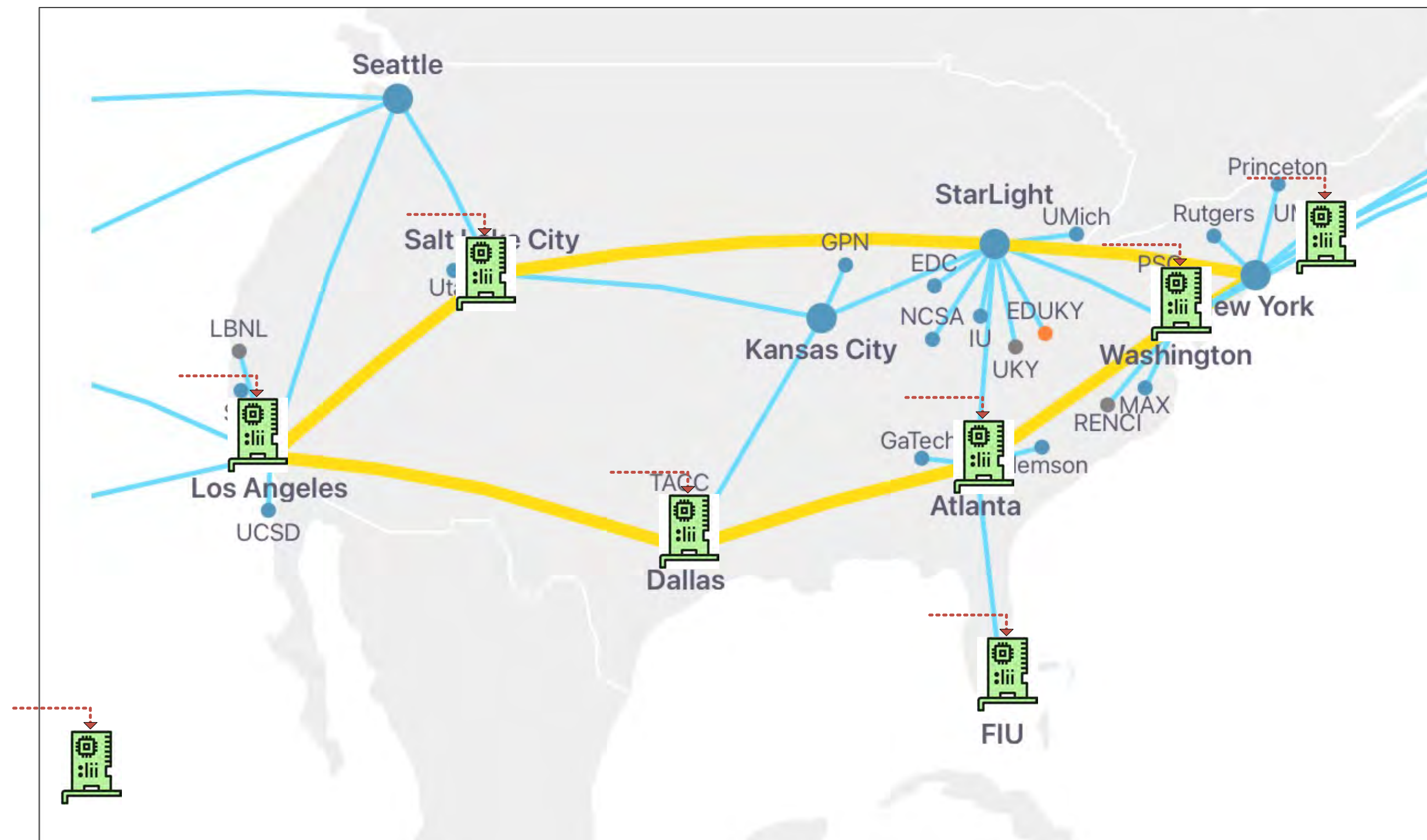


Cybersecurity, Management, and Science DMZs

17. Zeek IDS/IPS
18. Cybersecurity Fundamentals
19. perfSONAR 5
20. P4-perfSONAR
21. Network Management Tools
22. High-speed Networks: A Tutorial

DPU on FABRIC

- BlueField-3 DPUs are being installed on various FABRIC sites
 - Researchers working with US collaborators can generally access this testbed
- Currently the following sites have DPUs installed:
 - 100G: TACC, MICH, MASS, NCSA, DALL, FIU, UCSD, LOSA, KANS, NEWY, ATLA, SEAT, PSC, RUTG, HAWI
 - 400G: SALT, WASH
- These sites are available to the users (and projects that have the permissions)



SmartNIC and DPU Testbed Resources

You can access SmartNICs via a variety of general access testbeds:

USC-led workshops and training resources

<https://research.cec.sc.edu/cyberinfra/workshops>

FABRIC

100 GE connections and testbeds, BlueField-3 DPUs

<https://portal.fabric-testbed.net/>

CRNCH Rogues Gallery testbed

BlueField 2-3, Alveo FPGAs, MangoBoost SmartNICs

<https://crnch-rg.cc.gatech.edu/>

ORNL Wombat cluster

BlueField 2-3

<https://olcf.ornl.gov/olcf-resources/compute-systems/wombat>

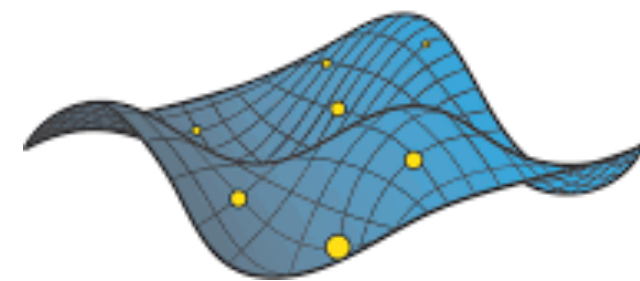
Xilinx HACC centers

AMD Alveos for OpenNIC work

<https://www.xilinx.com/support/university/xup-hacc.html>

Acknowledgements

- This work was supported by the U.S. National Science Foundation (NSF), under awards 2417823 and 2346726. Work at Georgia Tech was supported by NSF award 2316177.
- Elie Kfoury and Jorge Crichigno would also like to acknowledge the FABRIC team.
- Antonio J. Peña was partially supported by the Ramón y Cajal fellowship RYC2020-030054-I funded by MCIN/AEI/10.13039/501100011033 and, by “ESF Investing in your future”. Also co-funded by the Ministerio para la Transformación Digital y de la función pública, within the framework of the Plan de Recuperación Transformación y Resiliencia, and by the European Union – NextGenerationEU[†].



FABRIC



Questions and Attendee Feedback



Please fill out the MS Forms Survey and any conference surveys!